

Data Structures for Engineers and Scientists Using Python



Rakesh Nayak and Nishu Gupta



CRC Press
Taylor & Francis Group

Data Structures for Engineers and Scientists Using Python

The text covers the fundamentals of Python programming and the implementation of data structures using Python programming with the help of worked-out examples. It provides a learning tool for engineers as well as for researchers and scientists of advanced levels. The text further discusses important concepts such as polynomial manipulation, sparse matrices, implementation of stack using the queue model and topological sorting.

This book:

- Discusses the implementation of various data structures such as an array, stack, queue, tree and graph along with sorting and searching algorithms.
- Includes programming tips to highlight important concepts and help readers avoid common programming errors.
- Presents each concept of data structure with a different approach and implements the same using Python programming.
- Offers rich chapter-end pedagogy including objective-type questions (with answers), review questions and programming exercises to facilitate review.
- Covers fundamentals of Python up to object-oriented concepts including regular expression.

It is primarily written for senior undergraduate, graduate students and academic researchers in the fields of electrical engineering, electronics and communication engineering, computer engineering, and information technology.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Data Structures for Engineers and Scientists Using Python

Rakesh Nayak
and Nishu Gupta



CRC Press

Taylor & Francis Group

Boca Raton London New York

CRC Press is an imprint of the
Taylor & Francis Group, an **informa** business

Front cover image: Rakesh Nayak

First edition published 2024

by CRC Press

2385 NW Executive Center Drive, Suite 320, Boca Raton FL 33431

and by CRC Press

4 Park Square, Milton Park, Abingdon, Oxon, OX14 4RN

CRC Press is an imprint of Taylor & Francis Group, LLC

© 2025 Rakesh Nayak and Nishu Gupta

Reasonable efforts have been made to publish reliable data and information, but the author and publisher cannot assume responsibility for the validity of all materials or the consequences of their use. The authors and publishers have attempted to trace the copyright holders of all material reproduced in this publication and apologize to copyright holders if permission to publish in this form has not been obtained. If any copyright material has not been acknowledged please write and let us know so we may rectify in any future reprint.

Except as permitted under U.S. Copyright Law, no part of this book may be reprinted, reproduced, transmitted, or utilized in any form by any electronic, mechanical, or other means, now known or hereafter invented, including photocopying, microfilming, and recording, or in any information storage or retrieval system, without written permission from the publishers.

For permission to photocopy or use material electronically from this work, access www.copyright.com or contact the Copyright Clearance Center, Inc. (CCC), 222 Rosewood Drive, Danvers, MA 01923, 978-750-8400. For works that are not available on CCC please contact mpkbookspermissions@tandf.co.uk

Trademark notice: Product or corporate names may be trademarks or registered trademarks and are used only for identification and explanation without intent to infringe.

ISBN: 978-1-032-46368-1 (hbk)

ISBN: 978-1-032-84003-1 (pbk)

ISBN: 978-1-003-51075-8 (ebk)

DOI: 10.1201/9781003510758

Typeset in Sabon

by Deanta Global Publishing Services, Chennai, India

Contents

<i>About the authors</i>	ix
1 Introduction to Python	1
1.1 Variables, identifiers and keywords	1
1.2 Input and output	2
1.3 Indentation	5
1.4 Comment statement	5
1.5 Standard data types	5
1.6 Operators	14
1.7 Flow control statements	19
1.8 Functions	30
1.9 Classes and objects	39
1.10 Variables and methods	45
1.11 Public, private and protected variables	49
1.12 Methods	51
1.13 Class inside a class (inner class)	54
True-false questions	56
Fill-in-the-blank questions	57
Multiple-choice questions	57
Descriptive questions	62
Answers to true-false questions	62
Answers to fill-in-the-blank questions	62
Answers to multiple-choice questions	62
2 Fundamentals of data structures	63
2.1 Introduction	63
2.2 Classification of data structures	64
2.3 Descriptions of various data structures	66

2.4 Abstract data types (ADTs) 72

2.5 Algorithms 73

True-false questions 82

Fill-in-the-blank questions 83

Multiple-choice questions 83

Descriptive questions 86

Answers to true-false questions 86

Answers to fill-in-the-blank questions 86

Answers to multiple-choice questions 86

3 Arrays

87

3.1 Introduction 88

3.2 Multidimensional arrays 104

3.3 Applications 105

True-false questions 123

Fill-in-the-blank questions 123

Multiple-choice questions 123

Descriptive questions 130

Answers to true-false questions 130

Answers to fill-in-the-blank questions 130

Answers to multiple-choice questions 130

4 Linked list

131

4.1 Singly linked lists 131

4.2 Doubly linked lists 147

4.3 Circular linked lists 160

True-false questions 177

Fill-in-the-blank questions 178

Multiple-choice questions 178

Descriptive questions 181

Answers to true-false questions 181

Answers to fill-in-the-blank questions 182

Answers to multiple-choice questions 182

5 Stacks

183

5.1 Stacks 183

5.2 Stack operations 184

5.3 Stack (ADT) 186

5.4 Implementation 187

5.5 Applications of stack 209

<i>True-false questions</i>	222
<i>Fill-in-the-blank questions</i>	222
<i>Multiple-choice questions</i>	222
<i>Descriptive questions</i>	226
<i>Answers to true-false questions</i>	226
<i>Answers to fill-in-the-blank questions</i>	226
<i>Answers to multiple-choice questions</i>	226

6 Queues 227

6.1	<i>Queue</i>	227
6.2	<i>Queue operations</i>	228
6.3	<i>Queue (ADT)</i>	231
6.4	<i>Implementation</i>	232
6.5	<i>Circular queue</i>	251
6.6	<i>Double-ended queue</i>	257
6.7	<i>Priority queue</i>	262
	<i>True-false questions</i>	273
	<i>Fill-in-the-blank questions</i>	274
	<i>Multiple-choice questions</i>	274
	<i>Descriptive questions</i>	277
	<i>Answers to true-false questions</i>	277
	<i>Answers to fill-in-the-blank questions</i>	277
	<i>Answers to multiple-choice questions</i>	277

7 Trees 278

7.1	<i>Trees</i>	278
7.2	<i>Binary trees</i>	280
7.3	<i>Types of binary trees</i>	281
7.4	<i>Representation</i>	285
7.5	<i>Operations in a binary tree</i>	287
7.6	<i>Binary search trees</i>	295
7.7	<i>AVL trees (height-balanced trees)</i>	315
7.8	<i>Splay trees</i>	328
	<i>True-false questions</i>	336
	<i>Fill-in-the-blank questions</i>	336
	<i>Multiple-choice questions</i>	336
	<i>Descriptive questions</i>	339
	<i>Answers to true-false questions</i>	340
	<i>Answers to fill-in-the-blank questions</i>	340
	<i>Answers to multiple-choice questions</i>	340

8 Graphs 341

- 8.1 *Introduction* 341
- 8.2 *Graph representation* 348
- 8.3 *Traversals* 350
- 8.4 *AND/OR graphs* 359
- 8.5 *Bi-connected components* 361
- 8.6 *Shortest-path problem* 365
- 8.7 *Topological sorting* 367

True-false questions 369

Fill-in-the-blank questions 369

Multiple-choice questions 370

Descriptive questions 372

Answers to true-false questions 374

Answers to fill-in-the-blank questions 374

Answers to multiple-choice questions 374

9 Sorting and searching 375

- 9.1 *Introduction to sorting* 375
- 9.2 *Introduction to searching* 388

True-false questions 393

Fill-in-the-blank questions 393

Multiple-choice questions 394

Descriptive questions 395

Answers to true-false questions 396

Answers to fill-in-the-blank questions 396

Answers to multiple-choice questions 396

***Index* 397**

About the authors

Rakesh Nayak is a distinguished academician and author with a wealth of experience in the field of computer science and engineering. Currently serving as the assistant dean and head of the Department of Computer Science and Engineering at O P Jindal University in Raigarh, Chhattisgarh, India, Nayak has made significant contributions to the academic and administrative domains of technical education.

With a strong educational foundation, Nayak obtained his MCA from Indira Gandhi National Open University, India, in 2007, followed by an MTech (CSE) from Acharya Nagarjuna University, India, in 2010 and a PhD in computer science from Berhampur University, India, in 2013. His academic journey has equipped him with a deep understanding of computer science and engineering, paving the way for his impactful career in academia.

Nayak's professional trajectory showcases his commitment to education and research. Prior to assuming his current role, he held the position of professor in computer science and engineering at Vaagdevi Engineering College, Warangal, and Sri Vasavi Engineering College, Tadepalligudem. Before his tenure in these institutions, he served as a senior lecturer in mathematics for approximately eight years across various engineering and MCA colleges. His extensive experience in teaching and administrative roles spanning over 22 years reflects his dedication to nurturing the next generation of technical professionals.

Throughout his career, Nayak has demonstrated a passion for guiding and mentoring students, having supervised the research work of 12 MTech students. His commitment to academic excellence is further evidenced by his numerous publications in international journals and conferences, where he has shared his insights and research findings with the global academic community.

Beyond his academic pursuits, Nayak is also an accomplished author, having penned five books that contribute to the knowledge base of computer science and engineering. His multifaceted expertise in teaching, research and administrative leadership underscores his significant impact on the technical education landscape.

Nishu Gupta is a senior member of IEEE. He is a research scientist at the VTT Technical Research Centre of Finland, Oulu, Finland. Before this position, he was a postdoctoral fellow in the Department of Electronic Systems, Faculty of Information Technology and Electrical Engineering, at the Norwegian University of Science and Technology (NTNU) in Gjøvik, Norway. He has worked as research associate in the Smart Living Lab, at the University of Fribourg, Switzerland. Prior to this position, he was a postdoctoral fellow in the Department of Information Technology, Faculty of Computing and Information Technology, King Abdulaziz University in Rabigh, Saudi Arabia. He is also a visiting researcher at the University of Oviedo, Gijón, Spain, under the research group on Systems for Multimedia and the Internet of Things (SMIoT). He is a member of the Zero Trust Architecture working group of the MeitY-C-DAC-STQC project under “e-Governance Standards and Guidelines,” Ministry of Electronics and Information Technology (MeitY), Government of India. Before his current fellowship program, he served as an assistant professor in the electronics and communication engineering department at the College of Engineering and Technology, SRM Institute of Science and Technology, Kattankulathur, Tamil Nadu, India. Nishu Gupta received his PhD in 2016 from the Department of Electronics and Communication Engineering, MNNIT Allahabad, Prayagraj, India, which is an Institute of National Importance as declared by the Government of India. He specializes in the field of computer communication and networking. His major work is in the area of IoT-based enhanced safety applications in vehicular communication. He earned his MTech from Delhi Technological University, Delhi, India (formerly Delhi College of Engineering), and BTech from BBDNITM Lucknow, affiliated with U.P. Technical University, Lucknow, India.

Nishu was the recipient of the Best Paper Presentation Award at the 4th International Conference on Computer and Communication Systems held at Nanyang Technological University, Singapore, in 2019. He has published 5 patents and more than 65 research articles in reputed SCI- and Scopus-indexed journals. Dr. Gupta has supervised numerous theses at the master’s level and projects at the bachelor’s level in his main line of work. He has authored and edited several books with international publishers. Nishu is on the editorial board of various internationally reputed journals and transactions. He serves as a reviewer of various SCI-indexed journals and transactions. He was twice awarded for Outstanding Contribution in Reviewing. Gupta has chaired several international conferences and played key roles in successfully organizing various international events. He is in academic cum research collaboration with top academicians and researchers across the globe. He has served as head of the ECE department and chief coordinator of the Institute Innovation Cell, under MHRD-IIC, Government of India, at his previous organization, besides holding many other key positions in the academic, administration and research fields. His research interests include IoT; 5G/6G technologies; dynamic spectrum sensing, intelligent transportation, edge computing; etc.

Introduction to Python

LEARNING OBJECTIVES

After studying this chapter, the reader will be able to:

- Identify variables and keywords
- Read from the keyboard and display on the screen
- Understand different data types and their usage
- Understand different types of operators, their usage and precedence
- Understand different flow control statements and their usage
- Understand functions, their types and application
- Apply all the above concepts to write programs

A programming language is a formal computer language that is used to send commands and data to a machine. Many languages were created with specific goals in mind, such as data processing and scientific calculations. The Python programming language is a very versatile language that can be used for both data processing and scientific calculations.

In this chapter we will discuss some important concepts of Python that will be helpful in learning data structures using Python.

1.1 VARIABLES, IDENTIFIERS AND KEYWORDS

A variable is a space in computer memory that may be used to store values. The type of variable is defined by the data it contains, and Python automatically allocates memory space to each variable. Because of this, Python is known as a *dynamic typing language*. In Python, variables do not need to be declared before they may be utilized.

The name given to a variable or other entity, such as functions or objects, is its identifier. The terms “variables” and “identifiers” are not synonymous. The name of a variable acts as an identifier, but it also contains other properties such as value, type and scope. Readers should note the following:

1. The first thing to keep in mind is that Python is case sensitive. As a result, identifier X and identifier x are not the same thing.
2. Identifiers only comprise of alphabets, digits and underscores.
3. Identifiers must not begin with a number.
4. Identifier names and Python keywords cannot be the same.
5. Identifiers can only be characters, numbers or underscores. For instance, we may name an identifier ‘hello there’ and assign it a value.
6. When an underscore is used as the first character, it is understood as a special identifier.

1.1.1 Assigning values to variables

To reserve memory space, Python variables do not require explicit declaration. When you assign a value to a variable, the declaration occurs automatically. When assigning values to variables, the equal sign (=) is used.

Example

`i=5` refers to a variable ‘i’ that stores an integer data.

`i=5.5` refers to a variable ‘i’ that stores floating-point data. The same variable ‘i’ can be used as integer as well as a floating point number at different parts of the Python program. Python allows to assign a single value to several variables simultaneously.

Example

`i = j = k = 0` refers to the variables ‘i’, ‘j’ and ‘k’ and all of them are assigned to 0 in a single statement.

`i, j, k = 4, 5.5, ‘Hello’` refers to the variables where ‘i’ is assigned the value 4, ‘j’ is assigned the value 5.5 and ‘k’ is assigned the value ‘Hello’, and all the assignments are done in a single statement.

Keywords

Keywords are those reserved words that cannot be used as a variable because these words are used by Python and have specific meaning. Some of the keywords are listed in Table 1.1.

Table 1.1 Keywords

if	for	else	elif
as	except	is	return
assert	exec	in	try
not	and	or	pass
yield	or	continue	import

1.2 INPUT AND OUTPUT

To get the input from the keyboard, the *input()* function is called. The general format of the input function is

```
variable_name = input(["Prompt Message"])
```

The prompt message (optional) is something that is used to show a message to the user before he/she types anything. It is basically used for better understanding of the program. Output statements are the statements that are used to show the result on the screen. For that, we use the *print()* statement. A simple print statement looks like

```
print("object",[sep = " ", end = "\n", file= sys.stdout,flush = false])
```

All non-keyword arguments are converted to strings like *str()* and written to the stream, separated by comma '*sep*' and followed by '*end*'. Both *sep* and *end* must be strings; use of *sep* and *end* is optional. If no objects are given, *print()* will just print a blank line.

Example 1

```
name = input ("Enter your name :")
print("Your name is ", name)
```

Output

```
Enter your name : "Shiva"
Your name is Shiva
```

When we get an input from the keyboard by using *input()*, it takes strings. To make it numeric we need to do type casting.

Example 2

```
no1 = int(input("Enter Number 1 : "))
no2 = int(input("Enter Number 2 : "))
sum = no1 + no2
print("The sum is ", sum)
```

Output

```
Enter Number 1 : 33
Enter Number 2 : 55
The sum is 88
```

When the user's input is to be converted to the required type depending on the input, the *eval()* function attempts to evaluate a string in the same way that the interactive shell would evaluate it.

Example 3

```
no1 = eval(input("Enter number 1 : "))
no2 = eval(input("Enter number 2 : "))
sum = no1 + no2
print("The sum is ", sum)
```

Output 1

```
Enter number 1 : 2
Enter number 2 : 3
The sum is 5
```

Output 2

```
Enter number 1 : "Hello"
Enter number 2 : " How are you !"
The sum is Hello How are you !
```

There are many escape characters that are used to format a *print()* statement (Table 1.2).

Table 1.2 Escape characters

Escape character	Meaning
\n (New line)	Used to shift the cursor control to the new line.
\t (Horizontal tab)	Used to shift the cursor to a couple of spaces to the right in the same line.
\' (Apostrophe or single quotation mark)	Used to display the single-quotation mark.
\" (Double quotation mark)	Used to display the double-quotation mark.
\\ (Backslash)	Used to display the backslash character.

Example 4

```
print("I am learning Python \nand I am enjoying it")
print("I am learning Python \tand I am enjoying it")
print("I\'m learning Python and I\'m enjoying it")
print("I am learning \"Python\" and I am enjoying it")
print("I am learning Python \\ I am enjoying it")
```

Output

```
I am learning Python
and I am enjoying it
I am learning Python      and I am enjoying it
I'm learning Python and I'm enjoying it
I am learning "Python" and I am enjoying it
I am learning Python \ I am enjoying it
```

There are two keyword parameters that may be used with the *print()* method. It is required to understand the following guidelines in order to utilize it. A keyword argument is made up of three parts: a *keyword* that identifies the argument (*end* or *sep*), an *equal sign* (=) and a *value applied to that argument*. Any keyword argument must come after the final positional argument (this is very important).

The *end* keyword argument determines the characters the *print()* function sends to the output once it reaches the end of its positional arguments. This behavior can be changed too with the help of the second keyword argument that is named *sep* (derived from separator). Note that the argument's value may be an empty string also.

Example 5

```
print("I am learning Python","and","I am enjoying it.", sep =
      "...", end = "***")
```

Output

```
I am learning Python...and...I am enjoying it.***
```

1.3 INDENTATION

In Python, braces do not act as a distinguishing feature of a block of code while identifying class, function definitions or flow control. Instead, Python uses indentation of lines to indicate blocks of code. The lines are indented the same distance from the line containing the block of code. Python follows the indentation rule in its markup. The indentation of code blocks, rather than symbols, makes programs easier to read. Programs can be read more easily without symbols, and indentation indicates the type of statement. Code blocks can include single statements as well.

1.4 COMMENT STATEMENT

Whenever we want a line to be completely ignored by a compiler/interpreter, it is referred to as a *comment* statement. We use # (sharp or hash symbol) to comment a single statement or a part of a statement. Python ignores all remaining characters that appear after the hash symbol. Multiple lines may be commented using the triple quotation mark ("""" comment """).

Example 6

```
# This is a single line comment
nol = eval(input("Enter Something : ")) # Anything after HASH
    Symbol
    """ This is
    an example of
    multiple line comment"""
print("You have entered ",nol)
```

Output

```
Enter Something : "Shiva"
You have entered Shiva
```

1.5 STANDARD DATA TYPES

A data type indicates the type of data that the identifier is capable of storing. Because there are several data types available as shown next, the programmer can choose which one is most appropriate for their application. In Python, each data type is a class.

- Numeric
- Boolean
- String
- List
- Tuple
- Set
- Dictionaries

1.5.1 Numeric data type

Objects of numeric data types are created when we assign a numeric value to an identifier. Numerical values are stored in numeric data types. Python supports four different numerical data types:

- int (signed integers)
- long (long integers), they can also be represented in octal and hexadecimal.
- float (floating point real values)
- complex (complex numbers)

Example 7

[illegible]

Output

```
int data is 5
float data is 5.5
long integer data is 55555555555555555555
complex data is (4+5j)
```

1.5.2 Boolean data type

A binary variable that can have one of the two possible values – 0 (False or F) or 1 (True or T) – is called Boolean.

Example 8

```
a = True
b = False
a1 = bool(0)
b1 = bool(1)
print(a,a1)
print(b,b1)
```

Output

```
True False
False True
```

1.5.3 String data type

Strings are nothing more than a continuous combination of characters, numbers and symbols. A string is immutable; once a string is defined it cannot be changed. A pair of single quotes or a pair of double quotes can be used to assign a string in Python. The plus sign (+) is the string concatenation operator and the asterisk (*) is the repetition operator.

Example 9

```
str1 = "I am Learning Python"
str2 = 'I am enjoying it.'
no=2
print(str1)
print(str2)
print(str1 + str2)
print(str2 * no)
```

Output

```
I am Learning Python
I am enjoying it.
I am Learning PythonI am enjoying it.
I am enjoying it.I am enjoying it.
```

Substring of strings may be obtained by using the slice operator ([] and [:]), with indices starting at 0 while moving from left to right and starting at -1 while moving from right to left.

The general syntax is *string_Name=string[start : end : step]*, where the start is the starting index, the end is the ending index and step (optional) is the integer to skip before getting the next character. The start index must always be smaller than the end index, or else the function will return an empty string. Furthermore, the element in the end index is not included.

Example 10

```
str = "I am Learning Data Structure using Python"
print("The first character : ",str[0])
print("Sub string between index 0 and 13 : ",str[0:14])
print("Reverse Order : ",str[-1:-14:-1])
print("Every alternative character starting at index 0 : 
",str[0::2])
```

Output

```
The first character : I
Sub string between index 0 and 13 : I am Learning
Reverse Order : nohtyP gnisu
Every alternative character starting at index 0 : Ia erigDt
tutr sn yhn
```


There are several built-in methods available with string. Few frequently used methods are listed in Table 1.3.

Table 1.3 String methods

Method	Description
isalnum()	Returns true if the string has at least 1 character and all characters are alphanumeric and false otherwise.
isalpha()	Returns true if the string has at least 1 character and all characters are alphabetic and false otherwise.
isdigit()	Returns true if string contains only digits and otherwise false.
islower()	Returns true if the string has at least 1 cased character and all cased characters are in lowercase and false otherwise.
isnumeric()	Returns true if a unicode string contains only numeric characters and false otherwise.
isupper()	Returns true if the string has at least one cased character and all cased characters are in uppercase and false otherwise.
lower()	Converts all uppercase letters in string to lowercase.
max(str)	Returns the max alphabetical character from the string str.
min(str)	Returns the min alphabetical character from the string str.
replace(old, new [, max])	Replaces all occurrences of old in string with new or at most max occurrences if max given.
swapcase()	Inverts case for all letters in string.
upper()	Converts lowercase letters in string to uppercase.
zfill (width)	Returns original string left padded with zeros to a total of width characters; intended for numbers, zfill() retains any sign given (less one zero).
len(string)	Returns the length of the string.
find(str, beg=0, end=len(string))	Determines if str occurs in string or in a substring of string if starting index beg and ending index end are given returns index if found and -1 otherwise.

1.5.4 List

A list is a mutable sequence of objects. It means we can change (update, insert, delete) the objects in a list. It can have data of all different data types under one name.

The general syntax for a list is

List_Name = [object 1, object 2 , object 3 , ... , object n]

Just like elements of a string, the elements of a list can be accessed with an index. A sub-list can be obtained from a list by slicing.

Example 11

```

lst=[]
print("Empty List : ",lst)
str1 = "Python"
lst = list(str1)
print("List from a string : ",lst)
lst = [4,4.5,"Hello",[1,2,3],(9,7,6),{'a','b','c'},{1:'aa',2:'bb'}]
print("Type of list : ",type(lst))
print("Data in list : ",lst)
print("Third element in the list ",lst[3])
print("1st to 4th element in the list : ",lst[1:5])

```

Output

```

Empty List : []
List from a string : ['P', 'y', 't', 'h', 'o', 'n']
Type of list : <class 'list'>
Data in list : [4, 4.5, 'Hello', [1, 2, 3], (9, 7, 6), {'a', 'c', 'b'}, {1: 'aa', 2: 'bb'}]
Third element in the list [1, 2, 3]
1st to 4th element in the list : [4.5, 'Hello', [1, 2, 3], (9, 7, 6)]

```

A new list can also be created from more than one existing list by the “+” (concatenation) operation and “*” (repetition) operation.

Example 12

```

lst1 = [1,2,3]
lst2 = [7,8,9]
print("New list with concatenation is ", lst1 + lst2)
print("New list with repetition is ", lst1 * 2)

```

Output

```

New list with concatenation is [1, 2, 3, 7, 8, 9]
New list with repetition is [1, 2, 3, 1, 2, 3]

```

There are several built-in methods available with lists. Few frequently used methods are listed in Table 1.4.

1.5.5 Tuple

A tuple is an immutable sequence of objects. It can have data of all different data types under one name.

The general syntax for a list is

Tuple_Name = (object 1, object 2, object 3, ..., object n)

Or

tuple_name = object 1, object 2, object 3, ..., object n

Table 1.4 List methods

Method	Description
len()	This method returns the length of a list.
del(list_name[[start[:end]]])	If there is no parameter passed, it deletes the entire list.
max(List_name1,[List_name2])	This function returns the maximum in a list
min(List_name1,[List_name2])	This function returns the minimum in a list.
list.append(item)	This method adds an item to the end of the list.
insert(i, x)	This method inserts an item at a given position.
list.pop([i])	This method removes the item which is at a given location and returns it.
list.remove(x)	This method searches the item in the list and removes it if it is found.
list.extend(L)	This function extends a list by appending all its items into a given list.
list.index(x)	This function returns the location of the first occurrence of the searched item in the list.
list.count(x)	This function returns the number of times the item appears in the list.
list.sort reverse=True False, key=key)	This function sorts the items of the list.
list.reverse()	This function reverses the elements of the list in place.
list.copy()	This function returns a copy of the given list.
list.clear()	This function removes all items from the list.

Just like elements of a string, the elements of a tuple can be accessed with an index. A sub-tuple can be obtained from a list by slicing.

To create a tuple with a single item, a comma is used after the element. All the methods that are available with a list are the same in a tuple except for insert() and deleting a few elements from a tuple.

Example 13

```
tup = ()
print("Empty Tuple : ",tup)
tup = (1,)
print("Tuple with one element: ",tup)
tup = (4,4.5,"Hello",[1,2,3] ,(9,7,6) ,{'a','b','c'},{1:'aa',2:'bb'})
print("Type of Tuple : ",type(tup))
tup1 = tup[1:6:2]
print("1st element to 6th alternative element of tuple: ",tup1)
tup1 = (1,2,3)
tup2 = (7,8,9)
print("The tuple-2 is ",tup2)
tup3 = tup1 + tup2
print("New tuple with concatenation is ",tup3)
tup3 = tup1 * 3
print("New tuple with repetition is ",tup3)
```

Output

```

Empty Tuple : ()
Tuple with one element: (1,)
Type of Tuple : <class 'tuple'>
1st element to 6th alternative element of tuple: (4.5, [1, 2, 3], {'a', 'c', 'b'})
The tuple-2 is (7, 8, 9)
New tuple with concatenation is (1, 2, 3, 7, 8, 9)
New tuple with repetition is (1, 2, 3, 1, 2, 3, 1, 2, 3)

```

1.5.6 Set

A set is an unordered and unindexed collection of unique elements. A set in Python is defined as a collection of elements in curly brackets.

Set_Variable_Name = { item1, item2, item3,...,itemn}

Set_Variable_Name = set([item1, item2, item3,...,itemn])

Example 14

```

s = {1}
s1 = set()
print("Empty set : ",s,s1)
print("Type is : ",type(s),type(s1))
lst= [1,2,3,4,5]
set_A=set(lst)
print("The elements are ", set_A, " the type is ", type(set_A))

```

Output

```

Empty set : {1} set()
Type is : <class 'set'> <class 'set'>
The elements are {1, 2, 3, 4, 5} the type is <class 'set'>

```

There are several built-in set operations available. Few frequently used methods are listed in Table 1.5.

Example 15

```

set_A = {1,2,3}
set_B = {3,4,5}
set_C = set_A | set_B
set_D = set_A & set_B
set_E = set_A ^ set_B
set_F = set_A.symmetric_difference(set_B)
print("Set A :",set_A)
print("Set B :",set_B)
print("Union of Set A and Set B :",set_C)
print("Intersection of Set A and Set B :",set_D)
print("Exclusive-or of Set A and Set B :",set_E)
print("Symmetric difference of Set A and Set B :",set_F)

```

Output

```
Set A : {1, 2, 3}
Set B : {3, 4, 5}
Union of Set A and Set B : {1, 2, 3, 4, 5}
Intersection of Set A and Set B : {3}
Exclusive-or of Set A and Set B : {1, 2, 4, 5}
Symmetric difference of Set A and Set B : {1, 2, 4, 5}
```

Table 1.5 Set operations

Method	Description
set1.union(set2) or set1 set2	This method returns a set of all those elements that are in either of the sets.
Set1.intersection(Set2) or Set1 & Set2	This method returns all those elements that are in both sets.
set1.difference(Set2) or Set1 - Set2	This method returns all those elements that are in the first set but not in the second set.
Set1 ^ set2	This operation returns all those elements that are either in one set or the other set but not in both.
set1.symmetric_difference(Set2)	This method returns all those elements that are in either of the sets, set1 or set2, but not in both.
Set_name.add(element)	This method adds an element to the existing set.
Set_name.discard(element)	This method deletes an element from the set.
Set1.isdisjoint(Set2)	This method checks if any common elements are between two sets.
Set1.issubset(Set2)	This method checks if any one set is a subset of other sets.
Set1.issuperset(Set2).	This method checks if any one set is a superset of other sets.

1.5.7 Dictionary

A dictionary in Python is an unordered collection of items. A key is assigned to each item in a dictionary, and each key is connected with a value. The items in a dictionary are not arranged in any particular sequence. Instead of being regarded as a series of things, the dictionary is handled as a bag of stuff.

The general syntax is

Dictionary_Name = {key_1:value_1, key_2:value_2,...,key_n: value_n}

A dictionary starts and ends with a pair of curly brackets { }. The elements are filled within the curly brackets in the form of a key and a value pair separated by a colon. Each key-value pair is separated by comma. The elements inside a directory may have different types. The keys and the values in a directory may be numeric or string. Each key must be unique. It means

it is not possible to have more than one key of the same value. A dictionary is a one-way tool. It means, if a key is given, the corresponding value can be retrieved, but not vice versa. Refer to an item in a dictionary by its key.

Example 16

```
dict_A={}
print("Empty Dictionary with { }",dict_A," type is ",type(dict_A))
dict_B= dict()
print("The Empty dictionary with dict() ", dict_B, " and ", dict_B)
dict_C = {1:'RAMA', 2:'HARI', 3:'SHYAM', 4:'KRISHNA'}
print('The dictionary with { }', dict_C)
dict_D = dict([ [1,'DURGA'],[2,'SITA'],
[3,'RADHA'],[4,'SARASWATI']])
print("The dictionary is with dict() ", dict_D)
keys = [1,2,3,4]
values = ['RAMA', 'LAKSHMAN', 'BHARAT', 'SATRUGHNA']
dict_E = dict(zip(keys,values))
print("The dictionary with zip ", dict_E)
```

Output

```
Empty Dictionary with { } {} type is <class 'dict'>
The Empty dictionary with dict() {} and {}
The dictionary with { } {1: 'RAMA', 2: 'HARI', 3: 'SHYAM', 4:
'KRISHNA'}
The dictionary is with dict() {1: 'DURGA', 2: 'SITA', 3:
'RADHA', 4: 'SARASWATI'}
The dictionary with zip {1: 'RAMA', 2: 'LAKSHMAN', 3: 'BHARAT',
4: 'SATRUGHNA'}
```

There are various ways we can access the dictionary. Some of the ways are listed in Table 1.6.

Table 1.6 Dictionary methods to access key

<i>Method</i>	<i>Description</i>
<code>dictionary_Name[key]</code>	If the key of the dictionary is known, we can find its value.
<code>dictionary_Name.keys()</code>	This method returns a list built of all the keys within the dictionary.
<code>dictionary_Name.items()</code>	The method returns a list of tuples, where each tuple is a key–value pair.
<code>dictionary_name.get(key, [default=None])</code>	This method returns the value that is associated with the key.

Example 17

```
dict_A = {1:'RAMA', 2:'HARI', 3:'SHYAM', 4:'KRISHNA'}
print("The value with key = 2 ",dict_A[2])
print("The value with key=1 is ",dict_A.get(1))
print("The keys of the dictionary are ",dict_A.keys())
print("The items in the dictionary are ",dict_A.items())
```

Output

```
The value with key = 2 HARI
The value with key=1 is RAMA
The keys of the dictionary are dict_keys([1, 2, 3, 4])
The items in the dictionary are dict_items([(1, 'RAMA'), (2,
'HARI'), (3, 'SHYAM'), (4, 'KRISHNA')])
```

There are several built-in set operations available. Few frequently used methods are listed in Table 1.7.

Table 1.7 Dictionary methods

Method	Description
dictionary_Name.update(Another_Dictionary_Name)	This method updates a dictionary by using another dictionary.
sorted(dictionary_name).	This method returns a sorted key–value pair.
str(dictionary_Name).	This method returns a string of a given dictionary.
setdefault(key,[default_value=None])	This method is used to set a value of a key.

1.6 OPERATORS

An operator is a symbol in programming that generally symbolizes an action or process. Mathematical and logical symbols were used to create these symbols. An operator is a program that can manipulate a value or operand.

1.6.1 Arithmetic operators

Python provides all basic arithmetic operators. The symbols that are used for mathematical operations are +, -, *, /, **, %, //, and a unary minus sign (Table 1.8).

Table 1.8 Arithmetic operators

Operator	Meaning
+	Addition
-	Subtraction and unary minus
*	Multiplication
/	Division
**	Exponential
%	Modulo division
//	Integer division

Example 18

```

a = 10
b = 2
print("The value of a and b is ",a,b)
c = a + b
print(a," + ", b ," = ",c)
c = a - b
print(a," - ", b," = " ,c)
c = a * b
print(a," * ", b," = " ,c)
c = a / b
print(a," / ", b," = " ,c)
c = a ** b
print(a," ** ",b," = " ,c)
c = a // b
print(a," // ",b," = " ,c)

```

Output

```

The value of a and b is 10 2
10 + 2 = 12
10 - 2 = 8
10 * 2 = 20
10 / 2 = 5.0
10 ** 2 = 100
10 // 2 = 5

```

1.6.2 Assignment operators

The equal sign is used to assign a value to a variable. An assignment operator is a statement that assigns the right side value to the identifier that is on the left of the “=” sign, and the most basic form of this statement is *Variable_name=Expressions*. Other assignment operators are listed in Table 1.9.

Table 1.9 Assignment operators

Operator	Description
=	Assigns values from right-side operand to left-side operand.
+=	It adds right operand to the left operand and assigns the result to left operand.
-=	It subtracts right operand to the left operand and assigns the result to left operand.
*=	It multiplies right operand with the left operand and assigns the result to left operand.
/=	It divides left operand with the right operand and assigns the result to left operand.
%=	It takes modulus using two operands and assigns the result to left operand.
**=	Performs exponential (power) calculation on operators and assigns value to left operand.
//=	It performs floor division on operators and assigns value to the left operand.

Example 19

```
a = 10
print("The value of a is ",a)
a += 10
print("The value of a after += is ",a)
a -= 10
print("The value of a after -= is ",a)
a *= 10
print("The value of a after *= is ",a)
a /= 2
print("The value of a after /= is ",a)
a **= 2
print("The value of a after **= is ",a)
a //= 10
print("The value of a after //= is ",a)
```

Output

```
The value of a is 10
The value of a after += is 20
The value of a after -= is 10
The value of a after *= is 100
The value of a after /= is 50.0
The value of a after **= is 2500.0
The value of a after //= is 250.0
```

1.6.3 Comparison (relational) operators

Comparison operators compare the values on both sides and determine the relationship between them. They are also called relational operators. If the condition satisfies, it returns True, otherwise returns False (Table 1.10).

Table 1.10 Relational operators

Operator	Description
==	If the values of two operands are equal, then the condition becomes true.
!=	If values of two operands are not equal, then the condition becomes true.
<>	If values of two operands are not equal, then the condition becomes true.
>	If the value of the left operand is greater than the value of the right operand, then the condition becomes true.
<	If the value of the left operand is less than the value of the right operand, then the condition becomes true.
>=	If the value of the left operand is greater than or equal to the value of the right operand, then the condition becomes true.
<=	If the value of the left operand is less than or equal to the value of the right operand, then the condition becomes true.

Example 20

```

a = 10
b = 2
print("The value of a and b is ",a,b)
print(a," == ", b ,a == b)
print(a," != ", b, a != b)
print(a," > ", b, a > b)
print(a," < ", b, a < b)
print(a," >= ", b, a >= b)
print(a," <= ", b, a <= b)

```

Output

```

The value of a and b is 10 2
10 == 2 False
10 != 2 True
10 > 2 True
10 < 2 False
10 >= 2 True
10 <= 2 False

```

1.6.4 Logical operators

The meaning of these operators is the same as we use them in the English language. Table 1.11 lists the Python logical operators and as it can be seen, there are three operators in this set. They are *and*, *or* and *not*.

Table 1.11 Logical operators

Operator	Description
and	True if both operands are true.
or	True if either of the operands is true.
not	Complements the operand.

Example 21

```

a=20
b=10
c=30
if a >= b and a >= c:
    print( "a is big")
elif b >= a and b >= c:
    print ("b is big")
else:
    print ("c is big")

```

Output

```

c is big

```

1.6.5 Membership operators

There are two membership operators to choose from. They are *in* and *not in*. The *in* operator returns *True* if an item is part of a series, whereas the *not in* operator returns *False* if the item is not a part of a series.

Example 22

```
lst = [1,2,3,4,5]
no1 =3
no2 = 10
print(no1," is in ",lst,no1 in lst)
print(no2," is not in ",lst,no2 not in lst)
```

Output

```
3 is in [1, 2, 3, 4, 5] True
10 is not in [1, 2, 3, 4, 5] True
```

1.6.6 Identity operators

We examine if two variables or objects are identical, that is, if they point to the same memory address. The operators *is* and *is not* are called identity operators.

Example 23

```
x = 5
print(type(x) is int)
print(type(x) is not int)
```

Output

```
True
False
```

1.6.7 Bitwise operators

There are six bitwise operators: bitwise AND, bitwise OR, bitwise XOR, bitwise NOT(Complement), bitwise SHIFT TO THE LEFT and bitwise SHIFT TO THE RIGHT (Table 1.12).

Table 1.12 Bitwise operators

Operator	Description
&	Bitwise AND
	Bitwise OR
^	Bitwise XOR
~	Bitwise NOT (complement)
<<	Bitwise left shift
>>	Bitwise right shift

Example 24

```

x = 50
y = 2
print("x & y = ", x & y)
print("x | y = ", x | y)
print("x ^ y = ", x ^ y)
print("~x = ", ~x)
print("x << y = ", x << y)
print("x >> y = ", x >> y)

```

Output

```

x & y = 2
x | y = 50
x ^ y = 48
~x = -51
x << y = 200
x >> y = 12

```

1.6.8 Python operators precedence

The precedence of all the operators is followed as per Table 1.13.

Table 1.13 Precedence of operators

Operator	Meaning
()	Parentheses
**	Exponentiation
~,+, -	Complement, unary plus, unary minus
*,/, %, //	Multiply, divide, modulo, whole (floor) division
+, -	Addition, subtraction
<<, >>	Right, left bitwise shift
&	Bitwise and
^,	Bitwise exclusive OR, bitwise OR
<=, <, >, >=	Relational operators
<>, ==, !=	Equality operators
=, %=, /=, //=, -=, +=, *=, **=	Assignment operators
is, is not	Identity operators
in, not in	Membership operators

1.7 FLOW CONTROL STATEMENTS

The control flow of statements is divided into two categories: branching and looping. Conditional statements are another name for branching statements. They change the order in which program statements are executed. The statements that branch are:

1. if-else statement
2. if-elif-else statement

Looping structures, also known as iterating statements, are portions of code that are repeated until a termination condition is met. The looping statements are as follows:

1. for loop
2. while loop

If a condition is met when branching or looping, a certain block of instructions will be performed. In Python, indentation is used to express these blocks. The rule of indentation for expressing a block of code is known as off-side notation for coding. A tabbed piece of code is used as a delimiter, followed by a colon.

1.7.1 Conditional statements

Decision-making entails anticipating situations that may arise during program execution and describing actions to be done in response to such conditions. The conditional structures analyze several expressions and return *True* or *False* as the result. Based on the result of the conditional expression (True or False), we must decide which action to take or which statements to execute.

This is the syntax for an if statement in Python:

```
if Conditional_Expression :  
    Statement(s)1  
  
    else:  
        Statement(s)2
```

In the if-else statement notice that the keyword *if* is used to begin the if-else sentence. The conditional expression returns one of two values: True or False. A colon (:) is necessary. Following the colon, begins the body of the *if*, which consists of one or more statements. All of these statements must also be tabbed. Any sentence that is not tabbed and follows the body of the if-else clause is not included. If the conditional expression returns *True*, the statements in the if body are performed. If the conditional expression returns *False*, the lines in the else body are performed. The else clause in the if-else statement is optional (Figure 1.1).

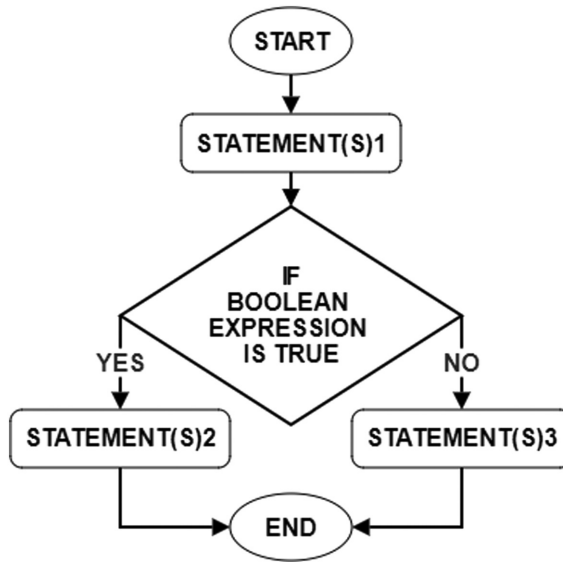


Figure 1.1 Flowchart for if condition

Example 25

```

a=10
b=15
if a < b:
    print (a, "is small")
else:
    print (b, "is small")
  
```

Output:

```
10 is small
```

1.7.2 The elif statement

When we need to verify several conditions, we write if–elif–else statements. It's worth noting that “else if” is spelled “elif” in Python (Figure 1.2).

This is the syntax for the if–elif–else statement in Python is:

```

if Conditional_Expression1 :
    Statement(s)1
elif Conditional_Expression2 :
    Statement(s)2
else:
    Statement(s)3
  
```

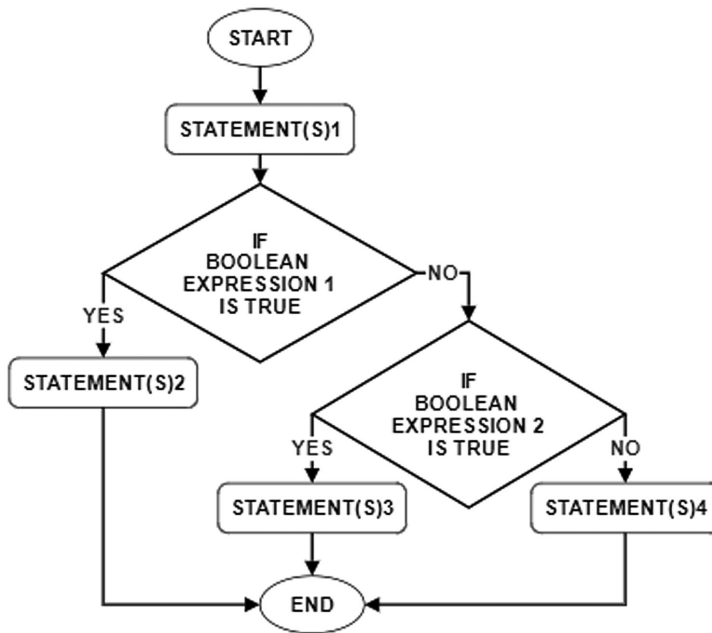


Figure 1.2 Flowchart for elif condition

Example 26

```
my_list1 = ["Hi", "How", "are", "You"]
my_list2 = ["I", "am", "Fine"]
my_input = input("Enter your String ")
if (my_input in my_list1):
    print(my_input, "is in L1st1")
elif (my_input in my_list2):
    print(my_input, 'is in List2')
else:
    print(my_input, 'is NOT available in any List')
```

Output 1

```
Enter your String Hi
Hi is in L1st1
```

Output 2

```
Enter your String am
am is in List2
```

Output 3

```
Enter your String Happy
Happy is NOT available in any List
```

1.7.3 Ternary operator

The ternary operator is a simple if–else statement with only one line of code. The ternary operator is used instead of multiline if–else expressions.

The syntax for a simple ‘ternary operator’ statement is:

Expression1 if Conditional_Expression else Expression2

Example 27

```
a = int(input("Enter First Number "))
b = int(input("Enter Second Number "))
print ("The bigger number is:", a) if a > b else print("The
bigger number is:",b)
```

Output

```
Enter First Number 10
Enter Second Number 5
The bigger number is: 10
```

1.7.4 Looping statement

Iterated (repeated) execution of statements is referred to as looping. In Python, there are two types of looping structures:

1. while loop
2. for loop

1.7.4.1 While loop

While the conditional expression is true, the while loop repeats a series of statements. Because the limit for terminating the loop is unknown in advance, this looping structure is referred to as indefinite.

The syntax for while loop is:

while conditional_Expression :

Statement(s)1

else:

Statement(s)2

In the while loop, notice that, the while loop is initiated with the keyword *while*. The conditional expression returns True or False. The conditional expression does not have to be enclosed by a pair of parentheses. A colon (:) is usually necessary. Following the colon, the following line begins the body of the while loop, which consists of one or more statements. All of these statements must also be tagged. Any statement that is not tabbed and

follows the body of the while loop is not a part of it. When the conditional expression evaluates to True, the statement(s)1 in the while-loop body is performed. Also, be aware that if the while conditional loop's expression constantly evaluates to true (never becomes false), the loop will never finish. A loop that never ends is referred to as an endless loop. The while-else loop's clause is optional. When the control variable's value is not in the collection of items, the statement(s)2 inside the else block is executed (Figure 1.3).

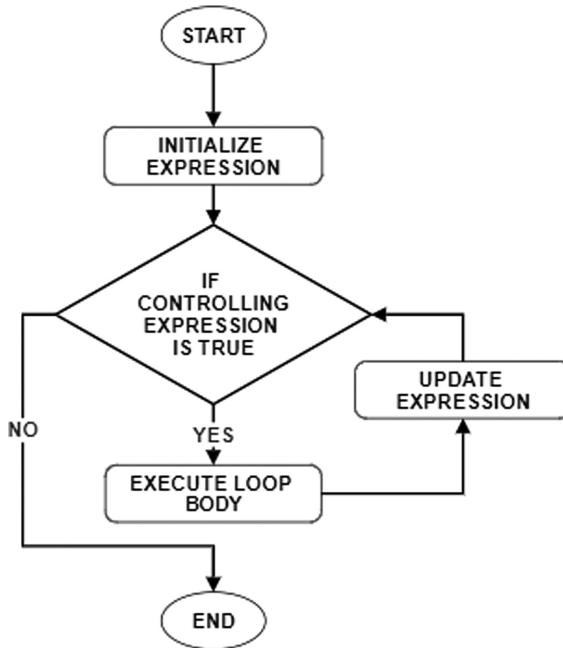


Figure 1.3 Flowchart for while loop

The while-loop is used when we need to repeatedly execute some statements until some conditions are met. We don't know in advance when this condition will be met.

Example 28

```

i = 0
while i < 5 :
    print(i, "I \m Learning Python")
    i = i + 1
else:
    print("inside the while-else")
    print(i, "I \m Enjoying it.")
  
```

Output

```

0 I 'm Learning Python
1 I 'm Learning Python
2 I 'm Learning Python
3 I 'm Learning Python
4 I 'm Learning Python
inside the while-else
5 I 'm Enjoying it.

```

Example 29

```

my_num = int(input("Enter a Number : "))
fact = 1
i = 1
while (i <= my_num):
    fact = fact * i
    i = i + 1
print("The factorial of ", my_num, " is ", fact)

```

Output

```

Enter a Number : 7
The factorial of 7 is 5040

```

1.7.4.2 For loop

Another method for iterating (repeating) statement execution is the for loop. The for loop is used in two situations: first, when we want to run over each item in a sequence and, second, when we want to repeat an action a specific number of times. This looping form is characterized as a definite iteration since there is a defined boundary for loop termination.

The syntax of the for loop is:

for Variable_Name in Collection_of_Items :

Statement(s)1

else:

Statement(s)2

It is important to remember that the for loop starts with the keyword *for*. The loop's control variable (or iterating variable) is any variable name following the for keyword; it counts the loop's turns automatically. The *in* keyword allows you to apply values to the control variable using the element mentioned in the collection of items. The range() function, which is responsible for producing all of the essential control variable values, may be used to get the collection of items variable. This method creates integer sequences from only numbers as input. The usage of a colon is always required. Following the colon, the body of the for loop, which consists of one or more statements. All of these claims must be tagged as well.

Statements that are not tabbed and follow the body of the for loop are not included in the for loop. The else condition in the for loop is optional. When the value of the control variable is not found in the collection of items, the statement(s) within the else block is performed.

When we know the maximum number of times the body of the loop will be run, we utilize the for loop (Figure 1.4).

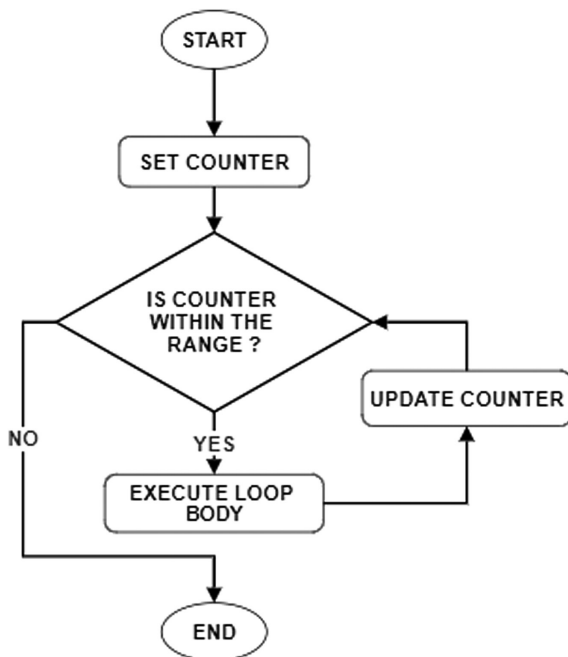


Figure 1.4 Flowchart for for loop

1.7.4.3 The range() function

The range function accepts three arguments, two of which are optional. The first argument is Start, which is the first number in the range; if this parameter is not specified, the default value is 0. The second parameter is End, which is the range's final value; the end value cannot be omitted. If the Increment/Decrement option is missing, the increment is set to 1.

Range function	Range object (output)
<code>range(10)</code>	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
<code>range(1, 10)</code>	1, 2, 3, 4, 5, 6, 7, 8, 9
<code>range(1, 10, 2)</code>	1, 3, 5, 7, 9
<code>range(10, 0, -1)</code>	10, 9, 8, 7, 6, 5, 4, 3, 2, 1
<code>range(10, 0, -2)</code>	10, 8, 6, 4, 2

<i>Range function</i>	<i>Range object (output)</i>
<code>range(2, 11, 2)</code>	2, 4, 6, 8, 10
<code>range(-5, 5)</code>	-5, -4, -3, -2, -1, 0, 1, 2, 3, 4
<code>range(1, 2)</code>	1
<code>range(1, 1)</code>	(empty)
<code>range(1, -1)</code>	(empty)
<code>range(1, -1, -1)</code>	1, 0
<code>range(0)</code>	(empty)

Example 30

```
for i in range(5):
    print(i, "I \'m Learning Python")
else:
    print("inside the for-else")
    print(i, "I \'m Enjoying it.")
```

Output

```
0 I 'm Learning Python
1 I 'm Learning Python
2 I 'm Learning Python
3 I 'm Learning Python
4 I 'm Learning Python
inside the for-else
4 I 'm Enjoying it.
```

Example 31

```
my_num = int(input("Enter a Number : "))
fact = 1
for i in range (1, my_num+1):
    fact = fact * i
print("The factorial of ", my_num, " is ", fact)
```

Output

```
Enter a Number : 7
The factorial of 7 is 5040
```

1.7.4.4 Continue, break and pass

The *continue* statement skips the rest of the code within the loop for the current iteration alone. The loop does not end, but instead continues with the following iteration. When the keyword *continue* appears, the control merely continues to execute for the next value of the control variable without doing anything. *Continue* is used in both the for loop and the while loop.

Example 32

```
for i in range(5):
    if(i == 3):
        continue
    print(i)
print("Out of Loop")
```

Output:

```
0
1
2
4
Out of Loop
```

The `break` statement ends the loop that contains it. When a `break` statement is encountered within a loop, control moves to the sentence immediately after the body of the loop.

Example 33

```
for i in range(5):
    if(i == 3):
        break
    print(i)
    i += 1
print("Out of Loop")
```

Output:

```
0
1
2
Out of Loop
```

The term *pass* refers to doing nothing (no code). It serves as a stand-in. It means that instead of nothing, we write *pass*.

Example 34

```
for i in range(5):
    if(i == 3):
        pass
    else:
        print(i)
print("Out of Loop")
```

Output:

```
0
1
2
4
Out of Loop
```

1.7.4.5 Nested loops

A nested loop is one that is contained within another loop. Any loop can be nested inside of another loop. This indicates that a `for` loop can be nested within another `for` loop, or a `for` loop can be nested inside a `while` loop, or

a while loop can be nested inside another while loop, or a while loop can be nested inside a for loop.

Example 35

```
lower = int(input("Enter lower range: "))
upper = int(input("Enter upper range: "))
if(lower < upper):
    print("Prime numbers between ",lower," and ",upper," are :")
    for num in range(lower, upper):
        p = 1
        for i in range(2, num):
            if(num % i) == 0:
                p = 0
                break
        if(p == 1):
            print(num, "is a Prime Number")
```

Output

```
Enter lower range: 8
Enter upper range: 25
Prime numbers between 8 and 25 are :
11 is a Prime Number
13 is a Prime Number
17 is a Prime Number
19 is a Prime Number
23 is a Prime Number
```

1.7.6 Iterator

The iterator (iter) is a distinct data structure in Python. It may “iterate” through the sequence, beginning at 0 and ending at the last item. Python supports iterating not only over sequences, but also over non-sequence data types (keys in a dictionary, lines in a file, and so on), as well as user-defined objects. An iterator has a method named next that is unique to it (). As with every looping structure, we increment the control variable to get to the next value. The iterator__next__() method is used to obtain the next item in an iterator. After all of the objects have been exhausted, the iterator throws a Stop exception.

Example 36

```
MyTup=(1, 'two', 3.0, 4j,"Five")
i = iter(MyTup)
print("The next item in Tuple is ",i.__next__())
```

Output

```
The next item in Tuple is 1
```

1.8 FUNCTIONS

There are various situations when we need to repeat a job several times while building software. Consider a program that computes a number's factorial at various times throughout the program. It would be inefficient and time-consuming to have to enter the same code for calculating the factorial every time we required it. The solution is to give each piece of code a name and then refer to it whenever it is required.

1.8.1 Defining a function

The general syntax for a function is:

```
def Function_Name([parameter list]) :  
    body_of_the_function  
    [return]
```

Take note of the following in a function: It is usually preceded by the term *def* (for “define”). Following *def* is the function's name (the rules for naming functions are the same as for naming variables). A pair of parentheses () is put next to the function name. Input parameters, which are optional, are specified inside the parentheses. It is possible to have a function with no input arguments if it is optional. If more than one parameter is specified, they are separated by a comma. The line must be terminated with a colon, which is always necessary. The function's body is made up of one or more statements. All of the statements within the function's body are tabbed. Any statement that is not tabbed will be excluded from the function. Finally, there is a return statement that is optional.

1.8.2 Calling a function

A function cannot work by itself. It only works when we give it a command. The next step is to call the function by name. When invoking the function, we should provide the necessary values in the parentheses.

Example 37

```
def sum(a,b):  
    c = a + b  
    print("The sum is ",c)  
  
A = int(input("Enter First number : "))  
B = int(input("Enter Second number : "))  
sum(A,B)
```

Output

```
Enter First number : 12
Enter Second number : 5
The sum is 17
```

1.8.3 Returning results from a function

By providing a “return” statement in the function body, we may get the function’s result or output. We don’t need to include a return statement in the body of a function if it returns nothing.

Example 38

```
def sum(a,b):
    c = a + b
    return c

A = int(input("Enter First number : "))
B = int(input("Enter Second number : "))
print("The sum is ",sum(A,B))
```

Output

```
Enter First number : 12
Enter Second number : 5
The sum is 17
```

1.8.4 Returning multiple values from a function

A function in Python can return several values. When a function computes several outcomes and wants to return the results, we may use the return statement as *return a, b*.

Here two values, *a* and *b*, are returned. The function returns these values as a tuple. To obtain these values, we may use two variables when invoking the function as: *x,y= functionName ()*.

Example 39

```
def Arith_Op(a,b):
    c = a + b
    d = a - b
    e = a * b
    f = a / b
    return c,d,e,f

A = int(input("Enter First number : "))
B = int(input("Enter Second number : "))
sum, sub, mul, div = Arith_Op(A,B)
print("The sum is ", sum)
print("The subtraction is ", sub)
print("The multiplication is ", mul)
print("The division is ", div)
```


Output

```
Enter First number : 15
Enter Second number : 3
The sum is 18
The subtraction is 12
The multiplication is 45
The division is 5.0
```

1.8.5 Pass by value

The phrase *pass by value* means that the function is given a copy of the variable value, and any changes to that value will not be reflected outside the function. In Python, object references are used to provide values to functions. Everything is considered an object in Python. All numbers, strings, tuples, lists and dictionaries are considered objects.

Example 40

```
def change(a):
    a = a + 5
    print("The value of a inside the function ",a)

a = int(input("Enter First number : "))
print("The value of a before calling the function ",a)
change(a)
print("The value of a after calling the function ",a)
```

Output:

```
Enter First number : 15
The value of a before calling the function 15
The value of a inside the function 20
The value of a after calling the function 15
```

1.8.6 Pass by reference

The phrase *pass by reference* refers to passing the variable's reference or memory address to the function. Because the function updates the variable value through memory address, the new value is reflected both inside and outside the function.

Example 41

```
def change(a,b):
    a.append(b)
    print("The list inside the function ",a)

lst = [5,10,15,20]
print("The list before calling the function ",lst)
change(lst,25)
print("The list after calling the function ",lst)
```

Output:

```
The list before calling the function [5, 10, 15, 20]
The list inside the function [5, 10, 15, 20, 25]
The list after calling the function [5, 10, 15, 20, 25]
```

1.8.7 Formal and actual arguments

A function may have some parameters. These parameters are useful to receive values from outside of the function. They are called “formal arguments.” When we call the function, we should pass data or values to the function. These values are called “actual arguments.”

In the previous example, *change(lst,25)* has *lst* and *25* as actual arguments, whereas *change(a,b)* has *a* and *b* as formal arguments. Sometimes actual arguments are simply known as arguments, whereas formal arguments are known as parameters.

The actual arguments are of four types: positional, keyword, default and variable.

1. Positional arguments

Assigning the *i*th argument to the *i*th parameter is called the positional parameter; such arguments are called positional arguments.

Example 42

```
def Function_with_Parameter(a,b):
    c = a + b
    print("The sum of",a,"and",b,"is",c)
Function_with_Parameter(5,10)
```

Output

The sum of 5 and 10 is 15

Here the position of the values of the argument is important. When we called the function for the first time the arguments were 5 and 10. These values are passed as parameters. Inside the function parameter *a* takes the value 5, and parameter *b* takes the value 10.

2. Keyword arguments

Python has another pattern for passing arguments, termed keyword argument passing, in which the meaning of the argument is determined by its name.

Example 43

```
def Function_with_Parameter(a,b):
    c = a + b
    print("The sum of",a,"and",b,"is",c)
Function_with_Parameter(a=5,b=10)
```

Output:

The sum of 5 and 10 is 15

Remember that the values supplied to the parameters are preceded by the names of the target parameters, followed by the equal sign (=) and lastly a value. It makes no difference in which order we supply the arguments. We are unable to utilize a name for a parameter that does not exist. If we don't, we'll get an error message. The value of each argument knows where it is going because of the name. In both situations, the outcome is the same.

3. Default arguments

Instead of sending value as an argument, we can assign a value to the parameter. Assume that the value of some argument is always the same. In that case, we can provide the argument a value. The rest of the code can be provided as an argument (may be positional or may be keyword argument). The term *default parameter* refers to such parameters.

Example 44

```
def Function_with_Parameter(a,b,c=40):  
    sum1 = a + b + c  
    print("The sum of",a,"",b,"and",c,"is",sum1)  
  
Function_with_Parameter(10,20)  
Function_with_Parameter(10,b=30)
```

Output:

```
The sum of 10 , 20 and 40 is 70  
The sum of 10 , 30 and 40 is 80
```

The function contains three arguments, as you can see. One of the parameters has the value 40 (i.e., $c = 40$). The remaining two parameters must be given to the function when it is called (may be positional or may be keyword argument).

4. Variable length arguments

The programmer might not be aware of how many values a function can receive at any given time. The programmer is unable to identify the number of arguments to provide in the function definition. A variable length parameter is utilized in the function definition for this reason. A variable length argument is one that may be filled with any number of values. In the function definition, the variable length parameter is written with a and asterisk (*) before it:

```
def add(farg, *args)
```

Example 45

```
def sum_of_numbers(*no):
    sum1 = 0
    for i in no:
        sum1 += i
    return sum1
print("The sum is", sum_of_numbers())
print("The sum is", sum_of_numbers(4))
print("The sum is", sum_of_numbers(34,2))
print("The sum is", sum_of_numbers(15,50,52))
```

Output

```
The sum is 0
The sum is 4
The sum is 36
The sum is 117
```

1.8.8 Recursive functions

Recursion is a problem-solving technique. It involves breaking down a problem into smaller and smaller sub-problems until we get to a small enough problem that can be solved quickly. A technique in which a function calls itself is known as recursion.

There are three recursion laws:

1. A recursive function must provide a base case that indicates when the function terminates.
2. A recursive function must progress toward the base case.
3. A recursive function must also call itself.

Example 46

```
def rec_fun_factorial(a):
    if a<1:
        return None
    if a == 1:
        return 1
    else:
        return rec_fun_factorial(a-1)*a
x = int(input("Enter a number: "))
print("The factorial of", x,"is", rec_fun_factorial(x))
```

Output:

```
Enter a number: 10
The factorial of 10 is 3628800
```

Observe that, the terminating condition is “if the value of ‘a’ is less than 1, it returns None and if the value of a is equal to 1” the factorial is 1. Otherwise we call the function recursively to find the factorial.

1.8.9 Anonymous function or lambdas

A lambda function is one that has no name. The name “lambda” or “anonymous” comes from the fact that they are not stated as other functions using the *def* keyword. Rather, the *lambda* keyword is used to generate them. Lambda functions are temporary functions that only need to be utilized where they were generated. They may be used everywhere a function is needed. Because of the desire from LISP programmers, the lambda functionality was added to Python.

The general syntax of the lambda function is:

lambda parameters : expression

It should be noted in a lambda function that the term *lambda* is used to start a lambda function. The arguments are placed next to the keyword, which is optional. After it, there is a colon. After the colon, the phrase is written that needs to be evaluated. The lambda function is a function that returns a value.

Example 47

```
First= lambda a,b : a + b
no1 = int(input("Enter First Number : "))
no2 = int(input("Enter Second Number : "))
print("The sum is ", First(no1,no2))
```

Output:

```
Enter First Number : 12
Enter Second Number : 4
The sum is 16
```

With lambda functions, code becomes shorter, cleaner and more legible. The lambda function returns the value of the expression implicitly, so we don't need to add a return statement directly. We can send a function as a parameter to another function on occasion. Lambda functions are the best option in these situations.

We can use lambda functions very commonly with inbuilt functions like *filter()*, *map()* and *reduce()*.

1.8.10 Function decorators

A decorator is a function that takes a parameter and returns another function. A decorator takes a function's result, modifies it and then returns it. As a result, decorators come in handy when a function requires some extra processing. In most cases, the following stages are involved in the development of decorators:

- A decorator function with another function name as an argument should be defined.

- Within the decorator function, we should define a function. The value of the function provided to the decorator function is modified or decorated by this function.
- The inner function that processed or decorated the value is returned.

The current functions are given new meaning by the decorator. In other words, decorators wrap one function around another. Decorators can also be used on functions that are specified by the user.

Decorator is a function that takes a function as an argument and extends the functionality of that function, returning a modified function with the additional functionality. The behavior of the original function remains unchanged while the functionality is extended.

Example 48

```
def improved_Div(func):
    def inner(a,b):
        if a < b and a==0:
            return func(0,b)
        elif b == 0:
            return func(0,a)
        elif a < b:
            return func(b,a)
        else:
            return func(a,b)
    return inner

@improved_Div
def Div(a,b):
    return a/b

n,m= eval(input("Enter two numbers(separated by comma):"))
print("The division is", Div(n,m))
```

Output 1

```
Enter two numbers(separated by comma):15,6
The division is 2.5
```

Output 2

```
Enter two numbers(separated by comma):0,6
The division is 0.0
```

Output 3

```
Enter two numbers(separated by comma):23,0
The division is 0.0
```

1.8.11 Function generators

A generator is a function that is responsible for generating a set of values. We may build generator functions in the same way we write regular functions, but the *yield* keyword is used to return values. Iterators are used to get one value at a time from a database.

The `yield` statement can be used to hold and return a succession of outcomes.

Example 49

```
def even(n):
    i = 1
    while (i <= n):
        if (i%2==0):
            yield i
        i += 1
j = even(10)
for k in j:
    print(k)
```

Output

```
2
4
6
8
10
```

When working with huge amounts of data, it is often preferable to utilize a generator rather than a function. This is due to the fact that a function returns a huge amount of data at once, all of which must be put into memory (sometimes the memory may not be sufficient). And it's possible that we'll only be dealing with one dataset at a time. In this case, we may utilize a generator that only produces one data at a time. Instead of collecting the full dataset, we may fetch one data at a time using a generator.

1.8.12 Modules

We also have the option of creating our own module. It's comparable to developing a standard program when we create our own modules. Two files are required. The first one is the module itself. The file name is the module name and must have a `.py` extension. The second file contains the code for the new module.

Let us create a module “`My_Module.py`”. It has two functions `MY_add(a,b)` and `My_mul(a,b)`.

Example 50

```
def MY_add(a,b):
    return a + b

def My_mul(a,b):
    return a * b
```

We can call the functions defined in `my_module.py` by importing the module.

Example 51

```
import my_module

a=5
b=10
x= my_module.MY_add(a,b)
print("The sum is",x)
y= my_module.My_mul(a,b)
print("The multiplication is", y)
```

1.9 CLASSES AND OBJECTS

A class defines data as well as functions. This is accomplished by creating a class that consists of two sections: a declaration part and an implementation part. The declaration part specifies variables and functions, whereas the implementation part defines functions whose prototypes were specified in the declaration section. This is known as encapsulation, and it means that the data and the functions are linked together. We know that a class is a template of an object.

The general syntax for a class is:

```
class ClassName:
    'class documentation string'
    class_suite
```

Notice that the term *class* is used to begin the definition. The keyword *class* is followed by an identifier, which is the class's name, and then a colon (:). Inside the class, the class suite is described. Everything in the class suite folder should be tabbed appropriately. The properties and behavior are both contained in the class suite.

The variables are called attributes, and the methods specified in the class are called behaviors. Each class is split into three sections:

Within its own namespace, a class has a name that uniquely identifies it. Each class has its own set of attributes.

A class consists of a collection of operations. It is the capacity to carry out certain tasks.

It is possible to leave the *class_suite* empty, a class with no attributes and no methods.

Example 52

```
class Two_Wheeler:
    Make= 'Hero'

    def __init__(self):
        self.Regd_no= 'PY 37 PQ 7898'
        self.owner= 'Srusti'
        self.kind= 'Scooter'

    def details(self):
        print ('Owner ',self.owner, 'has a',self.kind,'with Regd
no',self.Regd_no)
```

In this example, the *class_suit* contains attributes (variables) and methods (behaviors). Here *Make*, *Regd_no* and *owner* are attributes, whereas *__init__()* and *type()* are methods.

A class definition is nothing more than a design or template for all of the objects. Having a class is therefore pointless if no object is created. In other terms, an object is a class instance.

The presence of a class does not imply that all suitable objects will be generated automatically. We must build the object because the class cannot do it.

The general syntax for creating an object is: *Object_name=Class_name()*

The object provided is an instance of the class we requested. When we use the functional notation to assign a class, the newly generated object inherits all of the class's properties. Instantiation is the process of generating an object of the chosen class (as the object becomes an instance of the class).

Example 53

```
My_Byke = Two_Wheeler ()
My_Byke.details()
Two_Wheeler.details(My_Byke)
```

Output

```
Owner is : Srusti has a Scooter with no PY 37 PQ 7898
Owner is : Srusti has a Scooter with no PY 37 PQ 7898
```

There are two different ways we can use the methods defined in a class.

Method 1: Keep in mind that *My_Byke* is the object generated for the class *Two_Wheeler* in the first line. The second line, *My_Byke.details()*, invokes a method provided within the class. Remember that *My_Byke* is an object of *Two_Wheeler*, and the *details()* method of *My_Byke* accepts the parameter "self". It implies we wish to use a method that is part of the *My_Byke* object.

Method 2: The *details()* is a method defined in the *Two_Wheeler* class. We must give the object name as a parameter to access the methods specified within the class. There might be a lot of objects produced from the same class. The object name must be supplied as an argument to identify which

object is calling the method. The method *details()* is accessible by object *My Byke*, which is provided as an argument to access the method defined in the class *employee*, in the third line, *Two_Wheeler.details(My_Byke)*.

1.9.1 `__init__()` , `__new__()` and `__del__()`

There are two methods, `__init__()` and `__new__()`, used to initialize certain data and constructor of an object, whereas `__del__()` is used to destruct an object.

1.9.1.1 `__init__()`

This method's primary goal is the initialization of an object.

The general syntax is:

```
__init__(self [,arguments])
```

It's worth noting that the method's name is "`__init__()`" and it's preceded and followed by two underscores (`__`). The method's parameter is *self* and in addition to it a list of arguments may be supplied. It does not return any results.

The argument *self* is not passed to the method when it is called. Python does it automatically. The argument *self* refers to the object itself. The object that has invoked the method is this one. This indicates that the function `__init__()` accepts no parameters and should accept *self*. In the same way, a method that accepts just one argument will really accept two: *self* and the argument.

When an object is created, the first method that is automatically run is this one. The `__init__()` function is invoked every time an object is created. This function may be used to initialize variables in a class object.

There are two ways of assigning values to the variable in an object.

Method 1: Inside the `__init__()` method, we can assign the value to the variable statically.

Example 54

```
class Two_Wheeler:
    def __init__(self):
        self.Regd_no= 'PY 37 PQ 7898'
        self.owner = 'Srusti'
        self.kind = 'Scooter'

My_byke = Two_Wheeler()
print(My_byke.Regd_no,My_byke.owner,My_byke.kind)
```

Take note of the method `__init__(self)`. The value 'PY 37 PQ 7898' is set to `self.Regd_no`, 'Srusti' is set to `self.owner` and 'Scooter' is set to `self.kind` in

this case. We did not explicitly use the `__init__()` method while instantiating the object `My_byke`. Automatically, variables are assigned to the values supplied inside the method. The *self* refers to the object that is calling the method, which in this instance is `My_byke`.

Method 2: A value can be passed to the `__init__()` method that can assign the value to the variable dynamically.

Example 54

```
class Two_Wheeler:
    def __init__(self, regd, owner, kind):
        self.Regd_no = regd
        self.owner = owner
        self.kind = kind

My_byke = Two_Wheeler('PY 37 PQ 7898', 'Srusti', 'Scooter')
print(My_byke.Regd_no, My_byke.owner, My_byke.kind)
```

Observe that the `__init__(self, regd, owner, kind)` method has three more parameters in addition to the parameter *self*. We supplied the values for the `self.Regd_no`, `self.owner` and `self.kind` while creating the object `My_byke = Two_Wheeler('PY 37 PQ 7898', 'Srusti', 'Scooter')`.

The output is the same in both cases. The value of `Regd_no`, `owner` and `kind` is fixed for all the objects created in the first case, but the second method allows for more flexibility by supplying the value as an argument.

1.9.1.2 `__new__()`

The main purpose of this method is object creation. The general syntax is:

*object.__new__(cls[, *args, **kwargs])*

or

*super(class_name, cls).__new__(cls[, *args, **kwargs])*

It is worth noting that the method's name is "`__new__()`" and it's preceded and followed by two underscores (`__`). In the method, the argument is *cls*. A list of arguments can be supplied in addition to *cls*. It returns an object.

It is a static method that returns a new object instance that accepts *cls* as the first parameter. This approach is referred to as a "constructor." When the new class is ready to be instantiated, this method is invoked. This technique is used to modify an object's creation.

Example 55

```

class Two_Wheeler(object):
    def __new__(cls, regd, owner, kind):
        if regd.find('TS') != -1 or regd.find('ts') != -1:
            return object.__new__(cls)
        else:
            return None
    def __init__(self, regd, owner, kind):
        self.Regd_no= regd
        self.owner= owner
        self.kind = kind

    def display(self):
        print('Registration Number :',self.Regd_no, 'Owner :',self
            .owner, 'Kind :',self.kind )

My_byke= Two_Wheeler('TS 04 DE 1234','Aruna Kranthi','Byke')
My_byke.display()

```

Output

```
Registration Number : TS 04 DE 1234 Owner : Aruna Kranthi Kind : Byke
```

cls, regd, owner and kind are the arguments in the `__new__()` function. The method `__new__()` is a static method, as indicated by the cls argument. When the value of regd is *TS* or *ts*, the object is created. When the object is created, it returns the object together with its data, and when the object was not generated, it returns *None*.

Before the `__init__()` function is invoked, the `__new__()` method is called. It's because value initialization is only possible when an instantiation happens. When `__new__()` returns a class instance, the `__init__()` method is executed automatically. This instance is given as a parameter to the `__init__()` function. When utilizing the `__new__()` method, the `__init__()` function is not necessary to initialize some value to a variable.

Example 56

```

class Two_Wheeler(object):
    def __new__(cls, regd, owner, kind):
        if regd.find('TS') != -1 or regd.find('ts') != -1:
            self= object.__new__(cls)
            self.Regd_no = regd
            self.owner= owner
            self.kind = kind
            return self
        else:
            return None

    def __str__(self):
        return '{0}{1}'.format(self.__class__.__name__,
            self.__dict__)

My_byke= Two_Wheeler('TS 04 DE 1234','Nakshatra','Byke')
print(My_byke.Regd_no,My_byke.owner,My_byke.kind)
My_bykel= Two_Wheeler('OD 23 EE 1034','Aruna Kranthi','Scooter')
print(My_bykel)

```

Output

```
TS 04 DE 1234 Nakshatra Byke
None
```

In the `__new__()` method, notice how we generated objects based on the value of `Regd_no`. When an object is successfully created, the value given as a parameter is assigned to `self.regd_no`, `self.owner`, and `self.kind`. Then `self` is returned.

The variables of the object `My_byke` may be displayed using `print(My_byke.Regd_no, My_byke.owner, My_byke.kind)`. Because `My_byke1` isn't defined, it returns `None`, and attempts to display the value using `My_byke1.Regd_no` and `My_byke1.owner` will result in an `AttributeError`. When using the `__new__()` method to create objects, overloading the `__str__()` function is usually recommended.

1.9.1.3 __del__()

The goal of the destructor is to clean up the object before the garbage collector destroys it.

The general syntax is:

```
object.__del__(self):

# the destructor's body
```

When all references to an object are removed, this function is invoked. Garbage collection refers to the process of deleting unreferenced items from memory. When an object's reference count is 0, the Python interpreter runs the garbage collector to free it from memory.

Example 57

```
class Test:
    def __init__(self,a):
        self.no = a
        print("Object ",a, " is Created")

    def __del__(self):
        print("Object is Deleted")

obj1 = Test(1)
obj1 = 5

obj2 = Test(2)
del obj2

print("Program End")
```

Output

```
Object 1 is Created
Object is Deleted
Object 2 is Created
Object is Deleted
Program End
```

To destroy an object, we must either explicitly invoke the *del* method or erase the reference.

Observe how *obj1 = Test(1)* destroys the object it produced. As soon as *obj1* refers to 5, the destructor function is executed automatically. When *del obj2* is run, the object produced by *obj2 = Test(2)* is also deleted.

If more than one reference pointing to the same object, then the destructor cannot be called unless all the references are removed.

Example 58

```
class Test:
    def __init__(self,a):
        self.no = a
        print("Object ",a, " is Created")

    def __del__(self):
        print("Object is Deleted")

obj1 = Test(1)
obj2 = obj1
del obj1
print("Program End")
```

Output

```
Object 1 is Created
Program End
```

Observe that *obj1* and *obj2* both refer to the same object, which was created by *obj1 = Test (1)*. Because the object is still referenced by *obj2*, the command *del obj1* did not call the destructor.

1.10 VARIABLES AND METHODS

The variables and the methods are an important part of any class.

1.10.1 Variables (attributes)

Object variables and class variables are the two types of variables. The class variables are held by the class, but the object variables are owned by each object.

1.10.1.1 Object variables (instance variables)

Variables that vary from object to object are known as instance variables. Each object makes a duplicate of the variable. There are three ways to generate an instance variable.

1. Inside Initializer by using self-variable.
2. Outside the class, using the object reference variable.
3. Inside Instance Method by using self-variable.

Example 59

```
class Two_Wheeler:
    def __init__(self):
        self.Regd_no= 'PY 37 PQ 7898'

    def kin(self):
        self.kind = 'Scooter'

My_byke= Two_Wheeler()
My_byke.owner = 'Nakshatra'
My_byke.kin()
print("Owner : ", My_byke.owner, " has a ", My_byke.kind," with
redg no : ",My_byke.Regd_no)
```

Output

```
Owner : Nakshatra has a Scooter with redg no : PY 37 PQ 7898
```

In this example, we declare instance variables inside the initializer `__init__()` by using `self`. Once we create an object, automatically these variables will be added to the object. The statement `self.Regd_no= 'PY 37 PQ 7898'` defines a variable and assigns a value to it.

The value to the variables (attributes) can also be assigned after the object is created. It is done by referencing the object. The general syntax is ***Object_name.variable_name = value***. The statement `My_byke.owner = 'Nakshatra'` defines one more variable and assigns a value to it.

If any instance variable is declared inside the instance method, that instance variable will be added to the object once we call that method. Observe the method `kin()`, inside that method `self.kind = 'Scooter'` statement defines a variable `self.kind`. In order to access the variable, we need to call the method.

1.10.1.2 Class variables (static variables)

If the value of a variable does not change from object to object, it must be defined within the class but outside of any method. Variables of this sort are known as static variables or class variables. These variables are linked to the class object to which they belong and are not affected by any class

instances. Only one copy of a static variable is produced for a class, and it is shared by all objects of that class.

We can create a static variable in five different ways:

1. Inside the class.
2. Inside the constructor by using class name.
3. Inside instance methods by using class name.
4. Inside static methods by using class name.
5. Inside class methods by using either class name or *cls* variable.

Example 60

```
class Two_Wheeler:
    make = 'Hero'
    def __init__(self):
        self.Regd_no= 'PY 37 PQ 7898'
        Two_Wheeler.kind = 'Scooter'

    def colr(self):
        Two_Wheeler.color = 'Red'

    def capacity():
        Two_Wheeler.engine = '100 CC'

    @classmethod
    def seat(cls):
        cls.seater = 2

My_byke= Two_Wheeler()
My_byke.owner = 'Nakshatra'
My_byke.colr()
Two_Wheeler.capacity()
My_byke.seat()
print("Owner : ", My_byke.owner, " with redg no : ",My_byke.
    Regd_no)
print("Make :",My_byke.make,"Colour :", Two_Wheeler.col
    or,"Seater :",Two_Wheeler.seater)
```

Output:

```
Owner : Nakshatra with redg no : PY 37 PQ 7898
Make : Hero Colour : Red Seater : 2
```

In this example all the five ways of class variables are defined. The statement *make = 'Hero'* is inside the class definition but not inside any method that defines a class variable. Inside the *__init__()* method, the statement *Two_Wheeler.kind = 'Scooter'* defines a class variable.

Inside *colr(self)*, which is an instance method, the statement *Two_Wheeler.color = 'Red'* defines a class variable.

Inside *capacity()*, which is a static method, the statement *Two_Wheeler.engine = '100 CC'* defines a class variable. Inside *seat(cls)*, which is a static method, the statement *cls.seater = 2* defines a class variable.

1.10.1.3 Accessing variables

It is not enough to just declare variables and assign values to them; we must also be able to access them. In the software, there are several ways to access these variables.

The different methods for accessing variables are:

1. Accessing the variable by object reference.
2. Accessing the variable by class name.
3. Accessing the static variable outside the class by class name.

Example 61

```
class Two_Wheeler:
    make = 'Hero'
    def __init__(self):
        self.Regd_no= 'PY 37 PQ 7898'
        Two_Wheeler.kind = 'Scooter'

    def colr(self):
        Two_Wheeler.color = 'Red'

    @classmethod
    def seat(cls):
        cls.seater = 2

My_byke= Two_Wheeler()
My_byke.owner = 'Nakshatra'
My_byke.colr()
My_byke.seat()
print("Owner : ", My_byke.owner, " with redg no : ",My_byke.
Regd_no)
print("Make :",My_byke.make,"Colour :", Two_Wheeler.col
or,"Seater :",Two_Wheeler.seater)

My_byke1= Two_Wheeler()
My_byke1.Regd_no= 'TS 12 RR 3256'
My_byke1.owner = 'Srusti'
Two_Wheeler.make = 'Enfield'
Two_Wheeler.color = 'Black'
Two_Wheeler.seater = 1
print("Owner : ", My_byke.owner, " with redg no : ",My_byke1.
Regd_no)
print("Make :",My_byke1.make,"Colour :", Two_Wheeler.col
or,"Seater :",Two_Wheeler.seater)
```

Output

```
Owner : Nakshatra with redg no : PY 37 PQ 7898
Make : Hero Colour : Red Seater : 2
Owner : Nakshatra with redg no : TS 12 RR 3256
Make : Enfield Colour : Black Seater : 1
```

We can access the variable by object reference. The general syntax to access a variable is *object_name.variable_name*. Observe that we have defined a

class variable *make*. In order to access the variable we can use the object reference that is `My_byke.make` and `My_byke1.make`.

When accessing a static variable, we can use the class name instead of the object name. Because the static variable is shared by all the objects generated, we may access it using the class name. To access a static variable, use the general syntax *class_name.variable_name*. Observe that we have access to the same static variable by writing `Two_Wheeler.color` and `Two_Wheeler.seater`.

Static variables can also be accessed outside of the class. We may access a class variable in the same manner we assigned values to it. *class_name.variable_name* is the standard syntax for accessing a static variable outside the class. Observe that there is a class variables *make*, *colour* and *seater* defined inside the class changed to 'Enfield' , 'Black' and 1, respectively after the creation of the object. There may be several class variables defined for a class; we need to mention for which class variable the value is changed. It is also possible to define a class variable the same way we changed the value of a class variable.

1.11 PUBLIC, PRIVATE AND PROTECTED VARIABLES

Variables can be classified as public, private or protected. These are also called *access specifiers*.

Public variables are variables specified in a class that can be accessed using the dot operator from anywhere in the program. Private variables, on the other hand, are variables that have a single or double underscore prefix in the class definition. The strongly private variables are only accessible from within the class and not from anywhere else.

Unlike most programming languages, Python does not contain private, public or protected keywords; instead, they are distinguished by the absence of an underscore, a single underscore or a double underscore symbol before the variable or method.

If there is no underscore before a variable's or a method's name, it means the name is public. As a result, it is open to everyone and may be accessed from anywhere in the program.

If a variable or method has an underscore before its name, it means it is exclusively for internal usage and may be changed whenever the class wishes.

Python does not truly protect these names. As a result, these names can be accessed directly from other modules. Such variables are sometimes referred to as “weak private” or “protected.” These names can be used within the class, the child class and the package, but not outside of the package.

If a variable or function has a double underscore (__) before its name, it is for internal use only and may be changed only within the class. Such names are referred to as “strong private.”

Because we cannot access private members directly from another class, we can use such variable names many times in various methods. The main purpose of the double underscore (__) is to utilize names (variable/method) exclusively within the class if they are not to be used outside of the class.

Example 62

```
class Two_Wheeler:
```

```
    def __init__(self, regd, knd, mk):
        self.Regd_no= regd
        self._kind = knd
        self.__make = mk

    def show(self):
        print("From show() method:")
        print(self.Regd_no)
        print(self._kind)
        print(self.__make)
```

```
My_byke= Two_Wheeler('PY 37 PQ 7898', 'Scooter', 'Hero')
My_byke.show()
print("Data saved by My_byke Object \n", My_byke.__dict__)
print("\nFrom Outside the class")
print(My_byke.Regd_no)
print(My_byke._kind)
print(My_byke.__make)
```

Output

```
From show() method:
PY 37 PQ 7898
Scooter
Hero
```

```
Data saved by My_byke Object
{'Regd_no': 'PY 37 PQ 7898', '_kind': 'Scooter', '_Two__make': 'Hero'}
```

```
From Outside the class
PY 37 PQ 7898
Scooter
```

```
-----
AttributeError                                Traceback (most recent call last)
<ipython-input-38-d00836b25c11> in <module>()
    19 print(My_byke.Regd_no)
    20 print(My_byke._kind)
--> 21 print(My_byke.__make)
    22
```

```
AttributeError: 'Two_Wheeler' object has no attribute '__make'
```

Observe that the variable *Regd_no* has no underscore, the variable *kind* (*_kind*) has a single underscore and the variable *make* (*__make*) has two underscores. We can see that these variables can be accessed from within the class (*show()* method), and from outside the class the variables *My_byke.Regd_no* and *My_byke._kind* can be accessed. Whereas, accessing *My_byke.__make* outside the class raised *AttributeError*. The *My_byke.__dict__* shows the internal storage of data in the object *My_byke*.

1.12 METHODS

Under the object oriented paradigm, a function is known as a method. A method is a function that belongs to a certain class. Methods are specified inside the context of a class. The relationship between the class and the method is made explicit in this method declaration. The syntax for calling a function differs from that for calling a method.

There are three different types of methods:

1. Instance method
2. Class method
3. Static method

1.12.1 Instance method

The term *instance method* refers to a method that works with the instance variable. As an argument to the instance method, *self* must be provided. Other than *self*, we may or may not have any other arguments.

In the parameter list, the first argument is always *self*. Furthermore, if the method only has one argument, *self*, we do not supply any value for this parameter when calling the method. Similarly, a method that accepts just one argument will really accept two parameters, *self* and the *parameter*.

Example 63

```
class Two_Wheeler:
    def __init__(self, regd, knd, mk):
        self.Regd_no= regd
        self.kind = knd
        self.make = mk

    def show(self):
        print("From show() method:")
        print("Registration no : ",self.Regd_no)
        print("Kind of Two Wheeler : ",self.kind)
        print("Make of the Two-Wheeler : ",self.make)

My_byke= Two_Wheeler('PY 37 PQ 7898', 'Scooter', 'Hero')
My_byke.show()
```

Output

```
From show() method:
Registration no : PY 37 PQ 7898
Kind of Two Wheeler : Scooter
Make of the Two-Wheeler : Hero
```

In this, *self* is a part of the parameter, so `__init__()` and `show()` are instance methods.

1.12.2 Class method

We'll need a *class method* to deal with the class variable. In two ways, class methods vary from regular methods. They are first called by a class (not by the instance of the class). Second, the class method's first parameter is *cls* rather than *self*. In most cases, class methods are used to instantiate a class with arguments other than those given to the class constructor.

Example 64

```
class Two_Wheeler:
    make = 'Hero'

    def __init__(self, regd, knd):
        self.Regd_no= regd
        self.kind = knd

    def mk_value(cls):
        return cls.make

    def show(self):
        print("From show() method:")
        print("Registration no : ",self.Regd_no)
        print("Kind of Two Wheeler : ",self.kind)
        print("Two wheeler maker : ",Two_Wheeler.
mk_value(Two_Wheeler))

My_byke = Two_Wheeler('PY 37 PQ 7898', 'Scooter')
My_byke.show()
```

Output

```
From show() method:
Registration no : PY 37 PQ 7898
Kind of Two Wheeler : Scooter
Two wheeler maker : Hero
```

It's worth noting that the class variable *make* is specified. A class method is required to access this class variable. Because it accepts *cls* as an argument, the method *mk_value(cls)* is a class method. We can see that we have typed *Two_Wheeler.mk_value(Two_Wheeler)* to retrieve the returned value of this class method. It indicates that there is a class function *mk_value(cls)* in the employee class, and we must give the class name *Two_Wheeler* as a parameter.

1.12.3 Static method

Static methods are used when we require a method that has nothing to do with the instance or class variables. This method can be used to interact with a different class or object.

A *static method* is any functionality that belongs to a class but is not required by the objects. The main distinction between static and class methods is that static methods do not take any extra parameters. They perform the same functions as regular class methods.

Example 65

```
class Two_Wheeler:
    make = 'Hero'

    def __init__(self, regd, knd):
        self.Regd_no= regd
        self.kind = knd

    def mk_value(cls):
        return cls.make

    def show(self):
        print("From show() method:")
        print("Registration no : ",self.Regd_no)
        print("Kind of Two Wheeler : ",self.kind)
        print("Two wheeler maker : ",Two_Wheeler.
mk_value(Two_Wheeler))

    @staticmethod
    def print_info():
        print("\nFrom Static method")
        print("This is printed for static method")

My_byke = Two_Wheeler('PY 37 PQ 7898', 'Scooter')
My_byke.show()
Two_Wheeler.print_info()
```

Output

```
From show() method:
Registration no : PY 37 PQ 7898
Kind of Two Wheeler : Scooter
Two wheeler maker : Hero

From Static method
This is printed for static method
```

We can make a method static by using a decorator *@staticmethod* and the static method is accessed by *class_name.method_name()*. Observe that we have a method *print_info()*; this method is a static method, as we have used the decorator *@staticmethod*.

1.13 CLASS INSIDE A CLASS (INNER CLASS)

Defining a class inside another class is sometimes required. The term *inner class* describes such a class. It is self-evident that we require an outside class in order to define an inner. For instance, departments are found at every college. There is no department class unless there is a college class. As a result, the department class must be part of the college's inner circle.

We can create the object of the inner class in two ways:

1. The object of the inner class is created inside the outer class.
2. The object of the inner class is created outside the outer class.

Example 66

```
class Two_Wheeler:
    make = 'Hero'

    def __init__(self, regd, own):
        self.Regd_no= regd
        self.owner = own
        self.details = self.byke()

    class byke:
        def __init__(self):
            self.make = 'Hero'
            self.engine = '100 CC'
            self.kind = 'Scooter'

    class colr:
        def __init__(self):
            self.color = 'Red'

My_byke = Two_Wheeler('TS 77 AA 4325', 'Tusarika')
clr = Two_Wheeler.colr()
print("Owner : ", My_byke.owner, "Registration no : ", My_byke.
      Regd_no )
print("Make of the Two Wheeler : ", My_byke.details.make)
print("Two wheeler maker : ", My_byke.details.engine)
print("Two wheeler maker : ", My_byke.details.kind)
print("Color of the two wheeler : ", clr.color)
```

Output

```
Owner : Tusarika Registration no : TS 77 AA 4325
Make of the Two Wheeler : Hero
Two wheeler maker : 100 CC
Two wheeler maker : Scooter
Color of the two wheeler : Red
```

For the outer class *Two_Wheeler*, we have defined two inner classes: *byke* and *colr*. The object of inner class is created inside the outer class by *self*. *InnerClassObjectName = self.InnerClassName()*. The implementation is done by *self.details = self.byke()*.

The object of the inner class can also be created outside the outer class by *InnerClassObjectName = OuterClassName.InnerClassName()*. The implementation is done by *clr = Two_Wheeler.colr()*.

1.13.1 Accessing attributes of inner class

Accessing a name defined inside the inner method is a bit tricky. The different ways of accessing the attributes of inner class are:

- i. Accessing the attributes of inner class outside the class.
- ii. Accessing the attributes of inner class in the outer class.
- iii. Accessing the attributes of inner class outside the outer class.
- iv. Accessing the attributes of inner class when inner class and outer class have the same method.

Example 67

```
class Two_Wheeler:
    make = 'Hero'

    def __init__(self, regd, own):
        self.Regd_no= regd
        self.owner = own
        self.details = self.byke()

    def show(self):
        print("\nAttributes of inner class inside outer class:")
        print("Registration no : ",self.Regd_no)
        print("Owner of Two Wheeler : ",self.owner)
        print("Two wheeler maker : ",self.details.make)
        print("\nAttributes of inner class when inner and outer
            class have same method:")
        self.details.show()

class byke:
    def __init__(self):
        self.make = 'Hero'
        self.engine = '100 CC'
        self.kind = 'Scooter'

    def show(self):
        print("\nFrom show() method inside inner class:")
        print("Registration no : ",self.regd)
        print("Owner of Two Wheeler : ",self.own)
        print("Two wheeler maker : ",self.make)

class colr:
    def __init__(self):
        self.color = 'Red'

My_byke = Two_Wheeler('TS 77 AA 4325','Tusarika')
clr = Two_Wheeler.colr()
My_byke.show()

print("\nAttribute of Inner class From Outside the outer
    class:")
print("Two wheeler engine : ",My_byke.details.engine)
print("Two wheeler maker : ",My_byke.details.kind)
print("\nAttribute of Inner class Outside the outer class:")
print("Color of the two wheeler : ",clr.color)
```


Output

Attributes of inner class inside outer class:

```
Registration no : TS 77 AA 4325
Owner of Two Wheeler : Tusarika
Two wheeler maker : Hero
```

Attributes of inner class when inner and outer class have same method:

From show() method inside inner class:

```
Registration no : TS 77 AA 4325
Owner of Two Wheeler : Tusarika
Two wheeler maker : Hero
```

Attribute of Inner class From Outside the outer class:

```
Two wheeler engine : 100 CC
Two wheeler maker : Scooter
```

Attribute of Inner class Outside the outer class:

```
Color of the two wheeler : Red
```

Here the outer class is *Two_Wheeler*, and there are two inner classes *byke* and *clr*. The outer class *Two_Wheeler* has attributes 'Regd_no' and 'owner'. The inner class *byke* has attributes 'make', 'engine' and 'kind'. The inner class *clr* has attribute 'color'.

In order to access the attribute of the inner class outside the class, the general syntax is

ObjectofOuterClass.ObjectofInnerClass.AttributeNameofInnerObject.

This is implemented in *My_byke.details.engine* and *My_byke.details.kind*.

In order to access the attribute of the inner class outside the outer class, the general syntax is

self.ObjectofInnerClass.AttributeNameofInnerObject.

This is implemented in *self.details.make*.

We have already seen that an object of inner class can be created outside the outer class. In order to access the attribute of the inner class outside the outer class, the general syntax is

ObjectofInnerClass.AttributeNameofInnerObject.

This is implemented in *clr.color*.

If a method has the same name in both inner and outer class, we can call the inner class *self.innerClassObjectName.innerClassMethod()*. This is implemented in *self.details.show()*.

True–false questions

1. Curly braces are used to identify a block of code in Python.
 - a. True
 - b. False

2. `a,b,c = 1,2,3` is a valid statement.
 - a. True
 - b. False
3. Python variables do not need explicit declaration to reserve memory space.
 - a. True
 - b. False
4. Decision-making is anticipation of conditions occurring while execution of the program does not specify actions taken according to the conditions.
 - a. True
 - b. False
5. An `else` statement must come with an `if` statement.
 - a. True
 - b. False

Fill-in-the-blank questions

1. A _____ is a function that accepts a function as a parameter and returns a function.
2. _____ arguments are passed to a function in correct positional order.
3. An _____ is an object that stores a group of elements of the same datatype.
4. An expression is a combination of _____, _____ and _____ written according to the syntax of Python language.
5. _____ in Python are identified as a contiguous set of characters represented in the quotation marks. Python allows for either pairs of single or double quotes.

Multiple-choice questions

1. The value of the expression `16%7%4` will be evaluated to
 - a. 1
 - b. 2
 - c. 3
 - d. 0
2. Find the output of the following code

```
print(15 << 2)
```

 - a. `15 * 2`
 - b. `15 * (2+2)`
 - c. `15 * (2**2)`
 - d. `15*(2+3)`
3. Find the output of the following code:

```
x = 4
x == 3 + 2
print(x)
```

- a. 5
 - b. 4
 - c. True
 - d. False
4. The output of `print(float('100.0'))` is
- a. 100.0
 - b. 100
 - c. Error
 - d. None of the above
5. The right-side binding means the expression will be evaluated:
- a. From left to right
 - b. From right to left
 - c. In random order
 - d. None of the above
6. Find how many times “+” will be printed in the output of the following code:
- ```
i = 0
while i <= 5:
 i+=1
 if i % 2 == 0:
 break
 print("+")
```
- a. 0
  - b. 1
  - c. 2
  - d. 3
7. Find the output of the following code:
- ```
x = 10
if x > 5:
    x = x + 5
if x < 12:
    x = x + 5
if x == 15:
    x = x + 5
print(x)
```
- a. 10
 - b. 15
 - c. 20
 - d. 25

8. Find the output of the following code:

```
x = 10
y = 0
z = y < x and x > y or y > x and x < y
print(z)
a. True
b. False
c. TRUE
d. FALSE
```

9. Find the output of the following code:

```
my_list = ["pet", "dog", 35, "cat", 23]
count = 0
for item in my_list:
    if type(item) == str:
        continue
    count = count + 1
print(count)
a. 1
b. 3
c. 4
d. 2
```

10. Find the output of the following code:

```
for i in range(1):
    print("**")
else:
    print("$")
a. One * only
b. Two * only
c. One * and a $
d. Two * and a $
```

11. Find the output of the following code:

```
sum = 0
x = [i**2 for i in range(3)]
l = len(x)
for i in range(l):
    sum += x[i]
print(sum)
a. 2
b. 3
c. 4
d. 5
```

12. Find the output of the following code:

```
s = 'Python is easy'
s1 = s[-7:-5]
s2 = s[-4:-2]
print(s1 + s2)
a. isea
b. is easy
c. easyeasy
d. iseasyeasy
```

13. Find the output of the following code:

```
s = 'Python is Fun'
s1 = s[6:-4]
print(len(s1))
a. 2
b. 3
c. 4
d. 5
```

14. Find the output of the following code:

```
i = [10,20,[30,40],[50,60]]
count = 0
for i in range(len(l)):
    if type(l[i]) == list:
        count = count + 1
print(count)
a. 1
b. 2
c. 3
d. 4
```

15. Find the output of the following code:

```
a = ()
print(bool(a))
a. False
b. True
c. Depends on processor
d. Not defined
```

16. Find the output of the code:

```
d1 = {1:'one',2:'two'}
d1.pop('two')
a. ValueError
b. AttributeError
```

- c. KeyError
 - d. None of the above
17. Find the output of the following code:
- ```
set1 = {1,2,3,4}
set1.add(2)
print(set1)
```
- a. {1,2,3,4}
  - b. {1,2,2,3,4}
  - c. All of the above
  - d. None of the above
18. Function is defined inside
- a. Another function
  - b. Module
  - c. Class
  - d. All of the above
19. Consider the following code:
- ```
def get_names():
    names = ['Parrot','Owl','Peacock','Maina','Crow']
    return names[2:]
def update_names(elements):
    new_names = []
    for name in elements:
        new_names.append(name[:3].upper())
    return new_names
print(update_names(get_names()))
```
- Find the output.
- a. ['PEA']
 - b. ['PAR','PEA']
 - c. ['PAR','PEA','OWL']
 - d. ['PAR','PEA','OWL','MAI']
20. Consider the following code:
- ```
def calculate(amount = 6, factor = 3):
 if amount > 6 :
 return amount * factor
 else:
 return amount * factor *2
```
- Which among the following statements will give output 30?
- a. calculate()
  - b. calculate(10)
  - c. calculate(5,6)
  - d. calculate(6,3)

**Descriptive questions**

1. Write a Python program that calculates compound interest.
2. Write a Python program that swaps two numbers using a third variable.
3. Write a program that finds the greatest common divisor (GCD), or highest common factor (HCF), of two positive numbers.
4. Write a program to remove duplicate values from the dictionary.
5. Write a Python program that will add square brackets to a list. If the list contains a single item ['Peacock'] and we want three more square brackets [ ], the output should be [ [ [ 'Peacock' ] ] ] .
6. Write a Python program to remove an empty tuple(s) from a list of tuples.

**Answers to true–false questions**

1. False
2. True
3. True
4. False
5. False

**Answers to fill-in-the-blank questions**

1. decorator
2. Positional
3. array
4. variables, constants, operators
5. strings

**Answers to multiple-choice questions**

---

|     |   |     |   |     |   |     |   |     |   |
|-----|---|-----|---|-----|---|-----|---|-----|---|
| 1.  | b | 2.  | c | 3.  | b | 4.  | a | 5.  | b |
| 6.  | b | 7.  | c | 8.  | a | 9.  | d | 10. | c |
| 11. | d | 12. | a | 13. | b | 14. | b | 15. | a |
| 16. | c | 17. | a | 18. | d | 19. | a | 20. | b |

---

# Fundamentals of data structures

---

### LEARNING OBJECTIVES

After studying this chapter, learners will be able to:

- Know what data structures are
- Know the basic operations performed in a data type
- Know the difference between primitive and non-primitive data structures
- Know some well-known data structures
- Understand abstract data types

We will discuss the basics of data structure and abstract data types. Every computer program has two parts: an algorithm and a data structure. The algorithm tells us what the program will do and operates on variables, whereas the data structure is a group of memory locations used to retain the information used by the algorithm.

### 2.1 INTRODUCTION

Only the primitive data that are defined can be manipulated by a digital computer. The user does not need to apply any additional effort while manipulating the primitive data. However, in our real-world applications, we use a variety of data types other than primitive data. Manipulation of these data necessitates the completion of the following tasks.

1. **User data storage representation:** User data should be stored in a format that a computer can comprehend.
2. **Data retrieval:** Data saved on a computer should be retrieved in a way that the user can comprehend.
3. **User data transformation:** Various actions must be conducted on user data to translate from one form to another.



People who create and develop computer programs such as system software or application software must have a basic understanding of data structures. We know that data is represented by data values that are either temporarily stored in a program's data area or permanently recorded on a file. Different data values are frequently related to one another. These data values must be arranged for programs to create a connection. The term *data structure* refers to a structured collection of data. To access and process structured data, the programs must follow a set of rules.

Data structure = Organization of data \* Operations on the data

Data structure refers to the logical or mathematical paradigm of a data organization. Data can be structured in many different ways. The decision to adopt a data model is influenced by two factors. Firstly, it must have a rich structure that mimics the true relationship of the facts in the real world. Secondly, the structure should be simple enough that the data can be quickly processed as needed.

## 2.2 CLASSIFICATION OF DATA STRUCTURES

Data structures are generally divided into two broad categories:

1. Primitive data structures
2. Non-primitive data structures

Machine instructions work directly on primitive data structures. These data structures have different representations on different machines. The category includes integers, floating-point numbers, characters, strings, lists, tuples, sets, and dictionaries in Python.

The complexity of non-primitive data structures is higher than that of primitive data structures. The non-primitive data structures are made from primitive data structures. Non-primitive data structures are focused on the organized collection of homogeneous or heterogeneous data items.

The operations performed on the data structure must be considered while designing an effective data structure.

Further, the non-primitive data structure is categorized into linear data structures, non-linear data structures, and file structures.

The data elements in linear data structures are ordered in a linear sequence. In non-linear data structures, the data items arranged are not in a sequence (Figure 2.1).

The most commonly used operations in data structures are broadly categorized into seven types:

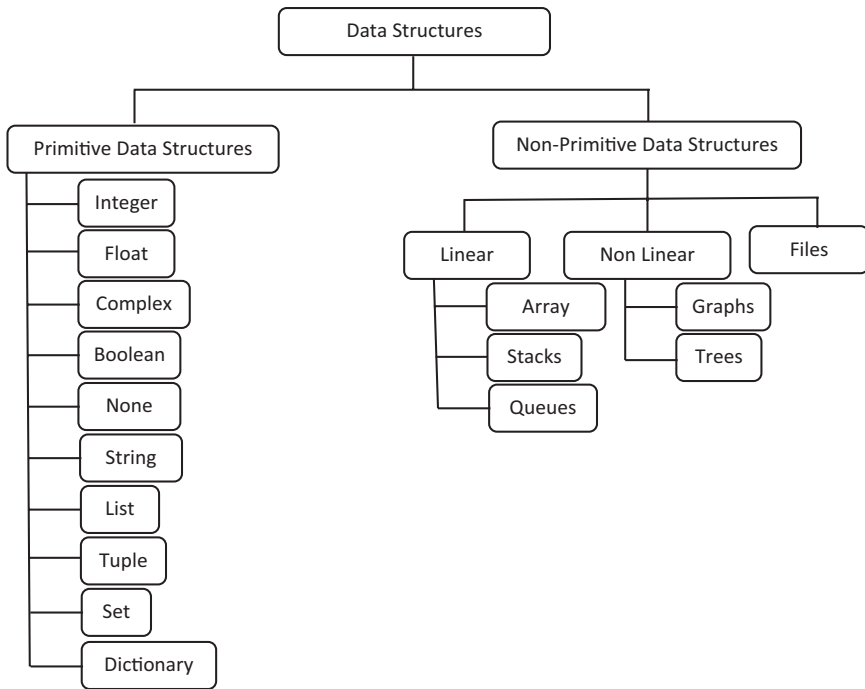


Figure 2.1 Data structure classification

- i. Create
- ii. Delete
- iii. Select
- iv. Update
- v. Search
- vi. Sort
- vii. Merge

The **Create** operation is used to reserve memory for program items. Declarative statements can be used to reserve the memory that can be done at compile time or runtime, depending on the programming language.

The **Delete** action releases memory reserved for a data item in the specified data structure.

The **Selection** operation is used to get a specific data item from a data structure.

The **Update** operation is used to make changes to a data item.

The **Search** procedure determines whether or not the targeted data item exists. It may also locate all the elements that satisfy a set of criteria.

**Sorting** involves putting data items in ascending or descending order.

**Merging** refers to the process of combining data items of two different sorted lists into a single list.

## 2.3 DESCRIPTIONS OF VARIOUS DATA STRUCTURES

Data structures can be classified into two categories: primitive and non-primitive.

### 2.3.1 Primitive data structures

Various primitive data structures come with the programming language. The primitive data structures that come with Python are discussed here.

#### 2.3.1.1 Integer

Numeric data types include integers, floats, complex numbers, and Booleans. Data in some variables are kept as whole numbers, for instance, the number of students in a class. An integer-type variable is used to save the value of such a sort of query. Integers are the only data types that do not support decimal places. In other words, only the complete number is required. It includes the values positive, negative, and zero. Integers such as 8, -432, and 0 are examples. Take a look at the expression  $x = 3 + 5$ . Python computes the right side of the equal sign, stores the result in memory, and connects the memory location with the identifier  $x$ . In other words, it assigns that location a name, and because the result of the right-hand expression is an integer, the type of this variable will be “int.” We have a variable in this scenario, and the identifier for this variable is  $x$ . A variable’s name can alternatively be referred to as  $x$ . The variable has an integer value of 8 and is of the type int. The statement `print(type(x))` returns a *class int*, indicating that its type is an integer, which can be confirmed.

#### 2.3.1.2 Float

Some variables have fractional values. When asked, “How tall are you?” we normally respond with 5.6 feet. We’ll need a floating-point representation to hold the height value. Floating point numbers are those that have a decimal point, for example, 43.9, -234.1, and 00.4. Take the statement  $y = 3 / 2$  as an example. Python calculates the right side of the equal sign, saves the result, and associates it with the identifier  $y$  in memory. Because the result is a floating-point number, this variable’s type will be float. The value of the variable is 1.5. This can be verified by “`print(type(y))`.”

### 2.3.1.3 Complex

One-dimensional numbers are integers and floats. We also require two-dimensional integers on occasion. Another numeric data format, complex number, is used to represent a two-dimensional number. The ordered pair  $x+yj$  is used to represent complex numbers, where  $x$  and  $y$  are numeric data. The real part is  $x$ , while the imaginary part is  $y$ . Complex numbers include  $9+8j$  and complex  $(9, 8)$ . The most common operations performed on complex data types are addition, subtraction, multiplication, and division.

### 2.3.1.4 Boolean

Boolean is a binary variable that can have one of two potential values: 0 (False or F) or 1 (True or T). For instance, if the inquiry is, “Are you happy?” The response is either yes or no. It means you’ll need a variable to hold data of the Boolean type. The operations used with the Boolean data type were AND, OR, and XOR.

Let's have a look at the bool type. Consider the proposition  $z = \text{False}$ . True or False are both keywords in Python, and the first letter is uppercase. Make sure that they are not typed as true or false (in lowercase letters).

Python computes the right side of the equal sign, stores the result in memory, correlates that memory address with the identifier  $z$ , and since the right-side result is a Boolean, the type is bool.

It's also worth noting that  $z$  was previously specified as “hello” and had the typed string.

However, with the sentence  $z = \text{False}$ , Python immediately assigns the value False to  $z$ . So, the prior value of  $z$ , “hello,” is no longer valid, and its new type is bool. In Python, this is done automatically, so we don't have to worry about it.

Python includes several other built-in data types, including string, list, tuple, dictionary, and set. We also have data structures like arrays that must be imported before they can be used.

### 2.3.1.5 None

This data type stands for “no value” or “null.” It has no meaning when it comes to “0,” “empty,” “False,” or “undefined.” It is a true data type; it can be assigned to variables and takes up memory like any other data type. When a function does not include a return statement and we want to print the value returned by the function, we use the “None” data type.

### 2.3.1.6 String

If you're asked, “What's your name?” the string type of variable is used to answer this question. It's simply a string of letters, numbers, and special

characters. Immutability is a property of the string data type. Concatenation, identifying substrings, and many other operations are performed on string-type data.

Now consider the statement `z = "hi."` Python calculates the right side once more, saves the result to memory, and connects that memory location with the identifier `z`. Because the right side contains a collection of characters, this variable's type is `str`, which stands for string. Python now has a variable with the name `z`, the value `"hi."`, and the type `str`.

### **2.3.1.7 List**

This sort of data is used to store numerous pieces of information at the same time under a single variable name. It is analogous to an array in a programming language such as C. C-array, on the other hand, is a homogeneous data type, whereas a list does not have to be of the same data type. The data in a list is enclosed by a pair of square braces `[ ]`.

### **2.3.1.8 Tuple**

The main difference between a tuple and a list is that a tuple is immutable. It is made up of unchanging and indexed data. The data in a tuple is enclosed by a pair of round brackets `( )`.

### **2.3.1.9 Set**

It's a collection of datasets that aren't in any particular sequence and aren't indexed. It's an immutable data type that stores one-of-a-kind information (no duplicates). A pair of curly braces `{ }` surrounds the data in a set.

### **2.3.1.10 Dictionary**

A dictionary is a specific data type in which data is stored in pairs of (key:value). It's an unsorted, changing, and indexed collection. Dictionary works similarly to a real-world dictionary. A dictionary's keys must be unique and immutable data types like strings, integers, and tuples, but the key-values can be repeated and of any kind.

## **2.3.2 Non-primitive data structures**

Non-primitive data structures, also known as composite or derived data structures, are data structures that are not built-in or predefined in a programming language. Unlike primitive data types that hold a single value, non-primitive data structures can hold multiple values and are composed of primitive data types or other non-primitive data structures. The

non-primitive data structures provide flexibility and efficiency in storing and manipulating complex data in programming languages.

### 2.3.2.1 Linear data structures

Linear data types are those data types that are ordered by nature. In other words, the elements are accessed one after another in a sequence.

### 2.3.2.2 Array

This data type holds multiple data of homogeneous data types together and is accessed by an index. In Python, an array can be implemented using primitive data types such as a list or a dictionary. It can also be implemented using a non-primitive data type by importing separate packages like NumPy and array. The elements in an array are accessed sequentially. We can implement an array with the help of a list or a dictionary.

### 2.3.2.3 Stack

A stack is also a linear data type, but it has a special way of inserting and deleting data. In a stack, the insertion and deletion take place at one end called the *top of the stack*. As a result, the element that is inserted last is to be deleted first so it is also called *last in, first out* (LIFO). For example, as shown in Figure 2.2, we can think of a stack of books on a table. We can remove a book from the top, and also we can put a book on the top of the stack. The operation of inserting and deleting is also known as *push* and *pop*, respectively. Another operation in the stack is *peek*, which returns the element at the top of the stack.

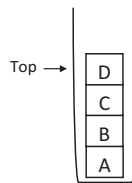


Figure 2.2 Stack

The concept of stack can be implemented using a list, dictionary, class, or queue module.

### 2.3.2.4 Queue

A queue is also a linear data type, but it has a special way of inserting and deleting data. In a queue, insertion occurs at one end, known as the *rear*,

while deletion occurs at the opposite end, known as the *front*. As a result, the element that is placed first is also the first to be deleted, hence the term *first in, first out* (FIFO). For example, we can think of people standing in a row waiting for a bus. The person who arrives first becomes the first person in the queue and they get onto the bus first (Figure 2.3).



Figure 2.3 Queue

The concept of queue can be implemented using a list, dictionary, class, or queue module.

### 2.3.3 Non-linear data structures

Non-linear data types are those data types that are not in a specific order. In other words, the elements are not accessed one after another in a sequence.

#### 2.3.3.1 Tree

This is a non-linear data structure in which there is no unique predecessor and successor. This data structure represents a hierarchical relationship between the elements. It is a collection element that is identified by a unique element called the root and the sub-trees are children of the root (Figure 2.4).

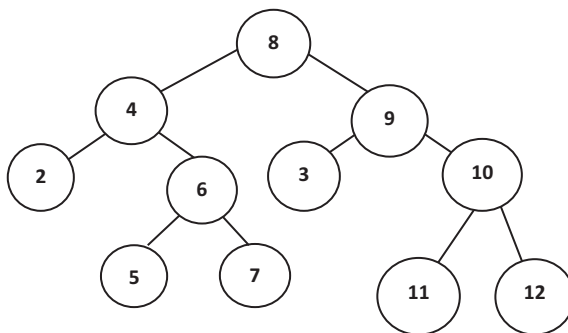


Figure 2.4 Tree

The concept of tree can be implemented using a list or a class.

### 2.3.3.2 Graph

It is a mathematical non-linear data structure capable of representing any kind of structure. A graph is made up of nodes (also known as vertices) that are linked together by edges (also called arcs) (Figure 2.5).

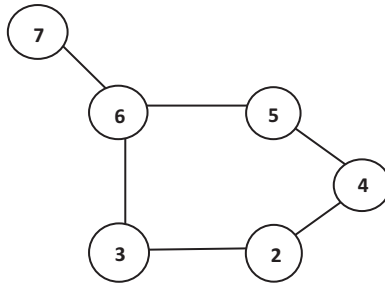


Figure 2.5 Graph

Graphs contain some key terms:

- When two nodes are joined by an edge, they are referred to as neighbors.
- The degree of a node is the number of other nodes to which it is connected (i.e., the number of neighbors that it has).
- A loop is an edge that connects one node to itself.
- A path is a series of nodes linked together by edges.
- A cycle is a closed path, that is, a path that begins and finishes at the same node (and no node is visited more than once).

### 2.3.3.3 File

In day-to-day life, all the information cannot be saved in terms of data. Sometimes a file is used to store voluminous data in an external storage device. A file is a collection of records with one or more fields. The commonly use organization for a file is:

- i. Sequential file
- ii. Relative file
- iii. Direct file
- iv. Index sequential file
- v. Index file

The basic operations that we perform with a file are the same as any other data structure.



## 2.4 ABSTRACT DATA TYPES (ADTS)

When it comes to solving problems with a computer, developing abstract models for the data and how the program manipulates data is a must. To sort a set of numbers, for example, we'll need an abstract model of a data structure array as well as a sorting method. To create a new model, we must first specify the abstract object we want to manage and then put the actions in place. We want to make sure that the objects are easy to use when solving a problem.

A mathematical model is an abstract data type (ADT). ADT is defined independently of the implementations. The model specifies a set of values as well as operations that can be performed on those values. A simple ADT example is a collection of numbers that can be read, sorted, searched, and printed. The ADT is a useful tool for representing the logical attributes of a data type. A data type is a collection of values and actions that are implemented with the help of a certain data structure. The basic mathematical concept that specifies the data type, pre-conditions (if any), and the operations are referred to as ADT. ADTs are generalizations of primitive data types, whereas procedures are generalizations of basic operations. The step-by-step development strategy is

- i. Mathematical model leads to informal algorithm
- ii. Abstract data type leads to algorithm
- iii. Data structures lead to program

Although most programming languages allow certain standard data types, they are insufficient for the majority of applications. Python has a derived data type known as class. However, just creating a class does not separate the operations applicable to objects of that kind. This is why the concept of ADT is useful; ADTs allow us to express whatever assumptions we have about the operations we perform on data objects.

Abstract data type is not a data structure, but it talks about the structure of the data. It tells how the data is stored and what to do with the data. These two responses describe the behavior of the data structure. A data structure is an implementation of an abstract data type.

Although the terms *data types*, *data structures*, and *abstract data type* are similar, they have distinct meanings. The data type of a variable in programming languages refers to the range of possible values for the variable. Basic types can be used to create composite data types. An abstract data type is a mathematical model that can be used to perform a variety of actions. We'll write algorithms in terms of ADTs, but to put them into practice, we'll need to figure out how to represent ADTs in terms of data types and operators. The mathematical models that underpin an ADT are expressed using data structures, which are collections of variables connected in various ways.

While establishing an abstract data type as a mathematical concept, we are not concerned with space or time efficiency. Those aren't important details.

The specification of an abstract data type includes:

1. *Instances*: The domain of values (data attributes) can be specified.
2. *Pre-conditions*: It is required when pre-conditions are mentioned.
3. *Operations*: It is a mathematical or logical process of deriving one data value from existing data values according to certain rules.

Consider the set as an ADT; the elements of the set are the domain of values, and the operations on the domain are union, intersection, complements, and so on.

The fundamental idea is that these operations are implemented once in the program, and then any other component of the program that needs to do something with the ADT may call the relevant function. If any implementation details need to be changed for whatever reason, it should be trivial to do so by simply changing the ADT operation functions. This change would go unnoticed by the rest of the software.

There is no rule defining which operations each ADT must support; this is a design decision. Some of the data structures we'll look at in this book are examples of ADT. We'll show how to implement each one, but if you do it well, the applications that use them won't need to know which one you used.

## 2.5 ALGORITHMS

A solution to a problem consists of the following steps:

1. Understand the problem
2. Design an algorithm
3. Convert the algorithm into a flowchart
4. Convert the flowchart into a computer program

In this section, we define an algorithm and study various methods to analyze its performance.

An algorithm is a technique used by a computer to solve a problem. An algorithm is a finite collection of instructions that performs a certain task. The following qualities must be present in all algorithms:

1. The algorithm must have zero or more inputs.
2. It must produce an output.
3. Each instruction must be clear and unambiguous.
4. The algorithm must terminate after a finite number of steps.

Algorithms are written in a language that is similar to programming. In a programming language, a program is the expression of an algorithm. There are a few distinct areas in the study of algorithms:

1. *Devise an algorithm*: For the design of an algorithm, there are various design approaches such as the greedy technique, divide and conquer technique, and dynamic programming. Some of the concepts might be used in fields other than computer science, such as operations research and electrical engineering.
2. *Validate an algorithm*: After it has been constructed, it is critical to ensure that the algorithm produces accurate results for all permissible inputs. This is referred to as algorithm validation. It is independent of the programming language in which it will be written in the future, which is the next stage in the algorithm's evolution. For the program to be legitimate, each statement must be stated, and all basic operations must be demonstrated accurately.
3. *Analyze an algorithm*: This activity, also known as performance analysis, involves deciding how much memory is required for program execution and the amount of time the central processing unit (CPU) is used. This enables us to compare one algorithm to another, both of which were created for the same goal and with the same inputs. This study also aids us in determining the algorithm's performance in the best, average, and worst cases.
4. *Test a program*: Debugging and profiling are the two steps of software testing. Debugging is the process of running a program on test data to see if the results are accurate, and if they aren't, fixing the program until it produces proper results for all legal inputs. The practice of running a program on datasets and measuring the time and space required to compute the results is known as profiling or performance measurement.

## 2.5.1 Algorithm specifications

### 2.5.1.1 Pseudo-code

A natural language like English can be used to specify an algorithm. For small and simple algorithms, graphic representations such as flowcharts are also used. We may also define algorithms in pseudo-code that resembles a computer language, making program translation simple. We may utilize conditional statements, conditional repeated statements, and finite repetitive statements while developing an algorithm using pseudo-code.

A sample algorithm:

```
Algorithm Max(a,n):
 result = a[0];
 for i in range (n):
 if(a[i] > result):
 result = a[i]
 return result
```

This method finds the largest value in an array of size  $n$ . In this case, *Max* is the name of the algorithm, and  $a$  and  $n$  are its parameters. The largest element in the array is returned as the “result” in the preceding procedure.

### 2.5.1.2 Recursion

If the same algorithm is invoked in the body, it is considered to be recursive. When algorithm A calls algorithm B and algorithm B calls algorithm A, it is said to be indirectly recursive.

The Towers of Hanoi problem is an example of a recursive algorithm.

Suppose we want to find the sum of the integers in an array.

```
algorithm Rsum(a,n):
 count=count+1
 if(n<=0):
 return 0
 else:
 count=count+1
 return Rsum(a,n-1) + a[n]
```

With a different set of parameters, the algorithm calls itself. The number of steps completed is shown by the count.

$$T_{\text{Rsum}}(n) = 2 + T_{\text{Rsum}}(n-1)$$

where  $T_{\text{Rsum}}(n)$  indicates the total number of steps to be executed while adding the elements in an integer array of size  $n$ . In the right-hand side of the equation, the 2 indicates the number of steps executed before we call  $\text{Rsum}(a, n-1)$ .

The recurrence relation can be solved as follows:

$$\begin{aligned} \text{TRsum}(n) &= 2 + \text{TRsum}(n-1) \\ &= 2 + 2 + \text{TRsum}(n-2) \\ &= \dots \\ &= 2(n) + \text{TRsum}(0) = 2n + 2 \end{aligned}$$

In general, if  $T(n)$  is the time taken by the algorithm when the input size is  $n$ , it can be expressed as

$$T(n) = c + T(n-1)$$

where  $c$  is some constant.

Another type of recurrence relation is

$$T(n) = a T(n/b) + f(n)$$

where the input is divided into  $b$  groups and for each group, we apply the same algorithm.  $f(n)$  is the number of steps or the time taken to combine the solutions of the sub-problems. Let us take a simple example.

$$T(1) = 1$$

$$T(n) = T(n/2) + c$$

We can solve the recurrence relation as follows:

$$T(n) = T(n/2) + c$$

Substituting for  $T(n/2)$ ,

$$\begin{aligned} T(n) &= (T(n/2^2) + c) + c \\ &= T(n/2^2) + 2c \end{aligned}$$

After  $k$  iterations

$$T(n) = T(n/2^k) + kc$$

Without any loss of generality, let us assume  $n/2^k = 1$  or  $k = \log_2 n$ . Then

$$T(n) = T(1) + c \log_2 n = 1 + c \log_2 n$$

Therefore,  $T(n) = O(\log_2 n)$ .

**Example**

$$T(n) = 2T(n/2) + n; T(1) = 1.$$

Substituting for  $T(n/2)$

$$\begin{aligned}
 &= 2[2(n/2^2) + n/2] + n \\
 &= 2 * 2T(n/2^2) + 2n
 \end{aligned}$$

After k iterations

$$= 2kT(n/2^k) + kn$$

Let us assume  $n = 2^k$  or  $k = \log_2 n$ .

$T(n) = nT(1) + n\log_2 n$ ; or  $T(n) = n + n \log_2 n$ . Since  $n\log_2 n$  dominates  $n$  as  $n$  becomes large, we write  $T(n) = O(n\log_2 n)$ . The big oh notation is explained later.

## 2.5.2 Performance analysis

The algorithm's efficiency may be assessed by measuring its performance. The performance is determined by the amount of storage space required for the program and data. This is referred to as space complexity, while the time it takes to run the program is referred to as time complexity. In this book, the focus will be more on data structures rather than performance analysis.

### 2.5.2.1 Space complexity

**Definition:** The space complexity of an algorithm is the amount of memory the algorithm needs to run until completion.

The space needed to run an algorithm consists of two parts:

1. Fixed part, which does not depend on inputs and outputs.
2. Variable part, which is the space needed by component variables.

$$S(p) = C + S_p$$

Algorithm MAX

```

{ result = a[i]
 for (i = 2 to n)
 { if (a[i] > result)
 result = a[i]
 } return result
}
```

In the preceding algorithm MAX, we need one word each for  $n$ ,  $result$ , and  $i$ , and  $n$  words for storing the array.

Therefore,  $S(p) \geq n+3$  words.

Consider the following recursive part of the program.

```
Algorithm RSum(a,n)
{
 if(n<= 0) return 0.0
 else return(RSum(a, n-1) + a[n])
}
```

Since the depth of recursion is  $n+1$  and each recursive call needs the value of  $n$ ,  $a[n]$ , and sum, we need  $S(p) \geq 3(n+1)$  words.

### 2.5.2.2 Time complexity

**Definition:** The time complexity  $T(P)$  for a program  $P$  is the sum of compile time and execution time.

The compile time is constant and does not depend on variable characteristics (int, float, long, etc.). Therefore,  $T(P) = C_p + C_a \text{ ADD}(n) + C_s \text{ SUB}(n) + \dots$ , where  $n$  denotes the instance characteristic and  $C_a$ ,  $C_s$ , ... represent the number of additions, subtractions, etc., that are performed on the instance characteristic  $n$ .

A programming step is a syntactically and semantically important piece of a program that executes independently of the instance characteristic. Consider the program that computes the sum of the objects in an  $n$ -dimensional array. A count variable is included in the program, which is incremented by one for each statement executed.

```
Algorithm sum(a, n)
{
 S=0.0;
 count = count+1 //count is initialized to 0.
 for i=1 to n do
 {
 count=count+1;
 S = S + a[i];
 count = count+1;
 }
 count=count+1; //last time for the for loop
 return S;
};
```

The value of count after the execution will be  $2n + 3$ , which indicates the number of steps executed.

Let us look at matrix addition.

```
Algorithm ADD(a,b,c,m,n)
// a, b are the matrices to be added. c is the result matrix. m>n
{
 for i=1 to m do
 for j=1 to n do
 c[i,j]=a[i,j] + b[i,j];
}
```

Let us calculate count by forming a table (see Table 2.1).

Table 2.1 Calculation of count after each statement is executed

| Statement             | s/e | Frequency    | Total steps  |
|-----------------------|-----|--------------|--------------|
| Algorithm MatAdd(a,b) | 0   | —            | 0            |
| {                     | 0   | —            | 0            |
| for i=1 to m do       | 1   | m + 1        | m + 1        |
| for j=1 to n do       | 1   | mn + m       | mn + m       |
| c[i,j]=a[i,j]+b[i,j]  | 1   | mn           | mn           |
| }                     | 0   | —            | 0            |
| Total =               |     | 2mn + 2m + 1 | 2mn + 2m + 1 |

### 2.5.2.3 Complexity analysis

The complexity analysis can produce three different outcomes: worst case, best case, and average scenario. The worst-case scenario occurs when the algorithm takes a large number of steps, and the best-case scenario occurs when the number of steps is the smallest. The average situation falls midway between these two extremes. In simple cases, the average case is calculated by looking at various inputs, calculating the number of steps necessary for each one, adding the number of steps required for all inputs, and dividing by the number of inputs. If we assume that the probability of occurrence of the  $i_{th}$  input is  $p(input_i)$ , then we may calculate

$$T_{ave} = \sum_i p(input_i) * steps(input_i)$$

assuming that the probability distribution of the inputs is known.

In the case of a sequential search, the complexity of the three instances is rather straightforward to estimate. However, as we'll see later, determining the average case behavior for all algorithms isn't always straightforward:

**Example:** Find the best, worst, and average cases to search for an element sequentially in an unordered array.

**Solution:** When the element to be searched is the last element in the array, the worst-case scenario happens, and when the element to be searched is the first element in the array, the best-case scenario occurs.

To find the average case, let us assume that the element we are looking for can be in any one of the cells of the array. In other words, the probability of occurrence in the first cell is  $\frac{1}{n}$ , in the second cell is  $\frac{1}{n}$ , and in the  $n$ th cell is  $\frac{1}{n}$ . Therefore, we may find the element in one trial, two trials, or  $n$  trials.

On the average, the number of trials needed is

$$\frac{(1 + 2 + 3 + \dots + n)}{n} = \frac{n + 1}{2}$$



**Example:** Take binary search, for example. Assume the input size is a power of two without sacrificing generality. If the element is in the center, a binary search can find it in one try. If it's in the middle of the left or right half, it'll take two attempts, or three tries if it's in the middle of the first quarter, or the middle of the second quarter, or the middle of the fourth quarter, etc., or  $\log_2 n$  tries if it's the first or last element.

Thus, the average number of possibilities is

$$1.1 + 2.2 + 4.3 + 8.4 + \dots + \frac{n}{2} \log_2 n$$

$$= \sum_0^{\log \log n - 1} 2^{(\text{power } i)} (i + 1)$$

and this has to be divided by  $n$ .

To express the complexity of an algorithm, we use Big Oh, Omega, Theta, Small Oh, and Little Omega and notations as defined and explained next.

### 2.5.2.3.1 Big oh

**Definition:** The function  $f(n)$  is  $O(g(n))$  if and only if there exists positive constants  $c$  and  $n_0$ , such that  $f(n) \leq cg(n)$  for all  $n$ ,  $n > n_0$ .

If  $f_1(n) \approx O(g_1(n))$  and  $f_2(n) = O(g_2(n))$ , then  $f_1(n) + f_2(n) = O(\max(g_1(n), g_2(n)))$  and

$$f_1(n) * f_2(n) = O((g_1(n)) * (g_2(n))).$$

Graphically, the relationship between  $f(n)$  and  $g(n)$  can be represented by the diagram in Figure 2.6.

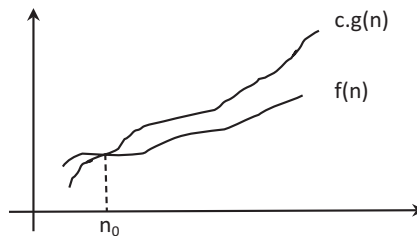


Figure 2.6 Big oh notation

**Example:** Let  $f(n) = 4n + 3$ ; then  $f(n) \leq 5n$  for all  $n \geq 3$ .

Thus,  $f(n)$  is  $O(n)$  with  $c = 5$  and  $n_0 = 3$ .

Let  $f(n) = 9n^2 + 5n + 3$ ; then  $f(n) \leq 10n^2$  for  $n \geq 6$ .

Therefore, we say  $f(n)$  is  $O(n^2)$  with  $c = 10$  and  $n_0 = 6$ .

**Theorem:** If  $f(n) = a_m n^m + \dots + a_1 n + a_0$ , then  $f(n)$  is  $O(n^m)$ .

**Proof:** Since the constants  $a_m, \dots, a_0$  can be positive or negative, we can write

$$f(n) \leq |a_m|n^m + \dots + |a_1|n + |a_0|$$

$$\leq n^m \sum_{i=0}^m |a_i| n^{i-m}$$

$< n^m \sum |a[m]|$  since  $(i - m)$ , the exponent of  $n$  in the summation is always negative and hence  $n^{i-m}$  is always less than 1. Let  $c = \sum |a[m]|$ . Then,  $f(n) \leq cn^m$  and  $f(n) = O(n^m)$ .

### 2.5.2.3.2 Omega

**Definition:** The function  $f(n)$  is  $\Omega(g(n))$  if and only if there exist positive constants  $c$  and  $n_0$ , such that  $f(n) \geq c \cdot g(n)$  for all  $n \geq n_0$  (see Figure 2.7).

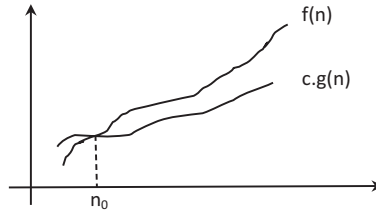


Figure 2.7 Omega notation

**Example:** Let  $f(n) = 4n + 3$ . Then  $f(n) \geq 4n$  for all  $n > 0$ . Thus,  $f(n)$  is  $\Omega(n)$  with  $c = 5$  and  $n_0 = 3$ .

**Example:** Let  $f(n) = 6 \cdot 2^n + n^2$ . Then  $f(n) \geq 2^n$  for all  $n \geq 1$ . Hence,  $f(n)$  is  $\Omega(2^n)$ .

We can generalize and say if  $f(n) = a_m n^m + \dots + a_1 n + a_0$ , then  $f(n)$  is  $\Omega(n^m)$ .

### 2.5.2.3.3 Theta

**Definition:** The function  $f(n)$  is  $\Theta(g(n))$  if and only if there exist positive constants  $c_1, c_2$ , and  $n_0$ , such that  $c_1 g(n) \leq f(n) \leq c_2 g(n)$  for all  $n \geq n_0$ .

**Example:** Let  $f(n) = 3n + 2$ . Then  $3n \leq 3n + 2 \leq 4n$  for all  $n \geq 2$ . Hence,  $f(n)$  is  $\Theta(n)$ .

We can generalize and say if  $f(n) = a_m n^m + \dots + a_1 n + a_0$ , then  $f(n) = \Theta(n^m)$ .

In general, exponential functions grow faster than polynomial functions and polynomial functions grow faster than logarithmic functions (Figure 2.8).



**Fill-in-the-blank questions**

1. \_\_\_\_\_ data types are those data types that are not in a specific order.
2. Machine instructions work directly on \_\_\_\_\_ data structures.
3. The function  $f(n)$  is \_\_\_\_\_ if and only if  $\text{Lt } (f(n)/g(n)) = 0$  as  $n \rightarrow \infty$ .
4. The performance is determined by the amount of \_\_\_\_\_ required for the program and \_\_\_\_\_.
5. The Towers of Hanoi problem is an example of a \_\_\_\_\_ algorithm.

**Multiple-choice questions**

1. Which one of the following is not true about a queue?
  - a. It is a primitive data type.
  - b. It is a non-linear data structure.
  - c. It is a hierarchical data structure.
  - d. All of the above.
2. An abstract data type (ADT) is
  - a. A small data type that cannot be instantiated.
  - b. The same as abstract class.
  - c. A data type for which only operation is defined on it can be used and nothing else.
  - d. All of the above.
3. Which of the following is true about a stack?
  - a. Data can be inserted only at one end.
  - b. Data can be inserted at both ends.
  - c. Data can be deleted from both ends.
  - d. None of the above.
4. Which one of the following is not the application of the queue data structure?
  - a. Data transfer asynchronously
  - b. Load balancing
  - c. Resource shared between various systems
  - d. Balancing of symbols
5. Traversal refers to
  - a. Putting the data in order
  - b. Reaching each element
  - c. Combining data elements
  - d. Finding a data element

6. Which of the following data structures allows deleting an element at one end and inserting it at the other end?
  - a. Stack
  - b. Queue
  - c. Array
  - d. Tree
7. Which of the following is a non-linear type data structure?
  - a. Stack
  - b. Queue
  - c. Array
  - d. None of the above
8. Which of the following is a linear type data structure?
  - a. Stack
  - b. Queue
  - c. Array
  - d. All of the above
9. Which of the following represents a hierarchical data structure?
  - a. Stack
  - b. Queue
  - c. Array
  - d. Tree
10. Stack is also called \_\_\_\_\_.
  - a. LIFO
  - b. FIFO
  - c. Tree
  - d. None of the above
11. Queue is also called \_\_\_\_\_.
  - a. LIFO
  - b. FIFO
  - c. Tree
  - d. None of the above
12. Which of the following data structures can't store non-homogeneous data elements?
  - a. Array
  - b. Stack
  - c. Queue
  - d. All of the above

- 
13. Which of the following is true about the characteristics of an abstract data type?
- Has a type
  - Has an operation
    - True, False
    - False, True
    - True, True
    - False, False
14. Push and pop operation refers to
- Array
  - Stack
  - Tree
  - Graph
15. Which of the following is a hierarchical data structure?
- Array
  - Stack
  - Tree
  - Graph
16. Which of the following is very useful in situations when data have to be stored and then retrieved in reverse order?
- Array
  - Stack
  - Tree
  - Graph
17. Linear arrays are also called \_\_\_\_\_.
- Straight line array
  - Vertical array
  - Horizontal array
  - One-dimensional array
18. Which of the following is an indexed data structure?
- Tree
  - Graph
  - Stack
  - Array
19. Match the following properties of an array with their descriptions.
- |                 |                                                                          |
|-----------------|--------------------------------------------------------------------------|
| a) Homogeneous  | i) The list size is constant.                                            |
| b) Ordered      | ii) There is a first and last element.                                   |
| c) Finite       | iii) There is a next and previous in the natural order of the structure. |
| d) Fixed length | iv) Every element is the same.                                           |

- a. a-i, b-ii, c-iii, d-iv
- b. a-ii, b-iii, c-iv, d-i
- c. a-iii, b-i, c-ii, d-iii
- d. a-iv, b-iii, c-ii, d-i

20. Which of the following is an application of stack?

- a. Infix to postfix conversion
- b. Finding factorial
- c. Tower of Hanoi
- d. All of the above

### Descriptive questions

- 1. What is a data structure?
- 2. What is the difference between primitive data and non-primitive data?
- 3. Explain the different operations on a data structure.
- 4. Write briefly about different data structures.
- 5. Explain the terms data type, data structures, and abstract data type.

### Answers to true-false questions

- 1. False
- 2. True
- 3. False
- 4. True
- 5. False

### Answers to fill-in-the-blank questions

- 1. Non-linear
- 2. primitive
- 3.  $O(g(n))$
- 4. storage space, data
- 5. recursive

### Answers to multiple-choice questions

- |       |       |       |       |       |
|-------|-------|-------|-------|-------|
| 1. d  | 2. c  | 3. a  | 4. d  | 5. b  |
| 6. b  | 7. d  | 8. d  | 9. d  | 10. a |
| 11. b | 12. d | 13. c | 14. b | 15. c |
| 16. b | 17. d | 18. d | 19. d | 20. d |

# Arrays

---

### LEARNING OBJECTIVES

After studying this chapter, readers will be able to:

- Know about arrays
- Use array as an abstract data type
- Implement array using list
- Implement array using dictionary
- Implement array using class
- Define and use multidimensional arrays
- Apply array to manipulate polynomials
- Apply array to manipulate sparse matrices

In Chapter 1 we saw that the non-primitive data structure is divided into linear, non-linear and files. Again the linear data structures are divided into array, stacks and queues. In this chapter we will focus on the array linear data structures. Linear data structures can be implemented using arrays. The sequential organization of an array is a fixed size and uses continuous memory locations and, hence, the access time to any data element is the same, however large may be the array size. However, in-between insertions and deletions are expensive since they require data shifting by several locations to keep the data consistent.

The elements are related to each other in a particular order or sequence. Some examples are:

1. even numbers less than or equal to 20  
= {0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20}
2. days = {Sunday, Monday, ..., Saturday}
3. colors in a rainbow = {violet, indigo, blue, green, yellow, orange, red}

The operations one needs to perform in an order are to traverse from left to right and from right to left.



In Python there are several built-in data structures, such as a list, a tuple, and a dictionary, that can be used to implement arrays.

### 3.1 INTRODUCTION

An array, in almost all computer languages, is a built-in data type. In other words, an array can be declared just as an integer or a floating point number. The restriction on an array is that all the elements we wish to store in the array must be of the same data type and have a predetermined size.

An array, as a data structure, may be defined as a set of pairs (index, value) such that with each index, a value is associated:

- Index → indicates location
- Value → indicates actual value stored at the location

Thus, we can define an array as a set of finite, ordered collections of homogeneous elements that provides direct access to any of its elements in constant time. The data type of the array is the data type of its elements. We can access the elements by using appropriate subscripts in the array.

#### 3.1.1 Array as an abstract data type

The following operations are provided with the abstract data type array (also see Table 3.1):

ADT Array Array\_name

- Instance: create array\_name(size)
- Pre-condition: At least one element required to perform any operation.
- Operations: Insert, Delete, Access

Table 3.1 Operation in Array datatype

| Operation                                                                    | Return |
|------------------------------------------------------------------------------|--------|
| Declare create(n)                                                            | array. |
| Access(array, i)                                                             | value. |
| Insert(array, index, value)                                                  | array. |
| Delete(array, index)                                                         | array. |
| Access(create(), i)                                                          | error. |
| Access(store(array(index, i, x), j)<br>= x if (i == j) else access(array, j) | value. |

The *create()* function creates a new array.

*access(array, i)* accesses the *i*th element in the array.

The third operation, *insert(array, index, value)*, stores the given value at the index position of the array and returns the updated array.

The operation *delete(array, index)* deletes the value at a given index position of the array and returns the updated array.

The operation *access(create(), i)* gives an error because *create()* creates an empty array and we cannot access the *i*th element from a newly created array.

The last operation is easy to understand. The *access()* function returns the value in the *i*th position if  $i == j$ , else returns the value in the *j*th position.

Python arrays can be implemented using a list, a tuple or a dictionary and also by importing the *array* module or *numpy* module. A list is a mutable sequential and heterogeneous data structure. We can enforce the storing of homogeneous data by restricting the user to entering a specific type of data. As the size of the list is not static, it violates the notion of predetermined size. But we can enforce an array size.

A tuple can also be used for defining an array. A tuple is an immutable sequence of heterogeneous data structures. As it is immutable, we cannot perform the delete or insert operation once the tuple is defined. In order to perform this operation, we need to convert the tuple to a list, perform the operation and convert it back to a tuple.

A dictionary can also be used for defining an array. A dictionary is a mutable sequential and heterogeneous data structure that has key-value pairs as the data. As the key is unique, it can be treated as the index of an array and the value can be treated as data of the array. We can enforce the storing of homogeneous data by restricting the user to entering a specific type of data. As the size of the dictionary is not fixed, it violates the notion of predetermined size. While inserting and deleting elements, we need to adjust the key of the dictionary.

It is worth mentioning that while implementing an array using *list* or importing an *array* module or importing a *numpy* module, except for integer data, all other data are not kept in consecutive memory locations.

### 3.1.2 Implementation of array using list

Let us create an array of integers using a list and implement all the operations through Python. In this implementation, we have not enforced the fixed size of the list.

### Python program 1

```
#create a list of integer
def create(n):
 lst=[]
 for i in range(n):
 no= int(input("Enter the number "))
 lst.append(no)
 return lst
```

The *create(n)* function creates a list with  $n$  elements. This method reads  $n$  integers appended to a list after the user enters all  $n$  integers and then the list is returned.

### Program continues...

```
def display(a):
 print("The data in the array are :")
 print(a)
```

The *display(a)* displays all the elements of the array.

### Program continues...

```
Access a data in the list from a given location
def access(a,i):
 if 0 <= i <= len(a):
 print("The element at position ",i," is ",a[i])
 else:
 print("The index is more than the size of array")
```

The *access(a, i)* method finds the element at  $i$ th location, if the location is not less than 0 or not greater than the size of the array.

### Program continues...

```
insert a data in the list at a given location
def insrt(a,i,x):
 if i < 0:
 print("Location is below the range")
 return a
 elif i > len(a) :
 print("Location is out of the range")
 return a
 else:
 a.insert(i,x)
 return a
```

The *insrt(a,i,x)* function adds an element at the *i*th location. This can be done by the *a.insert(i,x)* command while using a list.

**Program continues...**

```
Delete a data from the list at a given location
def delit(a,i):
 if i < 0:
 print("Location is below the range")
 return a
 elif i > len(a) :
 print("Location is out of the range")
 return a
 else:
 del a[i]
 return a
```

Deletion of an element which is at *i*th location can be done by the *del a[i]* command.

**Program continues...**

```
Main Program
n = int(input("Enter How many numbers in an array "))
Arr=create(n)
display(Arr)

print("\n--- Data at Location ---")
i = int(input("Element at which position element to get "))
access(Arr,i)

print("\n--- Insert at a Location ---")
j = int(input("Specify the Location to Insert "))
y = int(input("Specify the Data "))
Arr = insrt(Arr,j,y)
display(Arr)

print("\n--- Deletion at positon ---")
j = int(input("Specify the Location to Delete "))
Arr=delit(Arr,j)
display(Arr)
```

## Output

```
Enter How many numbers in an array 5
Enter the number 10
Enter the number 20
Enter the number 30
Enter the number 40
Enter the number 50
[10, 20, 30, 40, 50]

--- Data at Location ---
Element at which position element to get 2
The element at position 2 is 30

--- Insert at a Location ---
Specify the Location to Insert 4
Specify the Data 60
[10, 20, 30, 40, 60, 50]

--- Deletion at position ---
Specify the Location to Delete 3
[10, 20, 30, 60, 50]
```

Some key facts to remember regarding Python lists:

- It is possible for the list to be homogeneous or heterogeneous.
- Python lists are one-dimensional by default. We can, however, make an n-dimensional list. However, it will still be a 1D list holding another 1D list.
- A list's elements don't have to be in order in memory.

### 3.1.3 Implementation of array using dictionary

Let us create an array of integers using a dictionary and implement all the operations. In this implementation we have not enforced the fixed size of the array.

#### Python program 2

```
def create():
 A={}
 n=int(input("\nEnter the number of elements :"))
 print("\n")
 for i in range(n):
 print("Enter data at ",i,end=": ")
 A[i]=int(input())
 return A
```

The *create()* function first creates an empty dictionary. And then for every key, a corresponding value is appended. This is done by the *A[i]=int(input())* command, which restricts the user to enter an integer value only.

**Program continues...**

```
def display(A):
 for i in range(len(A)):
 print("A[" + str(i) + "] = " + str(A[i]))
```

The *display(A)* function displays the elements of the dictionary in the form of an array.

**Program continues...**

```
Access a data in the list from a given location
def access(A,i):
 if 0 <= i <= len(A):
 print("The element at position " + str(i) + " is " + str(A[i]))
 else:
 print("The index is more than the size of array")
```

The *insrt(A, i, x)* method inserts an element with value *x* at *i*th location.

| 0  | 1  | 2  | 3  | 4  |
|----|----|----|----|----|
| 10 | 15 | 20 | 25 | 30 |

Figure 3.1 Sample Array

The insert method takes three arguments: the array itself, the position where the value is to be inserted and the value (Figure 3.1). If we want to insert a value at the *i*th position, we have to move all the elements of the array one position to the right starting from the *i*th position. Suppose *i* = 2 (Figure 3.2).

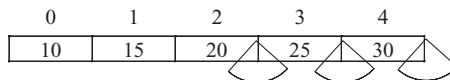


Figure 3.2 Movement of Data in Array-1

The size of the list will be increased by 1 after insertion of the element (Figure 3.3).

| 0  | 1  | 2 | 3  | 4  | 5  |
|----|----|---|----|----|----|
| 10 | 15 |   | 20 | 25 | 30 |

Figure 3.3 Movement of Data to a different position

We can now insert the element in the  $i$ th position. Let the element to be inserted be 17. The array now is as shown in Figure 3.4.

| 0  | 1  | 2  | 3  | 4  | 5  |
|----|----|----|----|----|----|
| 10 | 15 | 17 | 20 | 25 | 30 |

Figure 3.4 Insertion of Data at the  $i$ th location

**Program continues...**

```
def insrt(A,i,x):
 if i < 0:
 print("Location is below the range")
 return A
 elif i > len(A) :
 print("Location is out of the range")
 return A
 else:
 for j in range(len(A)-1,i-1,-1):
 A[j+1] = A[j]
 A[i]=x
 return A
```

The *delit*(A,  $i$ ) method deletes the element at the  $i$ th location (Figure 3.5).

| 0  | 1  | 2  | 3  | 4  | 5  |
|----|----|----|----|----|----|
| 10 | 15 | 17 | 20 | 25 | 30 |

Figure 3.5 Original Array

Suppose we want to delete the element in the second position. Delete the data 17 at the second location (Figure 3.6).

| 0  | 1  |  | 3  | 4  | 5  |
|----|----|--|----|----|----|
| 10 | 15 |  | 20 | 25 | 30 |

Figure 3.6 Deletion of element from array

Observe that the value for key = 2 is deleted. Now we need to reorder the keys and values by saving the data of the location. The key in a dictionary is unique and it is not adjusted dynamically. We need to manage it by the program.

All the elements to the right of the second location (the element in the second position) are shifted one position to the left. This can be done by overwriting the second cell with the element in the third cell, and the element in the third cell is overwritten with the element in the fourth cell and so on. Also we need to reorder the keys of the dictionary (Figure 3.7).

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 0  | 1  | 2  | 3  | 4  | 5  |
| 10 | 15 | 20 | 25 | 30 | 30 |

Figure 3.7 Array after left movement of data

Lastly, delete the last cell (Figure 3.8).

|    |    |    |    |    |
|----|----|----|----|----|
| 0  | 1  | 2  | 3  | 4  |
| 10 | 15 | 20 | 25 | 30 |

Figure 3.8 Array after deleting the last element

### Program continues...

```
Delete a data from the list at a given location
def delit(A,i):
 if I < 0:
 print("Location is below the range")
 return A
 elif I >= len(A) :
 print("Location is out of the range")
 return A
 else:
 del A[i]

 for j in range(I,len(A)):
 A[j]=A[j+1]
 del (A[len(A)-1])
 return A
```

### Program continues...

```
main Program (Driver Function)
Arr = create()
print("\nThe element in the array is ")
display(Arr)

print("\n--- Data at Location ---")
i = int(input("Element at which position element to get "))
access(Arr,i)

print("\n--- Insert at a Location ---")
j = int(input("Specify the Location to Insert "))
y = int(input("Specify the Data "))
Arr = insrt(Arr,j,y)
display(Arr)

print("\n--- Deletion at position ---")
j = int(input("Specify the Location to Delete "))
Arr=delit(Arr,j)
display(Arr)
```



## Output

```
Enter the number of elements :5

Enter data at 0: 1
Enter data at 1: 2
Enter data at 2: 3
Enter data at 3: 4
Enter data at 4: 5

The element in the array is
A[0] = 1
A[1] = 2
A[2] = 3
A[3] = 4
A[4] = 5

--- Data at Location ---
Element at which position element to get 3
The element at position 3 is 4

--- Insert at a Location ---
Specify the Location to Insert 4
Specify the Data 20
A[0] = 1
A[1] = 2
A[2] = 3
A[3] = 4
A[4] = 20
A[5] = 5

--- Deletion at position ---
Specify the Location to Delete 2
A[0] = 1
A[1] = 2
A[2] = 4
A[3] = 20
A[4] = 5
```

### 3.1.4 Implementation of array using import array

A one-dimensional array can be created by importing the *array* module.

#### Python program 3

```
from array import *
A = array('I', [1,2,3])
print(A)
```

To utilize the *array* module, we must first import it using the line *from array import \**. Here *A* is the name of the array, and the method *array()* has two parameters. The type code, in this example, *i* is the first parameter, and the data is the second. Different type codes are given in Table 3.2.

Table 3.2 Data types and their size

| Type code | C Type             | Python Type       | Minimum size in bytes |
|-----------|--------------------|-------------------|-----------------------|
| 'b'       | signed char        | Int               | 1                     |
| 'B'       | unsigned char      | Unicode character | 1                     |
| 'u'       | Py_UNICODE         | Int               | 2                     |
| 'h'       | signed short       | Int               | 2                     |
| 'H'       | unsigned short     | Int               | 2                     |
| 'l'       | signed int         | Int               | 2                     |
| 'L'       | unsigned int       | Int               | 2                     |
| 'l'       | signed long        | Int               | 4                     |
| 'L'       | unsigned long      | Int               | 4                     |
| 'q'       | signed long long   | Int               | 8                     |
| 'Q'       | unsigned long long | Int               | 8                     |
| 'f'       | Float              | Float             | 4                     |
| 'd'       | Double             | Float             | 8                     |

The program for accessing an array, inserting an element into the array and deleting an element from the array can be implemented as discussed in the list.

The Python array module provides the following advantages over Python lists:

- A list includes items of multiple data types, whereas the Python array module has elements of the same data type.
- To declare an array in Python, the array module must be explicitly imported, while in list, no module is required to be imported and no declaration is made.
- The Array module in Python may perform arithmetic operations.
- All items in the Python array module must be the same size.
- The term is most commonly used in the context of a prolonged succession of data items.
- Because addition, deletion and update operations are done on a single element at a time, modifying an array is complex.
- We'll need an explicit loop to print or access array items.
- In comparison to the list, it is a more compact in-memory size.

A list is the best solution if we need to store a reasonably brief sequence of elements and don't want to conduct any mathematical operations with it. Without importing any extra modules or packages, this data structure will let you store an ordered, mutable and indexed sequence of objects.

Consider utilizing an array if we have a long series of objects. This structure makes data storage more efficient. Use an array to conduct any numerical operations with your collection of objects. Arrays are widely used in data analytics and data science.

### 3.1.5 Implementation of array using NumPy

The module *Numpy* is a Python extension and is the most important module for scientific computing. *Numpy* arrays make it easier to do complex mathematical and other operations on enormous amounts of data. Such actions are often performed more quickly and with less code than utilizing Python's built-in methods. It performs quick and efficient operations on homogeneous data arrays.

The *numpy* package must be installed (if it is not already installed) before importing it. At the command prompt, type *pip install numpy* to install the package. It's a package for processing arrays in general.

#### Python program 4

```
import numpy as np
A = np.array([1,2,3], dtype = 'int')
print(A)
```

The package *numpy* must be imported before constructing an array. An alias (np) is used here to give a *numpy* shortcut. The function *np.array()* has two arguments to generate an array. The data (inside the square brackets) is the first argument, and the data type object (dtype) or type code of the array is the second argument. In this code, *dtype = 'int'* is used. This argument is optional for an array of integers, but it must be written for arrays of other data types.

In the example below, the type code is used as the second parameter. The second argument is optional for unsigned integers.

#### Python program 5

```
import numpy as np
A = np.array([1,2,3], 'I')
print(A)
```

The method for accessing array elements, inserting an element into the array and deleting an element from the array can be implemented as discussed in Section 3.1.2.

Some important points about *Numpy* arrays:

- *Numpy* arrays can also be used to construct an N-dimensional array in Python ().
- Arrays are homogeneous by default.
- It is possible to operate on individual elements.
- The *Numpy* array has a variety of functions, methods and variables that make calculating matrices easier.

*Numpy* arrays provide the following advantages over Python lists:

- *Numpy* arrays use less memory.
- When compared to the Python list, *Numpy* arrays are much faster.
- It is simple to use.
- It has many built-in optimized functions to perform array operations.
- Arrays are less versatile than lists. They have the ability to hold components of many data types, including strings. Furthermore, if you need to do mathematical computations on arrays and matrices, we should use *NumPy*.

We can implement an array using a list, Python *numpy* module or *array* module. Each method has its own benefits.

### 3.1.6 Implementation of array using class

We discussed class in Chapter 2. Now we will use class to implement an array. In this implementation, we have enforced the fixed size of the array.

#### Python program 6

```
Creating an Array
class Arr:
 def __init__(self):
 self.maxSize = 10
 self.size = 0
 self.items = list()
```

The name of the array class is *Arr* and it can hold elements up to the *max-Size* (initialized to 10). The *size* is also initialized to 0. It also has an empty list. All the methods associated with the class are declared as public.

Let us define each method:

**Program continues...**

```
Reading data for Array
def readArray(self):
 no = int(input("How many data to enter (Less than 10) "))
 if (no <= self.maxSize) :
 for I in range(no):
 data = input("Enter the data ")
 self.items.append(data)
 self.size += 1
 else:
 print("Number of Data is greater than Maximum size of array")
```

The *readArray ()* method reads character values into the list. It prompts how many characters to insert into the array. It checks if the input *array\_size* is greater than the *maxSize* and if so, prints the appropriate message and comes out of the method. Otherwise, it prompts the user to insert data into the array.

Suppose the data inserted into the array is as shown in Figure 3.9.

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 |
| A | B | C | D | E |

Figure 3.9 Given array

**Program continues...**

```
Inserting Data at Location
def addAtIndex(self,index,data):
 if ((index > self.size) or (self.size > self.maxSize)):
 print("Addition of",data, "at index", index,"is not
 possible")
 else:
 self.items.append(None)
 for i in range(self.size, index, -1):
 self.items[i] = self.items[i - 1]
 self.items[index] = data
 self.size += 1
```

Insertions and deletions into or from an array are costly operations since they involve data movement to the right, and to the left in the case of deletions.

The *addAtIndex()* method takes two arguments, the position where the value is to be inserted (*index*) and the value (*data*). We need to check if the *index* at which we want to insert the data is beyond the current array size or the current array size is greater than or equal to the maximum size of the array. If either of the conditions is true, we cannot enter the data into the

array so an appropriate message is displayed and comes out of the method. Otherwise we can insert the data into the array.

If we want to insert a value at the  $j$ th position, we have to move all the elements of the array one position to the right starting from the  $j$ th position. Suppose  $j = 2$  (Figure 3.10).

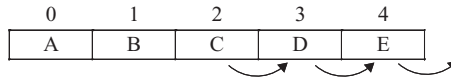


Figure 3.10 Movement of data in Array

When we insert an element into the array, the size of the array increases by 1. The elements are shifted one position to the right starting from the last element (the element at position  $\text{size} - 1$ ) (Figure 3.11).

| 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| A | B | C | C | D | E |

Figure 3.11 Array after data movement

We can now insert the element in the  $j$ th position. Let the element to be inserted be Z. The array now is as shown in Figure 3.12.

| 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| A | B | Z | C | D | E |

Figure 3.12 After insertion of data

**Program continues...**

```
#Searching
def search(self, data):
 print("Searching for Element...",data)
 if data in self.items:
 position = 0
 for i in range(self.size):
 if(self.items[i] == data):
 break
 else:
 position += 1
 print('Element {} found at position {}'.format(data, position))
 else:
 print('This element is not in the Array!')
```

The method *search()* searches for the data passed as parameters. It is quite straightforward. We will keep searching from index 0 until the current size

of the array. If it matches, come out of the looping structure and print an appropriate message for finding the element. Otherwise print an appropriate message for not finding the data.

Deletion of a data requires searching for data in the array and if found then delete the data and reduce the size by 1, otherwise print an appropriate message.

**Program continues...**

```
#Delete a Data
def deleteData(self, data):
 print("Deletion ...",data)
 if data in self.items:
 self.items.remove(data)
 self.size -= 1
 else:
 print('This element is not in the Array!')
```

Unlike any other programming language, we need not write any code for searching for data. Searching of data is performed simply by the *if data in self.items* statement.

Suppose we want to delete the element in the second position. All the elements to the right of Z (the element in the second position) are shifted one position to the left. This overwrites the second cell with the element in the third cell, and the element in the third cell is overwritten with the element in the fourth cell and so on, and the blank cell moves to the extreme right.

**Program continues...**

```
Delete an element at a location
def deleteAtLoc(self, loc):
 print("Deletion at location...",loc)
 if (loc == self.size):
 del self.items[loc]
 self.size -= 1
 else:
 print('This element is not in the Array!')
```

The *deleteAtLoc()* method takes one parameter, the location of data to be deleted. In Python deletion can be done by the *del* command.

**Program continues...**

```
Main Program
myArray = Arr()
myArray.readArray()
print(myArray.items,"Array with" ,myArray.size,"Elements")
```

---

```

#Inserting data at location
if (myArray.size == 0):
 data = input("Enter the data ")
 myArray.addAtIndex(0,data)
 print(myArray.items,"Array with" ,myArray.size,"Elements")
else:
 print("\nInsert Data at location ")
 loc = int(input("Enter the location "))
 data = input("Enter the data ")
 myArray.addAtIndex(loc,data)
 print(myArray.items,"Array with" ,myArray.size,"Elements")

Searching data
ele = input("\nEnter data to search ")
myArray.search(ele)

#Deleting Data
ele = input("\nEnter data to Delete ")
myArray.deleteData(ele)
print(myArray.items,"Array with" ,myArray.size,"Elements")

#Deleting data at a given location
loc = int(input("\nEnter location of data to Delete "))
myArray.deleteAtLoc(loc)
print(myArray.items,"Array with" ,myArray.size,"Elements")

```

In the main program, an object, *myArray* of the class *Arr*, is created. All the methods defined earlier are called to perform the operations.

## Output

```

How many data to enter (Less than 10) 4
Enter the data A
Enter the data B
Enter the data C
Enter the data D
['A', 'B', 'C', 'D'] Array with 4 Elements

Insert Data at location
Enter the location 2
Enter the data Z
['A', 'B', 'Z', 'C', 'D'] Array with 5 Elements

Enter data to search B
Searching for Element... B
Element B found at position 1

Enter data to Delete C
Deletion ... C
['A', 'B', 'Z', 'D'] Array with 4 Elements

```



```
Enter location of data to Delete 2
Deletion at location... 2
['A', 'B', 'D'] Array with 3 Elements
```

3.2 MULTIDIMENSIONAL ARRAYS

A one-dimensional array can be used to store the values of a function of one variable such as  $f(x) = 3x^4 + 8x^2 + 5x + 9$ ; the cells of the array taking values of  $f$  for different integer values of  $x$ . The values of  $f$  corresponding to the integer values of  $x$  from 0 to 5 can be stored in an array as shown in Figure 3.13.

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 |
| 9 | 5 | 8 | 0 | 3 | 0 |

Figure 3.13 Data values stored in one dimensional array

However, if we want to store the values of a function in two variables, say,  $x$  and  $y$  such as  $f(x,y) = x^2 + 5xy + 3y^2$ , a one-dimensional array is not very useful. We need a two-dimensional array, represented as a grid, the values in the grid corresponding to values of  $f$  for different values of  $x$  and  $y$  as shown in Figure 3.14.

|                |    |    |    |    |    |     |
|----------------|----|----|----|----|----|-----|
| Index<br>y↓ x□ | 0  | 1  | 2  | 3  | 4  | 5   |
| 0              | 1  | 3  | 12 | 27 | 48 | 75  |
| 1              | 5  | 9  | 23 | 43 | 69 | 101 |
| 2              | 32 | 15 | 36 | 52 | 92 | 129 |

Figure 3.14 Data values stored in multi-dimensional array

We can declare such an array as  $z[3][6]$ , with 3 and 6 indicating the number of rows and columns, and the values can be accessed as

$f[0][0] = 1, f[0][1] = 3, \dots, f[2][5] = 129.$

Two-dimensional arrays are called matrices. There are two methods of representing matrices in the computer memory:

- 1. row-major method
- 2. column-major method

In the row-major method, all rows are stored one row after another starting from the first row like:

```
0 3 12 27 48 75 1 9 23 43 69 101 4 15 36 52 92 129
```

In the column-major method, all columns are stored from the first column like:

0 1 4 3 9 15 12 23 36 27 43 52 48 69 92 75 101 129

The programmer need not know which method was used to store the two-dimensional array.

The same method of storage can be extended to multidimensional arrays (arrays of more than 2 dimensions). A three-dimensional array,  $A[m][n][p]$ , is represented as  $m \times n \times p$  dimensional matrices and these  $n \times p$  matrices are stored one after the other starting from  $m = 0$  in a row-major fashion. The element  $A[i][j][k]$ , therefore, occurs in the memory where the  $i$ th  $n \times p$  matrix is stored. Therefore, we have to leave the first  $(i-1)np$  locations starting from the first location where the three-dimensional matrix is stored, and search for  $A[i][j][k]$  in the next  $n \times p$  locations.

3.3 APPLICATIONS

Arrays have various applications and are widely used in programming and data structures. We will discuss two important applications, polynomial manipulation and sparse matrix implementation, in this section.

3.3.1 Polynomial manipulation

An array is a very useful data structure to implement polynomials in a computer. A polynomial in a single variable can be written as  $P(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$ . The degree of the polynomial is  $n$  and it contains  $(n + 1)$  terms. A polynomial of  $n$ th degree may therefore be represented by the array  $(a_0, a_1, a_2, \dots, a_n)$ .

Where the  $i$ th element of the array represents the coefficient of  $x^i$ . All the elements of the array may not be non-zero. For example, the polynomial  $3 - 2x - x^2 + x^4$  is represented by the array in Figure 3.15.

|   |    |    |   |   |
|---|----|----|---|---|
| 3 | -2 | -1 | 0 | 1 |
|---|----|----|---|---|

Figure 3.15 Polynomial in an Array

If we want to represent a ‘sparse’ polynomial  $5 + x^{98}$  we need an array of size 99 in which most of the elements are zero (Figure 3.16).

|   |   |   |   |   |   |   |   |   |    |
|---|---|---|---|---|---|---|---|---|----|
| 5 | 0 | 0 | 0 | 0 | 0 | - | - | - | 98 |
|---|---|---|---|---|---|---|---|---|----|

Figure 3.16 Sparse polynomial in Array

This is called a sparse polynomial since there are very few non-zero elements and the terms with 0 coefficients occupy a lot of space and may not contribute much to the operations on the polynomial.

The sparsity of a polynomial is the ratio between the number of zeros and the total number of elements. In the polynomial  $3 - 2x - x^2 + x^4$ , the number of zeros is 1 (as the  $x^3$  term is missing) and the total number of terms is 5. So the sparsity of the polynomial is  $1/5$ . Similarly, for the polynomial  $5 + x^{98}$ , the number of zero terms is 97 and the total number of terms is 99. So the sparsity is  $97/99$ .

We have discussed that every term in a polynomial is represented by a coefficient and an exponent. Python has many such constructs that are capable of representing a polynomial. We are using a dictionary to represent a polynomial.

A dictionary has two parts: a *key* and a *value*. In a dictionary, the key is unique so we will use it to represent the *exponent* and the value to represent the *coefficient* of a term in a polynomial.

We can represent the polynomial  $3 - 2x - x^2 + x^4$  in a dictionary as  $\{0:3, 1:-2, 2:-1, 3:0, 4:1\}$ .

A polynomial is an example of ordered data structure. The abstract data type for a polynomial is:

#### ADT Polynomial

Instance: create a polynomial

Pre-condition: At least one term is required to perform any operation.

Operations: Addition of two polynomial

Subtraction of two polynomial,

Multiplication of two polynomial and

Evaluation of a polynomial

Let us define a polynomial as an object belonging to a dictionary.

#### Python program 7

*# reads a polynomial*

```
def readPoly():
 p={}
 n=int(input("\nEnter Degree of Polynomial :"))
 print("\n")
 for i in range(n):
 print("Enter coefficient for x^",i,end=": ")
 p[i]=int(input())
 return p
```

The size of the polynomial is given in line 3. The for loop from lines 5 to 7 reads the coefficients and the corresponding exponents and creates the terms of the polynomial.

**Program continues...**

*# displays the polynomial in a readable form*

```
def print_poly(p):
 res=""
 for expo,coeff in p.items():
 if expo == 0:
 res = str(coeff)

 else:
 if int(coeff) == 0:
 pass
 else:
 res = res + ' + ' + str(coeff)+"x^"+str(expo)
 print(res)
```

The print\_poly(p) function displays on the screen the coefficients and exponents in the order of the terms of the polynomial.

**Program continues...**

*# adds two polynomials*

```
def add_poly(p1,p2):
 p3={}
 i=0
 j=0

 # while both polynomial have elements
 while (i < len(p1) and j < len(p2)):

 if (i < j):
 p3[i]=p1[i]
 i+=1

 elif (i > j):
 p3[j]=p2[j]
 j+=1

 else:
 p3[i]= p1[i] + p2[j]
 i+=1
 j+=1

 # if first polynomial exhausted
 if (i >= len(p1)):
 while (j != len(p2)):
 p3[j]=p2[j]
 j+=1
```

```
if second polynomial exhausted
if (j >= len(p2)):
 while (i != len(p1)):
 p3[i]=p1[i]
 i+=1
return p3
```

When we add two polynomials, the resultant polynomial is of the size of the larger of the two polynomials. Here p1 and p2 are two polynomials passed as parameters and defined in an empty dictionary, p3, to store the result. There are three cases we come across while adding two polynomials.

The first case is that while each polynomial has some terms, we will check each term of both polynomials. Starting with exponent 0, we will check if the exponent of a term in p1 is less than the exponent of the term in p2. If so, copy the coefficient of p1 to p3 at the exponent of p1. Otherwise, copy the coefficient of p2 to p3 at the exponent of p2. If the exponent of both terms are the same, add the coefficient of p1 and p2 to store it at the same location of p3.

The second case is that if there are some elements left out in polynomial p2 while there are no terms left in p1, copy the remaining terms of p2 to p3.

The third case is that if there are some elements left out in polynomial p1 while there are no terms left in p2, copy the remaining terms of p1 to p3. At last, the resultant polynomial, p3, is returned.

### **Program continues...**

#### *# multiplies two polynomials*

```
def mul_poly(p1,p2):
 p3={}
 #Initializing the Dictionary to 0
 for expol,coeff1 in p1.items():
 for expo2,coeff2 in p2.items():
 p3[expol+expo2]=0

 #Polynomial Multiplication
 for expol,coeff1 in p1.items():
 for expo2,coeff2 in p2.items():
 p3[expol+expo2]= p3[expol+expo2] + coeff1*coeff2

 return p3
```

Here we have taken an empty dictionary, p3, to store the product of two polynomials p1 and p2. The maximum terms possible in p3 is the sum of terms in p1 and terms in p2. Now assign the coefficient of each term of p3 to 0. The actual polynomial multiplication is done after that.

**Program continues...****# The main Program**

```

poly1=readPoly()
print("\nThe Polynomial-1 is ")
print_poly(poly1)

poly2=readPoly()
print("\nThe Polynomial-2 is ")
print_poly(poly2)

print("\nThe Sum of Polynomials is ")
poly3=add_poly(poly1,poly2)
print_poly(poly3)

print("\nThe Product of Polynomials is ")
poly3=mul_poly(poly1,poly2)
print_poly(poly3)

```

**Output**

Enter Degree of Polynomial :3

Enter coefficient for x<sup>0</sup>: 1

Enter coefficient for x<sup>1</sup>: 2

Enter coefficient for x<sup>2</sup>: -3

The Polynomial-1 is

1 + 2x<sup>1</sup> + -3x<sup>2</sup>

Enter Degree of Polynomial :6

Enter coefficient for x<sup>0</sup>: 0

Enter coefficient for x<sup>1</sup>: 0

Enter coefficient for x<sup>2</sup>: 0

Enter coefficient for x<sup>3</sup>: 6

Enter coefficient for x<sup>4</sup>: 4

Enter coefficient for x<sup>5</sup>: 8

The Polynomial-2 is

0 + 6x<sup>3</sup> + 4x<sup>4</sup> + 8x<sup>5</sup>

The Sum of Polynomials is

1 + 2x<sup>1</sup> + -3x<sup>2</sup> + 6x<sup>3</sup> + 4x<sup>4</sup> + 8x<sup>5</sup>

The Product of Polynomials is

0 + 6x<sup>3</sup> + 16x<sup>4</sup> + -2x<sup>5</sup> + 4x<sup>6</sup> + -24x<sup>7</sup>

The same program can be written with very few codes using the *numpy* package.

## Python program 8

```
import numpy as np

def display(p1):
 p={}
 # Convert the coefficient and exponents to a dictionary
 coeff = list(p1.c)
 coeff.reverse()
 n = len(coeff)

 for i in range(n):
 p[i]=coeff[i]

 # displaying in polynomial form
 res=""
 for expo,coeff in p.items():
 if expo == 0:
 res = str(coeff)
 else:
 if int(coeff) == 0:
 pass
 else:
 res = res + ' + ' + str(coeff)+"x^"+str(expo)

 print(res)

Constructing polynomial
p1 = np.poly1d([1, 2, 5, 3])
print("Polynomials-1")
display(p1)

p2 = np.poly1d([4, 9, 3])
print("Polynomials-2")
display(p2)

Addition of polynomials
add = np.polyadd(p1, p2)
print("Addition of two polynomials")
display(add)

Subtraction of polynomials
sub = np.polysub(p1, p2)
print("Subtraction of two polynomials")
display(sub)

Multiplication of polynomials
mul = np.polymul(p1, p2)
print("Multiplication of two polynomials")
display(mul)
```

---

```
Roots of polynomial
print("The roots of polynomial-1 are")
root=p1.r
print(root)

Solve polynomials at a value
print("The value of polynomial-1 at x=2 is ",p1(2))
print("The value of polynomial-2 at x=2 is ",p2(2))
```

We use built-in methods for adding and multiplying polynomials in the python *numpy* package. We have the `np.polyadd(p1, p2)` method for polynomial addition and the `np.polymul(p1, p2)` method for polynomial multiplication.

### Output

```
Polynomials-1
3 + 5x^1 + 2x^2 + 1x^3
Polynomials-2
3 + 9x^1 + 4x^2
Addition of two polynomials
6 + 14x^1 + 6x^2 + 1x^3
Subtraction of two polynomials
0 + -4x^1 + -2x^2 + 1x^3
Multiplication of two polynomials
9 + 42x^1 + 63x^2 + 41x^3 + 17x^4 + 4x^5
The roots of polynomial-1 are
[-0.63136114+1.9158304j -0.63136114-1.9158304j -0.73727772+0.j]
The value of polynomial-1 at x=2 is 29
The value of polynomial-2 at x=2 is 37
```

### 3.3.2 Sparse matrices

A matrix is the most frequently used data structure to solve many engineering and scientific problems. As we had studied earlier, a matrix can be represented in a row-major or a column-major method in a computer. In engineering problems such as finite element analysis or in scientific problems in the analysis of nuclear reactions, the matrices can be very large and most of the elements may be zero. Such matrices are called sparse matrices. In either row-major method or column-major method, a huge amount of space is used to represent such matrices, even though there are very few non-zero elements that really contribute to the solution of the problem. We need an alternate representation of such matrices for optimum utilization of memory and minimum use of computer time. Consider the matrix in Figure 3.17.



|   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|
| 2 | 0 | 0 | 0 | 0 | 0 | 8 | 9 | 0 | 0 | 0 |
| 3 | 5 | 0 | 0 | 0 | 0 | 3 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 1 | 9 | 7 | 0 | 0 | 0 | 0 |
| 1 | 3 | 4 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 6 |

Figure 3.17 Sparse matrix

It may be considered as a sparse matrix with 13 non-zero elements and 31 zeros. How many elements should be zero or what is the percentage of non-zero elements in the set of total elements to call a matrix sparse is not fixed. It depends on the programmer or the memory available. The sparse matrix shown in Figure 3.17 has 44 elements (integers including zeros) and hence, requires 44 memory locations in either row-major representation or a column-major representation. However, the 13 useful non-zero elements require a space of 13 memory locations only. Therefore, for the optimum utilization of space, we need an alternative representation.

Each element of a matrix is uniquely represented by its row number, its column number and its value. We might therefore store a matrix by a list of triples of the form (i, j, value). We might construct the list placing the three tuples in the increasing order of row numbers. These lists of tuples are placed one below the other in the increasing order of row numbers and with the same row number in the increasing order of column numbers is a matrix representation of the sparse matrix. This representation has  $3 \times$  (number of non-zero elements) elements.

Thus, the sparse matrix in Figure 3.17 can be stored as shown in Figure 3.18.

|       | 1 | 2  | 3 |
|-------|---|----|---|
| A[0]  | 0 | 0  | 2 |
| A[1]  | 0 | 6  | 8 |
| A[2]  | 0 | 7  | 9 |
| A[3]  | 1 | 0  | 3 |
| A[4]  | 1 | 1  | 5 |
| A[5]  | 1 | 6  | 4 |
| A[6]  | 2 | 4  | 1 |
| A[7]  | 2 | 5  | 9 |
| A[8]  | 2 | 6  | 7 |
| A[9]  | 3 | 0  | 1 |
| A[10] | 3 | 1  | 3 |
| A[11] | 3 | 2  | 4 |
| A[12] | 3 | 10 | 6 |

Figure 3.18 Sparse matrix representation in an array

This contains 13 rows and 3 columns, requiring 39 memory space, 5 less than what is required for the original matrix. For large matrices this makes a considerable difference. In the matrix in Figure 3.18, we have 13 rows and 3 columns. The number of rows is equal to the number of non-zero

elements. From the first row onward, we represent information about non-zero elements. The first row says that the first row and the first column in the sparse matrix contain the non-zero element 2. Similarly the seventh row says that the first row and third column in the sparse matrix contains 1. Some of the operations we wish to perform on these matrices are ‘transpose’, ‘addition’ and ‘multiplication’.

Let us represent the sparse matrix as a two-dimensional array:

### Python program 9

```
function read a matrix
def readMatrix():
 rows = int(input("How many rows "))
 columns = int(input("How many columns "))
 # Initialize a Zero-matrix
 matrix = [[0 for i in range(columns)]
 for j in range(rows)]
 # Reading data for the matrix
 for i in range(rows):
 for j in range(columns):
 print("Enter Matrix element",i,j, ":",end="")
 matrix[i][j] = int(input())
 return matrix
```

To create a sparse matrix we need to initialize a zero matrix with the required size. And then read the data for the sparse matrix.

### Program continues...

```
function display a matrix
def showMatrix(matrix):
 for column in matrix:
 for element in column:
 print(element, end =" ")
 print()
```

The function showMatrix(matrix) displays the elements of the matrix on the screen.

### Program continues...

```
function to convert the matrix into a sparse matrix
def ToSparseMatrix(matrix):
 sparseMatrix = []
 for i in range(len(matrix)):
 for j in range(len(matrix[0])):
 if matrix[i][j] != 0 :
 temp = []
```

```
 # appending row, column value and
 # element into the
 # sparse matrix
 temp.append(i)
 temp.append(j)
 temp.append(matrix[i][j])
 sparseMatrix.append(temp)

 return(sparseMatrix)
```

The function `ToSparseMatrix(matrix)` converts the matrix read earlier to a sparse matrix. We have taken an empty list, *sparseMatrix*, to store the sparse matrix. Whenever there is a non-zero value in the given matrix, append the row, column and the value to a *temp* list. And then append the *temp* list to the *sparseMatrix* list. So the *sparseMatrix* contains a list of lists.

The same program can be written with very few codes using *numpy* and *scipy* packages as the following program.

### Python program 10

```
import numpy as np
from scipy.sparse import csr_matrix
create a 2-D representation of the matrix
A = np.array([[0, 1, 0, 5],
 [0, 0, 2, 0],
 [0, 1, 0, 2]])

print("Dense matrix-1 :\n", A)
convert to sparse matrix representation
S1 = csr_matrix(A)
print("Sparse matrix-1: \n",S1)
```

### Output

```
Dense matrix-1 :
[[0 1 0 5]
 [0 0 2 0]
 [0 1 0 2]]
Sparse matrix-1:
(0, 1) 1
(0, 3) 5
(1, 2) 2
(2, 1) 1
(2, 3) 2
```

#### 3.3.2.1 Transpose

As we know, the transpose of a matrix is the matrix obtained by inter-changing rows and columns. In other words, the element in the *i*th row and

$j$ th column will be in the  $j$ th row and  $i$ th column in the transposed matrix. The transpose of the matrix in Figure 3.18 is shown in Figure 3.19.

|       | 1  | 2 | 3 |
|-------|----|---|---|
| A[0]  | 0  | 0 | 2 |
| A[1]  | 0  | 1 | 3 |
| A[2]  | 0  | 3 | 1 |
| A[3]  | 1  | 1 | 5 |
| A[4]  | 1  | 3 | 3 |
| A[5]  | 2  | 3 | 4 |
| A[6]  | 4  | 2 | 1 |
| A[7]  | 5  | 2 | 9 |
| A[8]  | 6  | 0 | 8 |
| A[9]  | 6  | 1 | 4 |
| A[10] | 6  | 2 | 7 |
| A[11] | 7  | 0 | 9 |
| A[12] | 10 | 3 | 6 |

Figure 3.19 Transpose of a matrix in sparse matrix

However, this does not correspond to the type of structure we had chosen (i.e., the increased row number and with the same row number increased column number) and it requires movement of rows and columns to bring it to the required form. To avoid this, we may proceed interchanging rows and columns starting from column 1 as follows:

*for all elements in column  $j$  ( $j$  ranging from 0 to number of columns)*

*place ( $i, j, value$ ) at ( $j, i, value$ )*

Since the rows are originally in order, the transposed matrix will be in the correct order (increased row number and with the same row number increased column number).

**Python program 9 continues...**

```
def transposeMatrix(matrix):
 matrix=np.array(matrix)
 matrix[:, [0, 1]] = matrix[:, [1, 0]]

 for i in range(len(matrix)):
 for j in range(len(matrix)-i-1):
 if ((matrix[j][0] > matrix[j+1]
 [0]) or (matrix[j][0] == matrix[j+1]
 [0] and matrix[j][1] > matrix[j+1][1])) :
 matrix[[j,j+1]] = matrix[[j+1,j]]

 showMatrix(matrix)
```

The above function transposes a sparse matrix. The statement `matrix[:,0, 1] = matrix[:,1, 0]` swaps the first column and second column of the sparse matrix. The two for loops arrange the rows in ascending order based on the elements of the first column. If two elements in the first column are the same, then it will put the rows in ascending order based on the values of the second column for those rows.

### 3.3.2.2 Sparse matrix addition

Let us write an algorithm to add two sparse matrices.

1. Let A and B be two matrices to be added and stored in C.
2. Let M and N be the number of rows in A and B, respectively. In other words, M – 1 and N – 1 are the number of non-zero elements in A and B, respectively.
3. The number of rows in C will be M + N – 1.
4. Two matrices can be added if and only if the number of rows and columns are the same.

`C[0][0] = A[0][0];`

`C[0][1] = A[0][1]`

`C[0][2] = M+N-2`

5. Let i, j and k be the indices into A, B and C, respectively, initialized to 1.

6. While `i <= M` and `j <= N`

if `(A[i][0] = B[j][0])` // if the row numbers are the same

if `(A[i][1] = B[j][1])` // column numbers are the same

{

`C[k,0] = A[i][0]`

`C[k,1] = A[i][1]`

`C[k][2] = A[i][2] + B[j][2]`

}

This says that “take the elements in the same row number and column number in matrices A and B, add them, and place it in the same row number and column number in C.”

7. Increment the indices: `i=i+1;j=j+1;k=k+1`

8. If `(A[i][0] = B[i][0])` and `A[j][1]<B[j][1]` copy the *i*th row of A into the *k*th row of C:

`C[k][0] = A[i][0];`

`C[k][1] = A[i][1];`

`C[k][2] = A[i][2];`

9. Increment `i=i+1;k=k+1`

10. Otherwise, i.e., `(A[i][0] = B[i][0])` and `A[j][1]>B[j][1]`)

11. `C[k][0] = B[j][0];`

`C[k][1] = B[j][1];` // copy the *j*th row of B into the *k*th row of C

`C[k][2] = B[j][2];`

```

12. Increment j=j+1;k=k+1;
}

```

In other words, add the elements if the row number and column numbers are the same; copy otherwise in the appropriate order.

### Python program 9 continues...

```

Addition of Sparse Matrix
def addSparse(m1,m2):
 resMatrix=[]
 l1=len(m1)
 l2=len(m2)
 i=0
 j=0

 while (i < len(m1) and j < len(m2)):
 temp=[]
 t1=[]
 t2=[]
 t1=m1[i]
 t2=m2[j]

 if ((t1[0] > t2[0]) or (t1[0] == t2[0])
 and (t1[1] > t2[1])):
 temp.append(t2[0])
 temp.append(t2[1])
 temp.append(t2[2])
 resMatrix.append(temp)
 j+=1
 elif((t1[0]< t2[0]) or (t1[0] == t2[0])
 and (t1[1] < t2[1])):
 temp.append(t1[0])
 temp.append(t1[1])
 temp.append(t1[2])
 resMatrix.append(temp)
 i+=1
 else:
 temp.append(t2[0])
 temp.append(t2[1])
 temp.append(t1[2] + t2[2])
 resMatrix.append(temp)
 i+=1
 j+=1
 if l1 < l2 :
 for i in range (j,l2):
 resMatrix.append(m2[i])

```

```
else:
 for i in range (i,ll):
 resMatrix.append(m1[i])

print("\nThe Addition of two Sparse Matrix is :")
showMatrix(resMatrix)
```

We have already discussed `readMatrix()`, `showMatrix(Matrix)`, `ToSparseMatrix(Matrix1)` and `transposeMatrix(matrix)`. Now we will discuss `addSparse(m1,m2)`. It takes two sparse matrices as input.

While there is some data in sparse matrix 1 and sparse matrix 2, take one row each time and perform the operations. Remember that each row of the sparse matrix contains a row, column and value of the original sparse matrix. The addition of sparse matrices has three cases.

The first case is when the row value of sparse matrix 1 is less than the row value of sparse matrix 2 or both of them are the same and the column value of sparse matrix 1 is greater than the column value of sparse matrix 2, then append the sparse matrix 2 to the resultant sparse matrix.

The second case is when the row value of sparse matrix 1 is greater than the row value of sparse matrix 2 or both of them are the same and the column value of sparse matrix 1 is less than the column value of sparse matrix 2, then append the sparse matrix 1 to the resultant sparse matrix.

Otherwise, append the row value of sparse matrix 2, column value of sparse matrix-2 and sum of the values of sparse matrix 1 and sparse matrix 2 to the resultant sparse matrix.

If the number of rows of sparse matrix 1 is exhausted, append the sparse matrix- 2 to the resultant sparse matrix. If the number of rows of sparse matrix 2 is exhausted, append the sparse matrix 1 to the resultant sparse matrix.

### Python program 9 continues...

```
#Main Program
rows = int(input("How many rows "))
columns = int(input("How many columns "))
print("\nRead Matrix-1: ")
Matrix1=readMatrix(rows,columns)
print("\nRead Matrix-2: ")
Matrix2=readMatrix(rows,columns)

#convert to Sparse Matrix
sm1= ToSparseMatrix(Matrix1)
sm2= ToSparseMatrix(Matrix2)

print("\nThe Sparse Matrix-1 : ")
showMatrix(sm1)
print("\nThe Sparse Matrix-2 : ")
showMatrix(sm2)
```

```
addSparse(sm1,sm2)

#Transpose of Sparse Matrix
print("\nThe Transpose of Sparse Matrix-1 is")
transposeMatrix(sm1)
print("\nThe Transpose of Sparse Matrix-2 is")
transposeMatrix(sm2)
```

## Output

```
How many rows 2
How many columns 3

Read Matrix-1:
Enter Matrix element 0 0 :0
Enter Matrix element 0 1 :0
Enter Matrix element 0 2 :7
Enter Matrix element 1 0 :0
Enter Matrix element 1 1 :6
Enter Matrix element 1 2 :7

Read Matrix-2:
Enter Matrix element 0 0 :0
Enter Matrix element 0 1 :0
Enter Matrix element 0 2 :1
Enter Matrix element 1 0 :2
Enter Matrix element 1 1 :0
Enter Matrix element 1 2 :0

The Sparse Matrix-1 :
0 2 7
1 1 6
1 2 7

The Sparse Matrix-2 :
0 2 1
1 0 2

The Addition of two Sparse Matrix is :
0 2 8
1 0 2
1 1 6
1 2 7

The Transpose of Sparse Matrix-1 is
1 1 6
2 0 7
2 1 7

The Transpose of Sparse Matrix-2 is
0 1 2
2 0 1
```



The same program can be written with very few codes using *numpy* and *scipy* packages as in the following program.

### Python program 11

```
import numpy as np
from scipy.sparse import csr_matrix

create a 2-D representation of the matrix
A = np.array([[0, 1, 0, 5],
 [0, 0, 2, 0],
 [0, 1, 0, 2]])

B = np.array([[0, 5, 0, 5],
 [0, 1, 2, 0],
 [0, 1, 0, 0]])

print("Dense matrix-1 :\n", A)
print("Dense matrix-2 :\n", B)

convert to sparse matrix representation
S1 = csr_matrix(A)
S2 = csr_matrix(B)
S = S1 + S2
print("Sparse matrix-1: \n",S1)
print("Sparse matrix-2: \n",S2)
print("Addition of two Sparse matrix: \n",S)
```

### Output

```
Dense matrix-1 :
[[0 1 0 5]
 [0 0 2 0]
 [0 1 0 2]]
Dense matrix-2 :
[[0 5 0 5]
 [0 1 2 0]
 [0 1 0 0]]
Sparse matrix-1:
(0, 1) 1
(0, 3) 5
(1, 2) 2
(2, 1) 1
(2, 3) 2
Sparse matrix-2:
(0, 1) 1
(0, 3) 5
(1, 2) 2
(2, 1) 1
(2, 3) 2
```

Addition of two Sparse matrix:

```
(0, 1) 2
(0, 3) 10
(1, 2) 4
(2, 1) 2
(2, 3) 4
```

### Example: Matrix multiplication

```
m = int(input("How many rows in Matrix1? "))
n = int(input("How many columns in Matrix1? "))
Mat1 = [[0 for x in range(n)] for x in range (m)]
for i in range(m):
 for j in range(n):
 Mat1[i][j] = int(input("Enter the elements of
Matrix 1: "))

m1 = int(input("How many rows in Matrix2? "))
n1 = int(input("How many columns in Matrix2? "))

if (n != m1):
 print("Matrix multiplication NOT possible!!!")
 exit()

Mat2 = [[0 for x in range(n1)] for x in range (m1)]
for i in range(m1):
 for j in range(n1):
 Mat2[i][j] = int(input("Enter the elements of
Matrix 2: "))

print("\nMatrix 1 is ")
for i in range(m):
 for j in range(n):
 print(Mat1[i][j], end = " ")
 print()

print("\nMatrix 2 is ")
for i in range(m1):
 for j in range(n1):
 print(Mat2[i][j], end = " ")
 print()

Mat3 = [[0 for x in range(m)] for x in range (n1)]
for i in range(m):
 for j in range(n1):
 for k in range(m1):
 Mat3[i][j] += Mat1[i][k] * Mat2[k][j]

print("\nMatrix multiplication is ")
for i in range(m):
 for j in range(n1):
 print(Mat3[i][j], end = " ")
 print()
```

## Output

```
How many rows in Matrix1? 2
How many columns in Matrix1? 3
Enter the elements of Matrix 1: 1
Enter the elements of Matrix 1: 2
Enter the elements of Matrix 1: 2
Enter the elements of Matrix 1: 3
Enter the elements of Matrix 1: 0
Enter the elements of Matrix 1: 4
How many rows in Matrix2? 3
How many columns in Matrix2? 2
Enter the elements of Matrix 2: 5
Enter the elements of Matrix 2: 0
Enter the elements of Matrix 2: 8
Enter the elements of Matrix 2: 1
Enter the elements of Matrix 2: 2
Enter the elements of Matrix 2: 3
```

Matrix 1 is

```
1 2 2
3 0 4
```

Matrix 2 is

```
5 0
8 1
2 3
```

Matrix multiplication is

```
25 8
23 12
```

## Example: Row sum of a matrix

```
Mat = [[501, 453, 664],
 [502, 444, 646],
 [503, 222, 999]]
for i in range(len(Mat)):
 sum = 0
 for j in range(len(Mat[i])):
 sum = sum + Mat[i][j]
 print("The sum of ", Mat[i], "is : ", sum)
```

## Output

The sum of [501, 453, 664] is : 1618

The sum of [502, 444, 646] is : 1592

The sum of [503, 222, 999] is : 1724

**True–false questions**

1. While using a list of integers, the memory location is consecutive.  
a. True                                      b. False
2. Numpy arrays use less memory.  
a. True                                      b. False
3. In a sparse matrix most of the elements are non-zero.  
a. True                                      b. False
4. Insertions and deletions into or from an array are costly operations  
a. True                                      b. False
5. All items in the Python Array module are not necessarily the same size.  
a. True                                      b. False

**Fill-in-the-blank questions**

1. An array is an ordered collection of \_\_\_\_\_ elements.
2. Numpy array can also be used to construct an \_\_\_\_\_ array in Python ().
3. Python lists are \_\_\_\_\_ by default.
4. A list is a mutable sequential and \_\_\_\_\_ data structure.
5. A list's elements don't have to be \_\_\_\_\_ in memory.

**Multiple-choice questions**

1. Find the output of the following code:

```
lst = [1,2,3,4,5]
print(lst[-1])
```

- a. 2
- b. 3
- c. 4
- d. 5

2. Find the output of the following code:

```
lst = [1,2,3,4,5]
print(list(lst[-3:-1]))
```

- a. [2, 3]
- b. [3, 4]
- c. [4, 5]
- d. [3, 5]

3. Find the output of the following code:

```
lst = [0,1,2,3,4,5]
lst.insert(0,1)
del lst[1]
print(lst)
```

- a. [0, 1, 2, 3, 4, 5]
- b. [1, 1, 2, 3, 4, 5]
- c. [2, 1, 2, 3, 4, 5]
- d. [3, 1, 2, 3, 4, 5]

4. Find the output of the following code:

```
lst = [0,1,2,3,4,5]
lst1 = lst
del lst1[1:2]
print(lst1)
```

- a. [0, 1, 3, 4, 5]
- b. [1, 2, 3, 4, 5]
- c. [0, 2, 3, 4, 5]
- d. [0,1, 3, 4, 5]

5. Find the output of the following code:

```
lst = [0,1,2,3,4,5]
lst1 = lst
del lst1[-1:-2]
print(lst1)
```

- a. lst and lst1 have the same length.
- b. It raises an error
- c. lst1 is a blank list.
- d. lst1 is longer than lst.

6. Find the output of the following code:

```
lst = [0,1,2,3,4,5]
lst1 = []
for i in lst:
 lst1.insert(0,i)
print(lst1)
```

- a. [0,1,2,3,4,5]
- b. [5,4,3,2,1,0]
- c. [0,0,0,0,0]
- d. None of the above

7. Find the output of the following code:

```
lst = [0,1,2,3,4,5]
for i in range(len(lst)):
 lst.insert(-1,lst[i])
print(lst)
```

- a. [1, 2, 3, 4, 5, 0, 0, 0, 0, 0, 0, 0]
- b. [1, 0, 0, 0, 0, 0, 0, 1, 2, 3, 4, 5]
- c. [0, 0, 0, 0, 0, 0, 0, 1, 2, 3, 4, 5]
- d. [0, 1, 2, 3, 4, 0, 1, 2, 3, 4, 0, 5]

8. Find the output of the following code:

```
lst = [[0,1,2,3,4,5] for i in range(3)]
print(lst[2][1])
```

- a. 0
- b. 1
- c. 2
- d. 3

9. Find the output of the following code:

```
my_list = [1,2,3,4,5,6]
count = 0
for item in my_list:
 if type(item) == str:
 continue
 count = count + 1
print(count)
```

- a. 2
- b. 4
- c. 6
- d. 0

10. Find the output of the following code:

```
m = 0
my_list_1= [1,2,5]
my_list_2= [1,3,2,6,5]
for x in my_list_1:
 for y in my_list_2:
 if x == y:
 m = m+1
print(m)
```

a. 3  
b. 4  
c. 6  
d. 0

11. Find the output of the following code:

```
n1 = [10,20,30,40,50]
n2 = [10,20,30,40,50]
print(n1 is n2)
print(n1 == n2)
n1 = n2
print(n1 is n2)
print(n1 == n2)
```

a. True,True,True,False  
b. True,False,True,True  
c. True,True,False,True  
d. False,True,True,True

12. Find the output of the following code:

```
prices = [30.5,'40.5',10.5]
total = 0
for price in prices:
 total += price
print(total)
```

- a. 81.5
- b. 30.1, 71.0, 81.5
- c. Expected indentation
- d. Raise an error ValueError

13. Find the output of the following code:

```
str1 = "I am learing Python"
str2 = str1.split('a')
print(str2)
```

- a. ['I ', 'm le', 'rning Python']
- b. ['Ia ', 'm lea', 'rning Python']
- c. ['I ', 'am le', 'arning Python']
- d. ['I ', 'am', 'learning', 'Python']

14. Find the output of the following code:

```
numbers = [0,1,2,3,4,5,6,7,8,9]
index = 0
while index < 10:
 print(numbers[index])
 if numbers(index) == 6:
 break
 else:
 index += 1
print(index)
```

- a. 0,1,2,3,4,5
- b. 1,2,3,4,5,6
- c. 0,1,2,3,4,5,6
- d. Raise an error

15. Find the output of the following code:

```
fruits = ['apple','orange','mango','banana']
for i in range(len(fruits)):
 fruits[i] = fruits[i][-1].upper()
print(fruits)
```



- a. ['B', 'A', 'N', 'A', 'N', 'A']
- b. ['A', 'O', 'M', 'B']
- c. ['E', 'E', 'O', 'A']
- d. ['L', 'G', 'G', 'N']

16. Find the output of the following code:

```
i = [10,20,[30,40],[50,60]]
count = 0
for i in range(len(l)):
 if type(l[i]) == list:
 count = count + 1
print(count)
```

- a. 1
- b. 2
- c. 3
- d. 4

17. Find the output of the following code:

```
x = [13,4,17,10]
w = x[1:]
u = x[1:]
y = x
u[0] = 50
y[1] = 40
print(x)
```

- a. [13, 40, 17, 10]
- b. [50, 40, 10]
- c. [13,4,17,10]
- d. [50,40,17,10]

18. Find the output of the following code:

```
a = ['a','b','c','d']
for i in a:
 a.append(i.upper())
print(a)
```

- a. ['a','b','c','d']
- b. ['A','B','C','D']
- c. Raise an error
- d. None of the above

19. Find the output of the following code:

```
numbers = [0,1,2,3,4,5,6,7,8,9]
```

```
i = '0'
```

```
index = 0
```

```
while(index < 10):
```

```
 if numbers[index] == 4:
```

```
 break
```

```
 else:
```

```
 index += 1
```

```
 i = i + str(index)
```

```
print(i)
```

- a. 1234
- b. 0123
- c. 12345
- d. 01234

20. Find the output of the following code:

```
my_list = [1,2,3,4,5,6]
```

```
count = 0
```

```
for item in my_list:
```

```
 if type(item != str):
```

```
 continue
```

```
 count += 1
```

```
print(count)
```

- a. 3
- b. 2
- c. 1
- d. 0

**Descriptive questions**

1. Consider the following matrix:

```
0 0 0 7 0 0 0 0
1 0 0 0 8 0 0 0
2 0 0 0 0 0 0 9
0 0 0 0 0 0 5 0
0 4 0 0 0 0 0 0
```

Represent this as a sparse matrix.

2. Create a list with the following elements and write a program to print it in the reverse order (without using the built-in function for reversing):

3 5 7 9 11 13 15

3. Write a program to delete the duplicate elements from a list.
4. How do you represent a polynomial in a computer? Write a program to add the following polynomials:
- $$4x^4 + 2x^3 + 3x + 5 \qquad 5x^5 + 3x^2 - 8$$
5. Write a program to remove alternate elements from a list and create a new list.
6. An array  $A = \{3, 5, 7, 9\}$  is given and it is full. Write a program to insert a new element 11 to a list at a location without using the insert() function.
7. Given a list of integers, write a program to find the largest and smallest elements in the list without using max() and min().
8. Write a program to find a sub-list of a list from a given list.

**Answers to true–false questions**

- 1. True
- 2. True
- 3. False
- 4. False
- 5. False

**Answers to fill-in-the-blank questions**

- 1. homogeneous
- 2. N-dimensional
- 3. one-dimensional
- 4. heterogeneous
- 5. in order

**Answers to multiple-choice questions**

- |     |   |     |   |     |   |     |   |     |   |
|-----|---|-----|---|-----|---|-----|---|-----|---|
| 1.  | d | 2.  | b | 3.  | b | 4.  | c | 5.  | c |
| 6.  | b | 7.  | d | 8.  | b | 9.  | c | 10. | a |
| 11. | d | 12. | c | 13. | a | 14. | d | 15. | c |
| 16. | b | 17. | a | 18. | c | 19. | d | 20. | d |

# Linked list

---

### LEARNING OBJECTIVES

After studying this chapter, readers will be able to:

- Know about linked list
- Implement singly linked list, doubly linked list and circular linked list
- Know different applications of linked list

A linked list is a linear collection of data objects called nodes, each pointing to the next node. A data element and an address to the next node may be present in each node. This data structure makes it simple to add and remove data objects. Other data structures such as stacks and queues can be implemented using the linked list data structure.

The most essential advantage of a linked list over an array is that the memory elements used to hold the data components do not have to be contiguous, hence the size of the linked list is not restricted to the contiguous memory locations available. Linked lists also allow for the insertion and deletion of nodes at any point in the list with a fixed number of operations.

Linked lists make it difficult to access data. As a result, identifying a node with a certain data element or the last node in a linked list may necessitate sequential scanning of the majority or all of the elements.

### 4.1 SINGLY LINKED LISTS

Singly linked lists contain nodes which have two fields: a 'data' field and a 'next' field. The 'next' field contains the address of the next node. Sometimes the address is also called a pointer to the next node (Figure 4.1).



*Figure 4.1* Node structure

We describe the node in the linked list as a class in Python with two attributes:

### Python program 1

```
class Node: # Definition of a Node
 def __init__(self, key):
 self.data = key
 self.next = None
```

This class definition creates an object of type Node. It has two attributes: data and next. Initially, the next is assigned to None and data is assigned to the key, which is provided while creating the object.

### 4.1.1 Singly linked lists: Creation

We first create the start node and keep on linking the new nodes. There is no limit to the number of nodes in the linked lists. The following Python program explains better the process of creation of the singly linked lists.

#### Program continues...

```
class SLinkedList:
Class definition of a Single Linked List
 def __init__(self):
 self.start = Node(None)
```

The class SLinkedList defines a single linked list with an attribute *start*, which is an object of type node. The *start* is a pointer that points to the starting node of the linked list.

#### Program continues...

```
Main Program
if __name__ == '__main__':
 list1 = SLinkedList()
 Ans = 'y'
 while (Ans == 'Y' or Ans == 'y'):
 x = input("Enter data to create the node :")
 NewNode = Node(x)
 if (list1.start == None):
 list1.start = NewNode
 else:
 temp = list1.start
 while(temp.next != None):
 temp = temp.next
 temp.next=NewNode
 Ans = input("Do you want to add More (Y | y) :")
```

```

print("The Elements in the Linked List are : ", end = "")
printval = list1.start
while printval is not None:
 print (printval.data ,end= " ")
 printval = printval.next
print("\n")

```

The main program creates an object of `SLinkedList()`, `list1`. When there is no element in the linked list, `start` points to *None*. When a new Node is created, the `start` points to the newly created node `list1.start = NewNode`. Otherwise, we take a variable *temp*, which initially points to the start node. If *temp* next is not *None*, move it to the next node. When such a node is found, assign the *NewNode* to the *temp.next*. Then print the elements in the linked list. This process continues as long as the value of *Ans* is “Y” or “y”.

A variable *printval* is assigned to the start node. As long as *printval* is not *None*, it will print the data part in it and then it is moved to the next node. Moving to next node is done by `printval = printval.next`.

A sample linked list

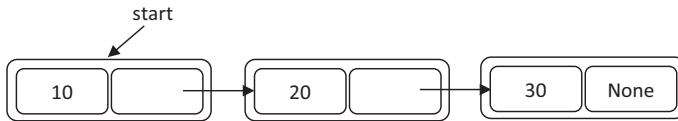


Figure 4.2 Linked List

## Output

```

Enter data to create the node :10
The Elements in the Linked List are : None 10

Do you want to add More (Y | y) :y
Enter data to create the node :20
The Elements in the Linked List are : None 10 20

Do you want to add More (Y | y) :y
Enter data to create the node :30
The Elements in the Linked List are : None 10 20 30

Do you want to add More (Y | y) :n

```

### 4.1.2 Operations in a singly linked list

Operations that need to be performed on the linked lists are traversal, insertion and deletion.

### i. Traversal

Traversal refers to visiting each node of a singly linked list. Before visiting each node in a linked list we must define the class of a node and class of a singly linked list.

#### Python program 2

```
class Node: # Definition of a Node
 def __init__(self, key=None):
 self.data = key
 self.next = None

class SLinkedList: # Class definition of a Single Linked List
 def __init__(self):
 self.start = Node(None)
```

While printing the elements of a singly linked list, each node of the linked list must be visited. To do that we need to take a pointer *temp*, which is assigned to *start*. While *temp* is not *None*, print the data part in *temp* and move the pointer to the next node in the singly linked list.

#### Program continues...

```
def listprint(self): # Printing the elements of the list
 print("The Elements in the Linked List are : ", end = "")
 temp = self.start
 while temp is not None:
 print (temp.data ,end= " ")
 temp = temp.next
```

### ii. Insertion

There are four positions where we can insert a node: in the front, at the end, after a given node and before a given node.

We have already created a linked list in the previous program with data elements 10, 20, 30.

#### 1. Insertion at the front

Let us insert a node with some data element in the front. While inserting an element as a front node, there are two cases.

Case 1: When there is no element in the singly linked list, start points to *None*. After creating a new node, the start points to the new node.

Case 2: When there is a singly linked list present, the new node's next points to the start node of the already existing singly linked list. And then the existing start node moves to the new node (Figure 4.3).

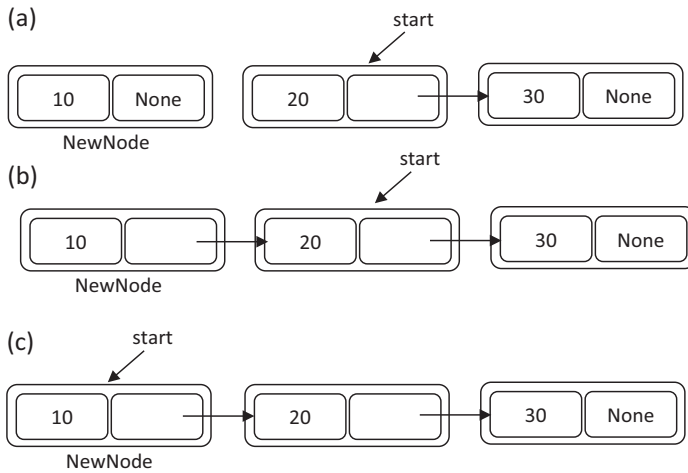


Figure 4.3 (a) Insertion at front. (b) Insertion at front. (c) Insertion at front

The Python program that does this is as follows:

**Program continues...**

```
def insert_At_Begining(self, newdata):
 NewNode = Node(newdata)
 if (self.start == None):
 self.start = NewNode
 else:
 NewNode.next = self.start
 self.start = NewNode
```

## 2. Insertion at the end

While inserting an element as a last node, there are two cases.

Case 1: When there is no element in the singly linked list, start points to *None*. After creating a new node, the start points to the new node.

Case 2: When there is a singly linked list present, assign a new pointer temp to start, move temp to the last node. Assign the next of temp to the new node (Figure 4.4).



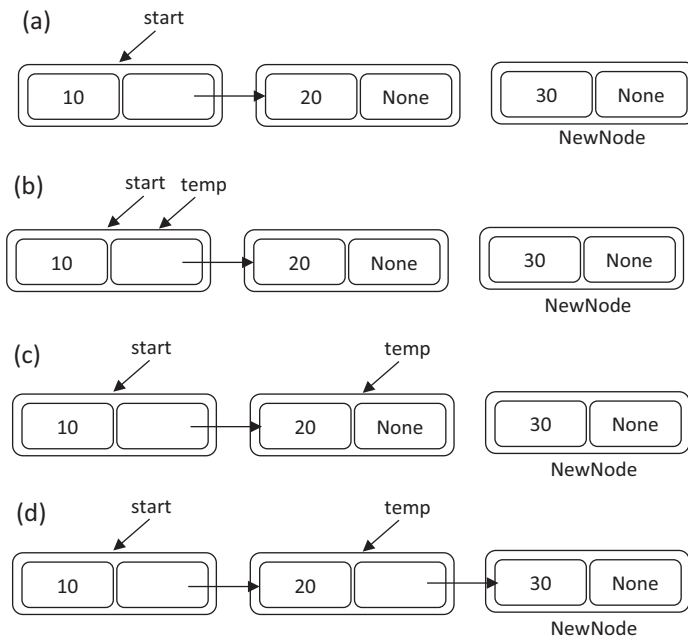


Figure 4.4 (a) Insertion at end. (b) Insertion at end. (c) Insertion at end. (d) Insertion at end

The function to do this can be expressed in the Python program as follows:

**Program continues...**

```
def insert_At_End(self, newdata):
 NewNode = Node(newdata)
 if (self.start == None):
 self.start = NewNode
 else:
 temp = self.start
 while(temp.next):
 temp = temp.next
 temp.next = NewNode
```

### 3. Insertion after a given node

Insertion in the middle is slightly more complicated since the link between two nodes is to be “broken” and reattached to the new node appropriately.

While inserting a node after a node with a given key, first, take a pointer *temp* and move the pointer to the location after which a new node to be inserted (Figure 4.5). There are two cases.

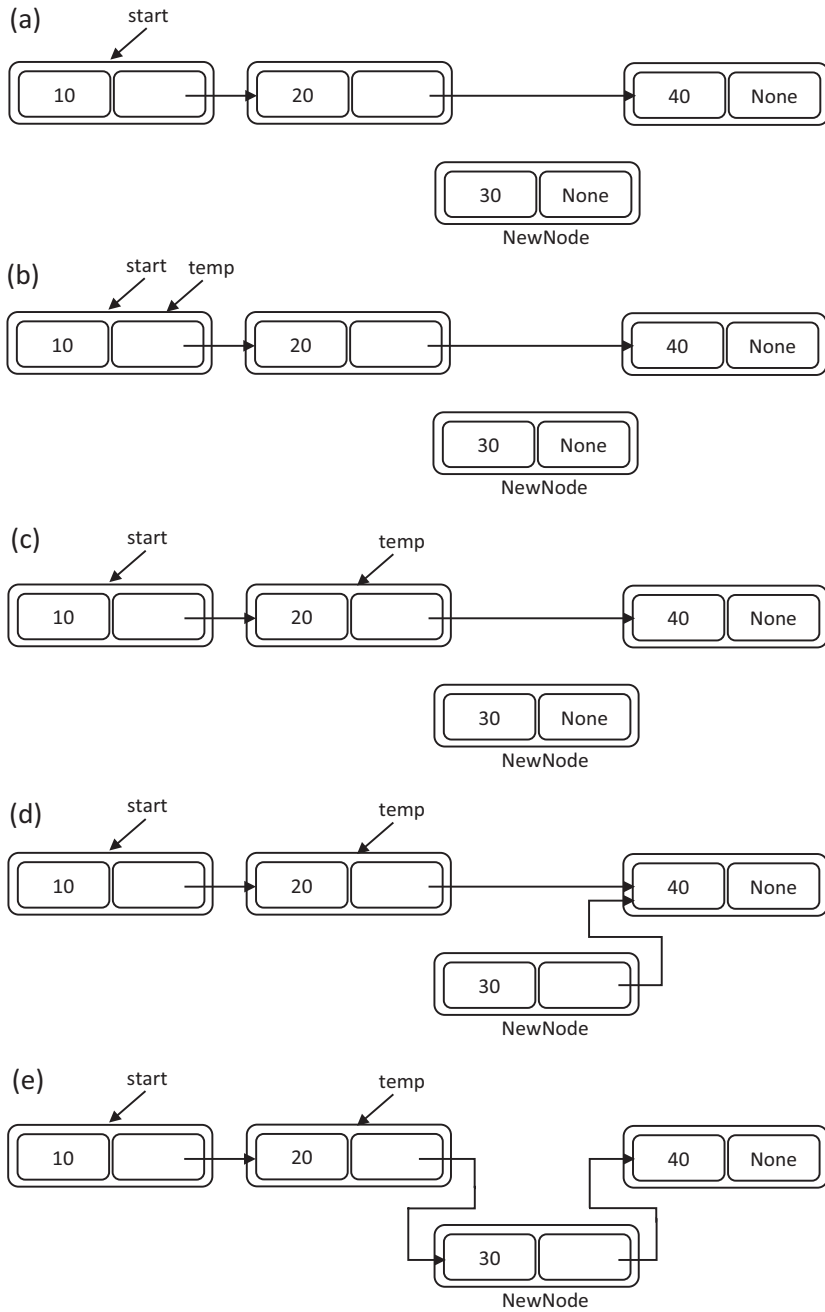


Figure 4.5 (a) New node created. (b) Assign temp to start node. (c) temp moved to the required node after which new node to be inserted. (d) Assign new node's next to temp's next node. (e) Assign temp's next to NewNode

Case 1: When the node with the given key is not found in the linked list, then print the appropriate message.

Case 2: When the node with the given key is found in the linked list, then make the new node's next point to the node to which temp's next is pointing and then make the temp's next point to the new node.

Let us write a method in Python which does this.

**Program continues...**

```
def insert_After(self, key, newdata):
 temp = self.start
 while(temp.data != key):
 temp = temp.next
 if (temp.data != data and temp.next == None):
 print(data, " Not Found")
 else:
 NewNode = Node(newdata)
 NewNode.next = temp.next
 temp.next = NewNode
```

#### 4. Insertion before a given node

While inserting a node before a node with a given key, assign *temp* to the *start*. If the data in the first node is the same as the key (before which new node to be inserted), use the *insert\_At\_Begining()* method.

If the node to be inserted is not the first node then take a pointer *temp*, which points to the first node while checking the data in the second node (that is temp's next data). If it matches with the key, insert the new node as a second node, otherwise move the *temp* pointer to the next one. There are two cases.

Case 1: If the key is not found in the linked list, then print the appropriate message.

Case 2: When the node with the given key is found in the linked list, then make the new node's next point to the node which temp's next is pointing to and then make the temp's next point to the new node.

The diagram is same as the previous one, except for the pointer.

Program continues...

```
def insert_Before(self, data,newdata):
 temp = self.start
 if (temp.data == data):
 self.insert_At_Begining(newdata)
 else:
 while(temp.next.data != data and temp.next !=
 None):
 temp = temp.next
 if (temp.next == None):
 print(data, " Not Found")
 else:
 NewNode = Node(newdata)
 NewNode.next = temp.next
 temp.next = NewNode
```

### iii. Deletion

Deletions from a linked list are comparatively easier than insertions. There are five types of deletions: deletion of the node in the front, deletion of the node at the end, deletion of a given node, deletion of the node after a given node and deletion of the node before a given node.

#### 1. Deletion of the node at the front

Deleting a node that is the first node of a linked list is quite simple. Take a pointer *temp* and assign *temp* to the *start*. Move the start to its next node. Now delete the node pointed by *temp* (Figure 4.6).

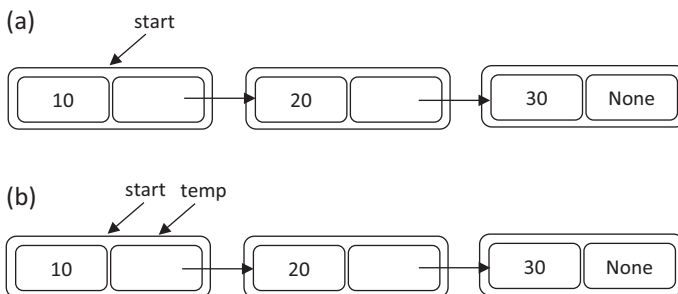


Figure 4.6 (a) Deletion of the node at the front. (b) Assign temp to start. (c) Move start to temp's next. (d) delete the node pointed by temp

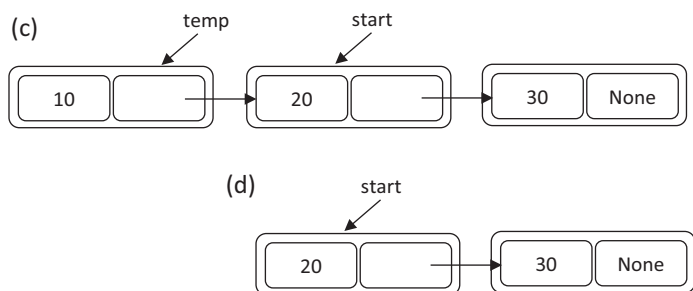


Figure 4.6 Continued

**Program continues...**

```
def del_first(self):
 temp = self.start
 self.start = temp.next
 del temp
```

**2. Deletion of the node at the end**

Deleting a node that is the last node of a linked list is quite simple. Take a pointer *temp* and assign *temp* to the *start* (Figure 4.7). There are two cases of deleting the last node.

Case 1: If the last node is the same as the first node. In other words, there is only one node present in the singlylinked list. Call the *del\_first()* method.

Case 2: If there is more than one element in the linked list, move the *temp* pointer to the last but one node using a while loop. Assign one more pointer *temp1* to *temp*'s next. Assign *temp*'s next to *None* and delete *temp1*.

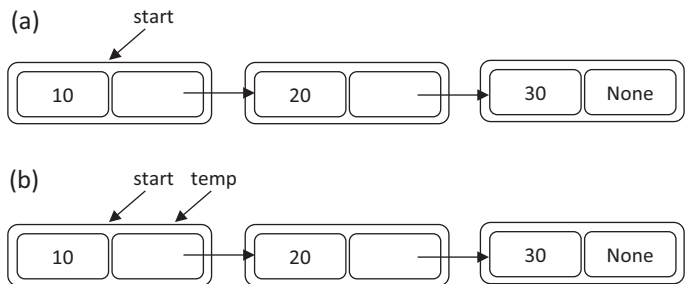


Figure 4.7 (a) Deletion of the node at the end. (b) Assign pointer *temp* to *start*. (c) move *temp* to last but one node. (d) Assign one more pointer *temp1* to *temp*'s next. (e) Assign *none* to *temp*'s next (f) Delete *temp1*

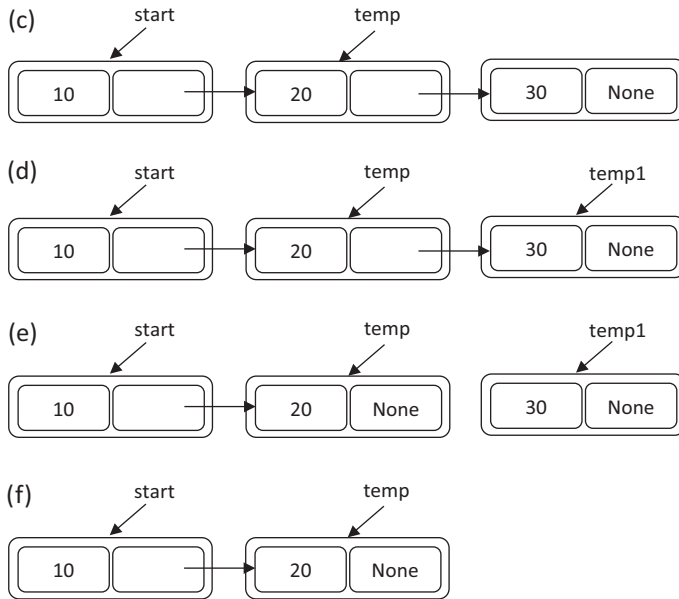


Figure 4.7 Continued

**Program continues...**

```
def del_last(self):
 temp = self.start
 if temp.next == None :
 self.del_first()
 return
 while(temp.next.next != None):
 temp = temp.next
 temp1 = temp.next
 temp.next = None
 del temp1
```

### 3. Deletion of the node

Deleting a node from the middle is a little more complicated since we have to first traverse the list up to the node we wish to delete. Let the node to be deleted contain the key. Depending on the presence of the key there are three cases.

Case 1: If the key is present in the first node, call the *del\_first()* method.

Case 2: If the key is not the first node, it can be anywhere in the linked list. To traverse the linked list, take two pointers *temp* and *temp1*. Assign *temp* to the *start* pointer and *temp1* to next of *temp*. Keep moving both the pointers *temp* and *temp1*, till *temp1*'s data matches with the *key*, then assign *temp*'s next to *temp1*'s next and delete *temp1* (Figure 4.8).

Case 3: If the key does not match with data of any node, display appropriate messages.

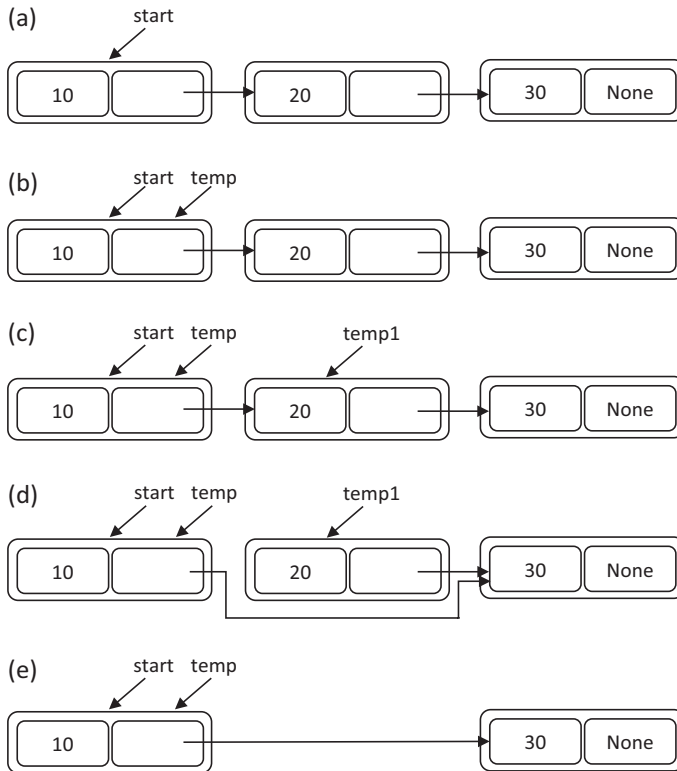


Figure 4.8 (a) Deletion of the node. (b) Assign pointer *temp* to *start*. (c) move *temp* to node previous to the node to be deleted. (d) Assign one more pointer *temp1* to *temp*'s next. (e) Assign *temp*'s next to *temp1*'s next (f) Delete *temp1*

The following code deletes the node containing the key.

**Program continues...**

```
def del_node(self, key):
 temp = self.start
 if (temp.data == key):
 self.del_first()
 return
```

```

temp1 = temp.next
while(temp1.data != key):
 temp = temp.next
 temp1 = temp1.next
if (temp1.data != key and temp1.next == None):
 print(data, " Not Found")
else:
 temp.next = temp1.next
 del temp1

```

#### 4. Deletion of the node after a given node

Depending on the presence of the key, there are three cases.

Case 1: If there is only node and the key matches with the data present in the node. As there is no node after the first node, display appropriate messages.

Case 2: If the key is not the first node, it can be anywhere in the linked list. To traverse the linked list, take a pointer *temp* and assign it to the *start* pointer. Keep moving the pointer until the temp's data matches with the *key*. Take one more pointer *temp1* and assign it to temp's next. Now assign temp's next to temp1's next and delete *temp1*.

Case 3: If the key does not match with data of any node, display appropriate messages.

The diagram is the same as the previous one.

**Program continues...**

```

def del_after(self, data):
 temp = self.start
 if (temp.data == data and temp.next == None):
 print("There is only one element in the list...")
 print("Next element can not be deleted..")
 return
 while(temp.data != data and temp.next !=None):
 temp = temp.next
 if (temp.data != data and temp.next == None):
 print(data, " data Not found")
 elif (temp.data == data and temp.next == None):
 print(data, "is the Last node. Next, node cannot be deleted")
 else:
 temp1 = temp.next
 temp.next = temp1.next
 del temp1

```



### 5. Deletion of the node before a given node

Depending on the presence of the key, there are four cases.

Case 1: If the key matches with the data present in the first node. As there is no node before the first node, before the node cannot be deleted, display appropriate messages.

Case 2: If the key is in the second node, call the *del\_first()* method.

Case 3: If the key is not the second node, it can be anywhere in the linked list. To traverse the linked list, take two pointers *temp* and *temp1*. Assign *temp* to the *start* pointer and *temp1* to next-next of *temp*. Keep moving both the pointers *temp* and *temp1*, until *temp1*'s data matches with the *key*. Take one more pointer *temp2* and assign *temp*'s next. For deleting, assign *temp*'s next to *temp2*'s next and delete *temp2* (Figure 4.9).

Case 4: If the key does not match with data of any node, display appropriate messages.

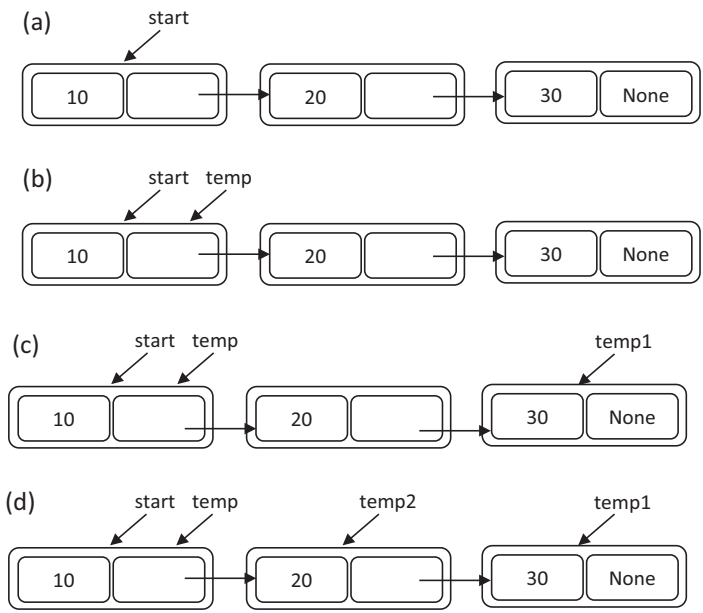


Figure 4.9 (a) Deletion of the node. (b) Assign temp to start. (c) Assign temp1 to last node. (d) Assign temp2 to last but 1 node (f) connect next of temp to next of temp2 (e) Deletion temp2

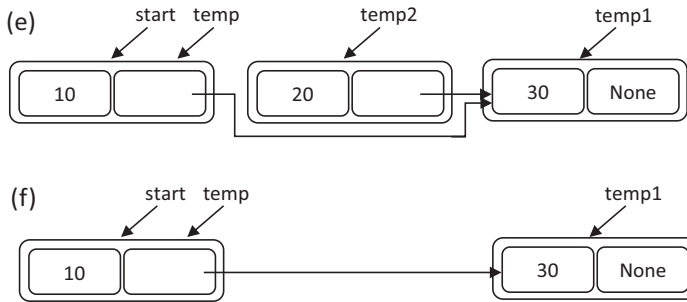


Figure 4.9 Continued

**Program continues...**

```
def del_before(self, data):
 temp = self.start
 if (temp.data == data):
 print("There is only one element in the list...")
 print("Previous element cannot be deleted..")
 return
 if(temp.next.data == data):
 self.del_first()
 return
 temp1 = temp.next.next
 while(temp1.data != data and temp1.next !=None):
 temp1 = temp1.next
 temp = temp.next

 if (temp1.data != data and temp1.next == None):
 print(data, " data Not found")
 else:
 temp2 = temp.next
 temp.next = temp2.next
 del temp2
```

**Program continues...**

```
def menu():
 print("\n~~~ MENU ~~~ ")
 print("1. Create a Single Linked List ")
 print("2. Insert at the Beginneing ")
 print("3. Insert at the End")
 print("4. Insert After a given Data ")
 print("5. Insert before a given Data")
```

```
print("6. Delete the First node")
print("7. Delete the Last node")
print("8. Delete the a given node")
print("9. Delete After a given Data")
print("10. Delete before a given Data")
print("11. Exit ")
opt = int(input("Enter a valid menu item ... "))
return opt

Main Program
if __name__ == '__main__':
 list1 = SLinkedList()
 i=1
 while (i > 0 and i <=10):
 i = menu()
 if i == 1:
 x = input("Enter data to create the first node :")
 list1.start = Node(x)
 list1.listprint()

 elif (i==2):
 x = input("Enter data to create the node :")
 list1.insert_At_Begining(x)
 list1.listprint()

 elif (i==3):
 x = input("\n Enter data to create the node :")
 list1.insert_At_End(x)
 list1.listprint()

 elif (i==4):
 x = input("\n Enter data to create the node :")
 y = input("Node to be created after data :")
 list1.insert_After(y,x)
 list1.listprint()

 elif (i==5):
 x = input("\n Enter data to create the node :")
 y = input("Node to be created before data :")
 list1.insert_Before(y,x)
 list1.listprint()

 elif (i==6):
 list1.del_first()
 list1.listprint()

 elif (i==7):
 list1.del_last()
 list1.listprint()
```

```

elif (i==8):
 x = input("Node to be deleted :")
 list1.del_node(x)
 list1.listprint()

elif (i==9):
 x = input("Node to be deleted after data :")
 list1.del_after(x)
 list1.listprint()

elif (i==10):
 x = input("Node to be deleted before data :")
 list1.del_before(x)
 list1.listprint()

else:
 print("End")

```

## Output

```

~~~ MENU ~~~
1. Create a Single Linked List
2. Insert at the Beginning
3. Insert at the End
4. Insert After a given Data
5. Insert before a given Data
6. Delete the First node
7. Delete the Last node
8. Delete the a given node
9. Delete After a given Data
10. Delete before a given Data
11. Exit
Enter a valid menu item ... 1
Enter data to create the first node :10
The Elements in the Linked List are : 10

```

In some implementations, an extra dummy node, usually called a *sentinel* node is added before the first data record and after the last data record. Every list, therefore, has two sentinel nodes, the first one and the last one, even when the list is empty. Since the reference to the first node gives access to the whole list, it is often called the “address” or “pointer”, or handle to the list.

## 4.2 DOUBLY LINKED LISTS

In doubly linked lists each node, besides containing a pointer to the next node, also contains a pointer to the previous node (Figure 4.10).



Figure 4.10 Node in a double linked list

A doubly linked list contains an extra pointer, usually called prev (previous), together with the next pointer and data (Figure 4.11).

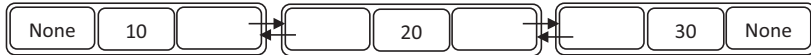


Figure 4.11 Structure of a doubly linked list

A node in a doubly linked list can be represented as:

### Python program 3

```
class Node: # Class Definition of a Node in Double Linked List
    def __init__(self, dataval=None):
        self.prev = None
        self.data = dataval
        self.next = None

class DLinkedList: # Class Definition of a Double Linked List
    def __init__(self):
        self.start = Node(None)

    def listprint(self): #Printing the elements of a DLL List
        print("The Elements in the Linked List are : ", end = "")
        printval = self.start
        while printval is not None:
            print (printval.data ,end= " ")
            printval = printval.next
        print("\n")
```

## 4.2.1 Operations in a doubly linked list

Operations that need to be performed on doubly linked lists are traversal, inserting and deleting.

### i. Traversal

Traversal refers to visiting each node in a data structure. Readers are advised to refer the traversal in singly linked list.

ii. Insertion

A node can be inserted in a doubly linked list in four different ways: at the front, at the end, after a given node and before a given node.

1. Insertion of a node at the front

When a node is added in the front, the newly added node becomes the *start* node.

Let us insert a node with some data element in the front. While inserting an element as a front node, there are two cases:

Case 1: When there is no element in the doubly linked list, the *start* points to *None*. After creating a new node with *prev* and *next* set to *None*, the *start* points to the new node.

Case 2: When there is a doubly linked list, follow these three steps to perform the operation (Figure 4.12).

- i. Set the *next* pointer of the new node to start.
- ii. Make the *prev* of the *start* node point to the new node
- iii. Move the *start* to the new node.

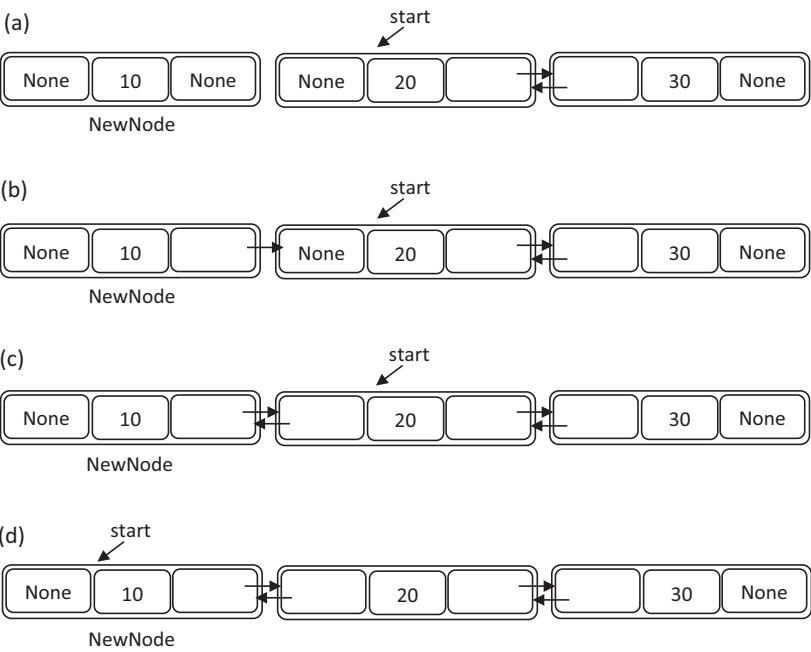


Figure 4.12 (a) create a new node (b) Assign next of new node to start (c) Assign previous of start to newnode (d) Assign start to new node

These steps can be converted to a Python program as follows:

**Program continues...**

```
def insert_At_Begining(self,newdata):
    NewNode = Node(newdata)
    if (self.start==None):
        self.start = NewNode
    else:
        NewNode.next = self.start
        self.start.prev = NewNode
        self.start = NewNode
```

**2. Insert a node at the end**

Insertion of a node as the last node in a double linked list is almost the same as insertion of a node as the last node in a single list. It also has two cases:

Case 1: When there is no element in the single linked list, start points to *None*. Call the `insert_At_Begining ()` method.

Case 2: When there is a doubly linked list present, assign a pointer *temp* to the *start*. Move *temp* to the node where next of the node is *None* (i.e., the last node). Assign *temp*'s *next* to New Node and then assign new node's *prev* to *temp* (Figure 4.13).

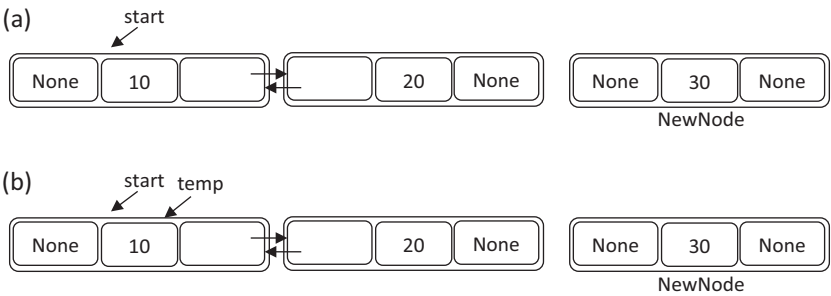


Figure 4.13 (a) Create a NewNode (b) Assign temp to start (c) Move temp till last node. (d) Assign next of temp to NewNode (e) Assign previous of NewNode to temp (e) Insertion of a node at the end in a doubly linked list

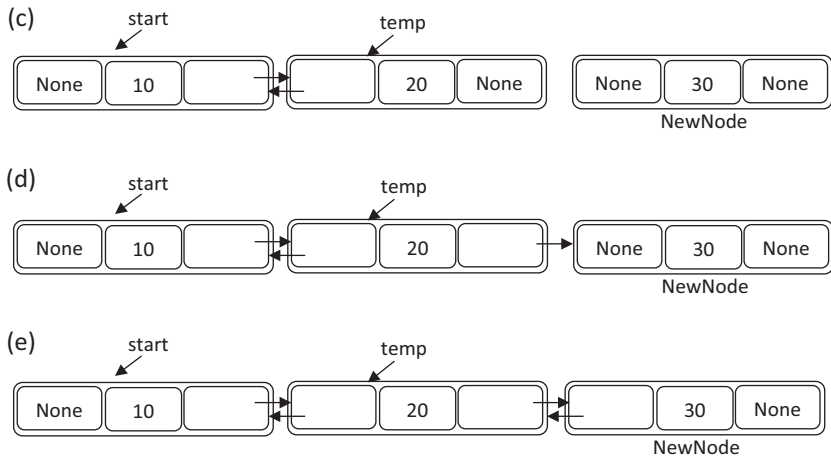


Figure 4.13 Continued

**Program continues...**

```
def insert_At_End(self,newdata):
    if (self.start == None):
        self.insert_At_Begining(newdata)
        return
    NewNode = Node(newdata)
    temp = self.start
    while(temp.next != None):
        temp = temp.next
    temp.next=NewNode
    NewNode.prev = temp
```

**3. Insert a node after a given node in a doubly linked list**

Insertion in the middle is slightly more complicated since the link between two nodes is to be “broken” and reattached to the new node appropriately.

While inserting a node after a node with a given key, first, take a pointer `temp` and move the pointer to the location after which a new node to be inserted. There are two cases:

Case 1: When the node with the given key is not found in the linked list, then print the appropriate message.



Case 2: When the node with the given key is found in the linked list, check if temp's next points to *None*. If so, insert the new node as last node by calling *insert\_At\_End()* method, otherwise follow these four steps (Figure 4.14):

- i. Set a new node's next point to the node to which temp's next is pointing.
- ii. Assign the new node's prev to temp.
- iii. Make the temp's next-prev point to the new node.
- iv. Set temp's next to the new node.

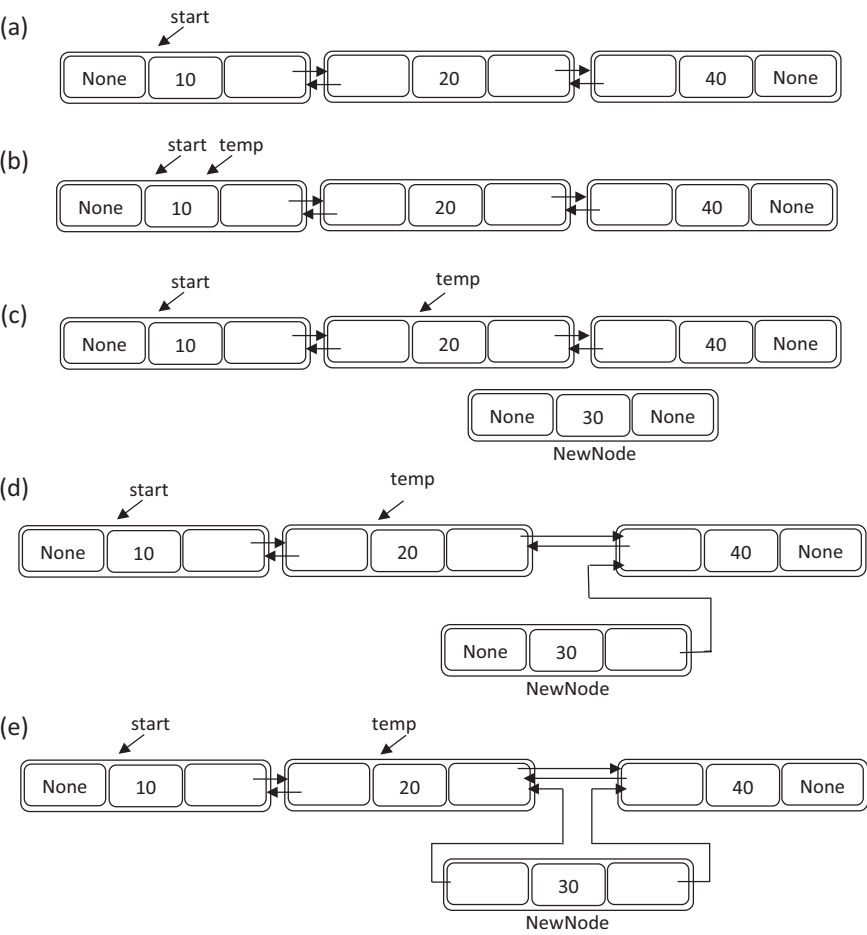


Figure 4.14 (a) Insert a node after a given node in a doubly linked list. (b) Assign Temp to start node (c) Move temp till required location (d) Assign newNode next to temp Next (e) Assign newnode previous to temp (f) Assign temp's next's previous to NewNode. (g) Assign temp's next to NewNode

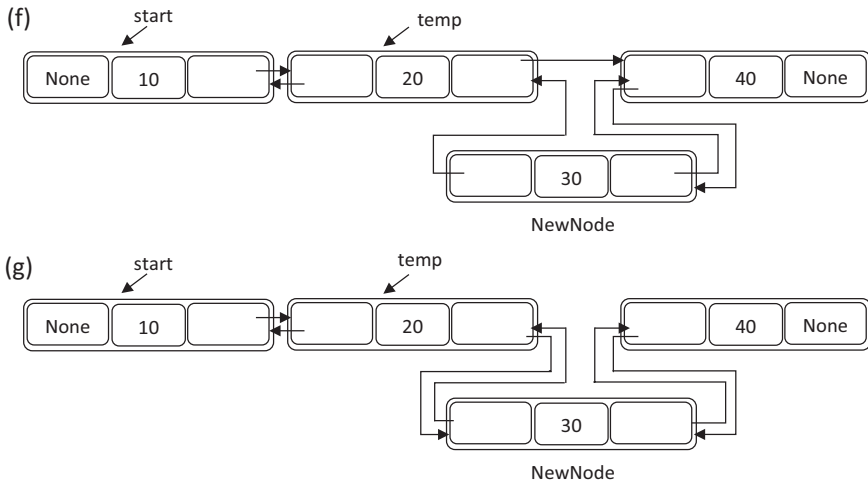


Figure 4.14 Continued

These steps can be executed through the *insert\_After()* method shown next.

**Program continues...**

```
def insert_After(self, key, newdata):
    temp = self.start
    while(temp.data != key and temp.next != None):
        temp = temp.next
    if (temp.data != key and temp.next == None ):
        print(key, " Not Found")

    elif (temp.data == key and temp.next == None):
        self.insert_At_End(newdata)
    else:
        NewNode = Node(newdata)
        NewNode.next = temp.next
        NewNode.prev = temp
        temp.next.prev = NewNode
        temp.next = NewNode
```

#### 4. Insert a node before a given node in a doubly linked list

While inserting a node before a node with a given key, assign *temp* to the *start*. If the data in the temp node is the same as the key (before which new node to be inserted), call *insert\_At\_Begining()* method.

If the node to be inserted is not the first node then make a pointer *temp* to check the data in the second node (that is *temp*'s next-data). If it matches with the key, insert the new node as a second node, otherwise move the *temp* pointer to the next one. There are two cases:

Case 1: If the key is not found in the linked list, then print the appropriate message.

Case 2: When the node with the given key is found in the linked list, then make the new node's next point to the node that the *temp*'s next is pointing to and then follow these steps.

- i. Set a new node's next point to the node to which *temp*'s next is pointing.
- ii. Assign the new node's prev to *temp*.
- iii. Make the *temp*'s next-prev point to the new node.
- iv. Set *temp*'s next to the new node.

The diagram is same as the previous one, except for the pointer.

**Program continues...**

```
def insert_Before(self, key, newdata):
    temp = self.start
    if (temp.data == key):
        self.insert_At_Begining(newdata)
    else:
        while(temp.data != key and temp.next != None):
            temp = temp.next
        if (temp.next == None ):
            print(key, " Not Found")
        else:
            NewNode = Node(newdata)
            NewNode.next = temp.next
            NewNode.prev = temp
            temp.next.prev = NewNode
            temp.next = NewNode
```

### iii. Deletion

Deletions from a linked list are comparatively easier than insertions. Once again there are five types of deletions: deletion of the first node, deletion of the node in the end, deletion of a given node, deletion of the node after a given node and deletion of the node before a given node.

## 1. Deletion of first node

Deleting a node which is the first node of a doubly linked list is quite simple. Take a pointer *temp* and assign it to the *start*. Move the *start* to its next node. Set the *start*'s *prev* to *None* and delete the node pointed by *temp*.

The diagram is almost the same as the deletion of the first node in the singly linked list.

**Program continues...**

```
def del_first(self):
    if(self.start != None):
        temp = self.start
        self.start = self.start.next
        temp.next = None
        del temp
    if (self.start != None ):
        self.start.prev=None
```

## 2. Deletion of the node at the end

Deleting a node that is the last node of a doubly linked list is quite simple. Take a pointer *temp* and assign *temp* to the *start*. There are two cases when deleting the last node.

Case 1: If the last node is the same as the first node. In other words, there is only one node present in the doubly linked list. Call the *del\_first()* method.

Case 2: If there are more than one element in the doubly linked list, move the *temp* pointer to the next to last one node using a while loop. Assign one more pointer *temp1*, pointing to *temp*'s next. Assign *temp*'s next to *None* and delete *temp1*.

The diagram is almost the same as deletion of the first node in the singly linked list.

**Program continues...**

```
def del_last(self):
    temp = self.start
    if (temp.next == None) :
        self.del_first()
        return
    while(temp.next.next != None):
        temp = temp.next
    temp1 = temp.next
    temp.next = None
    del temp1
```

### 3. Deletion of a given node

Deleting a node from the middle is a little more complicated since we have to first traverse the list up to the node we wish to delete. Let the node to be deleted contain the key. Depending on the presence of the key, there are three cases:

Case 1: If the key is present in the first node, call the *del\_first()* method.

Case 2: If the key is not the first node, it can be anywhere in the linked list. To traverse the linked list, take a pointer *temp* and assign it to the *start* pointer. Keep moving the *temp* pointer until *temp*'s data matches with the *key*.

- i. If the key is not found, print the appropriate message.
- ii. If the key is found at the last node, call the *del\_last()* method.
- iii. If the key is found in between, take one more pointer *temp1*, pointing to *temp*'s prev, assign *temp*'s next to *temp1*'s next, assign *temp1* to *temp*'s next-prev. Now delete *temp*.

**Program continues...**

```
def del_node(self, key):
    temp = self.start
    if (temp.data == key ):
        self.del_first()
        return
    while(temp.data != key and temp.next != None):
        temp = temp.next
    if (temp.data != key and temp.next == None ):
        print(data, " Not Found")
    elif(temp.data == key and temp.next == None):
        self.del_last()
    else:
        temp1=temp.prev
        print(temp1.data)
        temp1.next = temp.next
        temp.next.prev = temp1
        del temp
```

### 4. Deletion of the node after a given node

Depending on the presence of the key, there are two cases.

Case 1: If there is only node and the key matches with the data present in the node. As there is no node after the first node, display appropriate messages.

Case 2: If the key is not the first node, it can be anywhere in the linked list. To traverse the linked list, take a pointer *temp* and assign it to the *start* pointer. Keep moving the pointer until *temp*'s data matches with the *key*.

- i. If the key does not match with data of any node, display appropriate messages.
- ii. If the key matches with the last node, call the *del\_last()* method.
- iii. If the key matches with any other intermediate node, take one more pointer *temp1* and assign it to *temp*'s next. Assign *temp1*'s next-prev to *temp*, assign *temp*'s next to *temp1*'s next. Now delete the *temp1* node.

**Program continues...**

```
def del_after(self, key):
    temp = self.start
    if (temp.data == key and temp.next == None):
        print("There is only one element in the list...")
        print("Next element can not be deleted..")
        return
    while(temp.data != key and temp.next !=None):
        temp = temp.next

    if (temp.data != key and temp.next == None):
        print(key, " data Not found")

    elif (temp.data == key and temp.next == None):
        print(key, " is the Last node. Next, node cannot
        be deleted")

    elif(temp.data == data and temp.next.next == None):
        self.del_last()
    else:
        temp1=temp.next
        temp1.next.prev=temp
        temp.next = temp1.next
        del temp1
```

## 5. Deletion of the node before a given node

Depending on the presence of the key there are three cases.

Case 1: If the key matches with the data present in the first node. As there is no node before the first node, before the node cannot be deleted, display appropriate messages.

Case 2: If the key is in the second node, call the *del\_first()* method.

Case 3: If the key is not the second node, it can be anywhere in the linked list. To traverse the linked list, take a pointer *temp*. Assign *temp* to the *start* pointer. Keep moving *temp* pointers, until *temp*'s data matches with the *key*. There can be three options:

- i. If the key is not found, display appropriate message.
- ii. If *temp* is at the second node (*temp*'s *prev* is *None*), call the *del\_First()* method.
- iii. If *temp* is pointing to any other node, take one more pointer *temp1* and set it to *temp*'s *prev*.

**Program continues...**

```
def del_before(self, key):
    temp = self.start
    if (temp.data == key ):
        print("This is the first element in the list...")
        print("Previous element cannot be deleted..")
        return

    while(temp.data != key and temp.next !=None):
        temp = temp.next

    if (temp.data != key and temp.next == None ):
        print(data, " data Not found")

    elif(temp.data == key and temp.prev.prev == None):
        self.del_first()
    else:
        temp1=temp.prev
        temp1.prev.next = temp
        temp.prev = temp1.prev
        del temp1
```

**Program continues...**

```
def menu():
    print("\n~~~ MENU ~~~ ")
    print("1. Create a Double Linked List ")
    print("2. Insert at the Beginneing ")
    print("3. Insert at the End")
    print("4. Insert After a given Data ")
    print("5. Insert before a given Data")
    print("6. Delete the First node")
    print("7. Delete the Last node")
    print("8. Delete the a given node")
    print("9. Delete After a given Data")
    print("10. Delete before a given Data")
    print("11. Exit ")
```

---

```
opt = int(input("Enter a valid menu item ... "))
return opt

if __name__ == '__main__':
    list1 = DLinkedList()
    i=1
    while (i > 0 and i <=10 ):
        i = menu()
        if i == 1:
            x = input("Enter data to create the first node :")
            list1.start = Node(x)
            list1.listprint()

        elif (i==2):
            x = input("Enter data to create the node :")
            list1.insert_At_Begining(x)
            list1.listprint()

        elif (i==3):
            x = input("\n Enter data to create the node :")
            list1.insert_At_End(x)
            list1.listprint()

        elif (i==4):
            x = input("\n Enter data to create the node :")
            y = input("Node to be created after data :")
            list1.insert_After(y,x)
            list1.listprint()

        elif (i==5):
            x = input("\n Enter data to create the node :")
            y = input("Node to be created before data :")
            list1.insert_Before(y,x)
            list1.listprint()

        elif (i==6):
            list1.del_first()
            list1.listprint()

        elif (i==7):
            list1.del_last()
            list1.listprint()

        elif (i==8):
            x = input("Node to be deleted :")
            list1.del_node(x)
            list1.listprint()

        elif (i==9):
            x = input("Node to be deleted after data :")
            list1.del_after(x)
            list1.listprint()
```



```
elif (i==10):
    x = input("Node to be deleted before data :")
    list1.del_before(x)
    list1.listprint()
else:
    print("End")
```

### Advantages of doubly linked lists

1. The doubly linked list can be traversed in both directions.
2. We can access all the nodes starting from any node, not necessarily starting from the start.

### Disadvantages of doubly linked lists

1. Every node in the doubly linked list needs extra space.
2. All operations need to maintain two pointers.

## 4.3 CIRCULAR LINKED LISTS

In the last node of a singly linked list, we have a *None* reference to indicate the lack of further nodes. If we make it point to the first node (or the *start* node) of the list we have a *circularly linked list*. In other words, the last element contains the address of the first element. We take one more pointer, *end*, to point to the last node on a circular linked list (Figure 4.15).

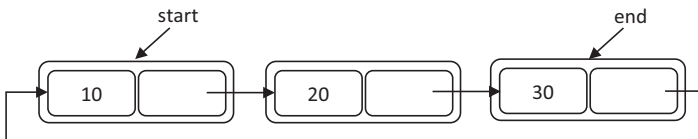


Figure 4.15 Circular linked lists

The node structure of a circular linked list and node structure of a singly linked list are the same. When we create a circular linked list, we take an additional point to point to the last node.

### Python program 4

```
class Node:
    def __init__(self, dataval=None):
        self.data = dataval
        self.next = None
```

---

```
class CLinkedList:
    def __init__(self):
        self.start = Node(None)
        self.end = Node(None)
        self.start.next = self.end
        self.end.next = self.start
```

### 4.3.1 Operations in a circular linked list

There are three operations that can be performed on a circular list: look-up, insertion and deletion.

#### i. Look-up

Let us say, we are looking for data item  $x$  in the circular list. We execute the following steps:

1. Start at the head node. (The head node can be any one of the nodes.)
2. Follow the links from the head node until we reach the node containing the desired value, if it exists.
3. If we reach back to the head node, report failure.

Let us first create a circular list. If we are creating the first node in the list, let us call it head. We define the node as a structure:

**Program continues...**

```
def listprint(self):
    print("The Elements in the Linked List are : ", end = "")
    temp = self.start
    if (self.start == None):
        print("There is no data..")
        return
    else:
        while(True):
            print(temp.data, end= " ")
            temp = temp.next
            if (temp == self.start):
                return
```

#### ii. Insertion

Insertion can be at the beginning, at the end, after a given node and before a given node.

Even though there is no beginning or end in a circular list, we identify one of the nodes as the head node, and the node pointing to the head node as the last node.

1. Insertion at the beginning

When a node is added in the front, the newly added node becomes the *start* node.

Let us insert a node with some data element in the front. While inserting an element as a front node, there are two cases:

Case 1: When there is no element in the circular linked list, the *start* and *end* points to *None*. After creating a new node with *next* of the new nodes pointing to itself, the *start* and *end* points to the new node (Figure 4.16).

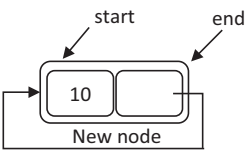


Figure 4.16 New node in circular linked lists

Case 2: When there is a circular linked list, follow these four steps to perform the operation (Figure 4.17).

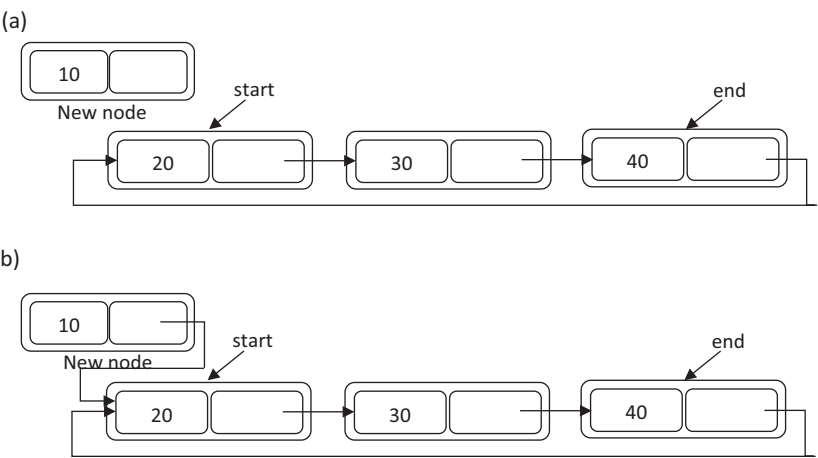


Figure 4.17 (a) Insert a node after a given node in a doubly linked list. (b) Assign NewNode's next to start. (c) Assign end's next to Newnode (d) Move start to Newnode

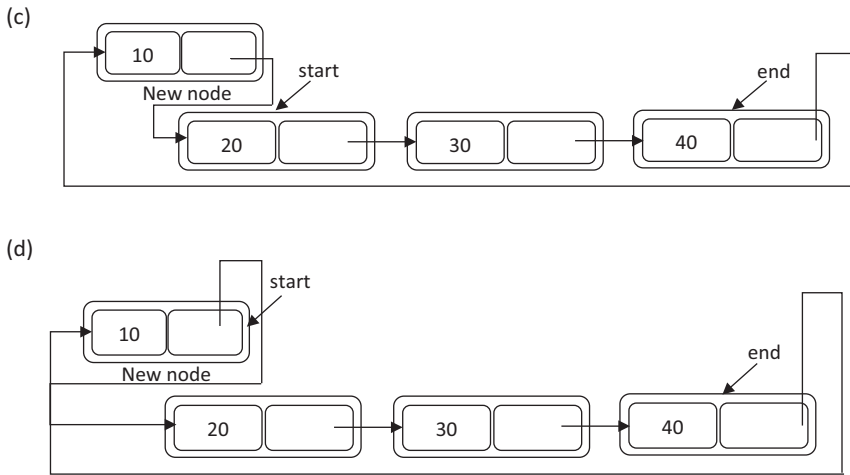


Figure 4.17 Continued

- i. Create a new node.
- ii. Set the *next* pointer of the new node to start.
- iii. Make the *next* of the *end* point to the new node.
- iv. Make a *start* point to the new node.

We execute the following Python code to insert a node with data item 10 in the beginning of the circular list with the variables declared earlier. The new node contains the data element 10.

**Program continues...**

```
def insert_At_Begining(self,newdata):
    NewNode = Node(newdata)
    if (self.start==None):
        self.start = NewNode
        NewNode=self.start
        self.end = self.start
        self.end.next = self.start
    else:
        NewNode.next = self.start
        self.start = NewNode
        self.end.next = self.start
```

## 2. Insertion at the end

When a node is added at the end, the newly added node becomes the *end* node.

Let us insert a node with some data element at the rear (Figure 4.18). While inserting an element as an end node, there are two cases:

Case 1: If the circular linked list is empty, then we simply add the new node as in case 1 of insertion at the beginning.

Case 2: When there is a circular linked list, follow these three steps to perform the operations (Figure 4.18).

1. Create a new node.
2. Point the end to the new node.
3. Point the end of the new node to start.

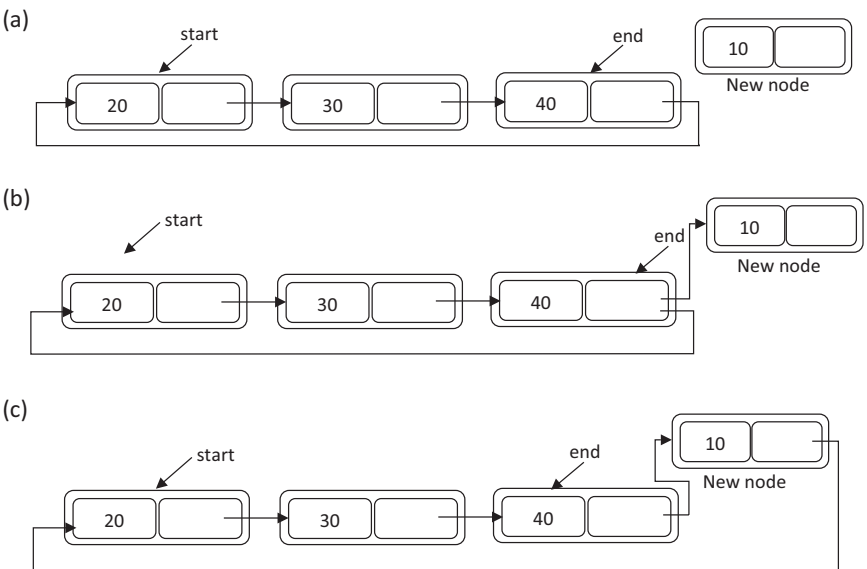


Figure 4.18 (a) Insertion at the end in circular linked lists. (b) Assign end's next to Newnode (c) Assign next of newnode to start

A program segment that does this with the usual notation is the following:

- The method takes two parameters: *self* (referring to the instance of the class) and *newdata* (the data to be inserted into the new node).
- A new node is created using the *Node* class, with *newdata* as its data.
- The code checks if the linked list is empty by verifying if *self.start* is *None*. If it is empty, the *insert\_At\_Begining* method is called to insert the new data at the beginning of the linked list. Then, the method returns.
- If the linked list is not empty, the next attribute of the new node (*NewNode.next*) is set to the current start of the linked list.
- The next attribute of the current end of the linked list (*self.end.next*) is set to the new node (*NewNode*).
- Finally, the end attribute of the linked list is updated to point to the new node (*self.end = NewNode*), making it the new end of the linked list.

**Program continues...**

```
def insert_At_End(self, newdata):
    NewNode = Node(newdata)
    if (self.start == None) :
        self.insert_At_Begining(newdata)
        return
    NewNode.next = self.start
    self.end.next = NewNode
    self.end = NewNode
```

### 3. Insertion after a given node in a circular linked list

Insertion in the middle is slightly more complicated since the link between two nodes is to be “broken” and reattached to the new node appropriately.

While inserting a node after a node with a given key, first, take a pointer temp and move the pointer to the location after which a new node is to be inserted. Let us assume that temp points to nodes having data 30. We are going to insert the new node after 30 as shown in Figure 4.19. There are two cases:

Case 1: When the node with the given key is not found in the linked list, then print the appropriate message.

Case 2: When the node with the given key is found in the linked list, check if temp’s next points to *start*. If so, insert the new node as last node by calling *insert\_At\_End()* method, otherwise follow these three steps:

- i. Set a new node's next point to the node to which temp's next is pointing.
- ii. Assign the new node's next to temp's next.
- iii. Make the temp's next point to the new node.

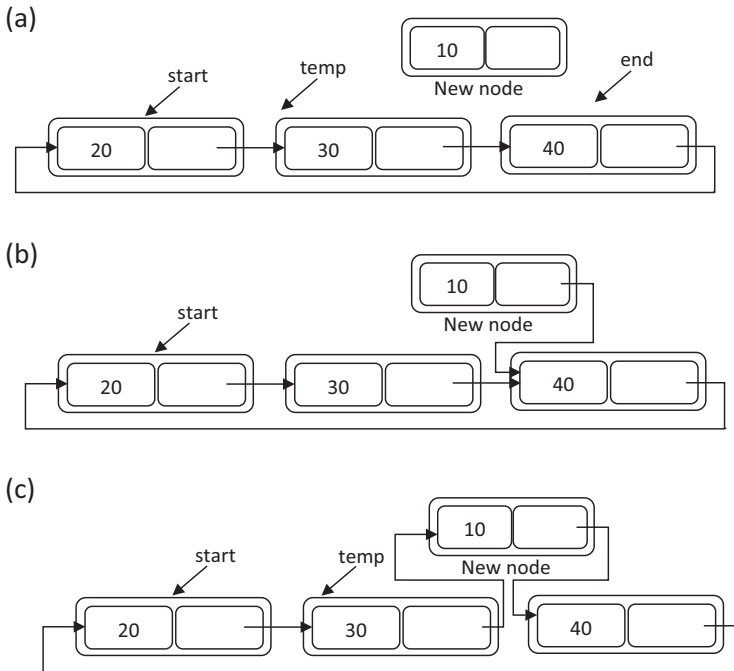


Figure 4.19 (a) Locate temp to desired location (b) Assign newnode's next to temp's next (c) Assign temp's next to Newnode

Program continues...

```
def insert_After(self, data, newdata):
    temp = self.start
    while(temp.data != data and temp.next != self.start):
        temp = temp.next
    if (temp.data != data and temp.next == self.start ):
        print(data, " Not Found")
    elif(temp.data == data and temp.next == self.start):
        self.insert_At_End(newdata)
    else:
        NewNode = Node(newdata)
        NewNode.next = temp.next
        temp.next = NewNode
```

The method takes three parameters: *self* (referring to the instance of the class), *data* (the data value after which the new node will be inserted), and *newdata* (the data to be inserted into the new node).

- A temporary variable *temp* is initialized with the start node of the circular linked list.
- A while loop is used to iterate through the linked list until either the data value is found or until we reach the start node again. The loop condition checks if *temp.data* is not equal to *data* and *temp.next* is not equal to the start node.
- If the loop terminates because the data value was not found (*temp.data != data* and *temp.next == self.start*), a message is printed indicating that the data value was not found.
- If the loop terminates because the data value was found and the next node is the start node (*temp.data == data* and *temp.next == self.start*), the *insert\_At\_End* method is called to insert the new data at the end of the circular linked list.
- If neither of the above conditions is met, it means the data value was found and it is not the last node. In this case, a new node *NewNode* is created using the *Node* class with *newdata* as its data.
- The *next* attribute of the new node (*NewNode.next*) is set to the next node of *temp*.
- The *next* attribute of *temp* is updated to point to the new node (*temp.next = NewNode*), effectively inserting the new node after *temp*.

#### 4. Insertion before a given node in a circular linked list

While inserting a node before a node with a given key, assign *temp* to the *start*. If the data in the temp node is the same as the key (before which new node to be inserted), call the *insert\_At\_Begining()* method.

If the node to be inserted is not the first node, then make a pointer *temp* to check the data in the second node (that is temp's next-data). If it matches with the key, insert the new node as a second node, otherwise move the *temp* pointer to the next one. There are two cases:

Case 1: If the key is not found in the linked list, then print the appropriate message.

Case 2: When the node with the given key is found in the linked list, then make the new node's next point to the node to which temp's next is pointing to. Let us assume that temp points to nodes having data 30. We are going to insert the new node before 40 as shown in Figure 4.19; follow these steps.

- i. Set a new node's next point to the node to which temp's next is pointing.



- ii. Assign the new node's next points to temp's next.
- iii. Make the temp's next point to the new node.

**Program continues...**

```
def insert_Before(self, data,newdata):
    temp = self.start
    if (temp.data == data):
        self.insert_At_Begining(newdata)
    else:
        while(temp.next.data != data and temp.next !=
            self.start):
            temp = temp.next
        if (temp.next.data != data and temp.next == self
            .start ):
            print(data, " Not Found")
        else:
            NewNode = Node(newdata)
            NewNode.next = temp.next
            temp.next = NewNode
```

- The method takes three parameters: *self* (referring to the instance of the class), *data* (the data value before which the new node will be inserted), and *newdata* (the data to be inserted into the new node).
- A temporary variable *temp* is initialized with the start node of the circular linked list.
- An initial check is performed to see if the data value of the start node is equal to the *data* parameter. If it is, the *insert\_At\_Begining* method is called to insert the new data at the beginning of the circular linked list.
- If the start node's data value is not equal to the *data* parameter, a while loop is used to iterate through the linked list until either the next node's data value is equal to *data* or until we reach the start node again. The loop condition checks if *temp.next.data* is not equal to *data* and *temp.next* is not equal to the start node.
- If the loop terminates because the data value was not found (*temp.next.data != data* and *temp.next == self.start*), a message is printed indicating that the data value was not found.
- If the loop terminates because the data value was found and it is not the last node, a new node *NewNode* is created using the *Node* class with *newdata* as its data.
- The *next* attribute of the new node (*NewNode.next*) is set to the next node of *temp*.
- The *next* attribute of *temp* is updated to point to the new node (*temp.next = NewNode*), effectively inserting the new node before *temp.next*.

### iii. Deletion

Deletions from a linked list are comparatively easier than insertions. There are five types of deletions: the first node, the node in the end, a given node, after a given node and before a given node.

#### 1. Deletion of first node

Deleting a node that is the first node of a circular linked list is quite simple. Take a pointer *temp* and assign it to the *start*. Move the *start* to its next node. Set the end's next start and delete the node pointed by *temp*.

The diagram is almost same as deletion of the first node in the singly linked list (Figure 4.20).

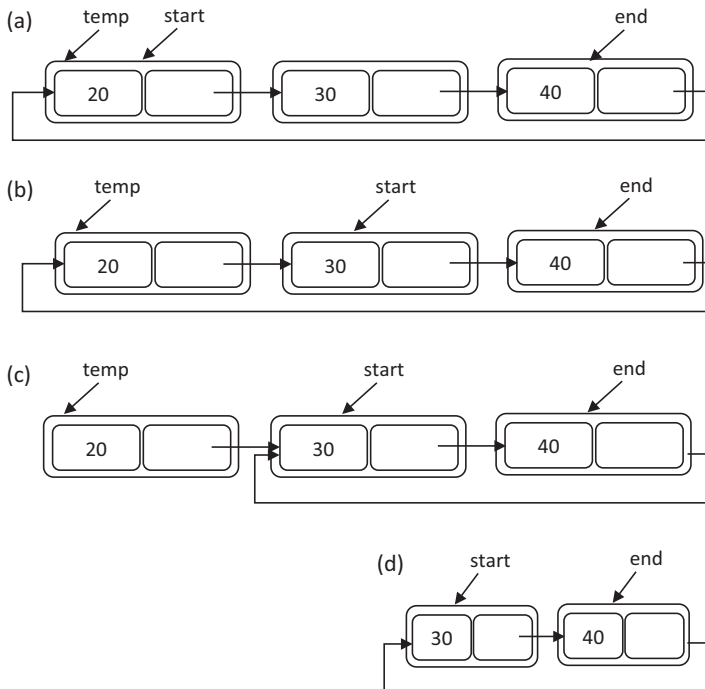


Figure 4.20 (a) Assign temp to start (b) Move start to temp's next (c) Assign end's next to start (d) remove temp

**Program continues...**

```
def del_first(self):
    temp = self.start
    if (self.start == self.end):
        self.start = self.end = None
        del temp
        return
    self.start = temp.next
    self.end.next = self.start
    del temp
```

- The method *del\_first* takes a single parameter *self* (referring to the instance of the class).
- A temporary variable *temp* is initialized with the start node of the circular linked list.
- An initial check is performed to see if the start node is also the end node (*self.start == self.end*). If it is, it means there is only one node in the circular linked list.
- In this case, both the start and end nodes are set to *None*, effectively removing the only node from the list.
- The temporary variable *temp* is deleted using the *del* keyword to free up the memory occupied by the node.
- The *return* statement is used to exit the method.
- If the start node is not the same as the end node (indicating that there are multiple nodes in the list), the following steps are executed:
- The *self.start* attribute is updated to point to the next node after the start node (*self.start = temp.next*). This effectively removes the first node from the list.
- The *next* attribute of the end node (*self.end.next*) is updated to point to the new start node, thereby maintaining the circular nature of the linked list.
- The temporary variable *temp* is deleted using the *del* keyword to free up the memory occupied by the node.

**2. Deletion of the node at the end**

Deleting a node that is the end node of a circular linked list is quite simple. Take a pointer *temp* and assign *temp* to the *start* (Figure 4.21). There are two cases while deleting the last node:

Case 1: If the end node is the same as the first node. In other words, there is only one node present in the doubly linked list. Call the *del\_first()* method.

Case 2: If there are more than one element in the doubly linked list, move the *temp* pointer until the temp's next to next is not the start node using a while loop. In other words, temp points the node before the end node. Assign one more pointer *temp1*, pointing to temp's next. Assign temp's next to start. Set end to temp and delete *temp1*.

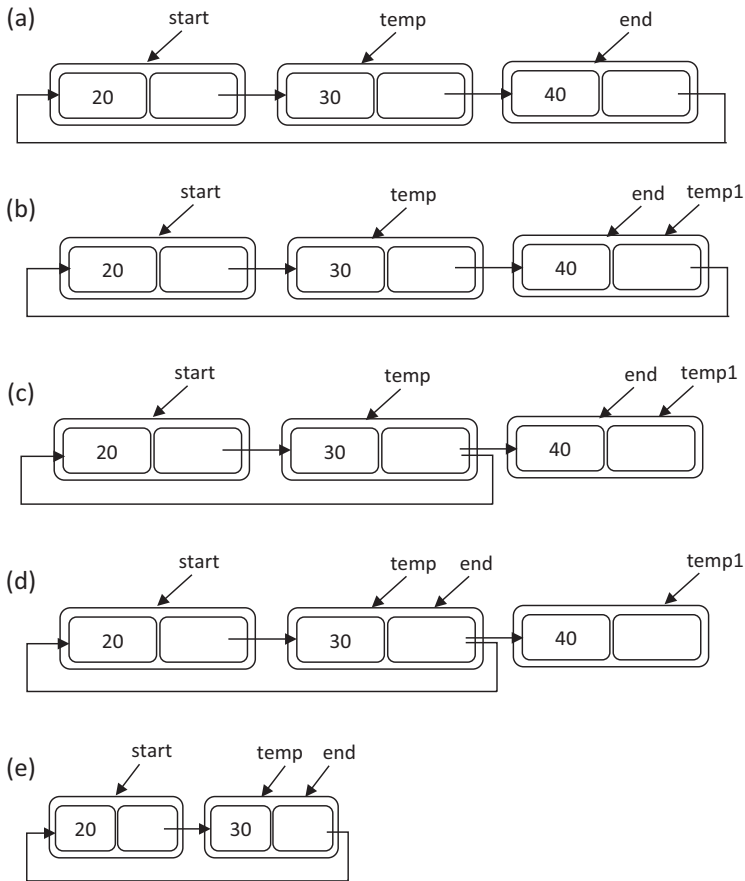


Figure 4.21 (a) Deletion of the node at the end in a circular linked list. (b) Assign temp1 to end. (c) Assign temp's next to start. (d) Assign end to temp. (e) Remove temp1

**Program continues...**

```
def del_last(self):
    temp = self.start
    if (self.start == self.end) :
        self.del_first()
        return
    while(temp.next.next != self.start):
        temp = temp.next
    temp1 = temp.next
    temp.next = self.start
    self.end = temp
    del temp1
```

### 3. Deletion of a given node

Deleting a node from the middle is a little more complicated since we have to first traverse the list up to the node we wish to delete (Figure 4.22). Let the node to be deleted contain the key. Depending on the presence of the key, there are four cases.

Case 1: If the key is present in the first node, call the `del_first()` method.

Case 2: If the key is not the first node, it can be anywhere in the linked list. To traverse the linked list, take a pointer `temp` and assign it to the `start` pointer. Keep moving the `temp` pointer until `temp`'s next data matches with the `key`. If the key is found at the last node, call the `del_last()` method.

Case 3: If the key is found in between, take one more pointer `temp1` (`temp1` to be deleted) pointing to `temp`'s next, assign `temp`'s next to `temp1`'s next. Now delete `temp1`.

Case 4: If the key is not found, print the appropriate message.

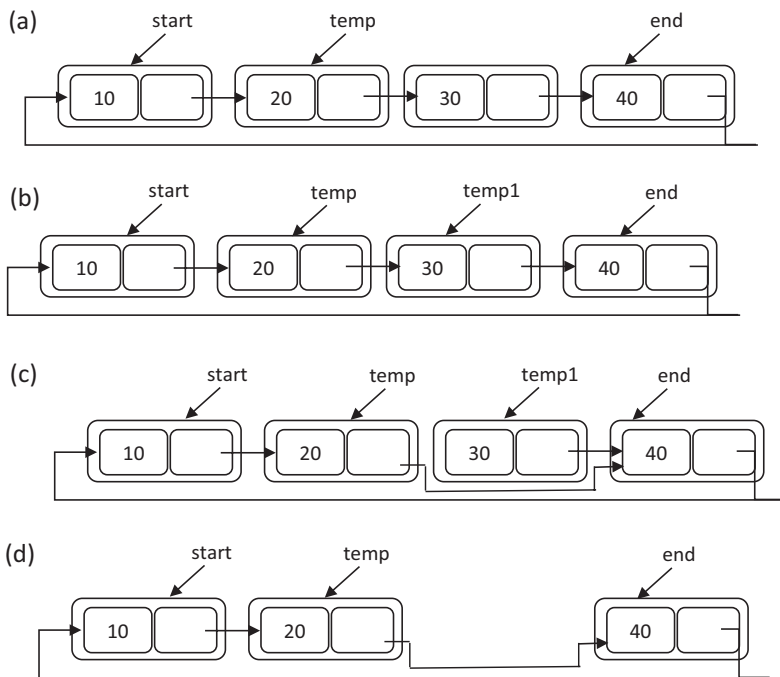


Figure 4.22 (a) Assign `temp` to desired location (b) Assign `temp1` to `temp`'s next (c) Assign `temp`'s next to `temp1`'s next (d) Delete `temp1`

**Program continues...**

```

def del_node(self, data):
    temp = self.start
    if (temp.data == data ):
        self.del_first()
        return
    while(temp.next.data != data and temp.next.next !=
self.start):
        temp = temp.next

    if (temp.next.data != data and temp.next.next ==
self.start):
        print(data, " Not Found")

    elif(temp.next.data == data and temp.next.next ==
self.start):
        self.del_last()
    else:
        temp1 = temp.next
        temp.next = temp1.next
        del temp1

```

**4. Deletion of the node after a given node**

Let the node to be deleted contain the key (Figure 4.22). Depending on the presence of the key, there are four cases.

Case 1: If the key is present in the first node, call the *del\_first()* method.

Case 2: If the key is not the first node, it can be anywhere in the linked list. To traverse the linked list, take a pointer *temp* and assign it to the *start* pointer. Keep moving the pointer until *temp*'s data matches with the *key*. If the key is found at the last node, call the *del\_last()* method.

Case 3: If the key matches with any other intermediate node, take one more pointer *temp1* and assign it to *temp*'s next. Assign *temp1*'s next-*prev* to *temp*, assign *temp*'s next to *temp1*'s next. Now delete the *temp1* node.

Case 4: If the key does not match with data of any node, display appropriate messages.

**Program continues...**

```

def del_after(self, data):
    temp = self.start
    if (temp.data == data and self.start == self.end ):
        print("There is only one element in the list...")
        print("Next element cannot be deleted..")

```

```
        return
    while(temp.data != data and temp.next != self.start):
        temp = temp.next

    if (temp.data != data and temp.next == self.start ):
        print(data, " data Not found")

    elif (temp.data == data and temp.next == self.start ):
        print(data, " is the Last node.. the next node is
        First node")
        ch = input("Do you to delete the first node (y | Y) : ")
        if (ch == 'y' or ch == 'Y'):

            self.del_first()
            return
    else:
        temp1 = temp.next
        temp.next = temp1.next
        del temp1
```

## 5. Deletion of the node before a given node

Depending on the presence of the key, there are four cases.

Case 1: If the key matches with the data present in the second node in the circular linked list, call the *del\_first()* method.

Case 2: If the key matches with the data present in the first node in the circular linked list, call the *del\_last()* method.

Case 3: If none of the above cases are met, the code enters a while loop that continues until either the data value is found or the next node becomes the starting node again.

- i. Inside the loop, *temp* is updated to the next node in the circular linked list.
- ii. After exiting the while loop, the code checks if the data value was found or if the next node is the starting node (indicating that the data value was not found).
- iii. If the data value was not found, it prints a message indicating that the data was not found.
- iv. Otherwise, it initializes *temp1* as the starting node and enters another while loop to find the node immediately before *temp* in the circular linked list.
- v. Once the previous node is found (*temp1*), it updates its *next* pointer to skip the node to be deleted (*temp*) and assigns *temp2* to the node to be deleted.
- vi. Finally, it deletes *temp2* using the *del* keyword.

**Program continues...**

```
def del_before(self, data):
    temp = self.start
    if ((temp.data == data and self.start == self.end)
        or temp.next.data == data ):
        self.del_first()
        return
    elif(temp.data == data and temp == self.start):
        self.del_last()
        return
    else:
        while(temp.data !=data and temp.next != self.start):
            temp = temp.next
        #while(temp.next.data != data and temp1.next.next
        !=self.start):

        # temp = temp.next
        if (temp.data != data and temp.next == self.start ):
            print(data, " data Not found")
        else:
            temp1 = self.start
            while(temp1.next.next != temp):
                temp1 = temp1.next

            temp2 = temp1.next
            temp1.next = temp
            del temp2
```

**Program continues...**

```
def menu():
    print("\n~~~ MENU ~~~ ")
    print("1. Create a Single Linked List ")
    print("2. Insert at the Beginneing ")
    print("3. Insert at the End")
    print("4. Insert After a given Data ")
    print("5. Insert before a given Data")
    print("6. Delete the First node")
    print("7. Delete the Last node")
    print("8. Delete the a given node")
    print("9. Delete After a given Data")
    print("10. Delete before a given Data")
    print("11. Exit ")
    opt = int(input("Enter a valid menu item ... "))
    return opt

if __name__ == '__main__':
    list1 = CLinkedList()
    i=1
    while (i > 0 and i <=10 ):
        i = menu()
```



```
if (i == 1):
    x = input("Enter data to create the first node :")
    list1.start = Node(x)
    list1.end= list1.start
    list1.end.next = list1.start
    list1.listprint()

elif (i==2):
    x = input("Enter data to insert at the beginning :")
    list1.insert_At_Begining(x)
    list1.listprint()

elif (i==3):
    x = input("\n Enter data to insert at the end :")
    list1.insert_At_End(x)
    list1.listprint()

elif (i==4):
    x = input("\n Enter data to create the node :")
    y = input("Node to be created after data :")
    list1.insert_After(y,x)
    list1.listprint()

elif (i==5):
    x = input("\n Enter data to create the node :")
    y = input("Node to be created before data :")
    list1.insert_Before(y,x)
    list1.listprint()

elif (i==6):
    list1.del_first()
    list1.listprint()

elif (i==7):
    list1.del_last()
    list1.listprint()

elif (i==8):
    x = input("Node to be deleted :")
    list1.del_node(x)
    list1.listprint()

elif (i==9):
    x = input("Node to be deleted after data :")
    list1.del_after(x)
    list1.listprint()

elif (i==10):
    x = input("Node to be deleted before data :")
    list1.del_before(x)
    list1.listprint()

else:
    print("End")
```

- ## Advantages of linked lists

**Fill-in-the-blank questions**

1. The \_\_\_\_\_ node always points to the first node of a linked list.
2. The number of pointers that must be changed when an element is inserted in the midst of a linked list is \_\_\_\_\_.
3. Insertion of an element at the ends of a linked list requires the modification of \_\_\_\_\_ pointer(s).
4. A \_\_\_\_\_ linked list stores the address of the head node in the next pointer of the last node.
5. Polynomial addition can be implemented using a \_\_\_\_\_.

**Multiple-choice questions**

1. In a circular singly linked list
  - a. Nodes are all linked together in some sequential manner.
  - b. There is no beginning and no end.
  - c. Nodes are arranged hierarchically.
  - d. Forward and backward traversal are permitted.
2. Each node in a linked list consists of two fields. One field is the data field. The other one is
  - a. Pointer to a char
  - b. Pointer to an integer
  - c. Pointer to the node
  - d. None of the above
3. How many null pointers exist in a doubly linked circular list?
  - a. 0
  - b. 1
  - c. 2
  - d. 3
4. An underflow condition in a linked list may occur while attempting to
  - a. Insert a new node when there is no free space for it
  - b. Delete a non-existent node
  - c. Delete a node from an empty list
  - d. Insert a new node in an empty list
5. A variant of a linked list where the last node points to the first node, is a
  - a. Singly linked list
  - b. Doubly linked list
  - c. Circular linked list
  - d. Multiple linked list

6. You are given pointers to the first node and the last node of a singly linked list. Which of the following operations is dependent on the length of the list?
  - a. Delete the first element of the list
  - b. Insert a new element as the first element
  - c. Delete the last element
  - d. Add a new element at the end of the list
7. Which of the following operations is performed more efficiently in a doubly linked list?
  - a. Deleting a node from a given location
  - b. Searching for a given element
  - c. Inserting a node after a given node
  - d. Traversing the list to process each node
8. What is the asymptotic time complexity to add an element to a linked list?
  - a.  $O(1)$
  - b.  $O(n)$
  - c.  $O(n^2)$
  - d. None of the above
9. In which type of linked list can traversals be performed in both directions?
  - a. Singly linked list
  - b. Doubly linked list
  - c. Circular linked list
  - d. All of the above
10. Which of the following statements is true?
  - a. The size of the linked list is dynamic and can be changed as needed.
  - b. Arrays have better cache locality than linked lists.
  - c. Random access of elements at a linked list is not possible.
  - d. All of the above.
11. Which of the following methods is best for locating an entry in the linked list at position  $k$ ?
  - a. Singly linked list
  - b. Doubly linked list
  - c. Circular linked list
  - d. Array implementation

12. Which of the following issues can be resolved with a linked list and two pointers?
- a. Finding the intersection of two linked lists
  - b. Finding the middle element of a linked list
  - c. Detecting a cycle in a linked list
  - d. All of the above
13. What is absent from a linked list node?
- a. A pointer to the linked list
  - b. Data
  - c. A pointer to the previous node
  - d. A pointer to the next node
14. Which of the following cannot be effectively accomplished with a linked list?
- a. Priority queue
  - b. Stack
  - c. Binary search
  - d. Queue
15. What drawback does a circular linked list have?
- a. A simple loop through the list will never end.
  - b. It is complex to implement.
  - c. It consumes more memory than a simple linked list.
  - d. All of the above.
16. What is a linked list's primary benefit over an array?
- a. In linked lists, random access is more effective.
  - b. It is simpler to implement linked lists.
  - c. Elements can be added to it in  $O(1)$  time.
  - d. A linked list requires less memory.
17. Which of these is an application of linked lists?
- a. To implement file systems
  - b. For separate chaining in hash tables
  - c. To implement non-binary trees
  - d. All of the above
18. One example of the \_\_\_\_\_ form of memory allocation is the linked list.
- a. Static
  - b. Dynamic
  - c. Compile time
  - d. None of the above

19. The linked list data structure provides significant cost savings in
- Space utilization
  - Computational time
  - Computational time and space utilization
  - None of above
20. Pointer to node X in a singly linked list is provided. Can we remove node X from the provided linked list, as there is only one pointer provided and no pointer to the head node?
- Potentially, if X is not the final node.
  - Potentially, if the linked list's size is even.
  - Potentially, if the linked list's size is odd.
  - Potentially, if X is not the first node.

### Descriptive questions

- Let A and B be two singly linked lists. Write a program to attach these two lists to construct a singly linked circular list
- Create a doubly-linked list with the following elements and write a program to print it in the reverse order: 3 5 7 9 11 13 15.
- If  $A[3][3][3]$  is an integer matrix, calculate the addresses of the following elements:

$A[1][1][1]$ ,  $A[1][2][2]$ ,  $a[2][1][2]$

- Write a program to delete the duplicate elements from an array.
- Write a program to remove alternate nodes from a list and create a new list.
- An array  $A = \{3, 5, 7, 9\}$  is given and it is full. Write a program to insert a new element 11 by doubling the array size.
- Given a linked list of integers, write a program to find the largest and smallest elements in the list.
- Write a program to find a sub-list in a given linked list.

### Answers to true–false questions

- True
- False
- False
- True
- False

### Answers to fill-in-the-blank questions

1. start
2. two
3. one
4. circular
5. linked list

### Answers to multiple-choice questions

- |     |      |     |   |     |   |     |   |     |   |
|-----|------|-----|---|-----|---|-----|---|-----|---|
| 1.  | a, b | 2.  | c | 3.  | a | 4.  | c | 5.  | c |
| 6.  | c    | 7.  | b | 8.  | b | 9.  | b | 10. | d |
| 11. | d    | 12. | d | 13. | a | 14. | c | 15. | a |
| 16. | c    | 17. | d | 18. | b | 19. | c | 20. | a |

# Stacks

---

### LEARNING OBJECTIVES

After studying this chapter, readers will be able to:

- Understand stack and its operations
- Implement stack using list
- Implement stack using queue module
- Implement stack using dictionary
- Implement stack using class
- Implement stack using queue module
- Apply stack to match delimiter
- Apply stack for infix–postfix conversion
- Apply stack for evaluation of the postfix expression
- Apply stack for recursion

We know that the linear data structures are divided into arrays, stacks and queues. In this chapter, we will focus on the stack as a linear data structure. Stack data structures can be implemented using arrays. Simple examples of a stack are a stack of plates in a restaurant and a stack of books in a library.

### 5.1 STACKS

The insert and delete operations are done at the same end of a stack, which is a special case of an ordered list. Stacks are used as a data structure in programming languages to achieve recursion, to transform an infix expression to a postfix expression and so on.

A stack is a restricted ordered list in which only one end, termed the *top* of the stack, allows insertions and deletions. The stack abstract data type has the data member *top*, which points to the topmost element in the stack. There are two basic operations: *push()* adds an element to the top of the stack and *pop()* removes an element from the top of the stack. The element



that is inserted last is deleted first. So a stack is also known as a *last-in-first-out (LIFO)* data structure.

If we describe the stack as  $S = \{a_1, a_2, \dots, a_n\}$ , then we consider  $a_1$  to be at the bottom of the stack and  $a_n$  as at the top of the stack and  $a_{i+1}$  as on the top of  $a_i$ .

A stack with four elements can be pictorially represented as in Figure 5.1.

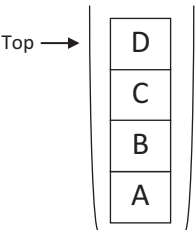


Figure 5.1 Stack

5.2 STACK OPERATIONS

The basic operations on a stack are the following:

| Description                       | Operation name |
|-----------------------------------|----------------|
| Push an element onto the stack    | push(element)  |
| Pop an element from a stack       | pop()          |
| Get the top element of the stack  | peek()         |
| Display the elements in the stack | display()      |

5.2.1 push()

The *push(S)* operation inserts an element on the top of the stack *S* (Figure 5.2). It follows these steps:

1. Let *top* be the index of the topmost element.
2. Check if the stack is full (if it is a restricted stack).
3. If stack is not full.
  - a. Increment *top* by one.
  - b. Top of the stack =  $S[top]$  (*S* is the list we use to create the stack).
  - c. Return stack.
4. Else report failure (*stack overflow*).

The maximum size of a Python list on a 32 bit system is around 536,870,912 elements. Since we are using a list to implement a stack, the condition *isFull()* is not required. If we implement a restricted list (a list with predefined size),

then *isFull()* is required to be implemented. We increment the 'top' by one and insert  $S[\text{top}]$  onto the stack.

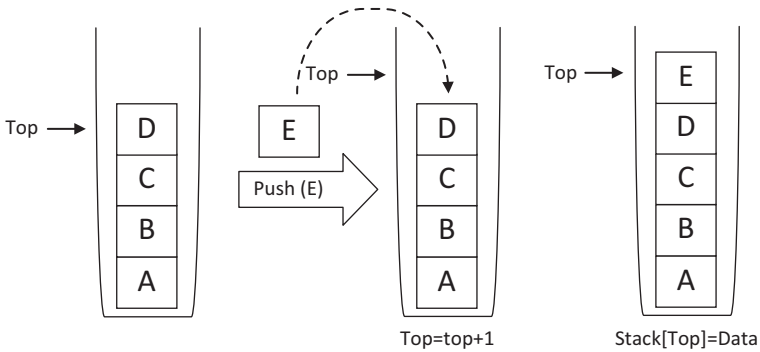


Figure 5.2 Stack operation push

### 5.2.2 pop()

We execute the following steps to perform the pop operation (also see Figure 5.3):

1. Let  $x$  be the variable to store the popped element.
2. If the stack is not empty.
  - a. Set  $x = A[\text{top}]$ .
  - b. Decrement top by one.
  - c. Return stack.
3. Else report failure (*stack underflow*).

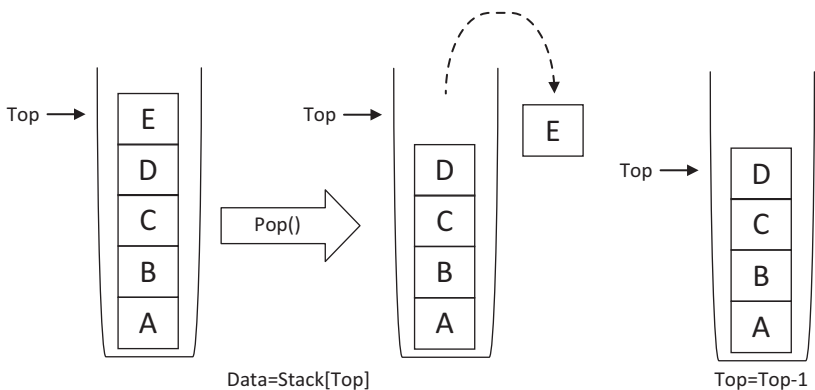


Figure 5.3 Stack operation pop

5.2.3 peek()

*peek()* is similar to *pop()* except that this function removes the element from the top of the stack without decrementing the top by one (Figure 5.4).

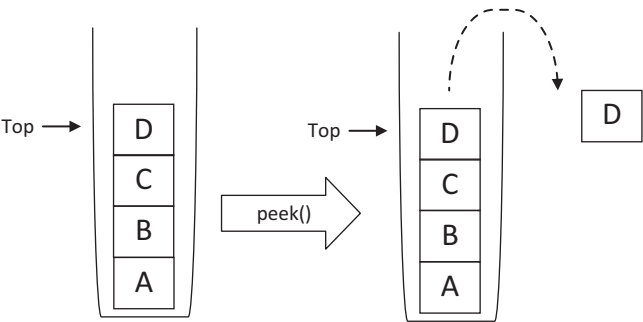


Figure 5.4 Stack operation peek

An error condition (*stack underflow*) is reported if we try to remove an element from an empty stack.

5.3 STACK (ADT)

Let us formally describe the structure of the stack together with a set of axioms that we expect for the stacks.

The abstract data type stack can be described by the following methods and axioms.

Methods: See Table 5.1.

Table 5.1 Stack Methods

| Method     | Description                                                          |
|------------|----------------------------------------------------------------------|
| Create()   | Creates an empty stack, say, S.                                      |
| push(i,S)  | Pushes i onto the stack: returns the resultant stack.                |
| pop(S)     | Pops the top element from the stack: returns the resultant stack.    |
| peek()     | Displays the top element: stack remains the same.                    |
| isEmpty(S) | Returns true if the stack is empty (top=-1 for an empty stack).      |
| isFull(S)  | Returns true if the stack is Full (Applicable for fixed size stack). |

Axioms: See Table 5.2.

Table 5.2 Stack axioms with return values

| Axiom                           | Return value |
|---------------------------------|--------------|
| <code>isEmpty(create())</code>  | True         |
| <code>isEmpty(push(e,S))</code> | False        |
| <code>pop(create())</code>      | Error        |
| <code>peek(create())</code>     | Error        |
| <code>peek(push(e,S))</code>    | Error        |

The first axiom says that when we create a stack, it is empty, hence the *isEmpty()* boolean function returns *true*.

The second axiom is easy to understand since when we push an element onto a stack, it is not empty, hence the *isEmpty()* function returns *false*.

We cannot *pop()* an element from an empty stack. Thus *pop(create())* gives an *error* since the *create()* function creates an empty stack and so is the case with *peek(create())*.

*peek(push(e,S))* returns *e* since after the push operation the topmost element is *e*. Also see Table 5.3.

Table 5.3 Stack methods with return

| Function                        | Return  |
|---------------------------------|---------|
| <code>create (S)</code>         | Stack   |
| <code>push(i,s)</code>          | Stack   |
| <code>pop(S)</code>             | Element |
| <code>getTop(S)</code>          | Element |
| <code>isEmpty(s)</code>         | Boolean |
| <code>isEmpty(create(S))</code> | True    |
| <code>isEmpty(push(E,S))</code> | False   |
| <code>pop(create(S))</code>     | Error   |
| <code>peek(create(S))</code>    | Error   |
| <code>peek(push(E,S))</code>    | Error   |

## 5.4 IMPLEMENTATION

There are various ways to implement a stack. It can be implemented using a list, dictionary or a class in Python.

### 5.4.1 Implementation using a list

The elements of the stack are the elements of a list. As a list is not a fixed-size data structure, stack is also not of fixed size while implementing. But we can restrict the size of the list, hence the size of stack can be made fixed.

Now let us implement a stack using list without restriction of the upper limit or the size of the stack.

### Python program 1

```
#Create an empty stack
def create():
    S=[]
    top = -1
    print("Empty Stack created")
    return S,top
```

The create() method creates an empty stack with the top pointing to -1. Here S = [] indicates an empty list.

### Program continues...

```
#Checks if the stack is empty
def isEmpty(top):
    if (top <= -1):
        return 1
    else:
        return 0
```

The isEmpty() method checks if the stack is empty. It checks if the top is pointing to -1. It returns 1 to indicate the stack is empty; otherwise returns 0 to indicate the stack is not empty (at least one element is there in the stack).

### Program continues...

```
#Insert data into stack
def push(S,top):
    no = int(input(("Enter a number to push onto the
        stack ")))
    top += 1
    S.append(no)
    return S,top
```

The push() method takes two parameters, the stack and the top. Every time an element is pushed onto the stack, the top is incremented by 1 and the element is appended to the list. As a list is a heterogeneous data structure, we force the user to enter an integer only.

**Program continues...**

```
#Delete from stack
def pop(S,top):
    if (isEmpty(top) == 1):
        print("Underflow: There is no element in the Stack")
    else:
        print("The element to pop is ",S[top])
        top -= 1
        return S,top
```

The pop() method takes two parameters, the stack and the top. Every time an element is popped from the stack, it checks if the stack is not empty. If the stack is empty, it shows an appropriate message, and if the stack is not empty, then the top of the stack is printed and the top is decremented by 1. At last the stack and the top is returned.

**Program continues...**

```
#Returns the top of the stack
def peek(S,top):
    if (isEmpty(top) == 1):
        print("Stack is Empty")
    else:
        print("The element at top is ",S[top])
```

The peek() method takes two parameters, the stack and the top. It just prints the element that is on the top of the stack.

**Program continues...**

```
#Displays the stack element
def display(S,top):
    print("Top is at index -> ",top)
    if (isEmpty(top) == 1) :
        print("\nStack is Empty")
    else:
        print("The elements in the Stack is")
        for i in range(top,-1,-1):
            print(i, "->",S[i])
```

The display() method takes two parameters, the stack and the top. This function prints the elements in the stack in the reverse order (the element inserted last is printed first).

**Program continues...**

```
def menu():
    print("\n~~~ MENU ~~~ ")
    print("1. Push an element ")
    print("2. Pop from stack ")
    print("3. Peek the stack")
    print("4. Display the stack")
    print("5. Exit ")
    opt = int(input("Enter a valid menu item ... "))
    return opt
```

The menu() method shows a menu to perform the operations defined for a stack. It returns a choice of operation to perform.

**Program continues...**

```
#Main Program
if __name__ == "__main__":
    St,top=create()
    i=1
    while (i > 0 and i < 5 ):
        i = menu()
        if i == 1:
            St,top = push(St,top)
            display(St,top)
        elif i == 2:
            St,top = pop(St,top)
            if (type(top) != str):
                display(St,top)
        elif i == 3:
            peek(St,top)
        elif i == 4:
            display(St,top)
        else:
            print("Exit")
```

**Output**

Empty Stack created

```
~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 1
Enter a number to push onto the stack 99
Top is at index -> 0
The elements in the Stack is
0 -> 99
```

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit

Enter a valid menu item ... 1

Enter a number to push onto the stack 88

Top is at index -> 1

The elements in the Stack is

1 -> 88

0 -> 99

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit

Enter a valid menu item ... 1

Enter a number to push onto the stack 77

Top is at index -> 2

The elements in the Stack is

2 -> 77

1 -> 88

0 -> 99

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit

Enter a valid menu item ... 3

The element at top is 77

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit

Enter a valid menu item ... 2

The element to pop is 77

Top is at index -> 1

The elements in the Stack is

1 -> 88

0 -> 99



```
~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 2
The element to pop is 88
Top is at index -> 0
The elements in the Stack is
0 -> 99

~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 2
The element to pop is 99
Top is at index -> -1

Stack is Empty

~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 2
Underflow: There is no element in the Stack
Top is at index -> -1

Stack is Empty

~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 6
Exit
```

Now let us implement a stack using list with a restriction of upper limit or the size of the stack. There is no change in the create(), pop(), peek(), isEmpty() and menu() functions and some changes in the push() and display() functions. A new function isFull() is also introduced. We are discussing only those functions that have some changes from Python program 1.

## Python program 2

```
def isFull(top,size):
 if (top >= size-1):
 return 1
 else:
 return 0
```

The `isFull()` method checks if the stack is full. It checks if the top is pointing to maximum size -1. It returns 1 to indicate the stack is full; otherwise returns 0 to indicate the stack is not full.

### Program continues...

```
def push(S,top,size):
 if (isFull(top,size) == 1):
 print("Overflow: No more place for the Stack")
 else:
 no = int(input(("Enter a number to push onto
 the stack ")))
 top += 1
 S.append(no)
 return S,top
```

The `push()` method takes two parameters, the stack and the top. Every time an element is pushed onto the stack, it checks if the stack is full. If the stack is full, it shows an appropriate message; otherwise the top is incremented by one and the element is appended to the list.

### Program Continues...

```
def display(S,top,size):
 print("Top is at index -> ",top)
 if (isEmpty(top) == 1) :
 print("\nStack is Empty")

 elif(isFull(top,size) == 1):
 print("\nStack is Full")
 else:
 print("The elements in the Stack is")
 for i in range(top,-1,-1):
 print(i, "->",S[i])
```

The `display()` method takes two parameters, the stack and the top. This function prints the elements in the stack in the reverse order (the element inserted last is printed first) if the stack is neither full nor empty.

## Output

```
Enter the size of the Stack 3
Empty Stack created

~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 1
Enter a number to push onto the stack 99
Top is at index -> 0
The elements in the Stack is
0 -> 99

~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 1
Enter a number to push onto the stack 88
Top is at index -> 1
The elements in the Stack is
1 -> 88
0 -> 99

~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 1
Enter a number to push onto the stack 77
Top is at index -> 2

Stack is Full

~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 1
Overflow: No more place for the Stack
Top is at index -> 2
```

Stack is Full

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit

Enter a valid menu item ... 3

The element at top is 77

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit

Enter a valid menu item ... 2

The element to pop is 77

Top is at index -> 1

The elements in the Stack is

1 -> 88

0 -> 99

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit

Enter a valid menu item ... 2

The element to pop is 88

Top is at index -> 0

The elements in the Stack is

0 -> 99

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit

Enter a valid menu item ... 2

The element to pop is 99

Top is at index -> -1

Stack is Empty

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack

```
4. Display the stack
5. Exit
Enter a valid menu item ... 5
Exit
```

### 5.4.2 Implementation of stack using a dictionary

Now let us implement a stack using dictionary with a restriction of upper limit or the size of the stack. We already know that a dictionary is a key-value pair. We are implementing the stack with a size restriction.

#### Python program 3

```
def create():
 S={}
 top = -1
 print("Empty Stack created")
 return S,top
```

In create() method, we have created an empty dictionary S={} and assigned top to -1.

#### Program continues...

```
def isFull(top,size):
 if (top >= size-1):
 return 1
 else:
 return 0
```

Just like the previous program, the isFull() function checks if the stack is full. When the location of the top is greater than or equal to maximum size -1, it returns 1 indicating the stack is full; otherwise it returns 0 indicating the stack is not full.

#### Program continues...

```
def isEmpty(top):
 if (top <= -1):
 return 1
 else:
 return 0
```

The isEmpty() function checks if the stack is empty. When the location of the top is less than or equal to -1, it returns 1 indicating the stack is empty; otherwise it returns 0 indicating the stack is not empty.

**Program continues...**

```
def push(S,top,size):
 if (isFull(top,size) == 1):
 print("Overflow: No more place in the Stack")

 else:
 no = int(input(("Enter a number to push onto the stack ")))
 top += 1
 S[top] = no
 return S,top
```

The `push()` function takes three parameters: the stack `S`, `top` and the size of the stack. If the stack is full, it prints an appropriate message; otherwise increment the `top` by one and insert the element on the top of the stack. It returns the stack and the top of the stack.

**Program continues...**

```
def pop(S,top):
 if (isEmpty(top) == 1):
 print("Underflow: There is no element in the Stack")
 else:
 print("The element to pop is ",S[top])
 del S[top]
 top -= 1
 return S,top
```

The `pop()` function takes two parameters: the stack `S` and top of the stack. If the stack is empty, it prints an appropriate message; otherwise, decrement the `top` by 1 and return the stack and the top.

**Program continues...**

```
def peek(S,top):
 if (isEmpty(top) == 1):
 print("Stack is Empty")
 else:
 print("The element at top is ",S[top])
```

The `peek()` method takes two parameters, the stack and the top. It just prints the element that is on the top of the stack.

**Program continues...**

```
def display(S,top,size):
 print("Top is at index -> ",top)
 if (isEmpty(top) == 1) :
 print("\nStack is Empty")

 elif(isFull(top,size) == 1):
 print("The elements in the Stack is")
 for i in reversed(list(S.keys())):
 print(i, "->",S[i])
 print("\nStack is Full")

 else:
 print("The elements in the Stack is")
 for i in reversed(list(S.keys())):
 print(i, "->",S[i])
```

The `display()` method takes two parameters: the stack and the top. This function prints the elements in the stack in the reverse order (the element inserted last is printed first) if the stack is neither full nor empty.

**Program continues...**

```
def menu():
 print("\n~~~ MENU ~~~ ")
 print("1. Push an element ")
 print("2. Pop from stack ")
 print("3. Peek the stack")
 print("4. Display the stack")
 print("5. Exit ")
 opt = int(input("Enter a valid menu item ... "))
 return opt
```

**Program continues...**

```
if __name__ == "__main__":
 size = int(input("Enter the size of the Stack "))
 St,top=create()
 i=1
 while (i > 0 and i < 5):
 i = menu()
 if i == 1:
 St,top = push(St,top,size)
 display(St,top,size)
 elif i == 2:
 St,top = pop(St,top)
 display(St,top,size)
 elif i == 3:
 peek(St,top)
```

```
elif i == 4:
 display(St,top,size)
else:
 print("Exit")
```

The output of the program is the same as the previous program.

### 5.4.3 Implementation of stack using a class

Now let us implement a stack using class with a restriction of upper limit or the size of the stack.

#### Python program 4

```
class Stack:
 def __init__(self,size):
 self.S = list()
 self.top = -1
 self.maxSize = size
 print("\nEmpty Stack Created")
```

Class 'Stack' is defined with three variables: a list, 'top' initialized to -1 and maximum size of the stack. All the methods in the class are public.

#### Program continues...

```
def isFull(self):
 if (self.top >= self.maxSize-1):
 return 1
 else:
 return 0
```

The `isFull()` method is a Boolean method that returns 1 (true) if the stack full condition is satisfied `self.top >= self.maxSize-1` and returns 0 (false) otherwise.

#### Program continues...

```
def isEmpty(self):
 if (self.top <= -1):
 return 1
 else:
 return 0
```

The `isEmpty()` method is a Boolean method that returns 1 (true) if the empty condition is satisfied `self.top <= -1` and returns 0 (false) otherwise.



**Program continues...**

```
def push(self):
 if (self.isFull() == 1):
 print("Overflow: No more place for the Stack")
 else:
 no = int(input(("Enter a number to push onto the stack ")))
 self.top += 1
 self.S.append(no)
```

The push() function first checks if the stack is full. If the stack is full, it prints an appropriate message; otherwise, increment the top by one and insert the element on the top of the stack.

The initial stack is shown in Figure 5.5.

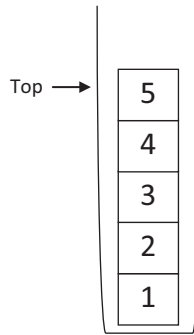


Figure 5.5 Stack push method

After executing push() method, the stack status is as shown in Figure 5.6.

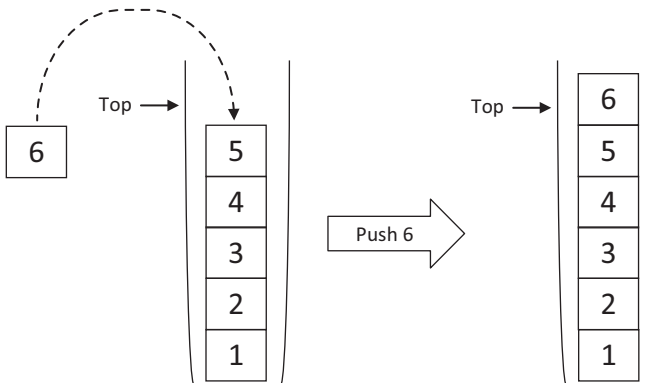


Figure 5.6 Stack push(6) method

**Program continues...**

```
def pop(self):
 if (self.isEmpty()==1):
 print("Underflow: There is no element in the Stack")
 else:
 print("The element to pop is ",self.S[self.top])
 self.top -= 1
```

The `pop()` function first checks if the stack is empty. If the stack is empty, it prints an appropriate message; otherwise it prints the element at the top of the stack and then decrements the top by one.

The initial stack is shown in Figure 5.7.

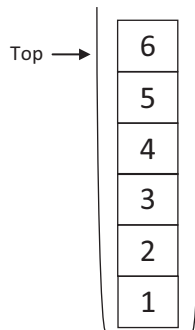


Figure 5.7 Sample stack

After the execution of `pop()`, the topmost element pops-out and leaves the stack as shown in Figure 5.8.

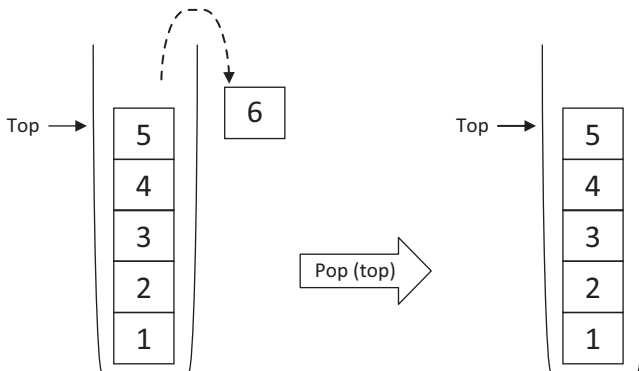


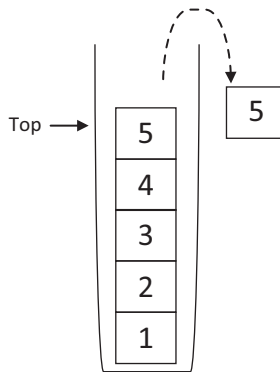
Figure 5.8 Sample stack pop operation

**Program continues...**

```
def peek(self):
 if(self.isEmpty()==1):
 print("Stack is Empty")
 else:
 print("The element at top is ",self.S[self.top])
```

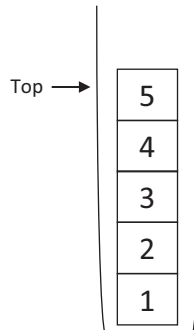
The `peek()` function first checks if the stack is empty. If the stack is empty, it prints an appropriate message; otherwise it prints the element at the top of the stack.

The initial stack is shown in Figure 5.9.



*Figure 5.9* Stack before execution of peek operation

The execution of `peek()` leaves the stack as shown in Figure 5.10.



*Figure 5.10* Stack after execution of peek operation

**Program continues...**

```

def display(self):
 print("Top is at index -> ",self.top)
 if(self.isEmpty()==1):
 print("\nStack is Empty")

 elif(self.isFull()==1):
 for i in range(self.top,-1,-1):
 print(i, "->",self.S[i])
 print("\nStack is Full")

 else:
 print("The elements in the Stack is")
 for i in range(self.top,-1,-1):
 print(i, "->",self.S[i])

```

**Program continues...**

```

def menu():
 print("\n~~~ MENU ~~~ ")
 print("1. Push an element ")
 print("2. Pop from stack ")
 print("3. Peek the stack")
 print("4. Display the stack")
 print("5. Exit ")
 opt = int(input("Enter a valid menu item ... "))
 return opt

```

**Program continues...**

```

if __name__ == "__main__":
 size = int(input("Enter the size of the Stack "))
 St = Stack(size)
 i=1
 while (i > 0 and i < 5):
 i = menu()
 if i == 1:
 St.push()
 St.display()
 elif i == 2:
 St.pop()
 St.display()
 elif i == 3:
 St.peak()
 elif i == 4:
 St.display()
 else:
 print("Exit")

```

## Output

```
Enter the size of the Stack 3
Empty Stack created

~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 1
Enter a number to push onto the stack 99
Top is at index -> 0
The elements in the Stack is
0 -> 99

~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 1
Enter a number to push onto the stack 88
Top is at index -> 1
The elements in the Stack is
1 -> 88
0 -> 99

~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 1
Enter a number to push onto the stack 77
Top is at index -> 2

Stack is Full

~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 1
Overflow: No more place for the Stack
Top is at index -> 2
```

Stack is Full

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit

Enter a valid menu item ... 3

The element at top is 77

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit

Enter a valid menu item ... 2

The element to pop is 77

Top is at index -> 1

The elements in the Stack is

1 -> 88

0 -> 99

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit

Enter a valid menu item ... 2

The element to pop is 88

Top is at index -> 0

The elements in the Stack is

0 -> 99

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit

Enter a valid menu item ... 2

The element to pop is 99

Top is at index -> -1

Stack is Empty

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack

```
4. Display the stack
5. Exit
Enter a valid menu item ... 5
Exit
```

### 5.4.4 Implementation of stack using queue module

#### Python program 5

```
import queue
def Push(s):
 if (not s.full()):
 data = input(("Enter a number to insert into the Stack "))
 s.put(data)
 else:
 print("There is NO SPACE in the Stack")
```

#### Program continues...

```
def Pop(s):
 if (not s.empty()):
 data=s.get()
 print("The Item Deleted is : ",data)
 else:
 print("There is NO DATA in the Stack")
```

#### Program continues...

```
def Display(s):
 if(s.full()):
 print("The Elements in the Stack are :")
 for i in s.queue:
 print(i)
 print("\nThe Stack is Full")
 elif(s.empty()):
 print("\nThe Stack is EMPTY")
 else:
 print("The Elements in the Stack are :")
 for i in s.queue:
 print(i)
```

#### Program continues...

```
def menu():
 print("\n~~~ MENU ~~~ ")
 print("1. Push into Stack ")
 print("2. Pop from Stack ")
```

```
print("3. Display the Stack")
print("4. Exit ")
opt = int(input("Enter a valid menu item ... "))
return opt
```

### Program continues...

```
if __name__ == "__main__":
 size = int(input("Enter the size of the Stack "))
 St = queue.LifoQueue(size)
 i=1
 while (i > 0 and i < 4):
 i = menu()
 if (i == 1):
 Push(St)
 Display(St)
 elif (i == 2):
 Pop(St)
 Display(St)
 elif (i == 3):
 Display(St)
 else:
 print("Exit")
```

### Output

```
Enter the size of the Stack 3

~~~ MENU ~~~
1. Push intoto Stack
2. Pop from Stack
3. Display the Stack
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Stack 99
The Elements in the Stack are :
99

~~~ MENU ~~~
1. Push intoto Stack
2. Pop from Stack
3. Display the Stack
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Stack 88
The Elements in the Stack are :
99
88
```



~~~ MENU ~~~

1. Push into Stack
2. Pop from Stack
3. Display the Stack
4. Exit

Enter a valid menu item ... 1

Enter a number to insert into the Stack 77

The Elements in the Stack are :

99

88

77

The Stack is Full

~~~ MENU ~~~

1. Push into Stack
2. Pop from Stack
3. Display the Stack
4. Exit

Enter a valid menu item ... 2

The Item Deleted is : 77

The Elements in the Stack are :

99

88

~~~ MENU ~~~

1. Push into Stack
2. Pop from Stack
3. Display the Stack
4. Exit

Enter a valid menu item ... 2

The Item Deleted is : 88

The Elements in the Stack are :

99

~~~ MENU ~~~

1. Push into Stack
2. Pop from Stack
3. Display the Stack
4. Exit

Enter a valid menu item ... 2

The Item Deleted is : 99

The Stack is EMPTY

~~~ MENU ~~~

1. Push into Stack
2. Pop from Stack
3. Display the Stack
4. Exit

Enter a valid menu item ... 3

The Stack is EMPTY

```

~~~ MENU ~~~
1. Push into Stack
2. Pop from Stack
3. Display the Stack
4. Exit
Enter a valid menu item ... 4
Exit

```

## 5.5 APPLICATIONS OF STACK

Stacks have various applications and are widely used in programming and data structures. In this section, we will discuss delimiter matching, infix-postfix conversion and evaluation of the postfix expression as examples of stacks.

### 5.5.1 Delimiter matching

Generally, a stack is very useful in situations when data have to be stored and retrieved in the reverse order. Let us consider the simple case where the delimiters in an arithmetic expression are '(', ')', '[', ']', '{', '}'. An arithmetic expression is said to be properly formed if the left delimiters and the corresponding right delimiters match. A Python program compiles correctly if and only if they match. Some examples where the delimiters match are

$$a = b + (c - d) + (e - f)$$

$$p[10] = h[(a + b)] + (j + k) - 1$$

A delimiter mismatch occurs in the following arithmetic expressions:

$$a = (b + c) - (d + e))$$

$$p[10] = h[(a + b) - (c + d)$$

A particular delimiter can be separated from its match by other delimiters. In other words, the first delimiter may be matched with the last delimiter; but this is done after the second delimiter is matched with the next to the last delimiter, etc. The delimiter matching algorithm reads a character from a Python program, then stores it in a stack if it is an opening delimiter. If it is a closing delimiter, it is compared with the delimiter popped off the stack. If they match, the processing continues. Otherwise the processing is discontinued and an error is signaled. The processing of the Python program ends successfully if the end of the program is reached and the stack is empty.

Let us write a simple algorithm to match delimiters in an arithmetic expression instead of the whole Python program.

**Algorithm delimiter matching (arithmetic expression)**

```
1. while (not end of expression)
2.   read a char ch from the expression
3.   if ( ch='(' or '[' or '{')
4.     push ch onto the stack
5.     continue // go to the beginning of the loop
6.   if (ch = ')' or ']' or '}')
7.     compare ch with top of stack element
8.     if they match pop the element from the stack
9.     continue
10.  else
11.    stop and report failure
12.  ignore all other characters
13.  end
```

Let us apply this algorithm to match the parentheses in the following expression:

$$a[5] + (b - \{c * (d - e) + (f - g)\})$$

See Table 5.4. The stack is empty and there is no more input to be read. Thus, the parentheses in the arithmetic expression match.

A simple program to illustrate use of stack for parentheses matching is given next. The program takes a string of parentheses (an arithmetic expression devoid of operands and operators) and checks for matching parentheses.

**Python program 6**

```
def isEmpty(top):
    if (top <= -1):
        return 1
    else:
        return 0

def push(Stack,i,top):
    top += 1
    S.append(i)
    return S,top

def pop(S,top):
    del S[top]
    return S,top-1

def peek(S,top):
    if (isEmpty(top) == 1):
        print("Stack is Empty: Mismatch")
    return 0
```

Table 5.4 Stack operation for delimiter matching

| Contents of the stack | Non-blank character read | Remaining exp                       |
|-----------------------|--------------------------|-------------------------------------|
| Empty                 | -                        | $a[5] + (b - \{c^*(d-e) + (f-g)\})$ |
| Empty                 | A                        | $[5] + (b - \{c^*(d-e) + (f-g)\})$  |
| Empty                 | [                        | $5] + (b - \{c^*(d-e) + (f-g)\})$   |
| [                     | 5                        | $] + (b - \{c^*(d-e) + (f-g)\})$    |
| [                     | ]                        | $+ (b - \{c^*(d-e) + (f-g)\})$      |
| Empty                 | +                        | $(b - \{c^*(d-e) + (f-g)\})$        |
| Empty                 | (                        | $b - \{c^*(d-e) + (f-g)\}$          |
| (                     | B                        | $- \{c^*(d-e) + (f-g)\}$            |
| (                     | -                        | $\{c^*(d-e) + (f-g)\}$              |
| (                     | {                        | $c^*(d-e) + (f-g)$                  |
| {{                    | C                        | $* (d-e) + (f-g)$                   |
| {{                    | *                        | $(d-e) + (f-g)$                     |
| {{                    | (                        | $d-e) + (f-g)$                      |
| {{{                   | D                        | $-e) + (f-g)$                       |
| {{{                   | -                        | $e) + (f-g)$                        |
| {{{                   | E                        | $) + (f-g)$                         |
| {{{                   | )                        | $+ (f-g)$                           |
| {{                    | +                        | $(f-g)$                             |
| {{                    | (                        | $f-g)$                              |
| {{{                   | F                        | $-g)$                               |
| {{{                   | -                        | $g)$                                |
| {{{                   | G                        | $)}$                                |
| {{{                   | )                        | $)}$                                |
| {{                    | }                        | $)$                                 |
| (                     | )                        | -                                   |
| Empty                 | -                        | -                                   |

```

else:
    return S[top]

if __name__ == "__main__":
    Expr=input("Enter an Expression ")
    S=[]
    top = -1

    for i in range(len(Expr)):
        if (Expr[i] == '('):
            S,top = push(S,Expr[i],top)

        elif(Expr[i] == '['):
            S,top = push(S,Expr[i],top)

        elif(Expr[i] == '{'):
            S,top = push(S,Expr[i],top)

```

```
elif(Expr[i] == ')'):  
    ele= peek(S,top)  
    if (ele == '(' and top != -1):  
        S,top = pop(S,top)  
    else:  
        break  
  
elif(Expr[i] == ']'and top != -1):  
    ele= peek(S,top)  
    if (ele == '['):  
        S,top = pop(S,top)  
    else:  
        break  
  
elif(Expr[i] == '}'and top != -1):  
    ele= peek(S,top)  
    if (ele == '{'):  
        S,top = pop(S,top)  
    else:  
        break  
  
else:  
    continue  
  
if (Expr != '' and top == -1 and ele != 0):  
    print("The Expression is well Parenthesized..")  
else:  
    print("Parenthesis Does not Matched..")
```

### Output 1

```
Enter an Expression a[5]+(b-{c*(d-e)+(f-g)})  
The Expression is well Parenthesized..
```

### Output 2

```
Enter an Expression (a+b)  
Parenthesis Does not Match..
```

## 5.5.2 Infix–postfix conversion

An arithmetic expression can be expressed in three ways:

1. The infix expression, which we commonly use:  
A + B is an infix expression where the operator occurs between operands.
2. The prefix expression where the operator precedes the operands:  
+AB is the prefix expression corresponding to the infix expression A+B.
3. The postfix expression where the operator follows the operands:  
AB+ is the corresponding postfix expression.

Postfix expressions are preferred by computers because they are free from operator precedence. However, we use infix expressions when writing a program, and for the ease of evaluations, these are converted to postfix expression by the computer.

A commonly agreed precedence of operators is in Table 5.5.

*Table 5.5 Precedence of operators with priority*

| <i>Operator</i>                               | <i>Priority</i> |
|-----------------------------------------------|-----------------|
| <i>*, unary -, not</i>                        | <i>4</i>        |
| <i>*, /, div, mod, and</i>                    | <i>3</i>        |
| <i>+, -, or</i>                               | <i>2</i>        |
| <i>&lt;, &lt;=, +, &lt; &gt;, &gt;=, &gt;</i> | <i>1</i>        |

The higher the priority, the higher is the precedence. The operators may be different in different languages. Notice that all relational operators have the same precedence.

When we have an expression where the adjacent operators have the same priority, we have to decide which one to evaluate first. If they are exponentiation operators, the evaluation takes place from right to left. For all other equal priority operators, the evaluation takes place from left to right. After the delimiter matching is done, for easy evaluation of an arithmetic expression, the computer converts the infix expression to a postfix expression using a stack.

### Example

Convert the infix expression  $A + B * C$  to a postfix expression (Table 5.6a).

*Table 5.6a Infix to postfix operation*

| <i>Token Read</i> | <i>Stack</i> | <i>Output</i> | <i>Comment</i>           |
|-------------------|--------------|---------------|--------------------------|
| None              | None         | None          | //initialise the stack   |
| A                 |              | A             | // if operand, output it |
| +                 | +            | A             | // if operator, stack it |
| B                 | +            | AB            | // operand: output it    |

The next token is `*`. We have to decide if this is to be pushed onto the stack or the operator `+` has to be popped and written to the output. Since `*` has higher precedence over `+` and has to be evaluated first, we push `*` onto the stack (Table 5.6b).

Table 5.6b Infix to postfix operation

|   |   |     |
|---|---|-----|
| * | + | AB  |
| C | + | ABC |

Since the input expression is exhausted, we output all the remaining elements on the stack and this gives the postfix expression as:

ABC\*+

**Example**

Convert the infix expression  $A/(B+C)*D$  to a postfix expression.

As in the previous example, we initialize a stack and read the tokens of the input infix expression from left to right (Table 5.7a).

Table 5.7a Infix to postfix operation

| Next token | Stack | Output |
|------------|-------|--------|
| None       | None  | None   |
| A          | None  | A      |
| /          | /     | A      |
| (          | /(    | A      |
| B          | /(    | AB     |
| +          | /(+   | AB     |
| C          | /(+   | ABC    |

When we read the next token, which is the right parenthesis ‘)’, we unstack down up to the left parenthesis and delete the left parenthesis from the stack (Table 5.7b).

Table 5.7b Infix to postfix operation

| Next token | Stack | Output  |
|------------|-------|---------|
| )          | /     | ABC+    |
| *          | /*    | ABC+    |
| D          | /*    | ABC+D   |
|            |       | ABC+D*/ |

When the input infix expression is exhausted, we output the contents of the stack and the postfix expression turns out to be

ABC+\*D\*

The following program in Python converts an infix expression to a postfix expression:

### Python program 7

```
def isEmpty(top):
    if (top <= -1):
        return 1
    else:
        return 0

def push(Stack,i,top):
    top += 1
    S.append(i)
    return S,top

def pop(S,top):
    i=S.pop(top)
    return S,top-1,i

def peek(S,top):
    if (isEmpty(top) == 1):
        print("Stack is Empty: Mismatch")
        return 0
    else:
        return S[top]

if __name__ == "__main__":
    Expr=input("Enter an Expression ")
    S=[]
    top = -1
    post = []
    operator = ['*','+','-','/','%']
    result = 0
    for i in range(len(Expr)):
        if ((Expr[i] == '(') or (Expr[i] in operator) ):
            S,top = push(S,Expr[i],top)

        elif(Expr[i] == ')'):
            ele= peek(S,top)
            while (ele != '(' and top != -1):
                S,top,item = pop(S,top)
                post.append(item)
            ele= peek(S,top)
            if (ele == '('):
                S,top,item = pop(S,top)
                result = 1
            else:
                result = 0
                break
```



```
elif (Expr[i] == '{' or Expr[i] == '}' or Expr[i] == '['
      or Expr[i] == ']'):
    result = 0
    break

else:
    post.append(Expr[i])

while(top != -1):
    S,top,item = pop(S,top)
    if (item == '('):
        result=0
        break
    post.append(item)
    result = 1

if (result == 1):
    print("The postfix expression is : ",end='')
    for i in post:
        print(i,end=' ')
else:
    print("Parenthesis NOT Matched..")
```

## Output

```
Enter an Expression A/(B+C)*D
The postfix expression is : A B C + D * /
```

### 5.5.3 Evaluation of the postfix expression

After checking for matching delimiters, the infix expression was converted into a postfix expression. The system now evaluates the postfix expression using a stack.

#### Algorithm for evaluation of a postfix expression

1. Initialize a stack
2. Read the next-token from the post-fix expression
3. If it is an operand push it on to the stack
4. If it is an operator pop required number of operands from the stack
5. Apply the operator to the operands and push the result on to the stack
6. If it is “#” indicating end of the stack
7. Pop the result
8. Else go to step 2
9. End.

#### Example

Evaluate the postfix expression 1 3 5 \*+

Table 5.8 Evaluate the postfix expression

| Next token | Stack | Remaining expression    |
|------------|-------|-------------------------|
| -          | -     | 1 3 5 *+##              |
| 1          | 1     | 3 5 *+                  |
| 3          | 1 3   | 5*+                     |
| 5          | 1 3 5 | *+                      |
| *          | 1 15  | + //result = 3*5 = 15   |
| +          | 16    | # // result = 1+15 = 16 |

See Table 5.8. Now we have reached the end of expression, so pop the contents of the stack. And the value of the postfix expression is 16.

A simple program in Python is given next to achieve this.

### Python program 8

```
def isEmpty(top):
    if (top <= -1):
        return 1
    else:
        return 0

def push(S,i,top):
    top += 1
    S.append(i)
    return S,top

def pop(S,top):
    i = S.pop(top)
    return S,top-1,i
```

The functions `isEmpty()`, `push()` and `pop()` are as defined earlier.

### Python program 9

```
def calculate(op1,op2,i):
    if i is '*':
        return float(op1)*float(op2)
    elif i is '/':
        return float(op1)/float(op2)
    elif i is '+':
        return float(op1)+float(op2)
    elif i is '-':
        return float(op1)-float(op2)
    else:
        return float(op1)%float(op2)
```

The `calculate(op1,op2,i)` function takes three parameters. The first two are two operands and the third one is an operator. Depending on the character received by the operator, the operation is performed.

**Program continues...**

```
if __name__ == "__main__":
    Expr=input("Enter an Expression ")
    S=[]
    top = -1
    operator = ['*','+','-','/','%']
    for i in range(len(Expr)):
        result = 0
        if (Expr[i] in operator):
            S,top,item = pop(S,top)
            S,top,item1 = pop(S,top)
            result= str(calculate(item1,item,Expr[i]))
            S,top=push(S,result,top)
        elif(Expr[i] in '0123456789'):
            S,top=push(S,Expr[i],top)
        else:
            pass

    S,top,item = pop(S,top)
    print("The result of postfix expression ",Expr, "
        is ", item)
```

The main program accepts a postfix expression in the variable `Expr`. Initialize an empty list `S` that is used as a stack. And a variable `top` is initialized to -1. All the arithmetic operators are listed in a list with variable name `operator`.

Read each character of the expression from left to right until the end of the expression. If an numeric value appeared, we pushed it onto the stack. If an operator appeared, we pop top two elements from the stack, perform the calculation and again push the result onto the stack. When the last element of the expression is reached, pop the element from the stack and show the result.

## Output

```
Enter an Expression 532+/8*
The result of postfix expression 532+/8* is 8.0
```

### 5.5.4 Recursion

Let us first understand what happens when a function is called. If the function has formal parameters, they are initialized with the actual parameters

passed to the function. In addition, the system has to know where to resume the execution of the program after the completion of the function call. In other words, the system has to remember where the function has been called from. This could be done by storing the return address in main memory, set aside to store return addresses. However, we do not know, in advance, how much space might be needed. Moreover, more information might be needed to be stored, besides the return address. Therefore, dynamic allocation using a run-time stack might be a better solution. For example, the local variables need to be stored. If a function *f* has a local variable *x* and it calls a function *g* that has a local variable also named *x*, the system must be able to distinguish them. Thus the state of each function including *main()* is characterized by the parameters passed to the function, its local variables and the return address from where we start further execution of the calling function. The data record that contains all this information is called the activation record and is stored on a run-time stack. This activation record exists as long as the corresponding function is executing. The activation record of a called function disappears before the activation record of the calling function disappears. Thus, the activation record of the *main()* function outlives all other activation records.

The activation record of any function contains the following information:

1. Values of the parameters of the function
2. Information about local variables
3. The return address
4. A pointer, called SP (stack pointer), to the callers activation record
5. The returned value of the function that is not declared as void

The activation record of the executing function is at the top of the stack. For example, if the main function calls *f1()*, *f1()* calls *f2()* and *f2()* calls *f3()*, the run-time stack, when *f3()* is being executed is as in Figure 5.11.

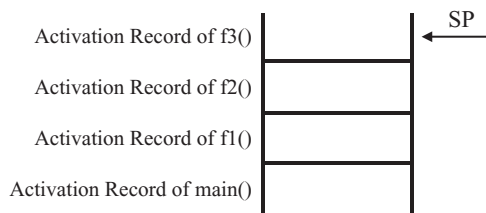


Figure 5.11 Activation record

Now, if we call a function that has the same name as the calling function, we call it recursion. To put it more precisely, it is an instantiation of a function calling another instantiation of the same function. These invocations

of the function are represented by different activation records. Let us take a simple example using a program written in Python:

### Python program 10

```
def rev():
    i=0
    ch=input()
    if (ch != 'z'):
        rev()
    print(ch)

if __name__ == "__main__":
    rev()
```

### Output

```
A
b
c
z
z
c
b
a
```

When the `rev()` function is called an activation record is created with the variable 'ch' and the return address. There is no need to store the result since no value is returned. And this is indicated by prefixing the function with void. The function `rev()` reads the first character and the contents of the run-time stack just before `rev()` calls itself for the first time (Figure 5.12a).

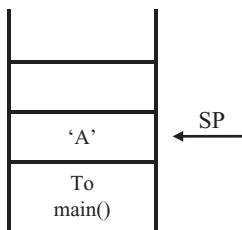


Figure 5.12a Contents of the run-time stack

The second character is read and after checking that it is not 'z', `rev()` is called again which reads character 'b'. Now the activation record for the second call is pushed onto the run-time stack (Figure 5.12b).

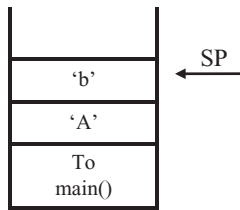


Figure 5.12b Contents of the run-time

In a similar manner, all the elements are pushed onto the stack and finally when 'z' is read the run-time stack is as shown in Figure 5.12c.

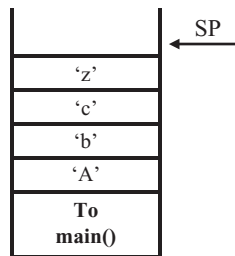


Figure 5.12c Contents of the run-time stack

When the system finds the end of line character, no other statement is executed. It then retrieves the return address from the activation record and discards this record by decrementing the stack pointer by the proper number of bytes.

Since the activation record of the fourth call to `rev()` is active, the value of `ch`, which is 'z', is output as the first character and the activation record for the fourth call is discarded by decrementing the stack pointer. 'c' is assigned to `ch` and then printed. Finally the activation record of the first call is reached and A is printed and what is seen on the screen is "zcbA".

Let us consider a non-recursive version of reversing a string:

### Python program 11

```
if __name__ == "__main__":
    Expr=input("Enter an Expression ")
    S=[]
    top = -1
    i=0
    while(i < len(Expr)):
        S,top = push(S,Expr[i],top)
        i+=1
```

1. The best data structure to check whether a arithmetic expression is well parenthesized or not is a
  - a. Queue
  - b. Stack
  - c. Tree
  - d. List
2. A single array  $A[1.. \text{Max}]$  is used to implement two stacks. The two stacks grow from opposite ends of the array. Variables  $\text{top1}$  and  $\text{top2}$  ( $\text{top1} > \text{top2}$ ) point to the location of the topmost element in each

stack. If the space is to be efficiently used, the condition for stack full is

- a.  $(\text{top1} = \text{Max}/2) \text{ and } (\text{top} = \text{max}/2+1)$
  - b.  $\text{top1} + \text{top2} = \text{Max}$
  - c.  $(\text{top1} = \text{Max}/2) \text{ and } (\text{top} = \text{max})$
  - d.  $\text{top1} = \text{top2} - 1$
3. A linear set of elements in which deletion can be done from one end (*front*) and insertion can take place only at the other end is known as a
- a. Queue
  - b. Stack
  - c. Tree
  - d. Branch
4. The end at which a new element gets added to a queue is called the
- a. Front
  - b. Rear
  - c. Top
  - d. Bottom
5. The advantage of recursion is that the
- a. Code size is less.
  - b. Time complexity is less.
  - c. Space complexity is less.
  - d. None of the above.

6. Consider the following code:

```
def f(n):
    i=1
    if (n >= 5):
        return n
    else:
        n = n + i
        i += 1
        return (f(n))
```

What would be the value returned by  $f(1)$ ?

- a. 5
  - b. 6
  - c. 7
  - d. 8
7. Any recursive function can be converted into an equivalent non-recursive function
- a. Always
  - b. Never
  - c. Sometimes
  - d. If the function is tail recursive



8. The Fibonacci function  $\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$  is an example of
- Direct recursion
  - Tree recursion
  - Linear recursion
  - Both a and b
9. With what data structure can reversing of string can be easily implemented?
- Array
  - List
  - Stack
  - All of the above
10. Find the output of the sample code.
- ```
def fun(n)
    Stack S // Say it creates an empty stack S
    while(n > 0):
        // This line pushes the value of n%2 to stack S
        push(S, n%2)
        n = int(n/2)
    // Run while Stack S is not empty
    while(!isEmpty(S)):
        print(pop(S)); // pop an element from S and print it
```
- Prints the value of  $\log n$
  - Prints the binary representation of  $n$
  - Prints the reverse of the number
  - All of the above
11. The following postfix expression with single-digit operands is evaluated using a stack:  $8\ 2\ 3\ ^\wedge\ /\ 2\ 3\ *\ +\ 5\ 1\ *\ -$
- Note that  $^\wedge$  is the exponentiation operator. The top two elements of the stack after the first  $*$  is evaluated are:
- 1,5
  - 3,2
  - 5,7
  - 6,1
12. If the elements A, B, C and D are placed in a stack and are deleted one at a time, in what order will they be removed?
- ABCD
  - DCBA
  - DCAB
  - ADBC
13. In linked list implementation of a queue, where can a new element be inserted?
- At the head of link list
  - At the tail of the link list
  - At the center position in the link list
  - None of the above

14. The following operations are performed on a stack: push(A), push(B), push(C), pop(), pop(), push(D), pop(), pop(). What is on the top of the stack?

- |      |         |
|------|---------|
| a. A | c. D    |
| b. C | d. NULL |

15. When several jobs arrive at the printer, the jobs join a

- |            |                      |
|------------|----------------------|
| a. Queue   | c. Stack             |
| b. Dequeue | d. None of the above |

16. A single array A[1..MAXSIZE] is used to implement two stacks. The two stacks grow from opposite ends of the array. Variables top1 and top2 (top1 < top2) point to the location of the topmost element in each of the stacks. If the space is to be used efficiently, the condition for "stack full" is

- |                                                                                              |                                                                                  |
|----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| a. $\text{top1} + \text{top2} = \text{MAXSIZE}$                                              | c. $\text{top1} = \text{top2} - 1$                                               |
| b. $\text{C} (\text{top1} = \text{MAXSIZE}/2)$<br>and $(\text{top2} = \text{MAXSIZE}/2 + 1)$ | d. $(\text{top1} = \text{MAXSIZE}/2) \text{ or } (\text{top2} = \text{MAXSIZE})$ |

17. Assume that the operators +, -,  $\times$  are left associative and  $\wedge$  is right associative. The order of precedence (from highest to lowest) is  $\wedge$ ,  $\times$ , +, -. The postfix expression corresponding to the infix expression  $a + b \times c - d \wedge e \wedge f$  is

- |                                          |                                        |
|------------------------------------------|----------------------------------------|
| a. $- + a \times bc \wedge \wedge def$   | c. $abc \times + de \wedge f \wedge -$ |
| b. $ab + c \times d - e \wedge f \wedge$ | d. $abc \times + def \wedge \wedge -$  |

18. The priorities of arithmetic operators +, \*,  $\wedge$ , / can be

- |               |               |
|---------------|---------------|
| a. 5, 1, 2, 9 | c. 1, 2, 5, 2 |
| b. 9, 5, 2, 1 | d. 5, 3, 3, 4 |

19. A normal stack of size MAX gets full when

- |                                  |                                                            |
|----------------------------------|------------------------------------------------------------|
| a. $\text{Top} = \text{MAX} - 1$ | c. $\text{Top} = -1$                                       |
| b. $\text{Top} = \text{NULL}$    | d. $\text{Top} = (\text{No of element} + 1) \% \text{MAX}$ |

20. A stack can be implemented using a

- |               |                     |
|---------------|---------------------|
| a. Dictionary | c. List             |
| b. Class      | d. All of the above |

**Descriptive questions**

1. Construct a stack pushing the elements 3 5 7 6 8 in that order. Find the status of the stack after the following operations:  
Push(9), pop(),pop(),pop(),push(10)
2. Convert the decimal number 24 to its binary equivalent using stack.
3. Convert  $Z - ((X * ((X + Y/J - 2)) + Y)/3)$  into a postfix expression.
4. Reverse the string ABCDFEGH using a stack.
5. Evaluate the following postfix expressions.
  - a. 70 14 4 5 15 3 /\* - - /6+
  - b. 3 5 6 \*+13-18 2/+
6. Write the equivalent infix expression for the following postfix expressions.
  - a. AB+CD-\*
  - b. AB-C-D\*
7. Transform the following infix expression into the equivalent infix expression: +A-BC

**Answers to true-false questions**

1. True
2. False
3. True
4. True
5. True

**Answers to fill-in-the-blank questions**

1. pop()
2. push()
3. -1
4. Same
5. False

**Answers to multiple-choice questions**

- |     |   |     |   |     |   |     |   |     |   |
|-----|---|-----|---|-----|---|-----|---|-----|---|
| 1.  | b | 2.  | d | 3.  | a | 4.  | b | 5.  | a |
| 6.  | a | 7.  | a | 8.  | d | 9.  | c | 10. | b |
| 11. | d | 12. | b | 13. | b | 14. | d | 15. | a |
| 16. | c | 17. | d | 18. | c | 19. | a | 20. | d |

# Queues

---

### LEARNING OBJECTIVES

After studying this chapter, readers will be able to:

- Understand queue and its operations
- Implementation of queue using list
- Implementation of queue using dictionary
- Implementation of queue using class
- Implementation of queue using queue module
- Understand circular queue
- Implementation of circular queue using class
- Understand double-ended queue
- Implementation of double-ended queue
- Understand a priority queue
- Implementation of priority queue using class
- Implementation of priority queue by importing PriorityQueue package of queue module
- Implementation of priority queue using heapq module

Linear data structures are divided into arrays, stacks and queues. In this chapter we will focus on the queue as a linear data structure. Queue data structures can be implemented using list, dictionary, class and the queue module. Simple examples of a queue are waiting for a bus in a bus stand and jobs that are taken for printing by a printer.

### 6.1 QUEUE

A queue is simply a waiting line found at railway stations, airports, cinema halls, etc. It grows when people join the queue and shrinks when people leave the queue. Unlike a stack, a queue is a data structure in which both ends are used: one for adding new elements and the other for removing them. The end where the elements join the queue is called the *rear* and the

end from where the elements are removed is called the *front*. A queue follows the FIFO (first in, first out) discipline (Figure 6.1).

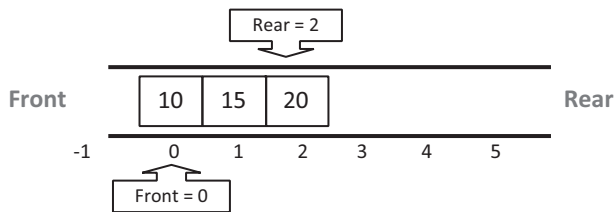


Figure 6.1 Sample queue

## 6.2 QUEUE OPERATIONS

The operations in Table 6.1 are used to manage a queue.

Table 6.1 Queue operations

| Operation                | Meaning                                                    |
|--------------------------|------------------------------------------------------------|
| <code>isEmpty()</code>   | Checks if the queue is empty                               |
| <code>isFull()</code>    | Checks if the queue is full                                |
| <code>enqueue(el)</code> | Put the element at the end of the queue                    |
| <code>dequeue()</code>   | Removes the first element in the queue                     |
| <code>first()</code>     | Returns the first element in the queue without removing it |

A series of enqueue and dequeue operations are shown in Figures 6.2–6.7e.

### 6.2.1 Enqueue (enter data into a queue)

The *enqueue(Q)* operation inserts an element at the rear of the queue *Q*. It follows the following steps:

1. Let *rear* be the index of the last element.
2. Check if the queue is full (if it is a restricted queue).
3. If the queue is not full:
  - a. Increment *rear* by one.
  - b. Assign the data at the *rear* index of queue.
  - c. Return queue.
4. Else report failure (*queue overflow*).

The maximum size of a Python list on a 32 bit system is around 536,870,912 elements. Since we are using a list to implement a queue, the condition *isFull()* is not required. If we implement a restricted list (a list with a

predefined size), then *isFull()* is required to be implemented. We increment the ‘top’ by one and insert  $Q[\text{rear}]$  into the queue (Figures 6.2 to 6.6).

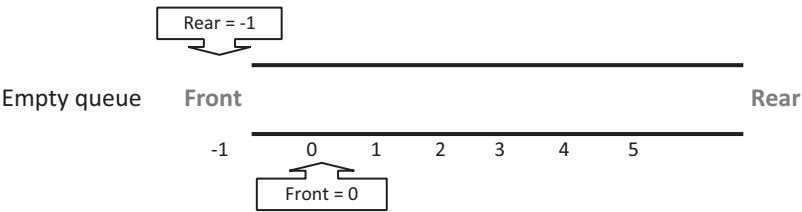


Figure 6.2 Empty queue

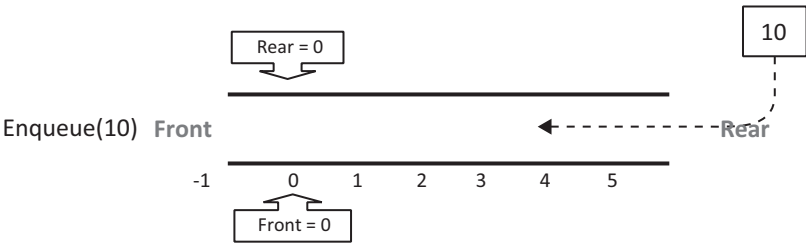


Figure 6.3 Before inserting 10 into the queue

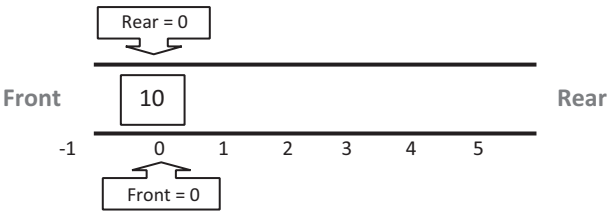


Figure 6.4 After inserting 10 into the queue

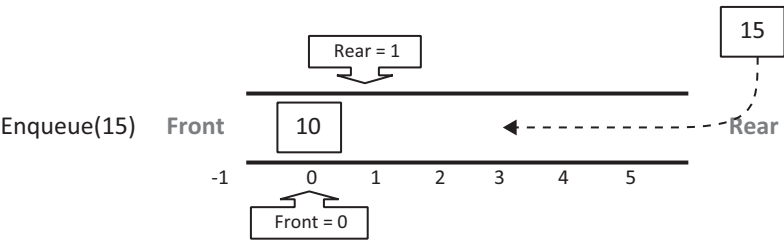


Figure 6.5 Before inserting 15 into the queue

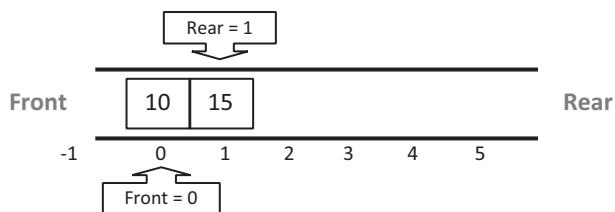


Figure 6.6 After inserting 15 into the queue

### 6.2.2 Dequeue (delete data from a queue)

We execute the following steps to perform the delete operation (Figures 6.7a to 6.7e):

1. Let  $x$  be the variable to store the deleted element.
2. If the queue is not empty:
  - a. Set  $x$  = front index of the queue.
  - b. Increment the front by one.
  - c. Return queue.
3. Else report failure (*queue underflow*).

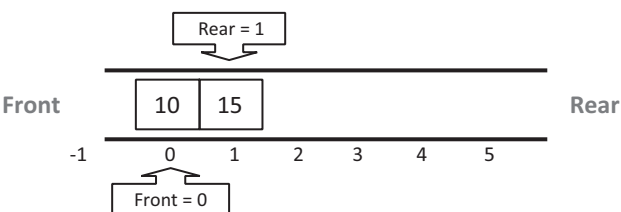


Figure 6.7a Initial queue

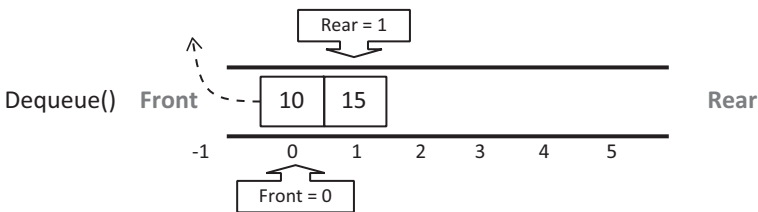


Figure 6.7b Before deleting 10 from the queue

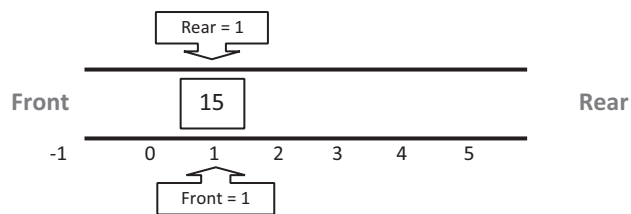


Figure 6.7c After deleting 10 from the queue

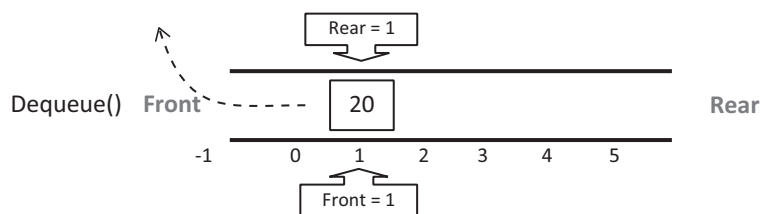


Figure 6.7d Before deleting 15 from the queue

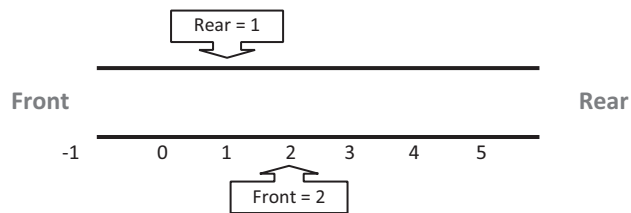


Figure 6.7e After deleting 15 from the queue

Since elements are entered at one end and removed from the other end, both ends are to be monitored.

Consider the ordered list  $L = [a_1, a_2, a_3, \dots, a_n]$ . If we assume  $L$  represents a queue,  $a_1$  is in the front end,  $a_n$  is at the rear and  $a_i$  is behind  $a_{i-1}$ .

Queues are very important data structures in time-sharing and distributed network systems where the users try to access the system resources simultaneously.

### 6.3 QUEUE (ADT)

The following are the operations and the set of axioms on the data structure queue.



Operations: See Table 6.2.

Table 6.2 Queue ADT operations

| <i>Operation</i>            | <i>Meaning</i>                                          |
|-----------------------------|---------------------------------------------------------|
| Create()                    | Creates an empty queue                                  |
| Add( <i>el</i> , <i>Q</i> ) | Adds the element “ <i>el</i> ” to the queue             |
| Delete( <i>Q</i> )          | Deletes the element from the front                      |
| getFront()                  | Outputs the front element without removing it           |
| isEmpty()                   | Returns “true” if the queue is empty; “false” otherwise |
| isFull()                    | Returns “true” if the queue is full; “false” otherwise  |

Axioms: See Table 6.3.

Table 6.3 Queue ADT axioms

| <i>Axiom</i>                          | <i>Returned value</i>                                       |
|---------------------------------------|-------------------------------------------------------------|
| isEmpty(create())                     | True                                                        |
| isEmpty(Add( <i>el</i> , <i>Q</i> ))  | False                                                       |
| deleteQ(create())                     | Error                                                       |
| getFront(create())                    | Error                                                       |
| getFront(Add( <i>el</i> , <i>Q</i> )) | <i>el</i> if isEmpty( <i>Q</i> ); else getFront( <i>Q</i> ) |

Axiom 1 says that when we create a queue, it is initially empty. However, after adding an element to a queue, obviously the queue is not empty, which is indicated by axiom 2. We cannot delete an element from an empty queue and thus deleteQ(create()) returns an error and so is the case with getFront(create()). Axiom 5 is obvious since when we add an element it joins the queue in the rear and thus getFront(Add(*el*,*Q*)) is the same as getFront(*Q*) unless *el* is added to an empty queue, in which case rear = front.

## 6.4 IMPLEMENTATION

There are various ways to implement a queue. It can be implemented using a list, dictionary or a class in Python.

### 6.4.1 Implementation using a list

The elements of the queue are the elements of a list. As a list is not a fixed-size data structure, while implementing, the queue is also not fixed size. But

we can restrict the size of the list, hence the size of the queue can be made fixed.

Now let us implement a queue using a list without a restriction of upper limit or the size of the queue.

### Python program 1

```
# To create an empty queue
def create():
    Q=[]
    front = 0
    rear = -1
    print("Empty Queue created")
    return Q,rear,front
```

The create() method creates an empty queue with the front pointing to 0 and rear pointing to -1. Here Q = [] indicates an empty list.

### Program continues...

```
# To check if the queue is empty or not
def isEmpty(front,rear):
    if (front > rear or rear == -1):
        front = 0
        rear = -1
        return 1
    else:
        return 0
```

The isEmpty() method checks if the queue is empty. It checks if the front is greater than the rear or the rear is at -1. It returns 1 to indicate the queue is empty; otherwise it returns 0 to indicate the queue is not empty (at least one element is there in the queue).

### Program continues...

```
# To insert an element into a queue
def enqueue(Q,rear):
    no = int(input(("Enter a number to push onto
        the Queue ")))
    rear += 1
    Q.append(no)
    return Q,rear
```

The enqueue() method takes two parameters: the queue and the rear. Every time an element is inserted into the queue, the rear is incremented by one and the element is appended to the list. As a list is a heterogeneous data structure, we force the user to enter an integer only.

**Program continues...**

```
# To delete an element from a queue
def dequeue(Q, front, rear):
    if (isEmpty(front, rear) == 1):
        front = 0
        rear = -1
        print("Underflow: There is no element in the Queue")
        return Q, front, rear
    else:
        print("The element to pop is ", Q[0])
        return Q, front+1, rear
```

The dequeue() method takes three parameters: the queue, the front and the rear. Every time an element is deleted from the queue, it checks if the queue is empty. If the queue is empty, it shows an appropriate message, and if the queue is not empty, then the front of the queue is printed and the front is decremented by one. At last, the front and the rear of the queue is returned.

**Program continues...**

```
#To display elements of a queue
def display(S, front, rear):
    print("Front is at index -> ", front)
    print("Rear is at index -> ", rear)
    if (isEmpty(front, rear) == 1) :
        print("\nQueue is Empty")
    else:
        print("The elements in the Queue is")
        for i in range(front, rear+1):
            print(i, "->", Q[i])
```

The display() method takes three parameters: the queue, the front and the rear. This function prints the elements in the queue (the element inserted first is printed first).

**Program continues...**

```
def menu():
    print("\n~~~ MENU ~~~ ")
    print("1. Insert to Queue ")
    print("2. Delete from Queue ")
    print("3. Display the Queue")
    print("4. Exit ")
    opt = int(input("Enter a valid menu item ... "))
    return opt
```

The menu() method shows a menu to perform the operations defined for a queue. It returns a choice of operation to perform.

### Program continues...

```
if __name__ == "__main__":
    Q,rear,front=create()
    i=1
    while (i > 0 and i <=3 ):
        i = menu()
        if i == 1:
            Q,rear = enqueue(Q,rear)
            display(Q,front,rear)
        elif i == 2:
            Q,front,rear = deque(Q,front,rear)
            if (front > rear):
                front = 0
                rear = -1
            if (type(front) != str):
                display(Q,front,rear)
        elif i == 3:
            display(Q,front,rear)
        else:
            print("Exit")
```

### Output

Empty Queue created

~~~ MENU ~~~

1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit

Enter a valid menu item ... 1

Enter a number to push onto the Queue 10

Front is at index -> 0

Rear is at index -> 0

The elements in the Queue is

0 -> 10

~~~ MENU ~~~

1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit

Enter a valid menu item ... 1

Enter a number to push onto the Queue 20

Front is at index -> 0

```
Rear is at index -> 1
The elements in the Queue is
0 -> 10
1 -> 20

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to push onto the Queue 30
Front is at index -> 0
Rear is at index -> 2
The elements in the Queue is
0 -> 10
1 -> 20
2 -> 30

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The element to pop is 10
Front is at index -> 1
Rear is at index -> 2
The elements in the Queue is
1 -> 20
2 -> 30

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The element to pop is 10
Front is at index -> 2
Rear is at index -> 2
The elements in the Queue is
2 -> 30

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The element to pop is 10
```

```

Front is at index -> 0
Rear is at index -> -1

Queue is Empty

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 4
Exit

```

Now let us implement a queue using a list with a restriction of an upper limit or the size of the queue. There is no change in the `create()`, `dequeue()`, `isEmpty()` and `menu()` functions and some changes in the `enqueue()` and `display()` functions. A new function `isFull()` is also introduced. We are discussing only those functions that have some changes to Python program 1.

## Python program 2

```

To check the Queue is full
def isFull(rear,front,size):
 if (rear >= size-1):
 return 1
 else:
 return 0

```

The `isFull()` method checks if the queue is full. It checks if the  $(\text{rear} - \text{front} + 1)$  is greater than the  $(\text{size} - 1)$ . It returns 1 to indicate the queue is full; otherwise returns 0 to indicate the queue is not full.

## Program continues...

```

To insert an element into a queue
def enqueue(Q,rear,front,size):
 if (isFull(rear,front,size) == 1):
 print("Overflow: No more place for the Queue")
 return Q,rear
 no = int(input(("Enter a number to insert into
the Queue ")))
 rear += 1
 Q.append(no)
 return Q,rear

```

The `enqueue()` method takes four parameters: the queue, the front, the rear and the size. Every time an element is inserted into the queue, it checks if the queue is full. If the queue is full, it shows an appropriate message; otherwise the rear is incremented by one and the element is appended to the list.

**Program continues...**

```
To display the elements of a Queue
def display(Q,front,rear):
 print("Front is at index -> ",front)
 print("Rear is at index -> ",rear)
 if (isEmpty(front,rear) == 1) :
 print("\nQueue is Empty")

 elif(isFull(rear,front,size)==1):
 for i in range(front,rear+1):
 print(i, "->",Q[i])
 print("\nQueue is Full")

 else:
 print("The elements in the Queue is")
 for i in range(front,rear+1):
 print(i, "->",Q[i])
```

The display() method takes three parameters: the queue, the front and the rear. This function prints the elements in the queue (the element inserted first is printed first) if the queue is neither full nor empty.

**Output**

```
Enter the size of the Queue 3
Empty Queue created

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 10
Front is at index -> 0
Rear is at index -> 0
The elements in the Queue is
0 -> 10

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 20
Front is at index -> 0
Rear is at index -> 1
```

The elements in the Queue is

0 -> 10

1 -> 20

~~~ MENU ~~~

1. Insert to Queue

2. Delete from Queue

3. Display the Queue

4. Exit

Enter a valid menu item ... 1

Enter a number to insert into the Queue 30

Front is at index -> 0

Rear is at index -> 2

0 -> 10

1 -> 20

2 -> 30

Queue is Full

~~~ MENU ~~~

1. Insert to Queue

2. Delete from Queue

3. Display the Queue

4. Exit

Enter a valid menu item ... 2

The element to delete is 10

Front is at index -> 1

Rear is at index -> 2

1 -> 20

2 -> 30

Queue is Full

~~~ MENU ~~~

1. Insert to Queue

2. Delete from Queue

3. Display the Queue

4. Exit

Enter a valid menu item ... 1

Overflow: No more place for the Queue

Front is at index -> 1

Rear is at index -> 2

1 -> 20

2 -> 30

Queue is Full

~~~ MENU ~~~

1. Insert to Queue

2. Delete from Queue

3. Display the Queue



```
4. Exit
Enter a valid menu item ... 2
The element to delete is 10
Front is at index -> 2
Rear is at index -> 2
2 -> 30

Queue is Full

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The element to delete is 10
Front is at index -> 0
Rear is at index -> -1

Queue is Empty

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 4
Exit
```

### 6.4.2 Implementation of queue using a dictionary

Now let us implement a queue using a dictionary with a restriction of upper limit or the size of the queue. We already know that a dictionary is a key-value pair. We are implementing the queue with size restriction.

#### Python program 3

```
To create an empty queue
def create():
 Q={}
 front = 0
 rear = -1
 print("Empty Queue created")
 return Q,front,rear
```

In the create() method, we have created an empty dictionary Q={} and assigned the front to 0 and rear to -1.

**Program continues...**

```
To check the queue is Full
def isFull(front,rear,size):
 if (rear >= size-1):
 return 1
 else:
 return 0
```

Just like the previous program, the *isFull()* function checks if the queue is full. When the rear is greater than or equal to (size -1), it returns 1 indicating the queue is full; otherwise it returns 0 indicating the queue is not full.

**Program continues...**

```
To check the queue is Empty
def isEmpty(front,rear):
 if (front > rear or rear == -1):
 return 1
 else:
 return 0
```

The *isEmpty()* function checks if the queue is empty. It returns 1, if the front is greater than rear or rear = -1. It is exactly the same as the previous Python program.

**Program continues...**

```
To insert into the queue
def enqueue(Q,front,rear,size):
 if (isFull(front,rear,size) == 1):
 print("Overflow: No more place in the Queue")
 else:
 no = int(input(("Enter a number to insert into the Queue ")))
 rear += 1
 Q[rear] = no
 return Q,rear
```

The *enqueue()* function inserts an element into a queue. It takes four parameters – the queue, the front, the rear and the size – and returns the queue and rear. The working is the same as the Python program 2 described earlier.

**Program continues...**

```
To delete from the queue
def dequeue(Q, front, rear):
 if (isEmpty(front, rear) == 1):
 print("Underflow: There is no element in the Queue")

 else:
 print("The element to delete is ", Q[front])
 del Q[front]
 front += 1
 return Q, front
```

The `dequeue()` function deletes an element. If there are no elements in the queue, it returns an appropriate message. Otherwise, it deletes the element that is at the front and increments the front by one and returns the queue and the front.

**Program continues...**

```
To display the elements of the queue
def display(Q, rear, size):
 print("Front is at index -> ", front)
 print("Rear is at index -> ", rear)
 if (isEmpty(front, rear) == 1) :
 print("\nQueue is Empty")

 elif (isFull(front, rear, size) == 1):
 print("The elements in the Queue is")
 for i in list(Q.keys()):
 print(i, "->", Q[i])
 print("\nQueue is Full")

 else:
 print("The elements in the Queue is")
 for i in list(Q.keys()):
 print(i, "->", Q[i])
```

The `display()` function shows the elements in the queue. If `isEmpty()` is true, it prints "Queue is empty". If `isFull()` is true, it prints all the elements of the queue; otherwise, it prints the elements in the queue.

**Program continues...**

```
def menu():
 print("\n~~~ MENU ~~~ ")
 print("1. Insert into Queue ")
 print("2. Delete from Queue ")
 print("3. Display the Queue")
```

---

```

print("4. Exit ")
opt = int(input("Enter a valid menu item ... "))
return opt

```

### Program continues...

```

if __name__ == "__main__":
 size = int(input("Enter the size of the Queue "))
 Q, front, rear = create()
 i = 1
 while (i > 0 and i < 4):
 i = menu()
 if i == 1:
 Q, rear = enqueue(Q, front, rear, size)
 display(Q, rear, size)
 elif i == 2:
 Q, front = dequeue(Q, front, rear)
 if (front > rear):
 front = 0
 rear = -1
 display(Q, rear, size)
 elif i == 3:
 display(Q, rear, size)
 else:
 print("Exit")

```

The output of the program is the same as the previous program.

## 6.4.3 Implementation of queue using a class

Let us implement queue using a class.

### Python program 4

```

Creating an Queue
class Queue:
 def __init__(self, size):
 self.maxSize = size
 self.front = 0
 self.rear = -1
 self.Q = list()
 print("\nEmpty Queue Created")

```

The initializer takes the object and the size as the parameter to create an empty queue. It assigns front = 0 and rear = -1. It creates:

**Program continues...**

```
def isFull(self):
 #if (self.rear-self.front >= self.maxSize-1):
 if (self. rear >= self.maxSize-1):
 return 1
 else:
 return 0
```

The method isFull() checks if the queue is full. It returns 1 if the queue is full and 0 otherwise.

**Program continues...**

```
def isEmpty(self):
 if (self.front == self.rear or self.rear == -1):
 return 1
 else:
 return 0
```

The method isEmpty() checks if the queue is empty. It returns 1 if the queue is empty and 0 otherwise.

**Program continues...**

```
def enqueue(self):
 if (self.isFull() == 1):
 print("Overflow: No more place for the Queue")
 else:
 no = int(input(("Enter a number to insert into the
 Queue ")))
 self.rear += 1
 self.Q.append(no)
```

The enqueue() method enters an element into the queue. If the queue is not full, increment the rear by one and append the new element to the queue.

**Program continues...**

```
def dequeue(self):
 if (self.isEmpty()==1):
 print("Underflow: There is no element in the Queue")
 else:
 print("The element to Delete is ",self.Q[self.front])
 self.front += 1
```

The dequeue() method deletes an element from the queue. If the queue is not empty, increment the front by one.

**Program continues...**

```

def display(self):
 print("Front is at index -> ",self.front)
 print("Rear is at index -> ",self.rear)
 if(self.isEmpty()==1):
 print("\nQueue is Empty")

 elif(self.isFull()==1):
 print("The elements in the Queue are")
 for i in range(self.front,self.rear+1):
 print(i, "->",self.Q[i])
 print("\nQueue is Full")

 else:
 print("The elements in the Queue is")
 for i in range(self.front,self.rear+1):
 print(i, "->",self.Q[i])

```

The display() method shows all the elements in the queue.

**Program continues...**

```

def menu():
 print("\n~~~ MENU ~~~ ")
 print("1. Insert into Queue ")
 print("2. Delete from Queue ")
 print("3. Display the Queue")
 print("4. Exit ")
 opt = int(input("Enter a valid menu item ... "))
 return opt

```

**Program continues...**

```

if __name__ == "__main__":
 size = int(input("Enter the size of the Queue "))
 Qu = Queue(size)
 i=1
 while (i > 0 and i < 4):
 i = menu()
 if i == 1:
 Qu.enqueue()
 Qu.display()
 elif i == 2:
 Qu.dequeue()
 if (Qu.front > Qu.rear):
 Qu.front = 0
 Qu.rear = -1
 print()

```

```
 Qu.display()
elif i == 3:
 Qu.display()
else:
 print("Exit")
```

The output of the program is the same as the earlier program.

#### 6.4.4 Implementation of queue using queue module

The queue module in Python can be used to generate a queue object. This is part of the Python standard library.

##### Python program 5

```
from queue import Queue
def enqueue(q):
 if (not q.full()):
 data = int(input(("Enter a number to insert into the Queue ")))
 q.put(data)
 else:
 print("There is NO SPACE in the Queue")
```

The enqueue() function first checks if the queue is full. This is done by the built-in method q.full(). Once this queue is full, insertion cannot be performed until all queue items have been deleted. If the queue is not full, insertion can be done by the q.get() method.

##### Program continues...

```
def dequeue(q):
 if (not q.empty()):
 data=Q.get()
 print("The Item Deleted is : ",data)
 else:
 print("There is NO DATA in the queue")
```

The dequeue() function first checks if the queue is empty. This is done by the built-in method q.empty(). Deletion cannot be performed until there is at least one element in the queue. If the queue is not empty, deletion can be done by the q.get() method.

##### Program continues...

```
def Display(q):
 if(q.full()):
 print("The Elements in the Queue are : [",end="")
 for i in q.queue:
 print(i," ",end="")
```

---

```

 print("]")
 print("The Queue is Full")

elif(q.empty()):
 print("The Queue is EMPTY")
else:
 print("The Elements in the Queue are : [",end="")
 for i in q.queue:
 print(i," ",end="")
 print("]")

```

The Display() function shows all the elements in the queue. All the elements can be fetched from q.queue.

### Program continues...

```

def menu():
 print("\n~~~ MENU ~~~ ")
 print("1. Insert to Queue ")
 print("2. Delete from Queue ")
 print("3. Display the Queue")
 print("4. Exit ")
 opt = int(input("Enter a valid menu item ... "))
 return opt

```

### Program continues...

```

if __name__ == "__main__":
 size = int(input("Enter the size of the Queue "))
 Q = Queue(size)
 i=1
 while (i > 0 and i < 4):
 i = menu()
 if (i == 1):
 enqueue(Q)
 Display(Q)

 elif (i == 2):
 dequeue(Q)
 Display(Q)

 elif (i == 3):
 Display(Q)
 else:
 print("Exit")

```

In queue, size is an integer that specifies the maximum number of items that can be added to the queue. Once this size is achieved q.full(), insertion cannot be performed until all queue items have been deleted. The queue size is unlimited if it is less than or equal to zero.



## Output

```
Enter the size of the Queue 3

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 10
The Elements in the Queue are : [ 10 ]

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 20
The Elements in the Queue are : [10 20]

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 30
The Elements in the Queue are : [ 10 20 30 ]
The Queue is Full

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The Item Deleted is : 10
The Elements in the Queue are : [20 30]

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The Item Deleted is : 20
The Elements in the Queue are : [ 30 ]
```

```

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The Item Deleted is : 30
The Queue is EMPTY

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 4
Exit

```

We can observe that except for the implementation of queue using the queue module, all the previous examples have one thing in common. Even though the queue had empty spaces, still we are not allowed to insert data into the queue as the rear of the queue is pointing to the last element (Figure 6.8a).

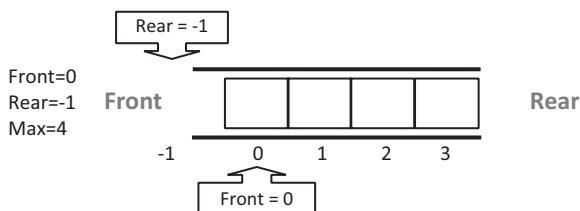


Figure 6.8a Empty queue

Insert an element 10 to the queue (Figure 6.8b).

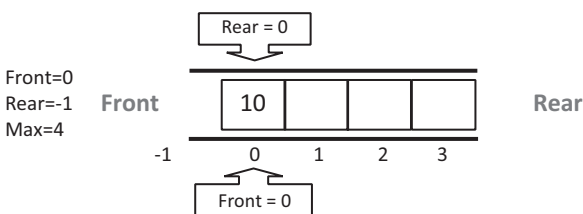


Figure 6.8b Insert of element 10 into queue

Insert an element 20 to the queue (Figure 6.8c).

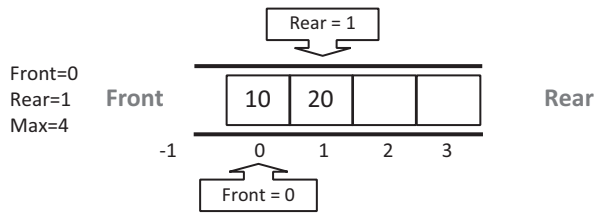


Figure 6.8c Insert of element 20 into queue

Insert an element 30 to the queue (Figure 6.8d).

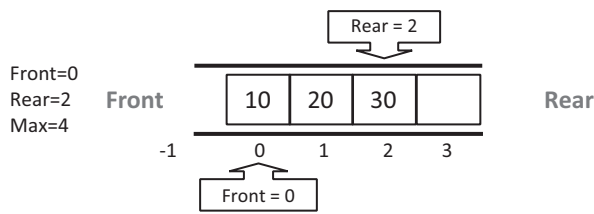


Figure 6.8d Insert of element 30 into queue

Insert an element 40 to the queue (Figure 6.8e).

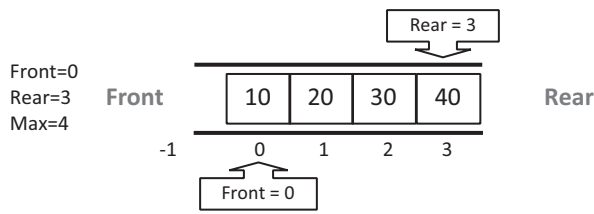


Figure 6.8e Insert of element 40 into queue

Now the queue is full. Delete an element from the queue (Figure 6.8f).

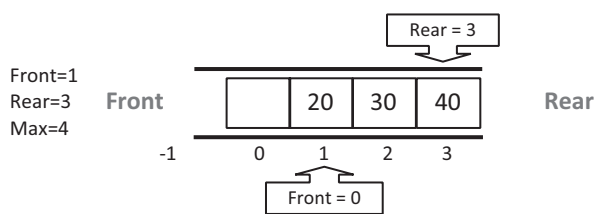


Figure 6.8f Deletion of element 10 from the queue

Now if we try to insert an element 50 to the queue, it will not be allowed since  $\text{rear} = \text{max} - 1$  is true. which is the queue-full condition. Thus, even though there are many vacant cells, we may get a queue-full condition.

There are two ways to avoid this problem:

1. After each deletion, shift all the data elements to the left so that the actual front is always  $-1$ , update rear to rear  $-1$  so that  $\text{rear} = \text{max} - 1$  is not always true. Obviously this is not a feasible solution since this involves a lot of frequent data movement and is time-consuming.
2. The other way is to choose alternate representation: either circular storage of data using the same array representation or use linked list representation.

## 6.5 CIRCULAR QUEUE

The data structure *circular queue* uses the same array representation but wraps around when a queue-full condition occurs using modular arithmetic. In other words, when the rear becomes equal to size, we reset rear to  $\text{rear} = \text{rear} + 1 \% \text{size}$ .

Using this representation, let us define the class MyCircularQueue.

### Python program 6

```
class MyCircularQueue():
    def __init__(self, S):
        self.size = S
        self.queue = [None] * S
        self.front = self.rear = -1
```

The `__init__()` method initializes the circular queue. It creates an empty list of size  $S$  and initializes the front and rear to  $-1$ .

### Program continues...

```
# Insert an element into the circular queue
def enqueue(self):
    data = int(input(("Enter a number to insert into the
    Queue ")))
    if ((self.rear + 1) % self.size == self.front):
        print("The circular queue is full\n")

    elif (self.front == -1):
        self.front = 0
        self.rear = 0
        self.queue[self.rear] = data
    else:
        self.rear = (self.rear + 1) % self.size
        self.queue[self.rear] = data
```

The enqueue() method adds an element at the rear. When the first element is added, rear and front are assigned to 0. Every time an element is added to the circular queue, it calculates the rear by  $(\text{self.rear} + 1) \% \text{self.size}$  and adds the element at the rear.

Let us assume the circular queue (Figure 6.9a).

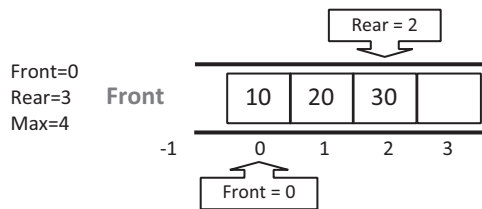


Figure 6.9a Initial circular queue

Let us add 40 to the circular queue. Calculate the index by  $(\text{rear} + 1) \% \text{size}$ , i.e.,  $(2+1) \% 4 = 3$ . So insert 40 at index 3 (Figure 6.9b).

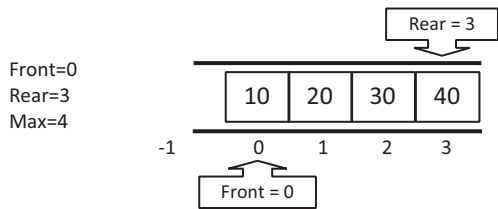


Figure 6.9b Insertion of 10 into a circular queue

Now delete an element from the circular queue (Figure 6.9c).

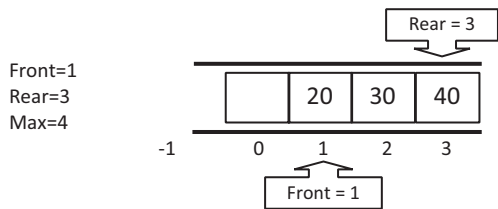


Figure 6.9c Deletion of element from circular queue

Now add one element 50 to the circular queue. Find the index by calculating  $(\text{rear} + 1) \% \text{size}$ , i.e.,  $(3+1) \% 4 = 0$ . Add the new element 50 at the 0th index (Figure 6.9d).

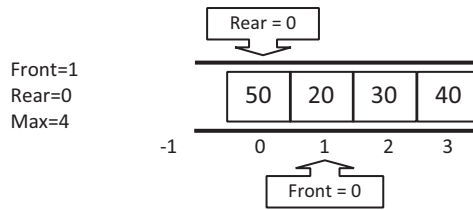


Figure 6.9d Insertion of element into circular queue

Program continues...

```
# Delete an element from the circular queue
def dequeue(self):
    if (self.front == -1):
        print("The circular queue is empty\n")

    elif (self.front == self.rear):
        self.queue[self.front] = None
        self.front = -1
        self.rear = -1
    else:
        self.queue[self.front] = None
        self.front = (self.front + 1) % self.size
```

The dequeue() method deletes an element from the circular queue that is at the front. When the front and rear are pointing to the same location, after deletion the circular queue becomes empty. So initialize the front and rear to -1. If not, the front is assigned to *None* and front is moved to  $\text{self.front} = (\text{self.front} + 1) \% \text{self.size}$ .

Let us assume the circular queue (Figure 6.10a).

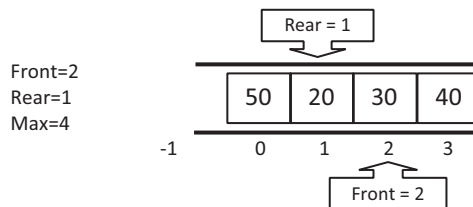


Figure 6.10a Initial circular queue

Now delete the element at the front of the circular queue. After deletion, the front points to the index  $(\text{front}+1) \% \text{size}$ , i.e.,  $(2+1) \% 4 = 3$  (Figure 6.10b).

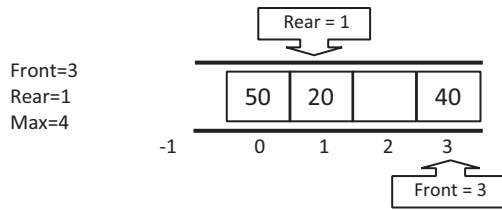


Figure 6.10b Deletion of element 30 from circular queue

Now delete the element at the front of the circular queue. After deletion, the front points to the index  $(\text{front}+1) \% \text{size}$ , i.e.,  $(3+1) \% 4 = 0$  (Figure 6.10c).

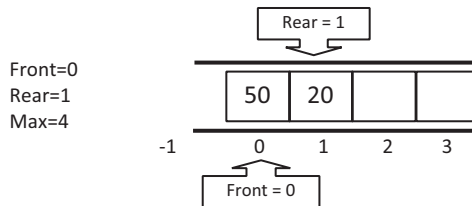


Figure 6.10c Deletion of element 40 from circular queue

**Program continues...**

```
def Display(self):
    print("Front is at index -> ",self.front)
    print("Rear is at index -> ",self.rear)
    if(self.front == -1):
        print("No element in the circular queue")
        print()

    elif((self.rear + 1) % self.size == self.front):
        print("Queue is Full")
        for i in range(0, self.size):
            print(self.queue[i], end=" ")
        print()

    else:
        for i in range(0, self.size):
            print(self.queue[i], end=" ")
        print()
```

The Display() method shows the elements in the circular queue. When front points to -1, it prints “No element in the circular queue”, and when  $(\text{rear} + 1) \% \text{size} = \text{front}$ , it shows the elements in the circular queue and prints

“Queue is Full”. In all other cases, it shows all the elements in the circular queue.

**Program continues...**

```
def menu():
    print("\n~~~ MENU ~~~ ")
    print("1. Insert to Circular Queue ")
    print("2. Delete from Circular Queue ")
    print("3. Display the Circular Queue")
    print("4. Exit ")
    opt = int(input("Enter a valid menu item ... "))
    return opt
```

**Program continues...**

```
if __name__ == "__main__":
    Size = int(input("Enter the size of the Circular Queue "))
    CQ = MyCircularQueue(Size)
    i=1
    while (i > 0 and i < 4 ):
        i = menu()
        if (i == 1):
            CQ.enqueue()
            CQ.Display()

        elif (i == 2):
            CQ.dequeue()
            CQ.Display()

        elif (i == 3):
            CQ.Display()
        else:
            print("Exit")
```

## Output

```
Enter the size of the Circular Queue 3
~~~ MENU ~~~
1. Insert to Circular Queue
2. Delete from Circular Queue
3. Display the Circular Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 10
Front is at index -> 0
Rear is at index -> 0
10 None None
```



~~~ MENU ~~~

1. Insert to Circular Queue
2. Delete from Circular Queue
3. Display the Circular Queue
4. Exit

Enter a valid menu item ... 1

Enter a number to insert into the Queue 20

Front is at index -> 0

Rear is at index -> 1

10 20 None

~~~ MENU ~~~

1. Insert to Circular Queue
2. Delete from Circular Queue
3. Display the Circular Queue
4. Exit

Enter a valid menu item ... 1

Enter a number to insert into the Queue 30

Front is at index -> 0

Rear is at index -> 2

Queue is Full

10 20 30

~~~ MENU ~~~

1. Insert to Circular Queue
2. Delete from Circular Queue
3. Display the Circular Queue
4. Exit

Enter a valid menu item ... 2

Front is at index -> 1

Rear is at index -> 2

None 20 30

~~~ MENU ~~~

1. Insert to Circular Queue
2. Delete from Circular Queue
3. Display the Circular Queue
4. Exit

Enter a valid menu item ... 2

Front is at index -> 2

Rear is at index -> 2

None None 30

~~~ MENU ~~~

1. Insert to Circular Queue
2. Delete from Circular Queue
3. Display the Circular Queue
4. Exit

Enter a valid menu item ... 1

Enter a number to insert into the Queue 40

```
Front is at index -> 2
Rear is at index -> 0
40 None 30

~~~ MENU ~~~
1. Insert to Circular Queue
2. Delete from Circular Queue
3. Display the Circular Queue
4. Exit
Enter a valid menu item ... 4
Exit
```

6.6 DOUBLE-ENDED QUEUE

A double-ended queue is also known as a deque. It defines a data structure where elements can be inserted and deleted at both ends. It supports both stack-like and queue-like capabilities. This hybrid data structure provides all the features of stacks and queues in a single data structure (Figure 6.11).

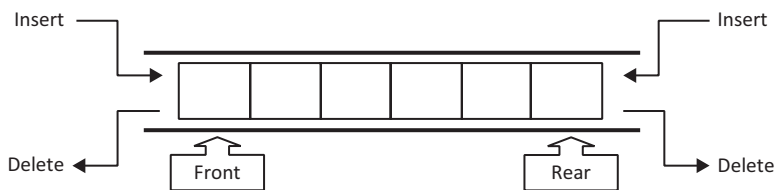


Figure 6.11 Double-ended queue

6.6.1 ADT of dequeue

Operations: See Table 6.4.

Table 6.4 Dequeue ADT operations

| Operation     | Meaning                                                  |
|---------------|----------------------------------------------------------|
| Create()      | Creates a deque that is empty                            |
| addFront(el)  | Adds a new element “el” to the front and returns nothing |
| addRear(el)   | Adds a new element “el” to the rear and returns nothing  |
| deleteFront() | Removes an element from the front and returns the item   |
| deleteRear()  | Removes an element from the rear and returns the item    |
| isEmpty()     | Checks if the deque is empty; returns 1 if true, else 0  |
| size()        | Returns the number of elements in the deque              |

**Axioms:** See Table 6.5.

Table 6.5 Axioms of dequeue operations

| Axiom                        | Returned value |
|------------------------------|----------------|
| isEmpty(create())            | True           |
| isEmpty(insertRear(el))      | False          |
| deleteFront(insertFront(el)) | El             |
| deleteFront(create())        | Error          |
| deleteRear(create())         | Error          |

As an example, let *d* be an empty deque (Table 6.6).

Table 6.6 Axioms of dequeue operations

| Deque operation | Deque content | Return value |
|-----------------|---------------|--------------|
| d.isEmpty()     | [ ]           | True         |
| d.addrear(4)    | [ 4 ]         | -            |
| d.addrear(5)    | [ 5 4 ]       |              |
| d.addFront(3)   | [ 5 4 3 ]     |              |
| d.deleterear()  | [ 4 3 ]       | 5            |
| d.deleteFront() | [ 4 ]         | 3            |
| d.isEmpty()     | [ 4 ]         | False        |

Deque can best be implemented using a doubly linked list.

The general deque class has some possible subtypes:

*Input-restricted deque* where deletions can be made from both ends and insertions can be made only from one end (Figure 6.12).

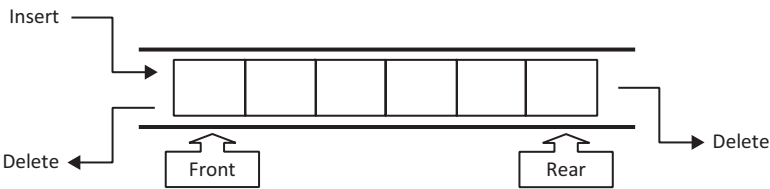


Figure 6.12 Input-restricted deque

*Output-restricted deque* where insertions can be made from both ends and deletions can be made from only one end (Figure 6.13).

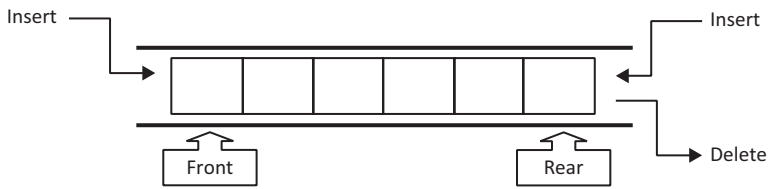


Figure 6.13 Output-restricted deque

The following Python program implements deque.

### Python program 7

```
import collections

def Insert(q):
    data = int(input(("Enter a number to insert into the
    Queue ")))
    side= input("Enter the side (F-> at front and R -> at
    rear) : ")
    if (side == 'F' or side == 'f'):
        q.appendleft(data)
    else:
        q.append(data)
```

In Python, collections are simply container data types, such as lists, sets, tuples and dictionaries.

Python's collections module implements specialized data structures as an alternative to the language's built-in container data types. Appends and pops from either end of a list are supported by a deque object. It uses less memory than a regular list object. When an item is removed from a regular list object, all items to the right are relocated one index to the left. As a result, it is extremely slow.

The Insert() function adds elements at both ends. There are two specialized methods, appendleft() and append(), available with the deque(). The function appendleft() adds an element at the front, whereas append() adds the element at the rear.

### Program continues...

```
def Delete(q):
    if (len(q) != 0):
        side= input("Enter from where to Delete (F-> at front and
        R -> at rear) : ")
        if (side == 'F' or side == 'f'):
            item = q.popleft()
            print("The element deleted is : ", item)
```

```
    else:
        item =q.pop()
        print("The element deleted is : ", item)
    else:
        print("There is NO DATA in the queue")
```

The Delete() function deletes elements from both ends. There are two specialized methods popleft() and pop(), available with the deque(). The function popleft() deletes the element at the front, whereas pop() deletes the element from the rear.

### Program continues...

```
def Display(q):
    if(len(q) == 0):
        print("The Queue is EMPTY")
    else:
        print("The Elements in the Queue are : [ ",end="")
        for i in q:
            print(i," ",end="")
        print("]")
```

### Program continues...

```
def menu():
    print("\n~~~ MENU ~~~ ")
    print("1. Insert to Queue ")
    print("2. Delete from Queue ")
    print("3. Display the Queue")
    print("4. Exit ")
    opt = int(input("Enter a valid menu item ... "))
    return opt
```

### Program continues...

```
if __name__ == "__main__":
    Q = collections.deque()
    i=1
    while (i > 0 and i < 4 ):
        i = menu()
        if (i == 1):
            Insert(Q)
            Display(Q)
        elif (i == 2):
            Delete(Q)
            Display(Q)
```

```
elif (i == 3):
    Display(Q)
else:
    print("Exit")
```

## Output

~~~ MENU ~~~

1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit

Enter a valid menu item ... 1

Enter a number to insert into the Queue 10

Enter the side (F-> at front and R -> at rear) : f

The Elements in the Queue are : [10]

~~~ MENU ~~~

1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit

Enter a valid menu item ... 1

Enter a number to insert into the Queue 20

Enter the side (F-> at front and R -> at rear) : f

The Elements in the Queue are : [ 20 10 ]

~~~ MENU ~~~

1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit

Enter a valid menu item ... 1

Enter a number to insert into the Queue 30

Enter the side (F-> at front and R -> at rear) : r

The Elements in the Queue are : [20 10 30]

~~~ MENU ~~~

1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit

Enter a valid menu item ... 2

Enter from where to Delete (F-> at front and R -> at rear) : f

The element deleted is : 20

The Elements in the Queue are : [ 10 30 ]

~~~ MENU ~~~

1. Insert to Queue
2. Delete from Queue

```
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
Enter from where to Delete (F-> at front and R -> at rear) : r
The element deleted is : 30
The Elements in the Queue are : [ 10 ]

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
Enter from where to Delete (F-> at front and R -> at rear) : R
The element deleted is : 10
The Queue is EMPTY

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 4
Exit
```

6.7 PRIORITY QUEUE

The use of a queue data structure ensures that items are processed in the order in which they are received.

Consider the setting of a hospital. In most cases, patients are treated in the order that they arrive. There may be times, however, when the first in, first out rule cannot be rigidly followed. For example, FIFO cannot be used if a patient arrives in a life-threatening condition following an accident. He must be attended immediately on priority basis.

In the world of computer science, an interactive program's print request takes precedence over a batch-processing program's print request. There are numerous instances when a work or a process is given a priority. Customers and processes with higher importance are pushed to the front of the queue using a particular data structure called a *priority queue*. A priority queue can be implemented in two ways. Let's give each element a priority (integer value); the smaller the integer, the greater the priority.

1. Maintain a separate queue of all the elements with the same priority. Let us assume there are jobs with three different priorities. Then there will be three different queues as shown in Figure 6.14.

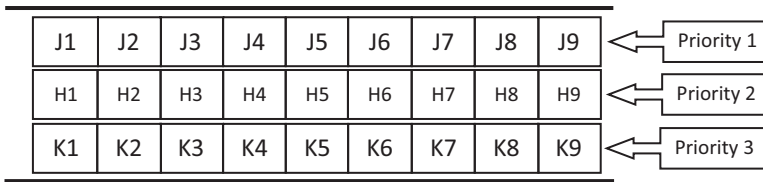


Figure 6.14 Priority queue

The jobs with priority 2 will be processed only when all the jobs of priority 1 are processed and the queue is empty. Jobs with priority 3 will be processed when the queues of priority 1 and 2 are empty. If higher priority jobs keep on coming, the lower priority jobs get delayed indefinitely.

2. The second method is to store all jobs as structures together with their priorities. All the jobs are stored as a queue of elements, with the highest priority job always in the front. When a new job arrives, it is inserted at the appropriate position in the queue, based on its priority.

When we store the jobs (elements) in a sorted list, we can easily delete them because the job with the highest priority is always at the front. However, putting the job in the right place is difficult since we must first locate it in the list.

If we keep the elements as an unsorted list, insertion is easy, since it can be put at the end of the list. However, deletion is difficult, since from the unordered list we have to pick up the job with highest priority.

6.7.1 Implementation of priority queue using class

We will make use of Section 6.4.3 where we implemented a queue using a class.

Python program 8

```
# Creating an Empty Priority Queue
class Queue:
    def __init__(self,size):
        self.maxSize = size
        self.front=0
        self.rear = -1
        self.priority= 0
        self.PQ = list()
        print("\nEmpty Queue Created")
```


The `__init__()` method initializes the priority queue as discussed in Section 6.4.3, apart from that a new attribute `priority` is assigned to 0. The priority queue is a list of tuples.

Program continues...

```
def isFull(self):
    if (self.rear >= self.maxSize-1):
        return 1
    else:
        return 0
```

Program continues...

```
def isEmpty(self):
    if (self.front > self.rear or self.rear == -1):
        return 1
    else:
        return 0
```

The methods `isFull()` and `isEmpty()` are as described earlier.

Program continues...

```
def enqueue(self):
    if (self.isFull() == 1):
        print("Overflow: No more place for the Queue")
    else:
        no = int(input(("Enter a number to insert into the\nQueue ")))
        p = int(input(("Enter priority of data (0 -> Low, 1->\nHigh ) ")))
        while (p not in [0,1]):
            p = int(input(("Enter priority of data (0 -> Low, 1->\nHigh ) ")))
        if (len(self.PQ) == 0):
            temp=(p,no)
            self.PQ.append(temp)
        elif (p ==1):
            j=0
            while (self.PQ[j][0] != 0 and len(self.PQ) > j ):
                j+=1
            temp=(p,no)
            self.PQ.insert(j,temp)
        else:
            temp=(p,no)
            self.PQ.append(temp)
        self.rear +=1
```

The enqueue() method adds an element to the priority queue. Along with the data to be inserted, we need to insert the priority with value 0 for low priority and 1 for high priority. This forms a tuple. Inserting the tuple into the priority queue has three cases.

Case 1: When there is no data in the priority queue, the tuple of the key and value pair is appended to the priority queue.

Case 2: When the priority queue contains some data, the tuple with priority 0 is appended to the queue.

Case 3: When a tuple with priority 1 is inserted, it is inserted before all the tuples having priority 0 but after any other tuple with priority 1.

Program continues...

```
def dequeue(self):
    if (self.isEmpty()==1):
        print("Underflow: There is no element in the Queue")
    else:
        print("The element to Delete is ",self.PQ[self.front])
        self.front += 1
```

The dequeue() method deletes an element that is at the front of the queue. It simply increments the front by one.

Program continues...

```
def display(self):
    print("Front is at index -> ",self.front)
    print("Rear is at index -> ",self.rear)
    if(self.isEmpty()==1):
        print("\nQueue is Empty")
    elif(self.isFull()==1):
        print("The elements in the Queue are")
        for i in range(self.front,self.rear+1):
            print(i, "->",self.PQ[i])
        print("\nQueue is Full")
    else:
        print("The elements in the Queue is")
        for i in range(self.front,self.rear+1):
            print(i, "->",self.PQ[i])
```

Program continues...

```
def menu():
    print("\n~~~ MENU ~~~ ")
    print("1. Insert into Queue ")
    print("2. Delete from Queue ")
    print("3. Display the Queue")
    print("4. Exit ")
    opt = int(input("Enter a valid menu item ... "))
    return opt
```

Program continues...

```
if __name__ == "__main__":
    size = int(input("Enter the size of the Queue "))
    Qu = Queue(size)
    i=1
    while (i > 0 and i < 4 ):
        i = menu()
        if i == 1:
            Qu.enqueue()
            Qu.display()
        elif i == 2:
            Qu.dequeue()
            if (Qu.front > Qu.rear):
                Qu.front = 0
                Qu.rear = -1
            print()
            Qu.display()
        elif i == 3:
            Qu.display()
        else:
            print("Exit")
```

Output

```
Enter the size of the Queue 3
Empty Queue Created
~~~ MENU ~~~
1. Insert into Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 10
Enter priority of data (0 -> Low, 1-> High ) 0
Front is at index -> 0
```

```
Rear is at index -> 0
The elements in the Queue is
0 -> (0, 10)

~~~ MENU ~~~
1. Insert into Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 20
Enter priority of data (0 -> Low, 1-> High ) 1
Front is at index -> 0
Rear is at index -> 1
The elements in the Queue is
0 -> (1, 20)
1 -> (0, 10)

~~~ MENU ~~~
1. Insert into Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 30
Enter priority of data (0 -> Low, 1-> High ) 0
Front is at index -> 0
Rear is at index -> 2
The elements in the Queue are
0 -> (1, 20)
1 -> (0, 10)
2 -> (0, 30)

Queue is Full

~~~ MENU ~~~
1. Insert into Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The element to Delete is (1, 20)
Front is at index -> 1
Rear is at index -> 2
The elements in the Queue are
1 -> (0, 10)
2 -> (0, 30)

Queue is Full
```

```
~~~ MENU ~~~
1. Insert into Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The element to Delete is (0, 10)
Front is at index -> 2
Rear is at index -> 2
The elements in the Queue are
2 -> (0, 30)

Queue is Full

~~~ MENU ~~~
1. Insert into Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The element to Delete is (0, 30)

Front is at index -> 0
Rear is at index -> -1

Queue is Empty

~~~ MENU ~~~
1. Insert into Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 4
Exit
```

6.7.2 Implementation of priority queue by importing PriorityQueue package of queue module

Here we have used the PriorityQueue package available in the queue module.

Python program 9

```
from queue import PriorityQueue

def enqueue(q):
    data = input(("Enter a number to insert into the Queue "))
    p = int(input(("Enter priority of data (0 -> Highest
        Priority ) ")))
    temp = (p,data)
    q.put(temp)
```

The queue module inserts a tuple (priority, data) by the *put()* method. The tuple having priority 0 has the highest priority.

Program continues...

```
def dequeue(q):
    if (not q.empty()):
        next_item = q.get()
        print("The Item Deleted is : ", next_item)
    else:
        print("Priority queue is Empty")
```

The queue module deletes the tuple (priority, data) with high priority data that is at the front of the queue by the *get()* method.

Program continues...

```
def Display(q):
    print(q.queue)
```

Program continues...

```
def menu():
    print("\n~~~ MENU ~~~ ")
    print("1. Insert to Queue ")
    print("2. Delete from Queue ")
    print("3. Display the Queue")
    print("4. Exit ")
    opt = int(input("Enter a valid menu item ... "))
    return opt
```

Program continues...

```
if __name__ == "__main__":
    PQ = PriorityQueue()
    i=1
    while (i > 0 and i < 4 ):
        i = menu()
        if (i == 1):
            enqueue(PQ)
            Display(PQ)
        elif (i == 2):
            dequeue(PQ)
            Display(PQ)
        elif (i == 3):
            Display(PQ)
        else:
            print("Exit")
```

PriorityQueue() creates a priority queue that is imported from the queue module. An integer can be passed to specify the maximum number of items that can be added to the queue. Once this size is attained, insertion will be halted until all queue items have been consumed. The queue size is unlimited if the integer passed is less than or equal to zero.

Output

```
~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 10
Enter priority of data (0 -> Highest Priority ) 1
[(1, '10')]
```

```
~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 20
Enter priority of data (0 -> Highest Priority ) 0
[(0, '20'), (1, '10')]
```

```
~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 30
Enter priority of data (0 -> Highest Priority ) 1
[(0, '20'), (1, '10'), (1, '30')]
```

```
~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The Item Deleted is : (0, '20')
[(1, '10'), (1, '30')]
```

```
~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
```

```

4. Exit
Enter a valid menu item ... 2
The Item Deleted is : (1, '10')
[(1, '30')]

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The Item Deleted is : (1, '30')
[]

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 4
Exit

```

6.7.3 Implementation of priority queue using heapq module

A priority queue is also implemented in Python by the heapq module. The tuple having priority 0 has the highest priority.

Python program 10

```

import heapq

def menu():
    print("\n~~~ MENU ~~~ ")
    print("1. Insert to Queue ")
    print("2. Delete from Queue ")
    print("3. Display the Queue")
    print("4. Exit ")
    opt = int(input("Enter a valid menu item ... "))
    return opt

```

Program continues...

```

if __name__ == "__main__":
    PQ = []
    i=1
    while (i > 0 and i < 4 ):
        i = menu()
        if (i == 1):
            data = input(("Enter a number to insert into the Queue "))

```



```
p = int(input(("Enter priority of data (0 -> Highest
Priority ) ")))
temp = (p,data)
heapq.heappush(PQ, (temp))
heapq.heapify(PQ)
print(PQ)
elif (i == 2):
    data=heapq.heappop(PQ)
    print("The Item Deleted is : ",data)
    if(PQ):
        print(PQ)
    else:
        print("No Data in Priority Queue")
elif (i == 3):
    print(PQ)
else:
    print("Exit")
```

The `heapq` module has a `heappush()` method to add a tuple with priority and data. Every time we insert a tuple, it is appended to the heap. So, after every insertion we need to apply the `heapify()` method to arrange the tuples in descending order of priority. The `heappop()` method deletes tuples from the front of the priority queue.

Output

```
~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 10
Enter priority of data (0 -> Highest Priority ) 2
[(2, '10')]

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 20
Enter priority of data (0 -> Highest Priority ) 1
[(1, '20'), (2, '10')]

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
```

```

4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 30
Enter priority of data (0 -> Highest Priority ) 0
[(0, '30'), (2, '10'), (1, '20')]

```

```

~~~ MENU ~~~

```

```

1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The Item Deleted is : (0, '30')
[(1, '20'), (2, '10')]

```

```

~~~ MENU ~~~

```

```

1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The Item Deleted is : (1, '20')
[(2, '10')]

```

```

~~~ MENU ~~~

```

```

1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The Item Deleted is : (2, '10')
No Data in Priority Queue

```

```

~~~ MENU ~~~

```

```

1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 4
Exit

```

True–false questions

1. Queues are also known as LIFO data structures.
 - a. True
 - b. False
2. A printer uses stack data structures.
 - a. True
 - b. False
3. While inserting an element onto a queue, we need to increment the front by one.
 - a. True
 - b. False

4. While deleting an element onto a queue, we need to increment the front by one.
 - a. True
 - b. False
5. A queue is empty when the front is greater than the rear.
 - a. True
 - b. False

Fill-in-the-blank questions

1. The _____ queue adds and deletes elements at both ends.
2. To add an element to a queue we have the _____ operation.
3. Before inserting an element to a queue, the _____ condition is checked.
4. The isEmpty condition is checked before the _____ operation.
5. $(Rear + 1) \% size == front$ is used to check the queue full condition in the _____ queue.

Multiple-choice questions

1. A linear list of elements in which deletion can be done from one end (front) and insertion can take place only at the other end is known as a
 - a. Queue
 - b. Stack
 - c. Tree
 - d. Branch
2. The end at which a new element gets added to a queue is called the
 - a. Front
 - b. Rear
 - c. Top
 - d. Bottom
3. With what data structure can a priority queue be easily implemented?
 - a. Array
 - b. List
 - c. Heap
 - d. All of the above
4. What are the advantages of priority queues?
 - a. Easy to implement
 - b. Processes with different priority can be efficiently handled
 - c. Applications with differing requirements
 - d. All of the above

5. Let the following circular queue accommodate a maximum six elements with the following data:
front = 2 rear = 4
queue = _____; L, M, N, _____, _____
What will happen after the ADD O operation takes place?
- a. front = 2 rear = 5
queue = _____; L, M, N, O, ____
- b. front = 3 rear = 5
queue = L, M, N, O, ____
- c. front = 3 rear = 4
queue = _____; L, M, N, O, ____
- d. front = 2 rear = 4
queue = L, M, N, O, ____
6. If the elements A, B, C and D are placed in a queue and are deleted one at a time, in what order will they be removed?
- a. ABCD
- b. DCBA
- c. DCAB
- d. ADBC
7. When several jobs arrive at the printer, the jobs join the
- a. Queue
- b. Dequeue
- c. Stack
- d. None of the above
8. A data structure in which elements can be inserted or deleted at/from both ends but not in the middle is a _____.
- a. Queue
- b. Circular queue
- c. Dequeue
- d. Priority queue
9. A circular queue is implemented using an array of size 10. The array index starts with 0, front is 6, and rear is 9. The insertion of the next element takes place at the array index.
- a. 0
- b. 7
- c. 9
- d. 10
10. A normal queue of size MAX gets full when
- a. $\text{Rear} = \text{MAX} - 1$
- b. $\text{Rear} = \text{front}$
- c. $\text{Rear} = \text{front} - 1$
- d. $\text{Rear} = (\text{rear} + 1) \% \text{MAX}$
11. A queue can be implemented using
- a. Arrays
- b. Linked lists
- c. Two stacks
- d. All the above

12. A circular queue of size MAX gets full when
- a. $\text{Rear} = \text{MAX} - 1$
 - b. $\text{Front} = \text{Rear} - 1$
 - c. $\text{Rear} = \text{Front} - 1$
 - d. $\text{Front} = (\text{Rear} + 1) \% \text{MAX}$
13. While implementing propriety queue using the heapq module, thr highest priority is
- a. -1
 - b. 1
 - c. 0
 - d. 2
14. The minimum number of stacks needed to implement a queue is
- a. 1
 - b. 2
 - c. 3
 - d. 4
15. Which of the following is true with respect to a double-ended queue?
- a. Deletion is done only at the rear.
 - b. Insertion is only done at the rear.
 - c. Deletion is done only at the front.
 - d. Insertion and deletion can be done at both ends.
16. Which of the following not an application of priority queue?
- a. Interrupt handling in OS
 - b. Undo operation in a text editor
 - c. Huffman code
 - d. Editing in a text editor
17. Which if the following is an advantage of a double-ended queue?
- a. Can be used for both stack and queue
 - b. To find the longest sub-string in a text
 - c. To avoid collision in a hash table
 - d. Job scheduling algorithm
18. Breadth-First Search of a graph uses a
- a. Stack
 - b. Queue
 - c. Array
 - d. None of the above
19. A list of size 10 is used to build a circular queue. The list index begins with 0 and ends with 9. The front index is 6, and the rear index is 9. The list index determines where the next element is inserted.
- a. 0
 - b. 7
 - c. 9
 - d. 10

20. Priority queue can be implemented using

- | | |
|-----------------|---------------------|
| a. Dictionary | c. Heapq module |
| b. Queue module | d. All of the above |

Descriptive questions

1. What is a circular queue? Write a Python program to insert an element in the circular queue. Write a Python function for printing the elements of the queue in reverse order.
2. Suppose the size of an array for implementing a circular queue is 100. Suppose the front is 50 and the rear is 99.
 - a. What are the values of front and rear after adding an element?
 - b. What are the values of rear and front after removing an element?
3. What are the advantages of circular queue over linear queue?
4. Write a Python program to create a queue using stack.
5. Write a Python program to find the number of elements in a queue.
6. Write a Python program that permits insertion at any vacant location at rear end.
7. Explain priority queue with the help of an example.
8. Explain how a circular queue is better than a linear queue.

Answers to true–false questions

1. False
2. True
3. False
4. True
5. True

Answers to fill-in-the-blank questions

1. double-ended
2. enqueue
3. isFull
4. deletion
5. circular

Answers to multiple-choice questions

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. a | 2. b | 3. d | 4. d | 5. a |
| 6. a | 7. a | 8. c | 9. a | 10. a |
| 11. d | 12. d | 13. c | 14. b | 15. d |
| 16. b | 17. c | 18. b | 19. a | 20. d |

Trees

LEARNING OBJECTIVES

After studying this chapter, the readers will be able to:

- Understand and implement a tree
- Perform different operations in a tree
- Understand binary search trees
- Know applications of binary search trees
- Understand and implement AVL trees
- Perform different operations on AVL trees
- Understand and implement splay trees
- Perform different operations on splay trees

We've learned about linear data structures like arrays, stacks and queues up to this point. In this chapter will concentrate on trees, which are non-linear data structures. The list, dictionary, class and queue modules can all be used to create tree data structures. Simple examples of trees include files and folders in a directory, decision-based algorithms, domain name system, representing a family tree and so on.

7.1 TREES

Data structures can be categorized into linear and non-linear structures. A data structure is said to be non-linear if its elements are not in a sequence. In other words, every element in a non-linear data structure does not have a unique successor and unique predecessor. Neither list nor dictionary is useful when we want a hierarchical structure of data, while stacks and queues do reflect hierarchy to some extent. They are, however, limited to one dimension. To overcome this limitation, we define a new data structure – *tree* – which consists of nodes (which are also called vertices) and edges (which are called arcs) with an inverted tree-like structure as shown in Figure 7.1.

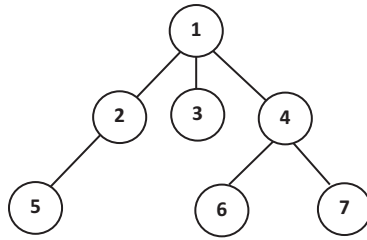


Figure 7.1 Tree

In the non-linear data structure tree, a node can have more than one successor but not more than one predecessor and in a *graph*, a node can have more than one successor and more than one predecessor as shown in Figure 7.2.

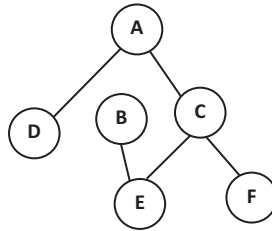


Figure 7.2 Graph

Definition: A tree can be defined as a finite set of one or more nodes such that

- i) One of the nodes in the set, say, R, is identified as the root.
- ii) The rest of the nodes of the set can be partitioned into k number of subsets T₁, T₂, ..., T_k such that each of these subsets also form a tree and are called the sub-trees of the root.

The following terminology is used in the description of a tree:

1. *Root*: the node of the tree that has no parent.
2. *Child*: a node that is directly connected to another node when moving away from the root.
3. *Degree*: the number of edges coming out of the node. Degree of the leaf node is always zero.
4. *Parent*: a node that is directly connected to another node when moving toward the root.
5. *Siblings*: a group of nodes with the same parent.

6. *Descendent*: a node reachable by repeatedly proceeding from parent to child.
7. *Ancestor*: a node reachable by repeatedly proceeding from child to parent.
8. *Leaf node* or *terminal node*: a node with no children.
9. *Internal node* or *non-terminal node*: a node with at least one child.
10. *Forest*: a set of disjoint trees.

In Figure 7.1, Node 1 is the root.

Node 2, Node 3 and Node 4 are children of Node 1; Node 5 is the child of Node 2; and Node 6 and Node 7 are children of Node 3.

Node 1 has degree 3; Node 2 has degree 1; Node 4 has degree 2; and Node 3, Node 5, Node 6 and Node 7 have degree 0.

Node 2 is the parent of Node 5, Node 4 is the parent of Node 6 and Node 7; and Node 1 is parent of Node 2, Node 3 and Node 4.

Node 2, Node 3 and Node 4 are siblings; Node 6 and Node 7 are siblings.

Node 2 and Node 5 are descendants of Node 1.

Node 1 is the ancestor of Node 5 and Node 2.

Node 3, Node 5, Node 6 and Node 7 are leaf nodes.

Node 2, Node 3 and Node 4 are the internal nodes.

7.2 BINARY TREES

A binary tree (BT) is an important type of tree that is often used in computer science and is characterized by the fact that each node can have a left child only, right child only or both (Figure 7.3). In other words, there is no node with more than two branches. (A branch is an edge coming out of the node.)

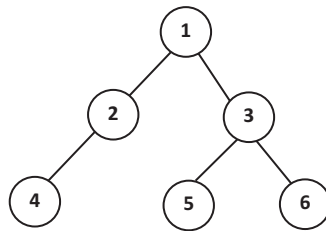


Figure 7.3 Binary tree

7.2.1 Abstract data type of a binary tree

The following are the operations and the set of axioms on the tree data structure:

Operations: See Table 7.1.

Table 7.1 Operations in a tree abstract data type

| Operation | Description |
|-----------------------|--|
| create() | Creates an empty binary tree |
| makeBT(lbt, data, rbt | BT (pointer to the root node [containing data] and lbt and rbt are pointers to the left sub-tree and right sub-tree, respectively, of the root node) |
| isEmpty() | True if BT = 0, else False |
| lchild() | BT |
| rchild() | BT |

Axioms: See Table 7.2.

Table 7.2 Axioms in tree abstract data type

| Operation | Result |
|--------------------------|--------|
| isEmpty(create()) | TRUE |
| isEmpty(makeBT(l, d, r)) | FALSE |
| Lchild(makeBT(l, d, r)) | L |
| Rchild(makeBT(l, d, r)) | R |
| Data(create()) | ERROR |

7.3 TYPES OF BINARY TREES

Different shapes of binary trees are named as follows:

Left-skewed binary tree: The right sub-tree is missing in every node of a tree (Figure 7.4).

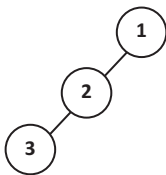


Figure 7.4 Left-skewed binary tree

Right-skewed binary tree: The left sub-tree is missing in every node of a tree (Figure 7.5).

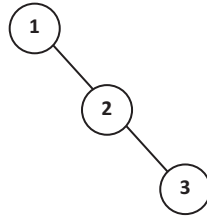


Figure 7.5 Right-skewed binary tree

Strictly binary tree: A binary tree in which every node has zero or two children (Figure 7.6).

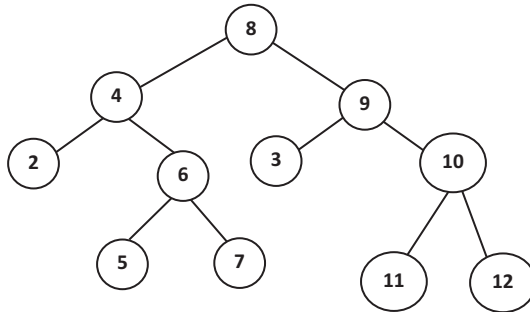


Figure 7.6 Strictly binary tree

Full binary tree: A binary tree in which all the interior nodes have two children and all the leaf nodes are at the same level. In a full binary tree, each node has two children or no child (Figure 7.7).

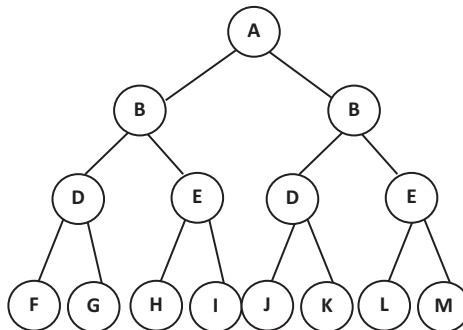


Figure 7.7 Full binary tree

Thus, the number of nodes in a full binary tree is

$$1 + 2 + 2^2 + 2^3 + \dots + 2^h = 2^{h+1} - 1$$

Thus, if n is the total number of nodes in a full binary tree

$$n = 2^{h+1} - 1$$

Therefore, $2^{h+1} = n + 1$ or

$$h = \log_2(n+1) - 1.$$

In Figure 7.7, $h = 3$ and hence there are $2^4 - 1 = 15$ nodes.

Complete binary tree: A binary tree in which all the levels are completely filled except the last level and all the nodes in the last level are as far left as possible (Figure 7.8).

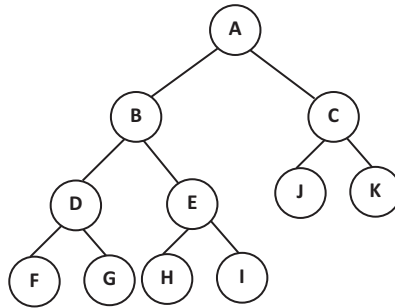


Figure 7.8 Complete binary tree

In other words, in a complete binary tree, the leaf nodes are at the same level or at two consecutive levels. Add children of a node from left to right.

A full binary tree is a complete binary tree but not vice versa.

If the tree is a complete binary tree (not a full binary tree) with n nodes, then

$$\log_2 n < h \leq \log_2(n+1)$$

Theorem: The maximum number of nodes at level i is 2^{i-1} , where $i \geq 1$.

Proof: The root is at level 1. The number of nodes is $2^0 = 1$, which is true.

Assume that the hypothesis is true for level $i - 1$, i.e., the number of nodes at level $i - 1$ is 2^{i-2} . Each node at level $i - 1$ can at maximum have two children.

Therefore, the maximum number of children at level i is $2 \times 2^{i-2} = 2^{i-1}$. Hence the theorem is proved.

Theorem: For any non-empty binary tree, $n_0 = n_2 + 1$, where n_0 is the number of leaves (nodes of degree 0) and n_2 is the number of nodes with degree 2.

Proof: Let n be the total number of nodes in the binary tree. Then, we have

$$n = n_0 + n_1 + n_2 \quad (7.1)$$

Now, every node except the root node has an edge coming into it. Thus, $n - 1 = E$ or $n = E + 1$ where E is the total number of edges.

Total number of edges = n_1 (nodes with degree 1) + n_2 (edges with degree 2) $\times 2$.

$E = n_1 + 2n_2$ which implies

$$n = 1 + n_1 + 2n_2 \quad (7.2)$$

Subtracting Equation 7.2 from Equation 7.1, we get

$$0 = n_0 - 1 - n_2$$

or

$$n_0 = n_2 + 1$$

Consider the tree structure in Figure 7.9.

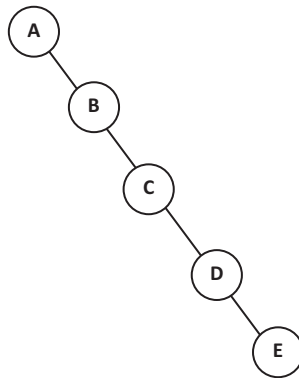


Figure 7.9 Binary tree with single leaf node

Here $n_2 = 0$ and $n_0 = 1$ and the theorem is true (Figure 7.10).

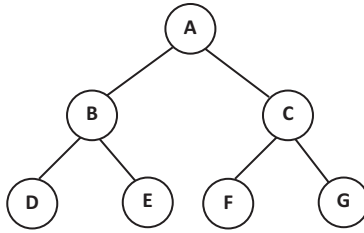


Figure 7.10 Binary tree with double leaf node

Here $n_2 = 3$ and $n_0 = 4$.

Therefore, $n_0 = n_2 + 1$.

Corollary: In a strictly binary tree, the number of leaves (n_0) = number of internal nodes (n_2) + 1.

Theorem: The number of vertices in a binary tree is one more than the number of edges.

Proof: Let there be one vertex in the binary tree. Then there are no edges and the theorem is true.

If there are two vertices, there can only be one edge. Again, the theorem is true. Let the theorem be true for vertices (v) and edges (e). Therefore, $v = e + 1$.

If we add a new vertex to the binary tree, connecting it to a vertex in the binary tree, then we are also adding a new edge. Thus, $v + 1 = e + 2$, which proves that the theorem is true for $v = v + 1$.

7.3.1 Some properties of binary trees

Level: The level of the root node is 1. If a node is at level i , then its children are at level $i + 1$.

Height or depth: The height or depth of a tree is defined as the maximum level of any of the nodes. The depth of a node is the number of edges between the root and the node. The height of a node is the number of edges between the node and the farthest leaf in the sub-tree rooted at the node.

7.4 REPRESENTATION

There are various ways of representing a binary tree. The first method to represent a binary tree can be represented with the help of a list of tuples. As a tuple, the node is declared as a structure with a data field and two

index fields. The index fields contain indices of the tuple in which the left and right children are stored. For example, consider the tree in Figure 7.11.

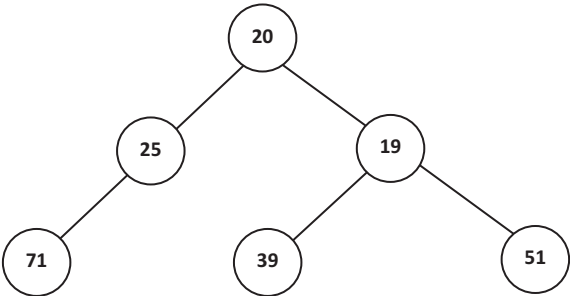


Figure 7.11 Sample binary tree

The root is always stored as the first node. The list of tuples for the tree in Figure 7.11 is shown in a tabular form where each row represents a tuple (Table 7.3).

Table 7.3 Tabular representation of tree

| Index | Data | Left index | Right index |
|-------|------|------------|-------------|
| 0 | 20 | 3 | 4 |
| 1 | 39 | None | None |
| 2 | 51 | None | None |
| 3 | 25 | 5 | None |
| 4 | 19 | 1 | 2 |
| 5 | 71 | None | None |

Tree = [(20, 3, 4), (39, None, None), (51, None, None), (25, 5, None), (19, 1, 2), (71, None, None)]

The second method to represent a binary tree would be to number the nodes of the tree starting with index 0 as the root node and number all the nodes successively level by level from left to right, considering it as a full binary tree and use these numbers as an index into the list. For example, we represent the tree given in Figure 7.11 as shown in Figure 7.12.

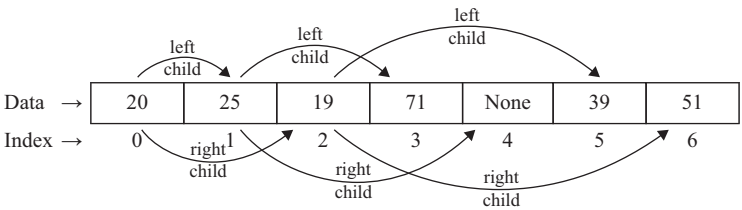


Figure 7.12 Array representation of tree

The *None* cell stores no information. If the tree is heavily skewed to one side there will be many such NULL cells, causing wastage of a lot of memory space. However, the list representation as described earlier has some advantages. For example, given the index of the node in the list, it is easy to find the index of its parent and vice versa. For example, if a node has index i , its children will be at $2i$ and $2i+1$ in the list and its parent will be at $\lfloor i/2 \rfloor$. Since the size of the list can change during the execution of the program, the tree can grow dynamically. A dynamic data structure is more efficient to represent a binary tree, since there is always scope for the tree to expand.

Python program 1

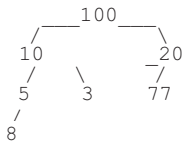
```
# Creating binary tree from given list
from binarytree import build

# List of nodes
nodes = [100, 10, 20, 5, 3, 77, None, 8]

binary_tree = build(nodes)
print('Binary tree from list :\n', binary_tree)
print('\nList from binary tree :', binary_tree.values)
```

Output

Binary tree from list :



List from binary tree : [100, 10, 20, 5, 3, 77, None, 8]

In the dynamic implementation, one more way to represent a node of a tree is an instance of a class composed of the data field and two pointer members, and the binary tree is a collection of these objects organized hierarchically.

7.5 OPERATIONS IN A BINARY TREE

Like any other data structure, insertion, deletion and traversal are the operations we perform in a tree.

7.5.1 Insertion into a binary tree

Insertion is an easy process if the node is to be inserted as a left child or a right child of a leaf node. Here the node containing 'L' is added as a leaf node to the tree in the tree given (Figures 7.13a and 7.13b).

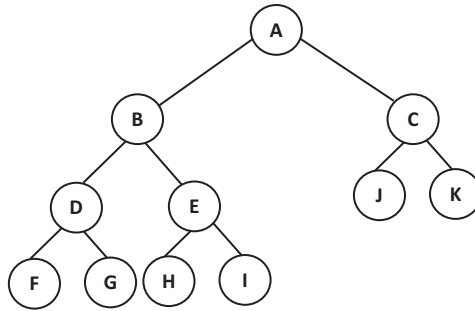


Figure 7.13a Insertion of binary tree

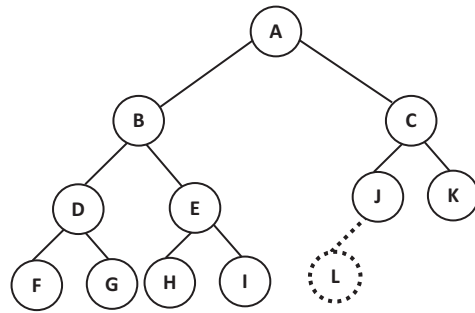


Figure 7.13b Insertion of binary tree

However, if we want to add the new node as an internal node, it becomes ambiguous. We do not know where to add the sub-tree that is replaced by the new node. It may be added as a left child or right of the new node. Therefore, we need a criterion to link this sub-tree. Binary search trees (discussed later in this chapter) provide such a criterion.

7.5.2 Deletion from a binary tree

Deletion is an easy process if the node is to be deleted as a left child or a right child of a leaf node. However, if we want to delete an internal node, it may give rise to ambiguity. When we delete an internal node, the tree will become disconnected. In order to make it connected, a node needs to be replaced for the deleted node. Therefore, we need a criterion to link the sub-tree. Binary search trees (discussed later in this chapter) provide such a criterion.

7.5.3 Binary tree traversals

Information is stored in the nodes of a binary tree. If we want to retrieve the information, we have to traverse the tree, reach that particular node and

extract the data from the node. Traversals can be specified by the ordering of three objects: the current node, its left sub-tree and its right sub-tree. There are several methods to traverse a tree and reach a particular node. For example, we may traverse the left sub-tree of a node first recursively, print the data in the node and then traverse the right sub-tree recursively. Let us call this LDR (Table 7.4).

Table 7.4 Binary tree traversals

| | |
|-----|--|
| DLR | Print Data.
Traverse left sub-tree recursively.
Traverse right sub-tree recursively. |
| LRD | Traverse right sub-tree recursively.
Traverse left sub-tree recursively.
Print Data. |
| DLR | Print Data.
Traverse right sub-tree recursively.
Traverse left sub-tree recursively. |

Similarly, we can define DLR, DRL, LRD and RLD. Let us apply these traversal methods to the following binary tree (Figure 7.14).

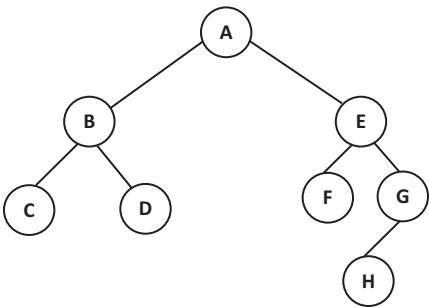


Figure 7.14 Sample binary tree

The following outputs are produced:

| | |
|------|-----------------|
| LDR: | C B D A F E H G |
| RDL: | G H E F A D B C |
| DLR: | A B C D E F G H |
| RLD: | H G F E D C B A |
| DRL: | A E G H F B D C |
| LRD: | C D B F H G E A |

We notice that: LDR and RDL are mirror images.

DLR and RLD are mirror images.

DRL and LRD are mirror images.

Ignoring the mirror images (or assuming that the left sub-tree comes before the right sub-tree) we call:

| | |
|-----|--------------------------|
| DLR | as pre-order traversal. |
| LDR | as in-order traversal. |
| LRD | as post-order traversal. |

Pictorially, the traversals can be represented as in Figure 7.15.

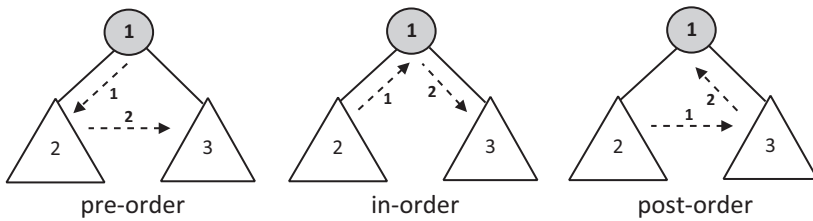


Figure 7.15 In-order, pre-order and post-order traversal

Example: Write the pre-order, in-order and post-order traversal for the given tree in Figure 7.16.

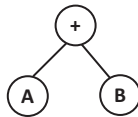


Figure 7.16 Given tree

- i. Pre-order traversal: + A B
- ii. In-order traversal: A + B
- iii. Post-order traversal: A B +

Python program 2

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
```

Program continues...

```
# Inorder traversal
def inorder(root):
    if root is not None:
        # Traverse left
        inorder(root.left)
        # Traverse root
        print(str(root.key), end=' -> ')
        # Traverse right
        inorder(root.right)
```

Program continues...

```
# preorder traversal
def preorder(root):
    if root is not None:
        # Traverse root
        print(str(root.key) , end=' -> ')
        # Traverse left
        preorder(root.left)
        # Traverse right
        preorder(root.right)
```

Program continues...

```
# postorder traversal
def postorder(root):
    if root is not None:
        # Traverse left
        postorder(root.left)
        # Traverse right
        postorder(root.right)
        # Traverse root
        print(str(root.key) , end=' -> ')
```

Program continues...

```
nodes =[10,20,30,40,50]
print('\nnodes in the binary tree :', nodes)

root = Node(nodes[0])
root.left = Node(nodes[1])
root.right = Node(nodes[2])
root.left.left = Node(nodes[3])
root.left.right = Node(nodes[4])
```

```
print("\nInorder traversal: ", end=' ')
inorder(root)

print("\nPreorder traversal: ", end=' ')
preorder(root)

print("\nPostorder traversal: ", end=' ')
postorder(root)
```

Output

```
nodes in the binary tree : [10, 20, 30, 40, 50]

Inorder traversal: 40 -> 20 -> 50 -> 10 -> 30 ->
Preorder traversal: 10 -> 20 -> 40 -> 50 -> 30 ->
Postorder traversal: 40 -> 50 -> 20 -> 30 -> 10 ->
```

7.5.3.1 Construction of binary tree traversals from known post-order and in-order traversal of a tree

Observe that in a pre-order traversal the root will always be at the first node, and in a post-order traversal the root will always be at the last node.

If two of the three traversals are known, the corresponding binary tree can be constructed. The basic steps for the formation of the tree with known in-order and post-order traversal is as follows:

- Step 1: In the post-order traversal, identify the last node as the root node.
- Step 2: In the in-order traversal, identify the root node somewhere at the middle.
- Step 3: After identifying the root node, the nodes in the left sub-tree and the right sub-tree of the root are the nodes to the left and right of the root node in the in-order traversal.
- Step 4: Repeat steps 1 through 3 to the node of the left sub-tree and the right sub-tree.

Example: Construct the tree of with known in-order and post-order traversals:

| | |
|-----------------------|-----------------|
| In-order traversal: | C B D A F E H G |
| Post-order traversal: | C D B F H G E A |

Solution: From the structure of the post-order traversal, we identify A as the root node. From the in-order traversal, we observe that the nodes C, B, D form the left sub-tree and F, E, G, H form the right sub-tree of the root node (Figure 7.17a).

| | |
|-----------------------|-----------------|
| In-order traversal: | C B D A F E H G |
| Post-order traversal: | C D B F H G E A |

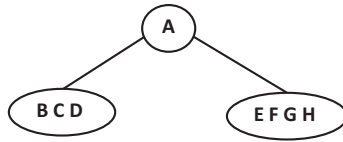


Figure 7.17a Tree with known in-order and post-order traversals

The nodes in the left sub-tree occur in post-order traversal in the order C, D, B indicating that B is the root of the left sub-tree. The nodes F, E, G, H occur in the right sub-tree in the order F, H, G, E, which implies that E is the root of the right sub-tree (Figure 7.17b).

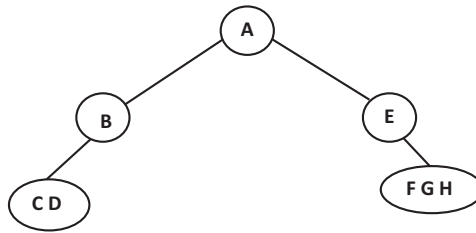


Figure 7.17b Tree with known in-order and post-order traversals

In-order traversal: C B D A F E H G
 Post-order traversal: C D B F H G E A

The nodes B, C, D occur in the order C, B, D showing that C is the left child of B and D is the right child of B. Similarly, we identify F is the left child of E, and G and H constitute the right sub-tree of E (Figure 7.17c).

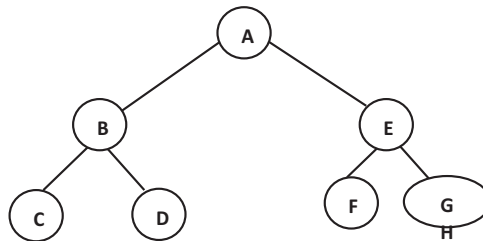


Figure 7.17c Tree with known in-order and post-order traversals

G and H occur in the order H and G in the pre-order traversal. Therefore, G is the root and H is its left child. The full binary tree is as shown in Figure 7.17d.

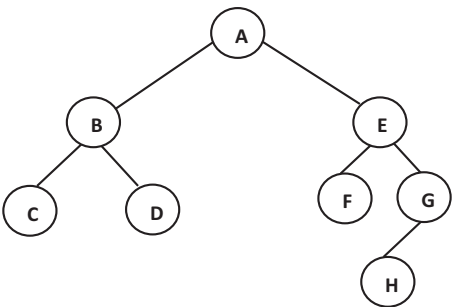


Figure 7.17d Tree with known in-order and post-order traversals

7.5.3.2 Construction of binary tree traversals from known pre-order and in-order traversal of a tree

The basic steps for the formation of the tree with known in-order and pre-order traversals is as follows:

- Step 1: In the pre-order traversal, identify the first node as the root node.
- Step 2: In the in-order traversal, identify the root node somewhere at the middle.
- Step 3: After identifying the root node, the nodes in the left sub-tree and the right sub-tree of the root are the nodes to the left and right of the root node in the in-order traversal.
- Step 4: Repeat steps 1 through 3 to the node of the left sub-tree and the right sub-tree.

Example: Construct a binary tree whose in-order and pre-order traversal are:

| | | | | | | | | | |
|---------------------|----|----|----|----|----|----|----|----|----|
| In-order traversal | 15 | 20 | 24 | 25 | 33 | 35 | 38 | 40 | 45 |
| Pre-order traversal | 25 | 20 | 15 | 24 | 35 | 33 | 40 | 38 | 45 |

Solution: From the pre-order traversal, we observe that 25 is the root. From the position of 25 in the in-order traversal, we notice that (15 20 24) and (33 35 38 40 45) constitute the left sub-tree and right sub-tree (Figure 7.18a).

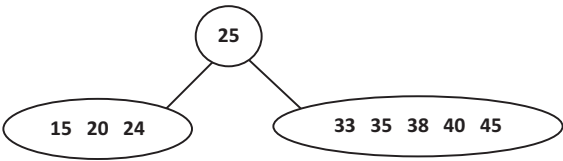


Figure 7.18a Construction of binary tree with given in-order and pre-order traversals

If we observe the three elements 15, 20, 24 in the pre-order traversal, they occur in the order 20, 15, 24, which shows that 20 is the root of the left sub-tree whose left and right children are 15 and 24, respectively. Further, if we observe that the elements constituting the right sub-tree, we notice that 35 is the root and 33 is the left child. Thus, we have so far Figure 7.18b.

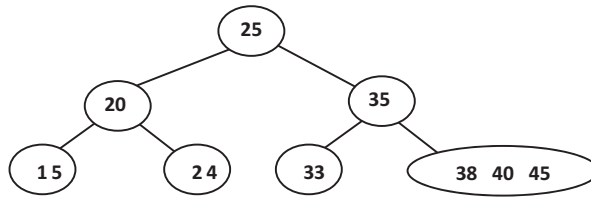


Figure 7.18b Construction of binary tree with given in-order and pre-order traversals

The elements 38, 40, 45 occur in the sequence 40, 38, 45 in the pre-order traversal. Therefore, 40 is the root, with 38 and 45 as the left and right children of 40, respectively. We finally have the tree as shown in Figure 7.18c.

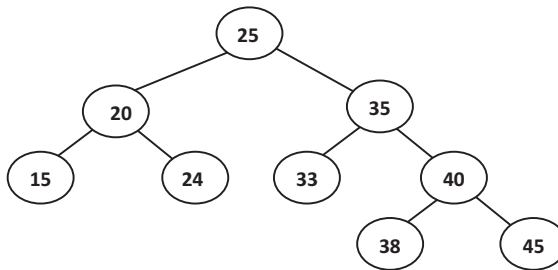


Figure 7.18c Construction of binary tree with given in-order and pre-order traversals

7.6 BINARY SEARCH TREES

The information stored in the nodes of the binary tree will be useful and easily accessible if the data is stored in an organized way. The *binary search tree* (BST) is such a structure where the data stored in the nodes follow a pattern (Figure 7.19).

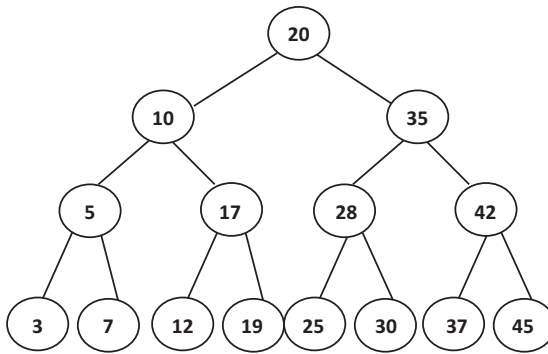


Figure 7.19 Binary search tree

Definition: A binary search tree is a binary tree that is either empty or each node satisfies the following properties:

1. The data value in the left child of the root is less than the data value in the root.
2. The data value in the right child of the root is greater than the data value in the root.
3. The left sub-tree and the right sub-tree of the root are also binary search trees.

Example: Construct the binary search tree as we read the following characters “E D U C A T I O N”.

Step 1: Construct a node with the first character as the root



Figure 7.20a Construction of binary search tree

Step 2: Read the next character D. Construct the node with this character as data. Compare the data with the data in the root. Since it is less (dictionary order) and the left child of the root is NULL, add it as the left child of the root (Figure 7.20b).

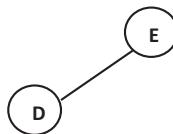


Figure 7.20b Construction of binary search tree

Step 3: The next character read is U. Construct a node with U as data. Since U is greater than E (data in the parent node) and the right child of the root is NULL, add it as the right child of the root (Figure 7.20c).

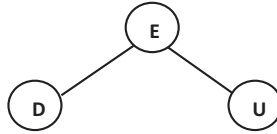


Figure 7.20c Construction of binary search tree

Step 4: We compare the next character C with the value in the root, i.e., E. Since it is less and the left child of the root is not NULL, we make the left child as the root and repeat the procedure. Therefore, we add C as the left child of D (Figure 7.20d).

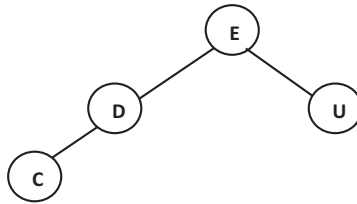


Figure 7.20d Construction of binary search tree

Step 5: We compare the next character A with the value in the root, i.e., E. Since it is less and the left child of the root is not NULL, compare A with D. Since A is less than D and the left child is not NULL, compare it with C. As A is less than C, we add A as the left child of C (Figure 7.20e).

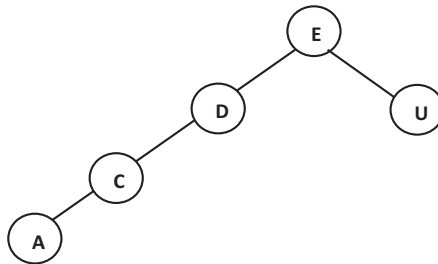


Figure 7.20e Construction of binary search tree

Step 6: We compare the next character T with the value in the root, i.e., E. Since it is greater and the right child of the root is not NULL, compare T with U. Since U is greater than T, we add T as the left child of U (Figure 7.20f).

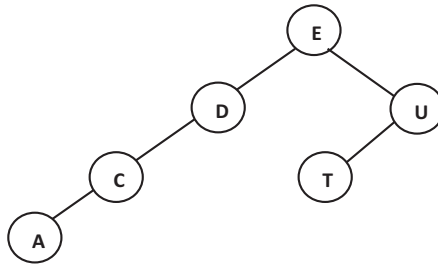


Figure 7.20f Construction of binary search tree

Step 7: We compare the next character I with the value in the root, i.e., E. Since it is greater and the right child of the root is not NULL, compare I with U. Since U is greater than I and the left child is not NULL, compare I with T. Since I is less than T, we add I as the left child of T (Figure 7.20g).

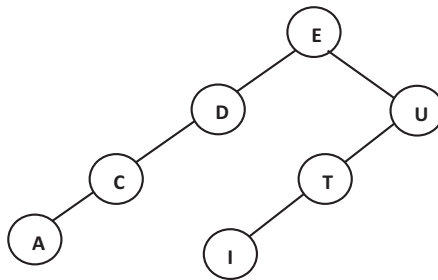


Figure 7.20g Construction of binary search tree

Step 7: We compare the next character O with the value in the root, i.e., E. Since the value is greater and the right child of the root is not NULL, compare O with U. Since U is greater than O and the left child is not NULL, compare O with T. Since O is less than T and the left child is not NULL, compare O with I. Since O' is greater than I, we add O as the right child of I (Figure 7.20h).

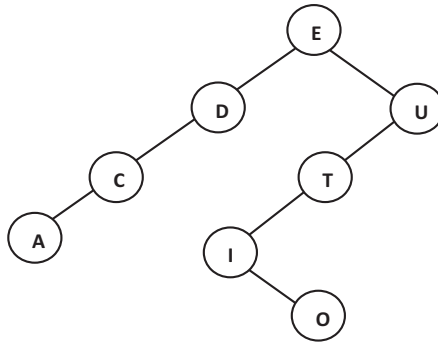


Figure 7.20h Construction of binary search tree

Step 8: We compare the next character N with the value in the root, i.e., E. Since it is greater and the right child of the root is not NULL, compare N with U. Since U is greater than N and the left child is not NULL, compare N with T. Since N is less than T and the left child is not NULL, compare N with I. Since N is greater than I, and the right child is not NULL, compare N with O. Since N is less than O and the left child is NULL, we add N as the left child of O.

Finally, we get the binary search tree as shown in Figure 7.20i.

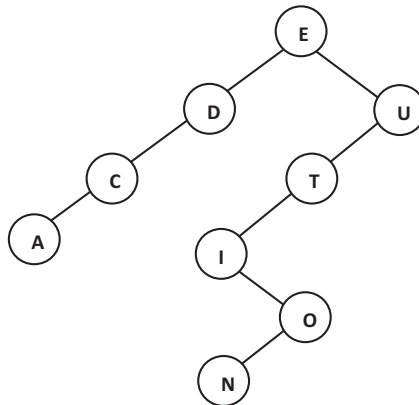


Figure 7.20i Construction of binary search tree

The following program in Python implements the above algorithm using a list to perform operations on a binary search tree.

Python program 3

```
from binarytree import build

# Inserting data into Binary Search Tree
def insert(tree,parent,element,level):
    if element in tree:
        print(element,"\nData already existing in the Binary
        Search Tree\n try any other number")
        return tree

    if (len(tree) < 2**((level+1)-1)):
        for i in range(2**((level+1) - len(tree))):
            tree.append(None)

    if (tree[parent] == None):
        tree[parent] = element
        return tree
    else:
        if (element < tree[parent]):
            parent = parent*2 + 1
            insert(tree,parent, element,level+1)
        else:
            parent=parent*2+2
            insert(tree,parent, element,level+1)
    return tree
```

Program continues

```
def display(tree):
    binary_tree = build(tree)
    print('The Binary Search Tree is :\n', binary_tree)
```

Search

When we want to insert a value in a binary search tree , we have to first check (or search the tree) if that value is already contained in a node since duplicate values are not allowed. We first compare the value with the value contained in the root. If the root contains the value we are searching for, the search is over. If not, we compare the value we are looking for with the value in the root. If it is less, we search the left sub-tree since all the key values in the left sub-tree are less than the key value in the root. If not, we search the right sub-tree. In either case, we search only one sub-tree as in the case of a binary search of a sorted array.

If the element we are searching is less than the “element” in the root, we search the left sub-tree of the root. Else we search the right sub-tree. If found, the pointer to the node is returned. The null pointer is returned if not found.

Example

Suppose we are searching for $x = 46$ in the binary search tree given in Figure 7.21. The path traced by the algorithm is shown by bold arrows, and not found is the result of the search.

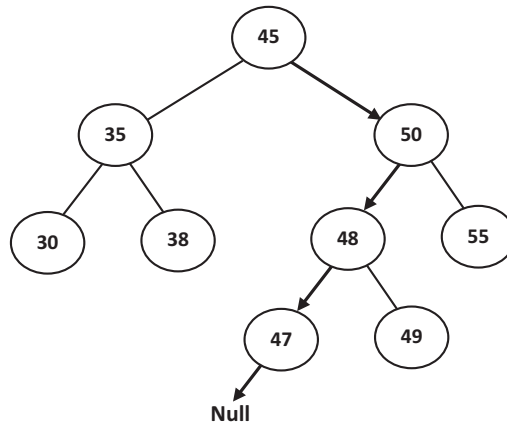


Figure 7.21 Searching in binary search tree

Program continues

```
#Searching a tree
def search(tree,loc, data):
    if (tree[loc] == data) :
        print(data,"Exist in the tree at location ",loc)

    elif(tree[loc] != None and data < tree[loc] ):
        loc = loc*2 + 1
        search(tree,loc, data)
    elif(tree[loc] != None and data > tree[loc] ):
        loc = loc*2 + 2
        search(tree,loc, data)
    else:
        print(data,"Doesnot Exist in the tree")
```

Maximum and minimum

To find the maximum key value in a binary search tree, identify the rightmost node, i.e., the farthest node we can reach following the right branches.

To find the minimum, identify the leftmost node, i.e., the farthest node we can reach by following the leftmost branches.

The following segments of a Python code identify the minimum.

Program continues

```
#Finding the Minimum in a tree
def getRightMin(tree,parent):
    if (tree[parent] != None):
        while(tree[parent*2 +1] != None):
            parent = parent*2+1
        return tree[parent]
    else:
        print("No Data in the tree")
```

Deletion

Three cases arise when we try to delete a node from a binary search tree:

1. The node we want to delete is a leaf node.
2. The node we want to delete has either a left child or a right child but not both.
3. The node we want to delete has both a left child and a right child.

Case 1: The leaf node is deleted, i.e., replaced with NULL. We free the memory that was used by the node we deleted. The node to be deleted is shaded, i.e., the node containing 3 is to be deleted (Figures 7.22a and 7.22b).

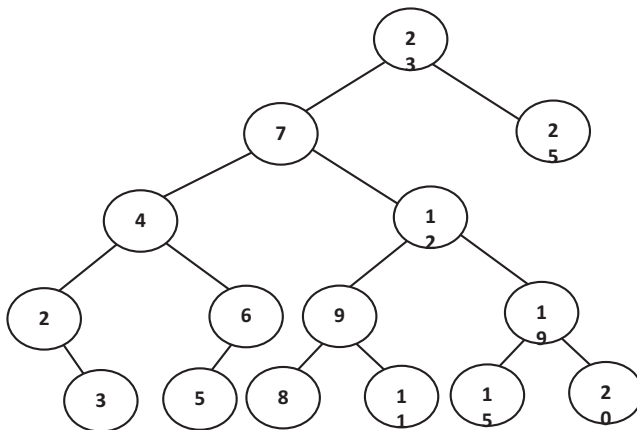


Figure 7.22a Binary search tree before deletion of 3

Case 2: If the node we want to delete has either a left child or a right child, promote the child to the position of the deleted node (Figure 7.22c and 7.22d).

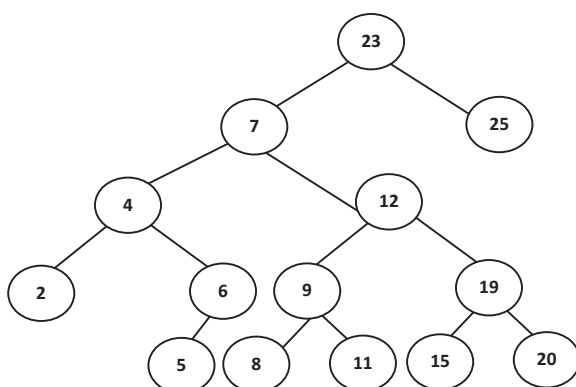


Figure 7.22b Binary search tree after deletion of 3

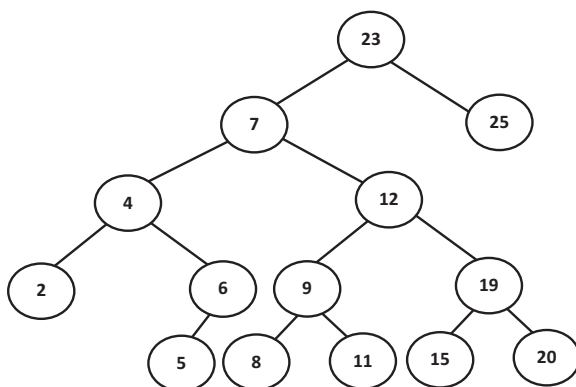


Figure 7.22c Binary search tree before deletion of 6

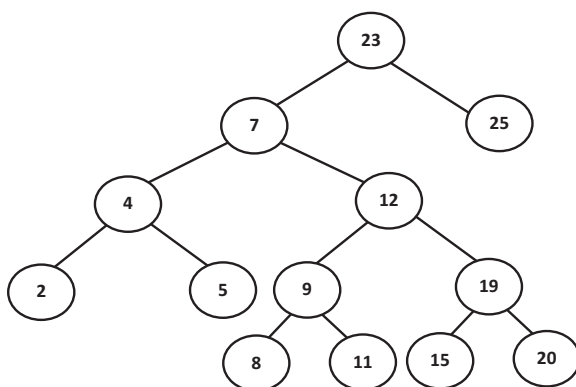


Figure 7.22d Binary search tree after deletion of 6

Case 3: If the node we want to delete has both the children, replace the node with its in-order successor and delete the successor that will be a leaf or a node with only a right child (Figures 7.22e and 7.22f).

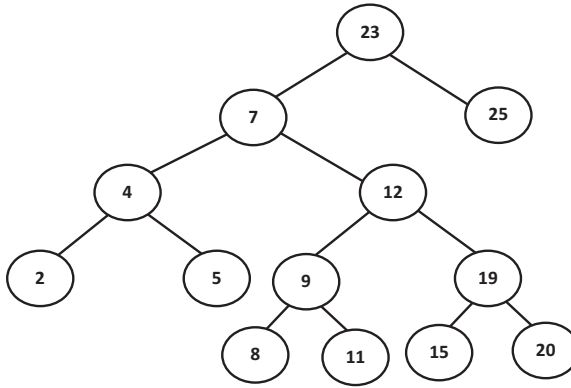


Figure 7.22e Binary search tree before deletion of 7

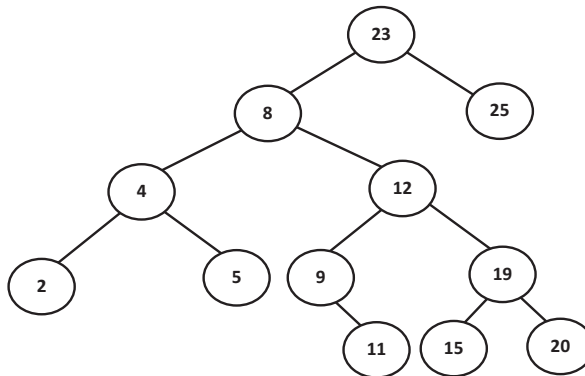


Figure 7.22f Binary search tree after deletion of 7

The following algorithm illustrates the process of deletion for all three cases.

Program continues

```

# Deletion of a Node from the BST
def delNode(tree,parent,data):
    if data in tree:
        i = tree.index(data)
    else:
        print(data,"Does not Exist in the tree")
        return
  
```

```

if(tree[i*2 +1] == None and tree[i*2 + 2] == None):
    tree[i]=None
elif(tree[i*2 +1 ] != None and tree[i*2 + 2] == None):
    tree[i] = tree[i*2 +1 ]
    tree[i*2+1] = None
elif(tree[i*2 +1 ] == None and tree[i*2 + 2] != None):
    tree[i] = tree[i*2 +2 ]
    tree[i*2+2] = None
elif(tree[i*2 +1 ] != None and tree[i*2 + 2] != None):
    min = getRightMin(tree,2*i+2)
    index = tree.index(min)
    tree[i] = min
    tree[index]= None
    if(tree[index*2 +2] != None):
        tree[index]=tree[index*2+2]
        tree[index*2+2]=None

```

Program continues

```

def menu():
    print("\n~~~ M E N U ~~~")
    print("1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Display")
    print("5. Exit")
    opt = int(input("Enter a valid menu item ... "))
    return opt

```

Program continues

```

if __name__ == "__main__":
    tree = [None]
    level=1
    parent = 0
    choice = 1
    while (choice > 0 and choice < 5 ):
        choice = menu()
        if (choice == 1):
            print("Enter 0 to exit Inserting Data into the BST ")
            x = int(input("Enter an element: "))
            while (x != 0):
                insert(tree,parent,x,level)
                #print(tree)
                x = int(input("Enter an element: "))
            display(tree)

```

```
elif(choice == 2):
    value = int(input("Enter element to search : "))
    search(tree,parent,value)

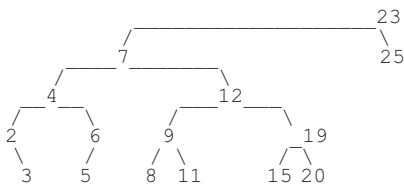
elif(choice == 3):
    value = int(input("Enter element to delete : "))
    delNode(tree,parent,value)
    display(tree)

elif(choice == 4):
    display(tree)

else:
    print("Exit")
```

Output 1

```
~~~ M E N U ~~~
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter a valid menu item ... 1
Enter 0 to exit Inserting Data into the BST
Enter an element: 23
Enter an element: 7
Enter an element: 25
Enter an element: 4
Enter an element: 12
Enter an element: 2
Enter an element: 6
Enter an element: 9
Enter an element: 19
Enter an element: 3
Enter an element: 5
Enter an element: 8
Enter an element: 11
Enter an element: 15
Enter an element: 20
Enter an element: 0
The Binary Search Tree is :
```



Output 2

```

~~~ M E N U ~~~
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter a valid menu item ... 2
Enter element to search : 6
6 Exist in the tree at location 8

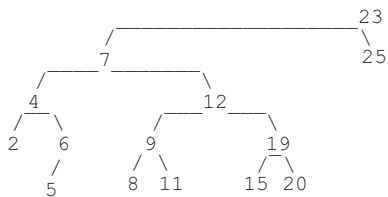
```

Output 3

```

~~~ M E N U ~~~
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter a valid menu item ... 3
Enter element to delete : 3
The Binary Search Tree is :

```

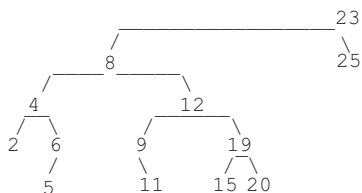


Output 4

```

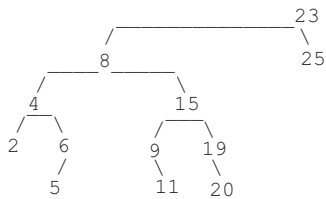
~~~ M E N U ~~~
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter a valid menu item ... 3
Enter element to delete : 7
The Binary Search Tree is :

```



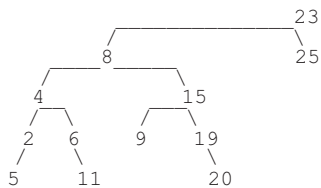
Output 5

```
~~~ M E N U ~~~
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter a valid menu item ... 3
Enter element to delete : 12
The Binary Search Tree is :
```



Output 6

```
~~~ M E N U ~~~
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter a valid menu item ... 3
Enter element to delete : 12
The Binary Search Tree is :
```



Output 7

```
~~~ M E N U ~~~
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter a valid menu item ... 5
Exit
```

The following program in Python implements the above algorithm using a class to perform operations on a binary search tree.

Python program 4

```
# Create a node
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
```

Program continues

```
#Search
def search(root, val):
    if (root is None or root.key == val):
        return root

    if (root.key < val):
        return search(root.right, val)

    return search(root.left, val)
```

Program continues

```
# Inorder traversal
def inorder(root):
    if root is not None:
        # Traverse left
        inorder(root.left)
        # Traverse root
        print(str(root.key), end=' -> ')
        # Traverse right
        inorder(root.right)
```

Program continues

```
# Insert a node
def insert(node, key):
    temp = search(node, key)
    if (temp != None):
        print("\n", temp.key, " Exists in the BST... Try another Number ")
        return node

    else:
        # Return a new node if the tree is empty
        if node is None:
            return Node(key)
```

```
    if key < node.key:
        node.left = insert(node.left, key)
    else:
        node.right = insert(node.right, key)

    return node
```

Program continues

```
# Find the inorder successor
def minValueNode(node):
    current = node

    while(current.left is not None):
        current = current.left
    return current
```

Program continues

```
# Deleting a node
def deleteNode(root, key):

    if root is None:
        return root

    # Find the node to be deleted
    if key < root.key:
        root.left = deleteNode(root.left, key)
    elif(key > root.key):
        root.right = deleteNode(root.right, key)
    else:
        # If the node is with only one child or
        # no child
        if root.left is None:
            temp = root.right
            root = None
            return temp

        elif root.right is None:
            temp = root.left
            root = None
            return temp

        # If the node has two children,
        # place the inorder successor in position of the
        # node to be deleted
        temp = minValueNode(root.right)

        root.key = temp.key

        # Delete the inorder successor
        root.right = deleteNode(root.right, temp.key)

    return root
```

Program continues

```
def menu():
    print("\n~~~ M E N U ~~~")
    print("1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Display")
    print("5. Exit")
    opt = int(input("Enter a valid menu item ... "))
    return opt
```

Program continues

```
if __name__ == "__main__":
    root = None
    choice = 1
    while (choice > 0 and choice < 5 ):
        choice = menu()
        if (choice == 1):
            print("Enter 0 to exit Inserting Data into the BST\n ")
            x = int(input("Enter an element: "))
            while (x != 0):
                root = insert(root, x)
                x = int(input("Enter an element: "))
            print("Inorder traversal: ", end=' ')
            inorder(root)

        elif(choice == 2):
            val = int(input("Enter an element to Search: "))
            temp = search(root, val)
            if (temp != None):
                print("\n", temp.key, " Exists in the BST")
            else:
                print("\n", temp.key, "Does not Exist in the BST")

        elif(choice == 3):
            value = int(input("Enter element to delete : "))
            root = deleteNode(root, value)
            print("Inorder traversal: ", end=' ')
            inorder(root)

        elif(choice == 4):
            print("Inorder traversal: ", end=' ')
            inorder(root)

        else:
            print("Exit")
```


Output

```
~~~ M E N U ~~~
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter a valid menu item ... 1
Enter 0 to exit Inserting Data into the BST

Enter an element: 6
Enter an element: 3
Enter an element: 6

6 Exists in the BST... Try another Number
Enter an element: 4
Enter an element: 0
Inorder traversal: 3 -> 4 -> 6 ->

~~~ M E N U ~~~
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter a valid menu item ... 2
Enter an element to Search: 4

4 Exists in the BST

~~~ M E N U ~~~
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter a valid menu item ... 3
Enter element to delete : 3
Inorder traversal: 4 -> 6 ->

~~~ M E N U ~~~
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter a valid menu item ... 4
Inorder traversal: 4 -> 6 ->
```

~~~ M E N U ~~~

1. Insert
2. Search
3. Delete
4. Display
5. Exit

Enter a valid menu item ... 6

Exit

### Successor and predecessor

The successor (predecessor) of a node with key value  $k$  in the binary search tree is the smallest (largest) key that belongs to the tree and is strictly greater than (less than  $k$ ):

- If  $x$  (node containing  $k$ ) has a right child, the successor is the minimum in the right sub-tree of  $x$ .
- Otherwise, the successor is the parent of the farthest node that can be reached by following the right branches backward.

**Example** Consider the binary search tree in Figure 7.23.

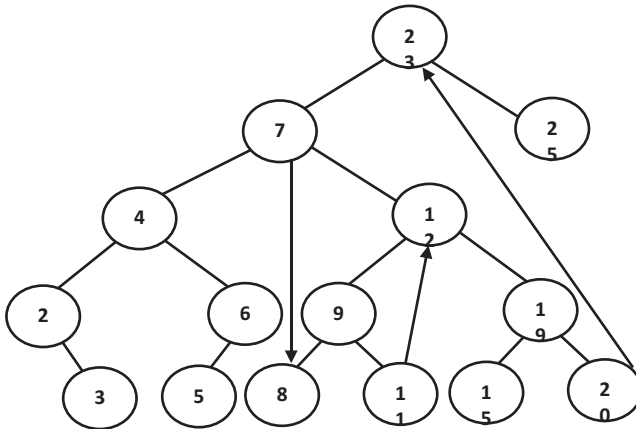


Figure 7.23 Successor and predecessor in a binary search

The successor of 7 is 8 since it is the left-most node of its right sub-tree or the minimum value in its right sub-tree. The successor of 11 is 12. Node 11 has no right sub-tree. Therefore, follow the right branches upward from 11 until there are no more right branches. We reach 9. The parent of 9, which is 12, is the in-order successor of 11. Similarly the successor of 20 is obtained by following the path backward as  $20 \rightarrow 19 \rightarrow 12 \rightarrow 7$  and obtaining the parent of 7, which is 23.

### 7.6.1 Applications of binary search trees

A binary search tree can be used to implement a sorting algorithm. If we insert a set of elements into a binary search tree and make an in-order traversal, we obtain a sorted list (Figure 7.24a).

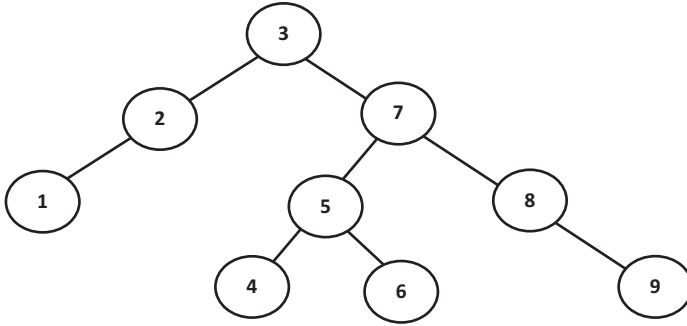


Figure 7.24a Given binary search tree

When we make an in-order traversal, we get the sorted sequence 1 2 3 4 5 6 7 8 9.

If we try to build a binary search tree from an already sorted list, we obtain a skewed binary search tree (Figure 7.24b).

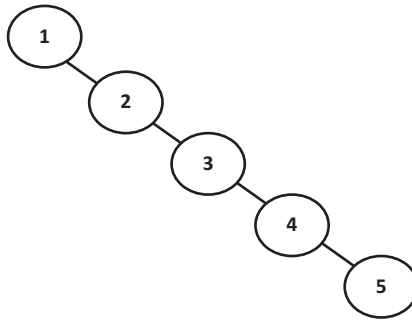


Figure 7.24b Given binary search tree

If there are  $n$  elements in the skewed binary search tree, we need  $n$  steps to reach the last node,  $n-1$  steps to reach the next to last node, etc. Therefore, on the average we need  $n/2$  steps and the search time is  $O(n)$ . We need to keep the height of the tree as small as possible to keep the average search time minimum since the search time is proportional to the height of the tree.

## 7.7 AVL TREES (HEIGHT-BALANCED TREES)

The performance of a search algorithm on a binary search tree depends on the shape of the tree and the shape of the tree depends on the data set. If the data is sorted, the binary search tree would be skewed, and the search would not be efficient as was pointed out in the previous section. On the other hand, if we can build, with the same data set, with minimum height, the search would be faster.

In this section, we define a special type of binary search tree, called the AVL tree (named after the researchers G.M. Adelson-Velsinki and E.M. Landis) in which the tree is nearly balanced. Let us first define a perfectly balanced tree.

**Definition:** A perfectly balanced tree is a binary search tree which is such that:

1. The height of the left sub-tree and right sub-tree of the root is the same.
2. The left sub-tree and the right sub-tree are also perfectly balanced trees.

Figure 7.25 shows a perfectly balanced tree.

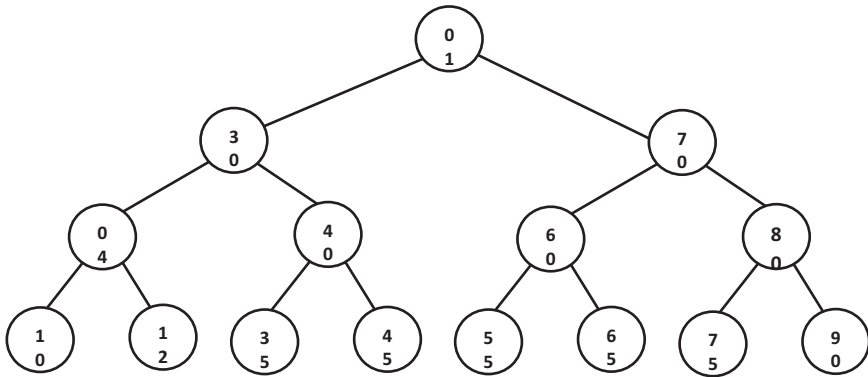


Figure 7.25 Perfectly balanced tree

From the definition of a perfectly binary search tree, we observe that if  $x$  is a node in the tree, the height of its left sub-tree is the same as the height of its right sub-tree. However, we need  $2^h - 1$  nodes to construct such a tree, but this need not be the case always.

**Definition:** An AVL tree (or a height-balanced tree) is a binary search tree such that:

1. The heights of the left sub-tree and right sub-tree of the root differ by at most 1.
2. The left sub-trees and right sub-trees are also AVL trees.

Let  $x$  be a node of the binary search tree. Let  $x_l$  and  $x_r$  be the heights of its left sub-tree and right sub-tree. Then we define the balance factor at node  $x$  as:

$$bf(x) = x_l - x_r$$

The permissible values  $bf(x)$  are  $\{-1, 0, +1\}$  for an AVL tree. Thus, the node in the AVL tree must contain besides the data value and left and right pointers the value of the balance factor.

The *constructor function* initializes the left and right pointers to NULL and the balance factor to 0.

### 7.7.1 Height of an AVL tree

**Proposition:** The height of an AVL tree  $T$  storing  $n$  key values is  $O(\log n)$ .

**Proof:** Let  $n(h)$  be the minimum number of internal nodes in an AVL tree of height  $h$ . We can see that  $n(1) = 0$  and  $n(2) = 1$ . For  $n \geq 3$ , an AVL tree of height contains the root node, an AVL sub-tree of height  $h - 1$  and an AVL sub-tree of height  $h - 2$ . Thus, we can write  $n(h)$  as

$$n(h) = 1 + n(h - 1) + n(h - 2)$$

Obviously,  $n(h - 1)$  is greater than  $n(h - 2)$ . Therefore,

$$\begin{aligned} n(h) &> 2n(h - 2) \\ &\geq 2^2 n(h - 4) \\ &\geq 2^3 n(h - 6) \\ &\geq \dots \\ &\geq \dots \\ &\geq 2^i n(h - 2i) \\ &\geq \dots, \text{ etc.} \end{aligned}$$

Let  $h - 2i = 2$ . Or  $i = (h - 2)/2 = h/2 - 1$ . Therefore,

$$n(h) > 2^{h/2-1}$$

taking logarithms,

$$h/2 - 1 < \log n \text{ or } h < 2\log n + 2.$$

Therefore,  $h$  is  $O(\log n)$ .

Most of the algorithms used for a binary search tree (such as searching, traversals, counting nodes) remain the same. However, insertions and deletions are different since after every insertion and deletion, the AVL property may be lost.

Examples of AVL trees are shown in Figure 7.26.

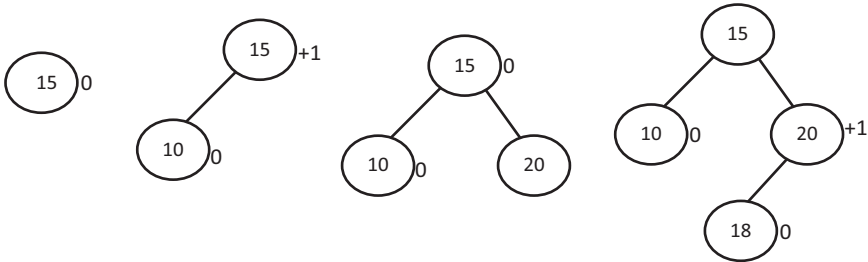


Figure 7.26 Height of AVL tree

### 7.7.2 Insertion

After inserting a node, as is done in any binary search tree, it is necessary to check that each of its ancestors satisfies the AVL property. This is achieved by considering the balance factor at each node. Since the insertion into a binary tree cannot increase the height by more than 1, the balance factor of each node lies in  $\{-2, 2\}$ . If the balance factors of all the nodes lie in  $\{-1, +1\}$ , no further operation is needed. However, if the balance factor at any node is more than  $+1$  or less than  $-1$ , the AVL property is lost and a rebalancing is needed. Let us trace the operations of insertion with an example.

Imagine the situation where we arrive at the structure in Figure 7.27a while constructing a binary search tree.

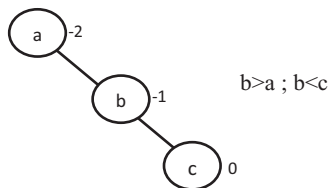


Figure 7.27a Insertion of element in an AVL tree

This does not satisfy the AVL property since the balance factor of the root node is  $-2$ . To fix this we have to perform a rotation. This is done using the following steps.

*LL rotation*

1. b becomes the new root.
2. a takes the ownership of the left child of B.
3. c takes the ownership of the right child of B.

The binary search property is not lost.  
The tree now becomes Figure 7.27b.

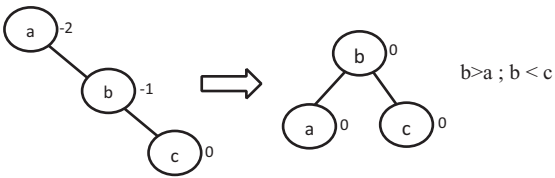


Figure 7.27b Insertion of element in an AVL tree

Suppose we have the mirror image situation of Figure 7.27c.

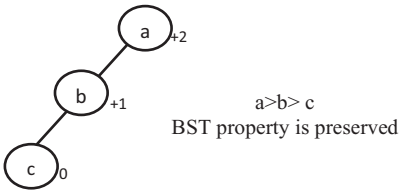


Figure 7.27c Insertion of element in an AVL tree

The balance factor of node a is +2 and hence does not satisfy the AVL property. We fix it with the following steps (RR rotation) (Figure 7.27d).

*RR rotation*

- Step 1: b becomes the new root.  
Step 2: a becomes the right child of b.  
Step 3: c continues to be the left child of b.

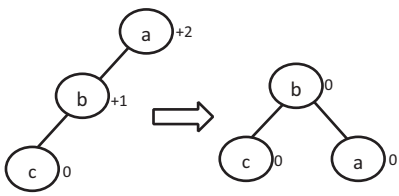


Figure 7.27d Insertion of element in an AVL tree

Both the AVL property and binary search tree property are restored.

Let us see what happens when the new node is the left child of the right sub-tree (see Figure 7.27e).

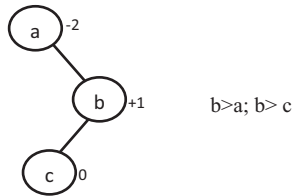


Figure 7.27e Insertion of element in an AVL tree

The balancing factor of  $a$  is 2 and hence AVL structure is not maintained. The new node is added as the left child of the right sub-tree. Now we have to be a little careful.

We initially make a rotation about  $b$  which yields, and then we make the familiar RR rotation to arrive at Figure 7.27f which satisfies both AVL and binary search tree properties

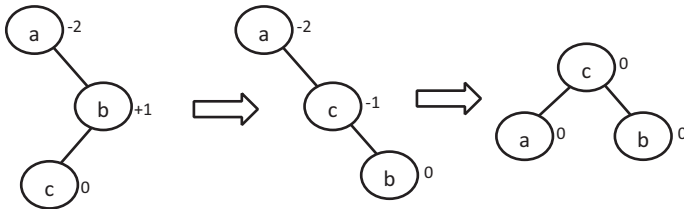


Figure 7.27f Insertion of element in an AVL tree

These two rotations together is called an *RL rotation*.

Similarly, we use *LR rotation* to convert the following binary search tree, where a new node is added to the right of a left sub-tree (Figure 7.27g). The sub-tree rooted at  $a$  does not satisfy the AVL property. To bring it back to

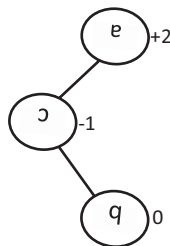


Figure 7.27g Insertion of element in an AVL tree



AVL structure, we first rotate about c, which makes b take the position of c, and c becomes the left child of b, and then rotate about a. This double rotation is called LR rotation.

We arrive at Figure 7.27h.

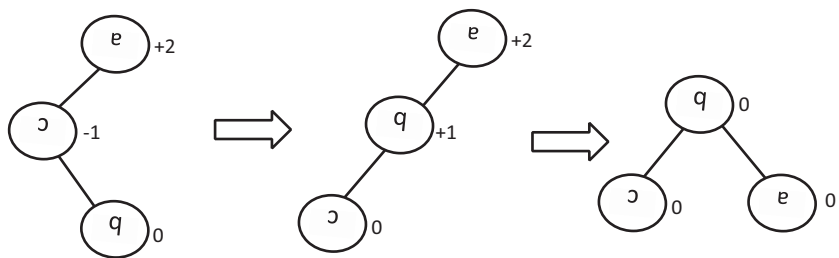


Figure 7.27h Insertion of element in an AVL tree

If we observe closely, we need two rotations if the balance factor at the root (where the balance factor is +2 or -2) and the balance factor of its child are of opposite signs. If the balance factor of the root is +2 and its child has balance factor -1, we first make a left rotation (which makes the sign of balance factors the same) and then a right rotation (LR rotation). If it is the other way (root -2 and its child +1), we first make a right rotation (making the signs of the balance factors the same) and then a left rotation (RL rotation).

Consider a full-sized example to construct a binary search tree with AVL property when the following elements are used to construct the tree (Figures 7.28a to 7.28f), read in order:

50, 17, 12, 23, 9, 14, 19, 72, 54, 76, 67

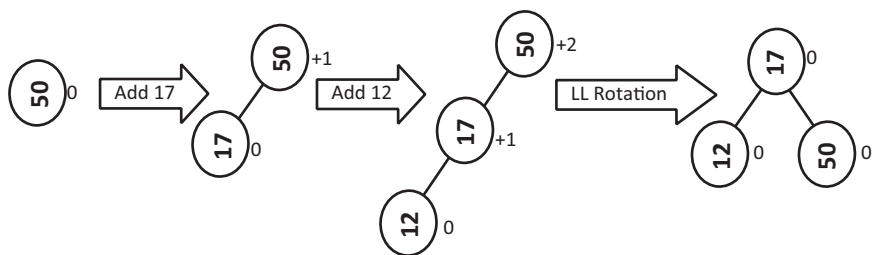


Figure 7.28a Insertion of element in an AVL tree

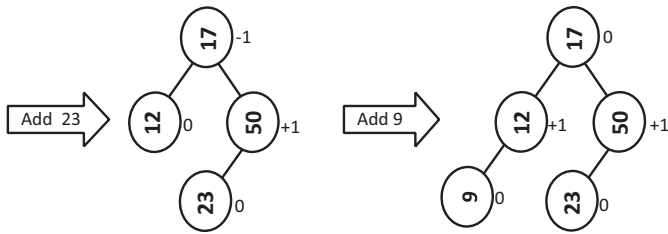


Figure 7.28b Insertion of element in an AVL tree

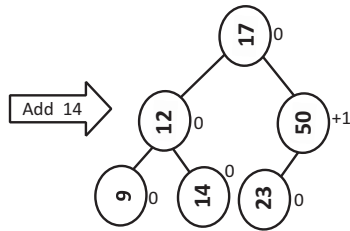


Figure 7.28c Insertion of element in an AVL tree

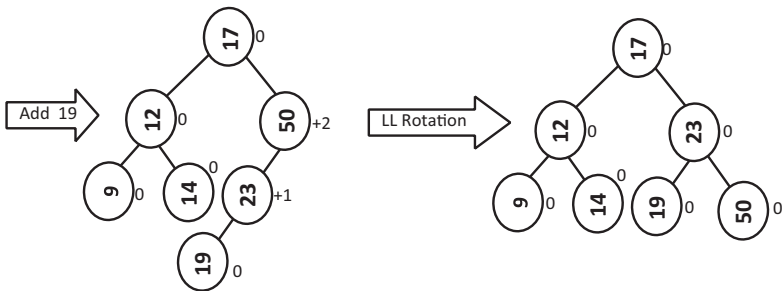


Figure 7.28d Insertion of element in an AVL tree

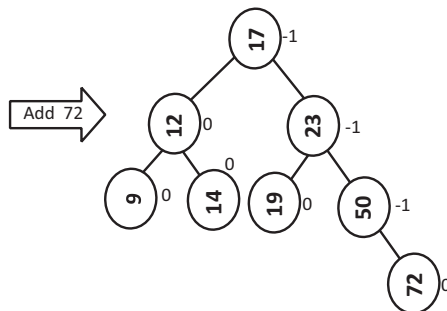


Figure 7.28e Insertion of element in an AVL tree

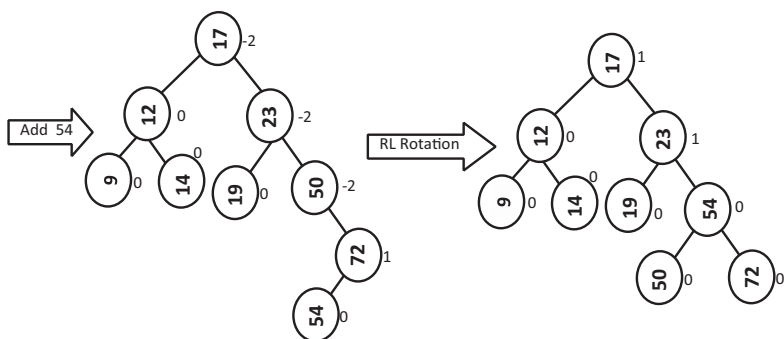


Figure 7.28f Insertion of element in an AVL tree

### 7.7.3 Deletion

As with insertion, the deletion of a node may cause the violation of AVL property. The sub-tree shown in Figure 7.29a is strictly in the AVL form.

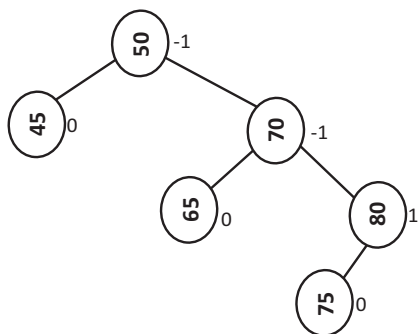


Figure 7.29a Deletion of element from an AVL tree

If we delete node 65, it is no longer an AVL tree since node 70 has balance factor (-2) and also node 50 has balance factor -2 as shown in Figure 7.29b.

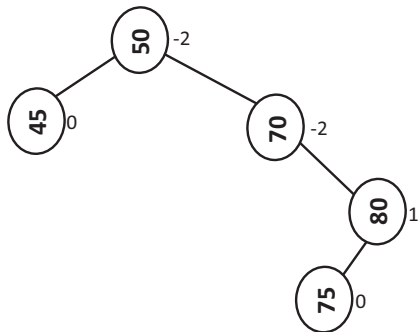


Figure 7.29b Deletion of element from an AVL tree

A restructuring is needed to make it an AVL tree. Unlike in the case of insertion where the restructuring is done locally, in deletion, the imbalance may percolate upward up to the root. Therefore, we follow the path backward up to the root, restructuring the sub-trees wherever needed using the rotations described earlier.

Let us take a concrete example. Consider deleting 8 from the AVL tree in Figure 7.29c.

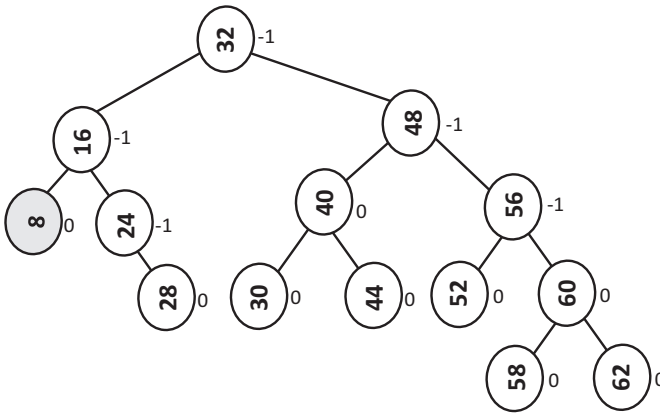


Figure 7.29c Deletion of element from an AVL tree

We start at 32, follow the path  $32 \rightarrow 16 \rightarrow 8$ . After deletion of 8, we move back to the root along the same path. Observe that the parent of 8, i.e., 16 has a balance factor  $-2$ . We locally restructure the sub-tree to obtain the tree in Figure 7.29d.

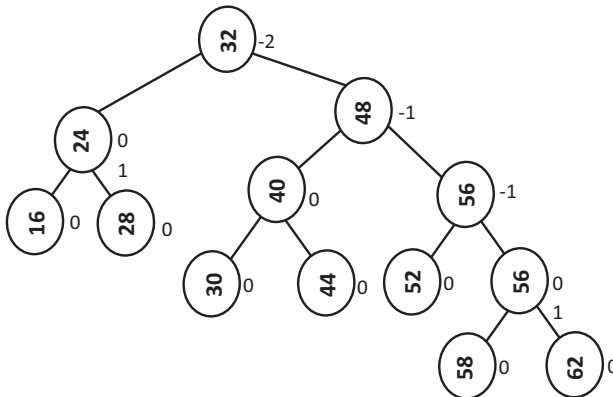


Figure 7.29d Deletion of element from an AVL tree



---

```

    # Update the balance factor and balance the tree
    balanceFactor = self.getBalance(root)
    if balanceFactor > 1:
        if key < root.left.key:
            return self.rightRotate(root)
        else:
            root.left = self.leftRotate(root.left)
            return self.rightRotate(root)

    if balanceFactor < -1:
        if key > root.right.key:
            return self.leftRotate(root)
        else:
            root.right = self.rightRotate(root.right)
            return self.leftRotate(root)

    return root

# Function to delete a node
def delete_node(self, root, key):

    # Find the node to be deleted and remove it
    if not root:
        return root
    elif key < root.key:
        root.left = self.delete_node(root.left, key)
    elif key > root.key:
        root.right = self.delete_node(root.right, key)
    else:
        if root.left is None:
            temp = root.right
            root = None
            return temp
        elif root.right is None:
            temp = root.left
            root = None
            return temp
        temp = self.getMinValueNode(root.right)
        root.key = temp.key
        root.right = self.delete_node(root.right,
                                     temp.key)

    if root is None:
        return root

    # Update the balance factor of nodes
    root.height = 1 + max(self.getHeight(root.left),
                          self.getHeight(root.right))

    balanceFactor = self.getBalance(root)

```

```
# Balance the tree
if balanceFactor > 1:
    if self.getBalance(root.left) >= 0:
        return self.rightRotate(root)
    else:
        root.left = self.leftRotate(root.left)
        return self.rightRotate(root)
if balanceFactor < -1:
    if self.getBalance(root.right) <= 0:
        return self.leftRotate(root)
    else:
        root.right = self.rightRotate(root.right)
        return self.leftRotate(root)
return root

# Function to perform left rotation
def leftRotate(self, z):
    y = z.right
    T2 = y.left
    y.left = z
    z.right = T2
    z.height = 1 + max(self.getHeight(z.left), self.
        getHeight(z.right))
    y.height = 1 + max(self.getHeight(y.left), self.
        getHeight(y.right))
    return y

# Function to perform right rotation
def rightRotate(self, z):
    y = z.left
    T3 = y.right
    y.right = z
    z.left = T3
    z.height = 1 + max(self.getHeight(z.left), self.
        getHeight(z.right))
    y.height = 1 + max(self.getHeight(y.left), self.
        getHeight(y.right))
    return y

# Get the height of the node
def getHeight(self, root):
    if not root:
        return 0
    return root.height

# Get balance factore of the node
def getBalance(self, root):
    if not root:
```

---

```

        return 0
    return self.getHeight(root.left) - self.getHeight
        (root.right)

def getMinValueNode(self, root):
    if root is None or root.left is None:
        return root
    return self.getMinValueNode(root.left)

# Print the tree
def display(self, currPtr, indent, last):
    if currPtr != None:
        print(indent,end="")
        if last:
            print("R----",end="")
            #sys.stdout.write("R----")
            indent += "        "
        else:
            print("L----",end="")
            #sys.stdout.write("L----")
            indent += "|        "
        print(currPtr.key)
        self.display(currPtr.left, indent, False)
        self.display(currPtr.right, indent, True)

myTree = AVLTree()
root = None
nums=[]
print("Enter 0 to exit Inserting Data into the AVL Tree ")
print("Enter the node of AVL Tree ")
x = int(input("Enter an element: "))
while(x!=0):
    nums.append(x)
    x = int(input("Enter an element: "))
nums=set(nums)
for num in nums:
    root = myTree.insert_node(root, num)
myTree.display(root, "", True)

key = int(input("Enter an element to Delete: "))
if(key not in nums):
    print(key,"Doesnot Exist")
else:
    root = myTree.delete_node(root, key)
    print("After Deletion: ")
    myTree.display(root, "", True)

```



## Output

```
Enter 0 to exit Inserting Data into the AVL Tree
Enter the node of AVL Tree
Enter an element: 5
Enter an element: 6
Enter an element: 2
Enter an element: 9
Enter an element: 1
Enter an element: 0
R----2
    L----1
    R----6
        L----5
        R----9
Enter an element to Delete: 5
After Deletion:
R----2
    L----1
    R----6
        R----9
```

## 7.8 SPLAY TREES

The main concern in balancing trees is to keep them from becoming lopsided and ideally allowing leaves to occur only at one or two levels. Therefore, if a newly arriving element disturbs the balance of the tree, a restructuring of the tree is done locally (AVL method). This restructuring is needed to reduce the average access time. However, not all elements are accessed with the same frequency. If an element at the tenth level of the tree is accessed infrequently, it may not be a worthwhile exercise to rebalance the tree. However, if the same element is frequently accessed, it makes a great difference if it is at the second level. The average access time will be greatly reduced if the frequently accessed elements are closer to the root. Therefore, the strategy in self-balancing trees should be to restructure the tree by moving up the elements frequently accessed.

One strategy to find the frequently accessed elements is to assume that the element being accessed has a good chance of being accessed again soon (a good example is the loop variable in a for loop). Therefore, it may be moved up the tree with no further restructuring.

Splaying is a method to move the recently accessed element to the root. This move-to-the-root-strategy has three cases depending on the links between the child, parent and the grandparent. Let X be the recently accessed element and let P be its parent and G its grandparent.

Case 1: X's parent P is the root. Perform a single rotation about the root (P) making X the root. This is called a “zig” step (also called singular splay). It is the same as a single right rotation (Figure 7.30).

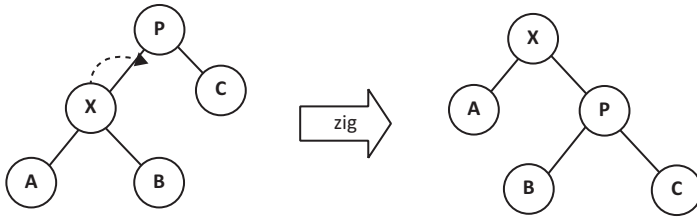


Figure 7.30 Zig operation in splay tree

Its mirror situation is known as a “zag” step. It is the same as a single left rotation (Figure 7.31).

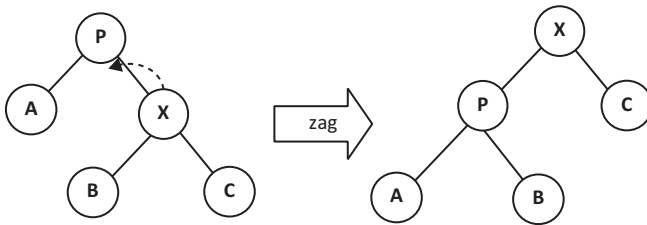


Figure 7.31 Zag operation in splay tree

Case 2: X is the left child of its parent P and P is the left child of its parent G (Figure 7.32). Or the mirror situation where X is the right child of its parent P and P is the right child of its parent G. This is called a homogeneous configuration. First rotate P about G and then rotate X about P. This is called

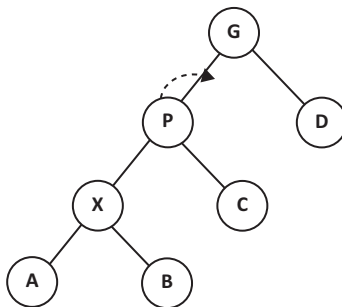


Figure 7.32 Zig-zig operation in splay tree

the “zig-zig” step. It is also known as a double zig (double right rotation) (Figure 7.33a).

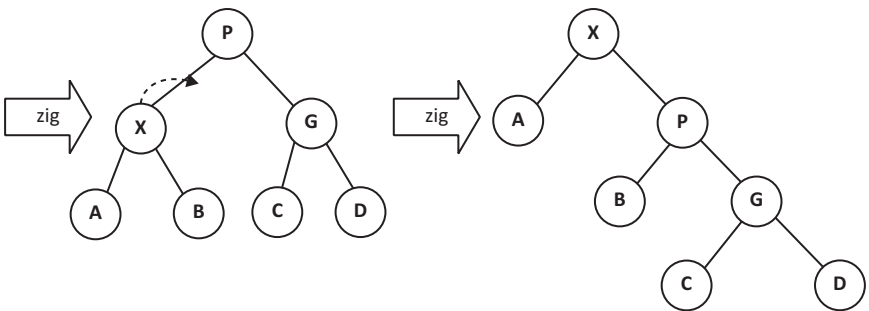


Figure 7.33a Zig-zig operation in splay tree

Same as Figure 7.33b.

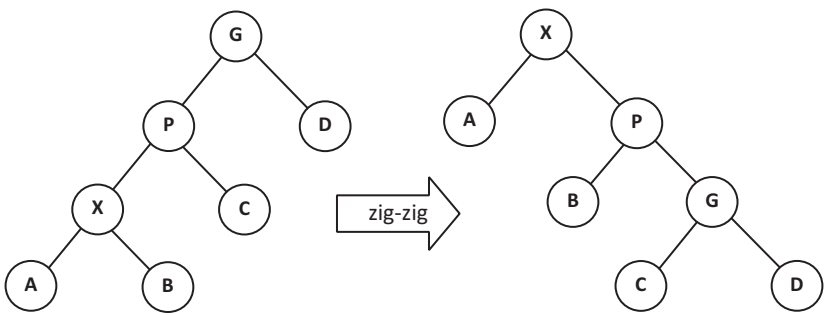


Figure 7.33b Zig-zig operation in splay tree

Its mirror situation is known as a “zag-zag” step. It is the same as a double left rotation (Figure 7.34a).

Same as Figure 7.34b.

Case 3: X is the left child of its parent P and P is the right child of its parent G. Or its mirror situation where X is the right child of P and P is the left child of G. This is called a heterogeneous configuration. First, perform a rotation of X about P and then about G. This is called a “zig-zag” step (Figure 7.35a).

Same as Figure 7.35b.

Its mirror situation is known as a “zag-zig” step. It is the same as left-right rotation (Figure 7.36a).

Same as Figure 7.36b.

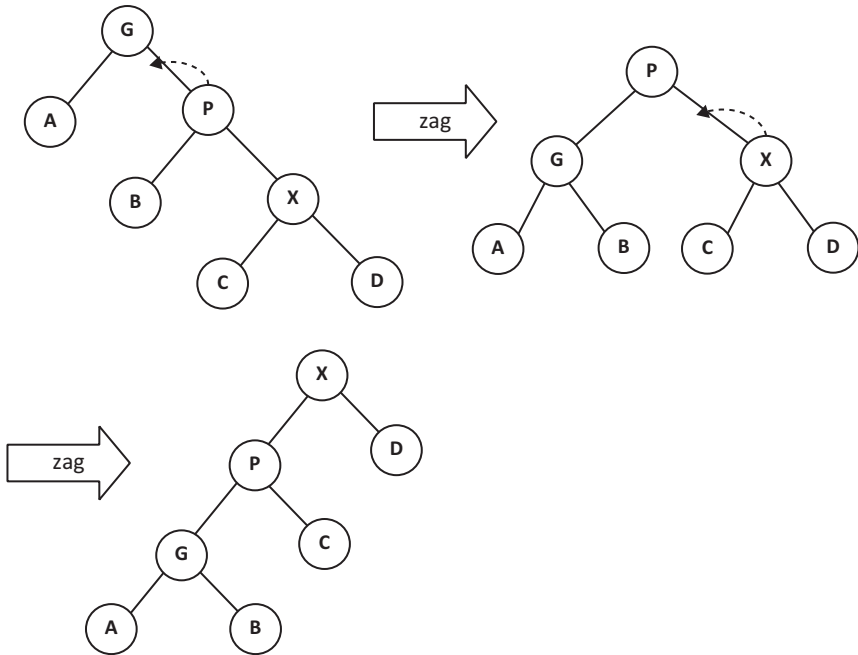


Figure 7.34a Zig-zig operation in splay tree

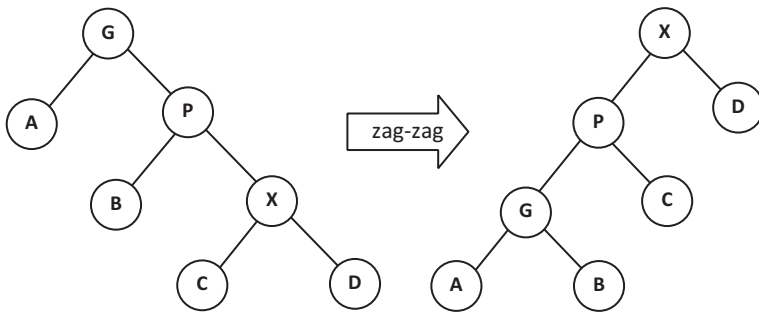


Figure 7.34b Zig-zag operation in splay tree

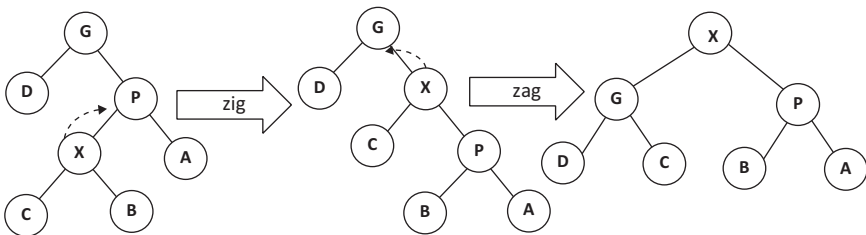


Figure 7.35a Zig-zag operation in splay tree

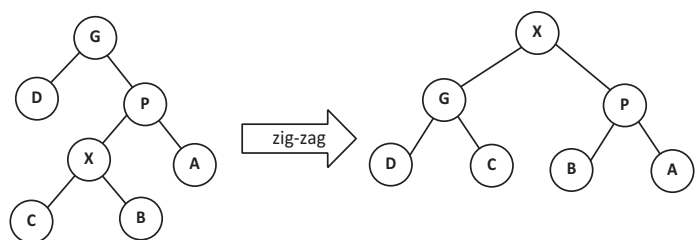


Figure 7.35b Zig-zag operation in splay tree

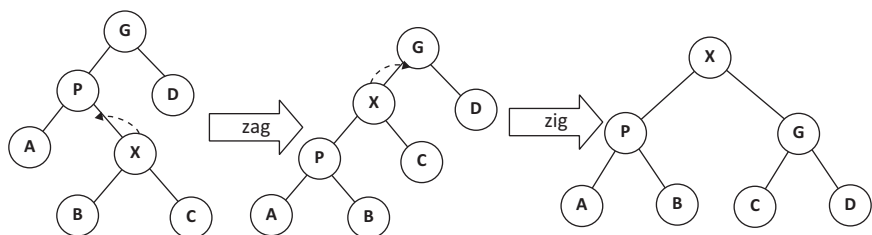


Figure 7.36a Zag-zig operation in splay tree

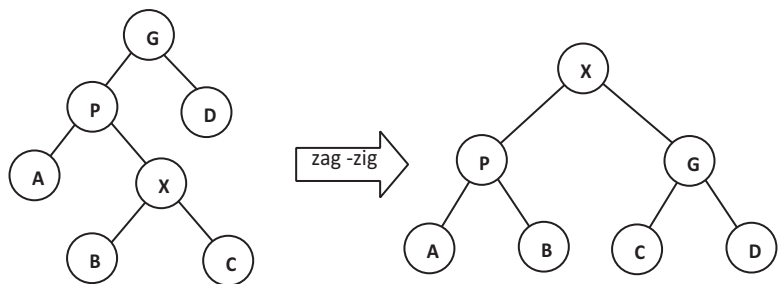


Figure 7.36b Zag-zig operation in splay tree

### 7.8.1 Insertion: Interchange B and C

1. Insert the element X as we do in a binary search tree.
2. Perform a splay (one of the three cases may occur).
3. The newly inserted element becomes the root.

**Deletion:** We use the same method as in a normal binary search tree. We splay the parent of the deleted node.

**Example:** With 5, 10, 15, 6, 9, 7, 11, 4 form a splay tree inserting the elements in the same order (Figures 7.37a to 7.37g).

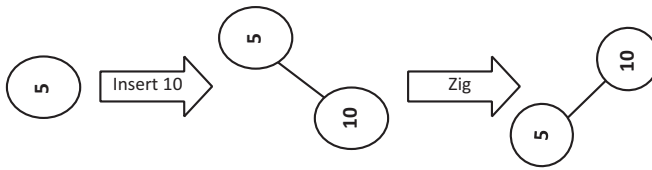


Figure 7.37a Insertion of 10 into a splay tree

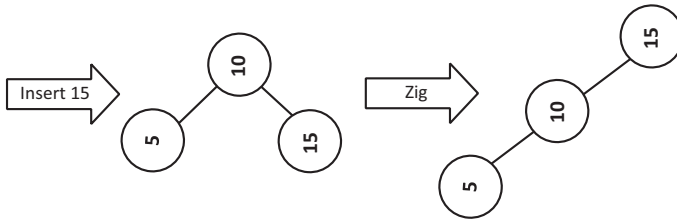


Figure 7.37b Insertion of 15 into a splay tree

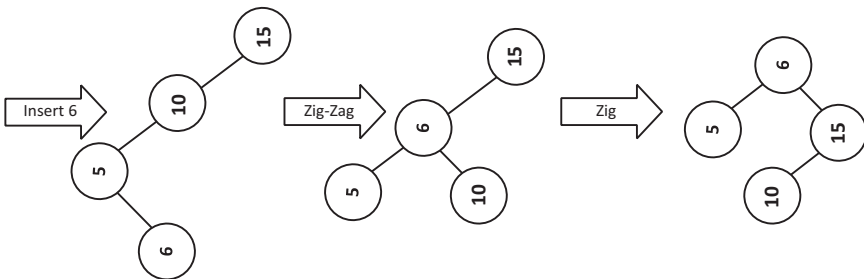


Figure 7.37c Insertion of 6 into a splay tree

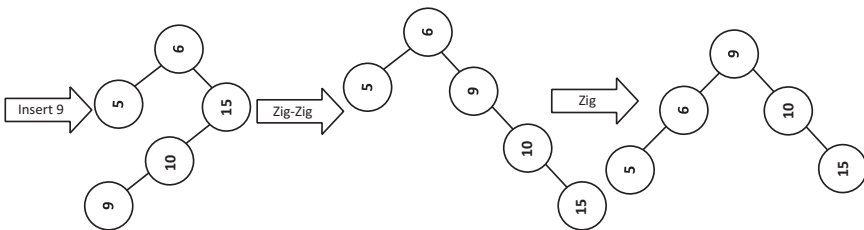


Figure 7.37d Insertion of 9 into a splay tree

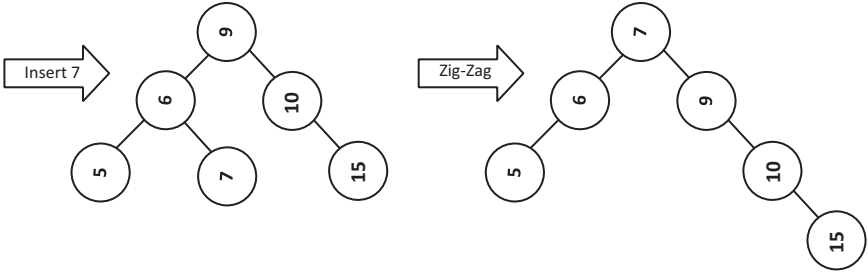


Figure 7.37e Insertion of 7 into a splay tree

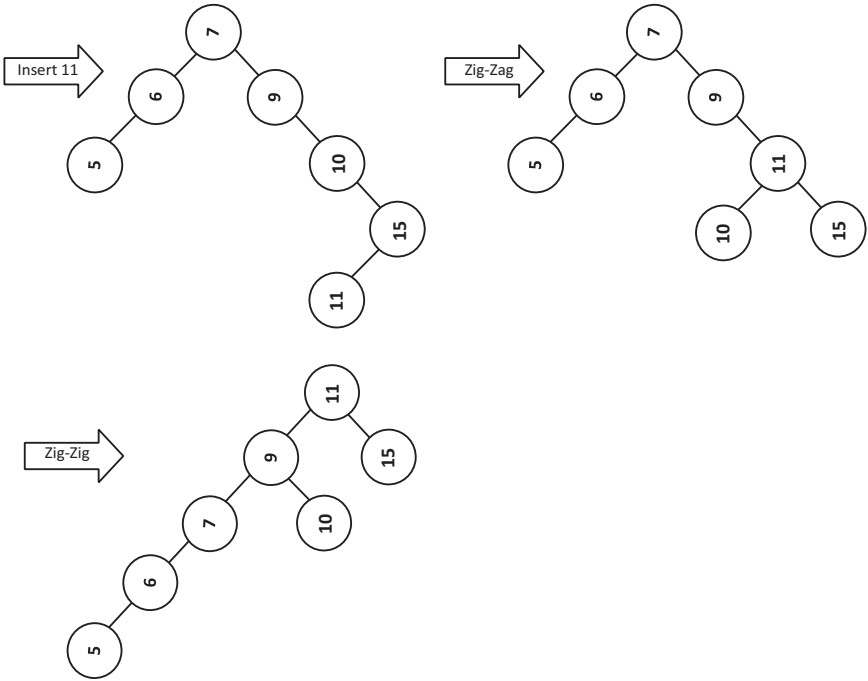


Figure 7.37f Insertion of 11 into a splay tree

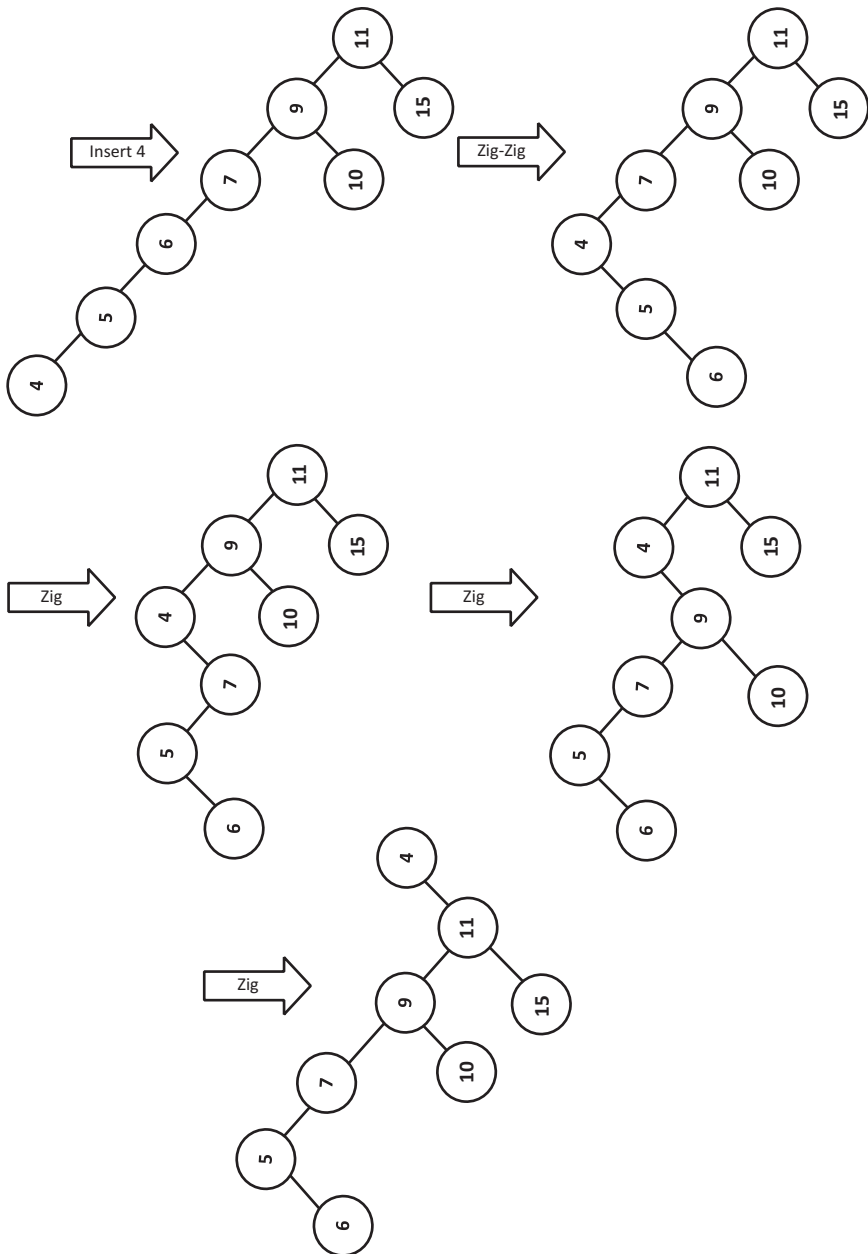


Figure 7.37g Insertion of 4 into a splay tree





- 
4. If a new node is inserted as the left child to a node that is the right child of its parent, then we make an
- a. RL rotation
  - b. LR rotation
  - c. LL rotation
  - d. RR rotation
5. We need a double rotation
- a. When the parent of the newly inserted and its grandparent differ in sign in their balance factors
  - b. When the balance factors are same
  - c. When both of them are in the same sub-tree
  - d. None of the above
6. In a splay tree, the newly accessed node is
- a. A leaf node
  - b. A node with only right child
  - c. A node with only left child
  - d. The root
7. If the key values 2, 3, 7, 8, 9, 1 are inserted into a splay tree, the root contains
- a. 2
  - b. 7
  - c. 1
  - d. 9
8. Why is a splay tree called the perfect tree?
- a. Space efficiency
  - b. Searching is quick
  - c. Easier to program and fast access time
  - d. Easy to construct
9. Why are splay trees preferred?
- a. Easier to program and faster access to recently accessed items
  - b. Space efficiency
  - c. Difficult to program
  - d. Quick searching
10. Which of the following rotation in AVL tree known as single rotation?
- a. LL and RR
  - b. LL and LR
  - c. LR and RL
  - d. RR and RL

11. A binary search tree is generated by inserting integers one by one in a particular order. Which traversal prints the nodes in the same order?
- a. Pre-order
  - b. In-order
  - c. Post-order
  - d. Level-order
12. What is the height of a complete binary tree with  $n$  nodes?
- a.  $\log(n+1) - 1$
  - b.  $\log n$
  - c.  $\log(n+1) + 1$
  - d.  $\log n - 1$
13. What is a splay operation?
- a. Moving leaf node
  - b. Moving root to leaf
  - c. Moving parent to child node
  - d. Moving node to root
14. A full binary tree with  $2n + 1$  nodes contains
- a.  $n$  leaf nodes
  - b.  $n$  internal nodes
  - c.  $n - 1$  leaf nodes
  - d.  $n - 1$  internal nodes
15. The minimum number of key values in a node of a B-tree with  $m+1$  links is
- a.  $\lceil m/2 \rceil$
  - b.  $\lfloor m/2 \rfloor$
  - c.  $m+1$
  - d.  $m-1$
16. The minimum number of key values in the root in a B-tree is
- a. 2
  - b. 3
  - c. 4
  - d. 5
17. The disadvantage of splay tree is
- a. Splay operations are difficult
  - b. Splay tree performs unnecessary splay when a node is only being read
  - c. No disadvantage
  - d. The height of a splay tree can be linear when accessing elements in non-decreasing order
18. The advantage of a B-tree is
- a. The number of disc accesses is reduced
  - b. The tree is balanced
  - c. Both of them
  - d. None of them

19. Which of the following is a self-adjusting or self-balancing binary search tree?

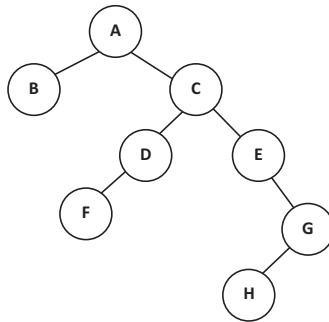
- a. Splay tree
- b. Red-black tree
- c. AVL tree
- d. All of the above

20. If 2000 numbers are used to construct a binary search tree, the maximum height of the tree is

- a. 1
- b. 11
- c. 1999
- d. 2000

### Descriptive questions

1. List the nodes in the binary tree given below in
  - i. Pre-order traversal
  - ii. In-order traversal
  - iii. Post-order traversal



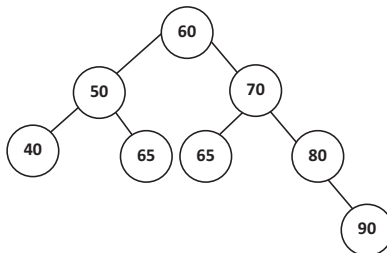
2. The nodes visited in the pre-order traversal and in-order traversal are given below.

Pre-order: ABCDEFGHIJKLM

In-order: CEDFBAHJIKGML

Draw the binary tree.

3. Insert node 100 in the following AVL tree. The resulting tree must be an AVL tree.



What is the balance factor of the root after insertion?

4. Starting from an empty AVL tree, insert the following key values:  
24, 39, 35, 47, 58, 36, 71, 100 in that order

**Answers to true–false questions**

1. True
2. False
3. True
4. True
5. False

**Answers to fill-in-the-blank questions**

1. height
2.  $2^{i-1}$
3. maximum level
4. leaf node
5. left-skewed tree

**Answers to multiple-choice questions**

- |     |   |     |   |     |   |     |   |     |   |
|-----|---|-----|---|-----|---|-----|---|-----|---|
| 1.  | c | 2.  | a | 3.  | b | 4.  | a | 5.  | a |
| 6.  | d | 7.  | d | 8.  | c | 9.  | a | 10. | a |
| 11. | d | 12. | b | 13. | d | 14. | b | 15. | a |
| 16. | a | 17. | d | 18. | d | 19. | d | 20. | c |

# Graphs

---

### LEARNING OBJECTIVES

After studying this chapter, readers will be able to:

- Know about graphs and the types
- Represent a graph
- Perform graph traversal
- Know the AND/OR graph
- Know the bi-connected component
- Understand topological sorting

One disadvantage of the tree data structure is that it is completely hierarchical in that it represents only parent–child relationships and only indirectly a sibling relationship. When we lift this limitation, we get the data structure graph. Generally, no restriction is imposed on the number of vertices nor on the number of connections a vertex can have with other vertices. We shall define the graph more precisely before describing several applications of graphs.

A simple graph  $G = (V, E)$  consists of a non-empty set of vertices  $V$  and possibly an empty set of edges, each edge being a set of two vertices from  $V$ .

### 8.1 INTRODUCTION

A graph can be defined (and represented) as a set of vertices and a set of edges. An edge is a connection between two vertices. The graph shown in Figure 8.1 is a set of vertices

$$V = \{1, 2, 3, 4, 5, 6\}$$

and a set of edges

$$E = \{ (1, 2), (1, 5), (2, 3), (3, 4), (4, 5), (4, 6) \}$$

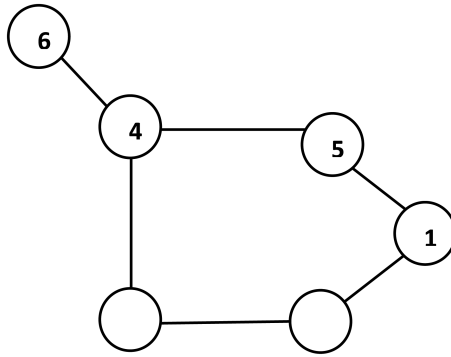


Figure 8.1 A graph

- *Undirected graphs:* In an undirected graph, the order of vertices in the pairs representing the edge set does not matter (as in the graph in Figure 8.1). In undirected graphs, the edges are drawn as lines between pairs of vertices. The adjacency relation in an undirected graph is symmetric. So, if  $u \sim v$  is a relation representing the fact that  $(u, v)$  is an edge, then  $(v \sim u)$  also holds.
- *Directed graphs:* In a directed graph, the order of vertices in the representation is important. If  $u \sim v$  is true (representing the fact that  $u$  to  $v$  is an edge),  $v \sim u$  may not be true, which means that  $v$  to  $u$  may not be an edge. We usually use arrows to represent edges in a directed graph. An arrow is drawn from  $u$  to  $v$  only if  $(u, v)$  is in the edge set. A simple path in a graph from  $u$  to  $v$  is defined as a set of vertices  $\{u, v_1, v_2, \dots, v_n, v\}$  such that the edges  $(u, v_1), (v_1, v_2), \dots, (v_n, v)$  are all in the edge set and  $v_1, v_2, \dots, v_n$  are all distinct.

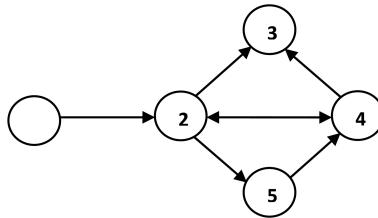


Figure 8.2 Directed graph

For the graph shown in Figure 8.2, the vertex set is  $\{1, 2, 3, 4, 5\}$  and the edge set is  $\{(1, 2), (2, 3), (2, 4), (2, 5), (4, 2), (4, 3), (5, 4)\}$ .

- *Vertex-labeled graph:* Each vertex in addition to the label it carries also contains some data. In the graph in Figure 8.3, the labels to the vertices are shown outside the nodes and the data is shown inside the nodes.

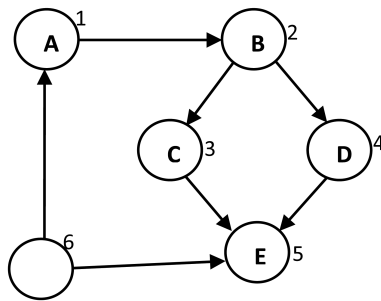


Figure 8.3 Vertex-labeled graph

We can show the data in the vertex set, like

$$V = \{(1, A), (2, B), (3, C), (4, D), (5, E), (6, F)\}$$

and the edge set as

$$E = \{(1, 2), (2, 3), (2, 4), (3, 5), (4, 5), (6, 1), (6, 5)\}$$

- *Cyclic graph*: A cyclic graph is a directed graph with at least one cycle. A cycle in a graph is a directed path from any vertex to itself. In Figure 8.4, there are two cycles and they are

$$2 \rightarrow 4 \rightarrow 5 \rightarrow 7 \rightarrow 6 \rightarrow 2$$

and

$$4 \rightarrow 3 \rightarrow 7 \rightarrow 6 \rightarrow 2 \rightarrow 4$$

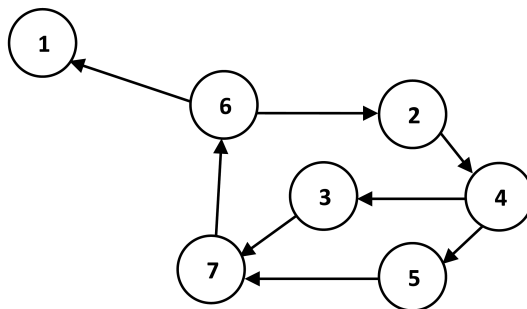


Figure 8.4 Cyclic graph



- *Edge-labeled graph*: In an edge-labeled graph, the edges are associated with labels. The labels are shown as a triple  $(u, v, X)$  showing an edge from  $u$  to  $v$  with label  $X$ .

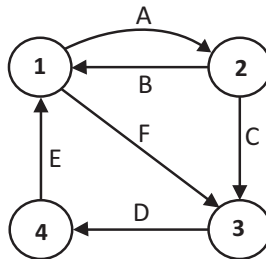


Figure 8.5 Edge-labeled graph

In the graph in Figure 8.5, the vertex set is  $\{1, 2, 3, 4\}$  and the edge set is  $\{(1, 2, A), (2, 1, B), (2, 3, C), (3, 4, D), (4, 1, E), (1, 3, F)\}$ , where

A is the label associated with  $(1, 2)$ .

B is the label associated with  $(2, 1)$ .

C is the label associated with  $(2, 3)$ .

D is the label associated with  $(3, 4)$ .

E is the label associated with  $(4, 1)$ .

F is the label associated with  $(1, 3)$ .

- *Weighted graphs*: If the labels associated with the edges are numbers indicating the importance of the edge, we call the graph a weighted graph (Figure 8.6). The numbers can be distances between two cities where the nodes represent cities or the cost associated with the completion of phase I of a project and taking it to phase II, where the nodes represent different phases of a project.

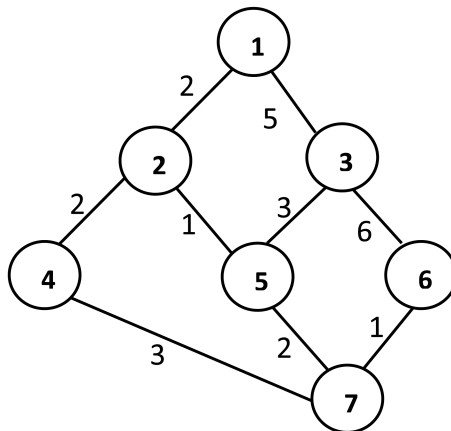


Figure 8.6 Weighted graph

- *Directed acyclic graphs (DAGs)*: A DAG is a directed graph with no cycles.

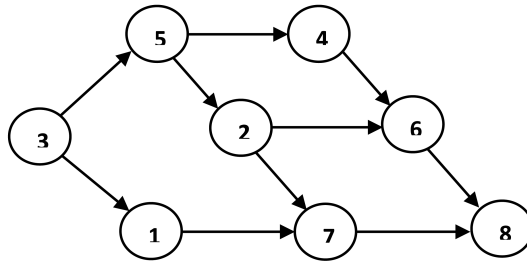


Figure 8.7 Directed acyclic graph

The vertex set is  $\{1, 2, 3, 4, 5, 6, 7, 8\}$ .

The edge set is  $\{(1, 7), (2, 6), (3, 1), (3, 5), (4, 6), (5, 2), (5, 4), (6, 8), (7, 2), (7, 8)\}$ .

The graph shown in Figure 8.7 is a directed acyclic graph since it contains no cycles. Vertices with only in-arrows are called sinks. In Figure 8.7, vertex 8 is a sink.

- *Adjacent vertex*: Graphs are often used to represent physical entities (a network of roads, relationships between people, etc.) inside a computer. A good choice of representation depends on the operations that the program needs to perform on the graph to solve a problem.

In a graph, two vertices are called adjacent if there is an edge connecting them. The degree of a vertex in a graph is the number of edges associated with it.

Consider the graph in Figure 8.8. It is a weighted undirected graph.

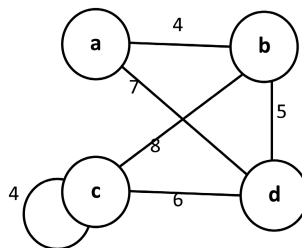


Figure 8.8 Adjacent vertex

- The degree of a is 2.
  - The degree of b is 3.
  - The degree of c is 4.
  - The degree of d is 3.
- The self-loop counts for two edges.

**Theorem:** Let  $G$  be an undirected graph with  $V$  vertices and  $N$  edges, then

$$2N = \sum_{v \in V} \text{degree}(v)$$

**Proof:** If  $u \sim v$  is an edge, each of the vertices is of degree 1. An edge therefore contributes 2 to the total degree of the graph. Hence, if there are  $N$  edges, their contribution to the total degree is  $2N$ .

In a directed graph, we define the in-degree of a vertex as the number of edges directed toward it and the out-degree as the number of edges directed away from it (Figure 8.9).

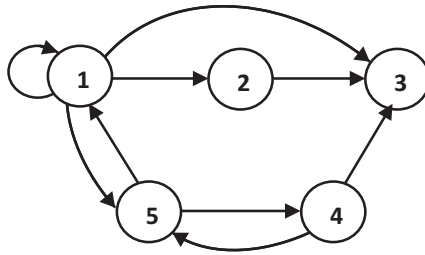


Figure 8.9 Directed graph

- The in-degree of 1 is 2
- The out-degree of 1 is 3.
- The in-degree of 2 is 1.
- The out-degree of 2 is 1.
- The in-degree of 3 is 3.
- The out-degree of 3 is 0.
- The in-degree of 4 is 1.
- The out-degree of 4 is 2.
- The in-degree of 5 is 2.
- The out-degree of 5 is 2.

**Theorem:** Let  $G$  be a directed graph (or a multi-graph) with  $V$  vertices and  $N$  edges. Then

$$2N = \sum_{v \in V} \text{in-degree}(v) + \sum_{v \in V} \text{out-degree}(v)$$

This is just an extension of the earlier theorem. The in-degree counts the edges into a node and the out-degree counts the edges going out of a node and the sum, therefore, gives the total number of edges, counted twice, i.e.,  $2N$ .

- **Complete graph:** The graph in which all the nodes have the same degree is called a complete graph.

A complete graph, with  $n$  vertices, denoted as  $K_n$ , is a simple graph in which there is an edge between every pair of vertices (Figure 8.10).

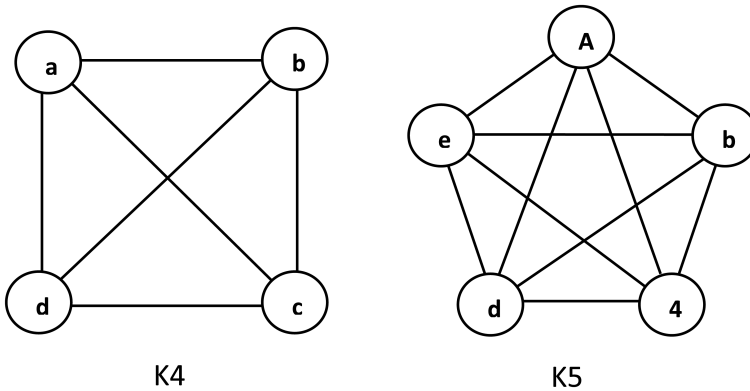


Figure 8.10 Complete graph

- **Connectivity:** The path in a graph is a sequence of distinct vertices connected by edges. Vertex  $v$  is reachable from  $u$ , if there is a path from  $u$  to  $v$ . A graph is connected if there is a path between every pair of vertices.

### Example

Given a graph with seven vertices, three of them of degree 2 and four of them of degree 1, is the graph connected?

**Answer:** No

**Reason:** The number of edges is equal to  $[(3 \times 2) + (4 \times 1)]/2 = 5$ . The minimum number of edges, if all of them are connected, is six.

If every pair of vertices is connected in a directed graph, then it is said to be strongly connected.

A directed graph is weakly connected if the underlying undirected graph is connected.

**Theorem:** In an undirected graph, there are at most  $n(n-1)$  edges.

**Proof:** We shall prove the theorem by induction. If  $V = 1$ , there are no edges. Hence the theorem is true.

Let  $V = n$ . Then the number of edges is equal to  $n(n-1)/2$  (hypothesis).

Let  $V = n+1$ . The number of edges added is maximum  $n$ , when the new vertex is connected to all other  $n$  vertices. Therefore, the maximum number of edges when  $V = n + 1$  is  $n(n-1)/2 + n = (n+1)n/2$ .

## 8.2 GRAPH REPRESENTATION

There are two ways to represent a graph:

- Adjacency list
- Adjacency matrix

### 8.2.1 Adjacency list

Adjacency list representation is used when the number of edges is less than  $V \times \log V$ . The adjacency list representation of a graph  $G$  consists of an array of linked lists, one for each vertex. Consider the graph in Figure 8.11.

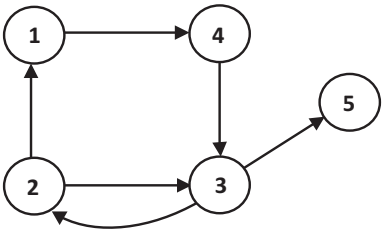


Figure 8.11 Directed graph

The corresponding representation is in Figure 8.12.

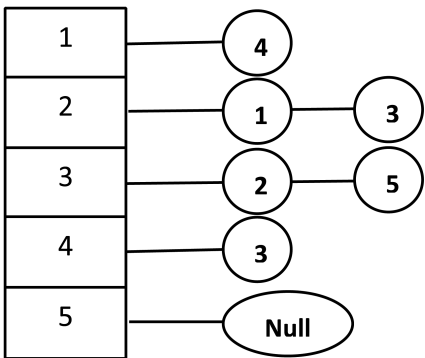


Figure 8.12 Adjacency list

Vertices in adjacency lists are stored in an arbitrary order. A potential disadvantage of adjacency list representation is there is no easy way to find if there is an edge between two vertices.

### 8.2.2 Adjacency matrix

If  $M$  is the matrix representing a graph  $G$ , then

$$M_{ij} = \begin{cases} 1 & \text{if there is an edge between vertices } i \text{ and } j \\ 0 & \text{otherwise} \end{cases}$$

|   | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 1 | 0 |
| 2 | 1 | 0 | 1 | 0 | 0 |
| 3 | 0 | 1 | 0 | 0 | 1 |
| 4 | 0 | 0 | 1 | 0 | 0 |
| 5 | 0 | 0 | 0 | 0 | 0 |

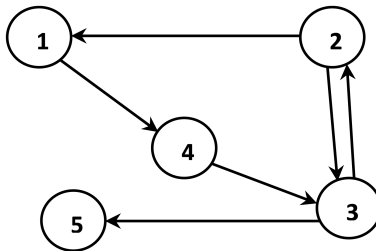


Figure 8.13 Adjacency matrix

The graph shown in Figure 8.12 can be represented by the matrix in Figure 8.13.

A tree can be considered as a connected graph with no cycles.

A connected graph with  $V$  vertices and  $V - 1$  edges must be a tree.

A weighted graph is also represented as an adjacency matrix where, if there is an edge, the entry is the weight and not 1. The adjacency matrix of the weighted graph is shown in Figure 8.14.

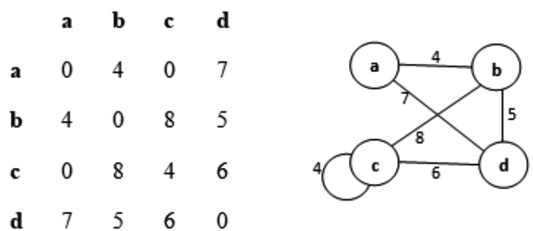


Figure 8.14 Adjacency matrix for a weighted graph

8.3 TRAVERSALS

In tree traversals, we start at the root and follow the edges until we encounter a null node. A node is never visited twice since there are no cycles. However, in graph traversals it is possible we visit the same node many times by following the edges. Hence it is necessary to keep track of nodes already visited and ignore them the second time. This may be accomplished by maintaining a Boolean array “visited,” which may be of the size of the number of nodes, one entry for each node of the graph, and is initialized to false. This array keeps track of the nodes already visited so far and ignores them when we reach them again.

8.3.1 Depth-first search

The depth-first search (DFS) visits all child vertices, recursively, before visiting the sibling vertices. In other words, it traverses the depth of any particular path before exploring its breadth. Often a stack is used for implementing the algorithm. There is nothing called a “root” vertex in a graph. We arbitrarily choose a vertex and start the search from there. It then iteratively visits adjacent unvisited vertices until it can no longer find an unvisited vertex from its present position. In other words, all adjacent vertices of the current position are already visited. The algorithm, then, “backtracks” along the previously visited vertices until it finds a vertex that is not yet visited. It follows the new path backtracking whenever necessary (if it finds dead ends) and ends only when the algorithm backtracks to the root.

Algorithm 1

- 1.Algorithm dfs(G,v)
- 2.Mark v visited and push it onto a stack
- 3.While stack is not empty do

4. Pop a vertex from the stack
5. Call it  $u$
6. Mark it visited, if not already marked visited.
7. Push the adjacent vertices of  $u$  onto the stack, after marking them visited.
8. Go to step 4
9. End while

Consider the graph in Figure 8.15.

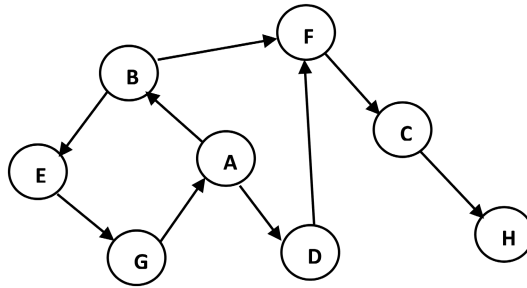


Figure 8.15 Directed graph

Let us start from A. Mark it *visited*. Push it onto the stack (Figure 8.16a). *Visited* is a Boolean array of the size of the number of nodes; one entry for each node. True if the node is visited, false otherwise.

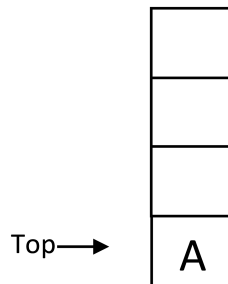
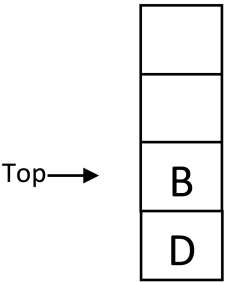


Figure 8.16a A is in the stack

Pop an element from the stack, show it in the output, and push its adjacent vertices D, B in that order onto the stack (any of the adjacent sides can be pushed onto the stack, but we push them in alphabetical order) after marking them visited. The stack position now is as in Figure 8.16b.

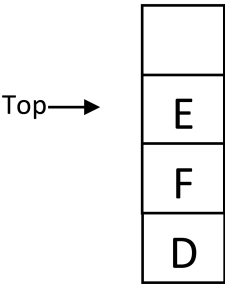




Output: A

Figure 8.16b A is popped and D, B are in the stack

We have B on top of the stack, pop it out and show it in the output. Its adjacent nodes are A, E, F. A is already visited. Push F and E onto the stack, in that order, after marking them visited. The stack position now is as in Figure 8.16c.



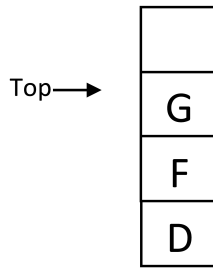
Output: A B

Figure 8.16c A and B are popped and D, F, E are in the stack

The top element E in the stack is popped out and shown in the output. Its adjacent is G. It is marked as visited and pushed onto the stack. Now the stack is as shown in Figure 8.16d.

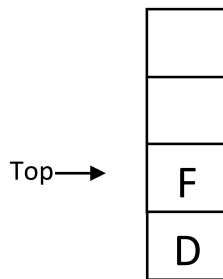
We have G on top of the stack. Pop it out and show it in the output. As there is no element to push, the position of the stack is as in Figure 8.16e.

The top element F in the stack is popped out and shown in the output. Its adjacent is C. It is marked as visited and pushed onto the stack. Now the stack is as shown in Figure 8.16f.



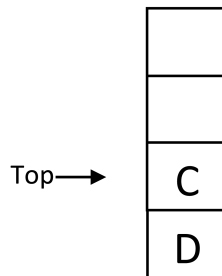
Output: A B E

Figure 8.16d A, B and E are popped and D, F, G are in the stack



Output: A B E G

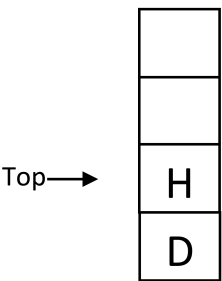
Figure 8.16e A, B, E and G are popped and D, F are in the stack



Output: A B E G F

Figure 8.16f A, B, E, G and F are popped and D, C are in the stack

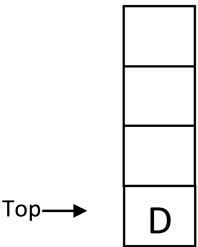
The top element C in the stack is popped out and shown in the output. Its adjacent is H. It is marked as visited and pushed onto the stack. Now the stack is as shown in Figure 8.16g.



Output: A B E G F C

Figure 8.16g A, B, E, G, F and C are popped and D, H are in the stack

We have H on top of the stack. Pop it out and show it in the output. As there is no element to push, the position of the stack is as in Figure 8.16h.



Output: A B E G F C H

Figure 8.16h A, B, E, G, F, C and H are popped and D is in the stack

We have D on top of the stack. Pop it out and show it in the output. As there is no element to push, the stack is empty.

The sequence of nodes contained in the output is

A B E G F C H D

### Python program I

```
graph = {
    'E' : ['B', 'G'],
    'G' : ['A', 'E'],
    'B' : ['A', 'E', 'F'],
    'A' : ['B', 'D', 'G'],
    'D' : ['A', 'F'],
    'F' : ['B', 'C', 'D'],
    'C': ['F', 'H'],
    'H': ['C']
}

visited = [] # List for visited nodes.
queue = []    #Initialize a queue

def bfs(visited, graph, node): #function for BFS
    visited.append(node)
    queue.append(node)

    while queue:          # Creating loop to visit each node
        m = queue.pop(0)
        print (m, end = " ")

        for neighbour in graph[m]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

# Driver Code
print("Following is the Breadth-First Search")
bfs(visited, graph, 'A')    # function calling
```

### Output

Following is the Depth-First Search

```
A
B
E
G
F
C
H
D
```

## 8.3.2 Breadth-first search

In a breadth-first search (BFS), we start at any node and visit all the adjacent nodes. We use the data structure queue to determine the next node to be visited. We also maintain a Boolean array *visited* to keep track of all

the nodes processed so far. To illustrate the procedure, let us use the same graph we used to illustrate the depth-first traversal (Figure 8.17). We use a queue instead of a stack.

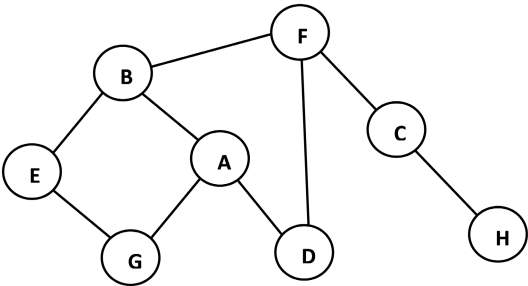


Figure 8.17 Undirected graph

We start from A. Mark it visited. Insert into the queue. Now remove A from the queue, display in the output and insert its adjacent vertices B D G in alphabetical order into the queue and mark them as visited.

The status of the queue now is shown in Figure 8.18a.

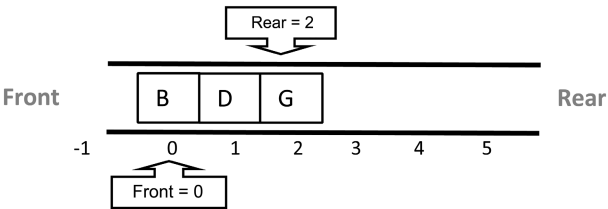


Figure 8.18a B D G are in the queue

Visited ☹ A G D B  
Output ☹ A

B is removed from the queue, and is put in the output. Its adjacent nodes E and F are inserted into the queue in that order and marked as visited. The current status of the queue and the array ‘visited’ are shown in Figure 8.18b.

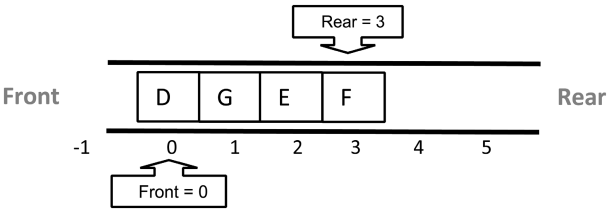


Figure 8.18b D G E and F are in the queue

Visited ☹️                      A G D B E F  
 Output ☹️                      A B

The next vertex to be processed is D. It is removed from the queue and added to the output and its adjacent vertices A and F are already visited. The status of the queue at this position is shown in Figure 8.18c.

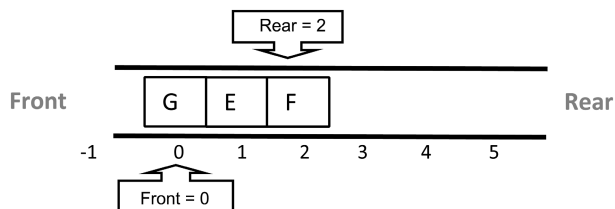


Figure 8.18c G E and F are in the queue

Visited ☹️                      A G D B E F  
 Output ☹️                      A B D

The next vertex in the queue is G. It is removed from the queue and added to the output (Figure 8.18d).

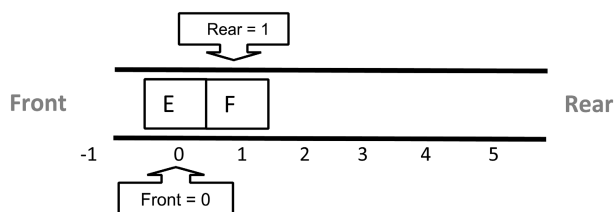


Figure 8.18d E and F are in the queue

Visited ☹️                      A G D B E F  
 Output ☹️                      A B D G

G's neighbors are already visited. The next vertex E is taken out from the queue and added to the output (Figure 8.18e). The status of the queue, the array visited and output are

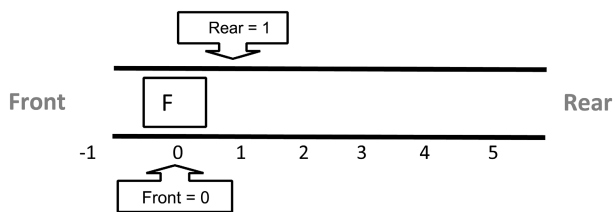


Figure 8.18e F is in the queue

Visited ☹                    A G D B E  
Output ☹                    A B D G E F

Remove F from the queue and add to the output. Its neighbor C is added to the queue (Figure 8.18f).

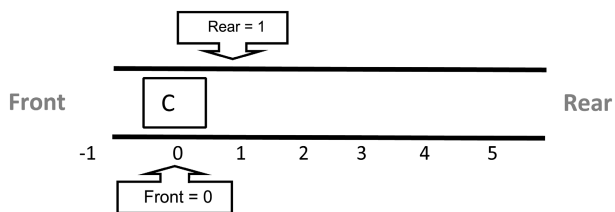


Figure 8.18f C is in the queue

Visited ☹                    A B D G E F C  
Output ☹                    A B D G E F

Remove C from the queue and add to the output. Its neighbor H is added to the queue after marking it visited (Figure 8.18g).

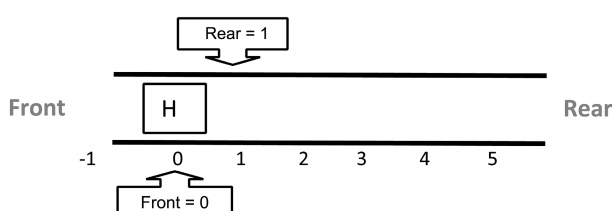


Figure 8.18g H is in the queue

Visited ☹                    A B D G E F C H  
Output ☹                    A B D G E F C

Lastly, H has no unvisited neighbors; it is added to the visited and output. Now the queue is empty and the path of the breadth-first search is

A B D G E F C H

### Python program 2

```
graph = {
    'E' : ['B','G'],
    'G' : ['A', 'E'],
    'B' : ['A','E','F'],
    'A' : ['B','D','G'],
    'D' : ['A','F'],
    'F' : ['B','C','D'],
    'C': ['F','H'],
    'H': ['C']
}

visited = [] # List for visited nodes.
queue = []    #Initialize a queue

def bfs(visited, graph, node): #function for BFS
    visited.append(node)
    queue.append(node)

    while queue:
        # Creating loop to visit each node
        m = queue.pop(0)
        print (m, end = " ")

        for neighbour in graph[m]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

# Driver Code
print("Following is the Breadth-First Search")
bfs(visited, graph, 'A')    # function calling
```

### Output

Following is the Breadth-First Search  
A B D G E F C H

## 8.4 AND/OR GRAPHS

AND/OR graphs are very useful for problem-solving. Suppose we want to solve problem A. Let us represent this as the root of a tree.



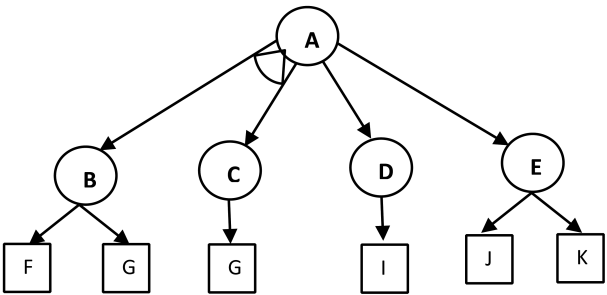


Figure 8.19a AND/OR graph

The tree structure in Figure 8.19a implies that, to solve problem A, solve both problems B and C or solve D or solve E. The tree structure can further be extended breaking down the problems B, C, D, E into sub-problems. The terminal square nodes represent solvable problems without further decomposition. As we can see, AND/OR graphs are not necessarily trees.

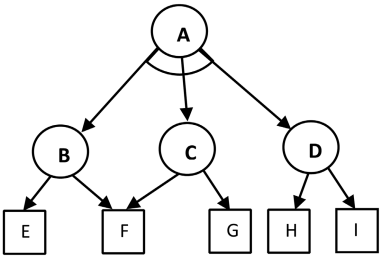


Figure 8.19b AND/OR graph

The diagram in Figure 8.19b is not a tree. Problem A can be solved by solving B, C, D. To solve B we have to solve E or F and the solution to F is also needed to solve C. Thus there is a cycle and hence is not a tree.

Let us assume that there is a cost associated with each edge. Therefore, the cost of the solution to a problem represented as an AND/OR graph is the sum of the edge costs. Consider the example in Figure 8.19c.

To solve P1 we have to solve P2 or P3 or P4. To solve P2 we have to solve P5 and P6. To solve P3 we have to solve both P6 and P7. Therefore, if we obtain the solution of P1 through P2, the cost is  $1 + 1 + 2 = 4$ . If we obtain the solution through P3, the cost is  $3 + 2 + 1 = 6$ . And through P4, it is 8. Therefore, the minimum cost path is  $P1 \rightarrow P2 \rightarrow (P5 + P6)$ .

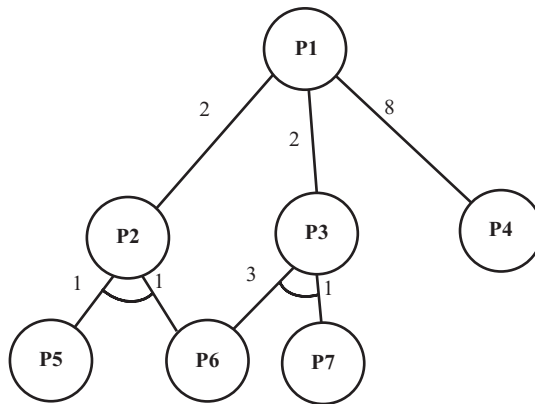


Figure 8.19c AND/OR graph

## 8.5 BI-CONNECTED COMPONENTS

Bi-connected components, also known as 2-connected components, are a type of maximal sub-graph in graph theory that maintains connectivity even after removing a vertex. They are connected at cut vertices, separating vertices or articulation points. These components are crucial in graph algorithms and network analysis, helping to identify critical points, analyze robustness, and identify bridges or articulation points affecting a graph's connectivity.

### 8.5.1 Connectivity

An undirected graph is said to be connected if there is a path between any two vertices. A graph is said to be  $n$ -connected if there are at least  $n$  different paths between any two vertices and the paths having no common vertices. A special type of graph is a 2-connected or bi-connected graph in which there are at least two different paths between any two vertices. A graph is not bi-connected if a vertex can be found that always has to be included in a path between at least two different vertices, say  $a$  and  $b$ . In other words, if this vertex is removed (along with incident edges), there is no way to reach from  $a$  to  $b$  and vice versa. This means that the graph is split into two or more sub-graphs. Such vertices are called articulation points or cut-vertices. If an edge causes its removal by a split of the graph into two sub-graphs, the edge is called a bridge. The sub-graphs obtained by removing the articulation points are called bi-connected components.

Consider the graph in Figure 8.20 (Horowitz, Sahni, Rajsekharan et al.).

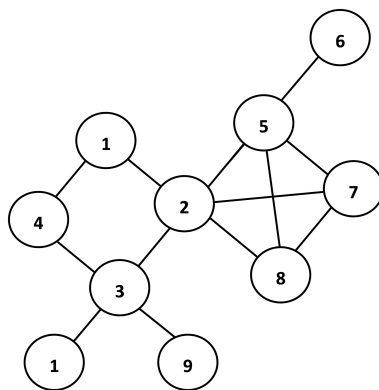


Figure 8.20 Bi-connected graph

Removal of vertex 2 creates two sub-graphs (Figure 8.21a).

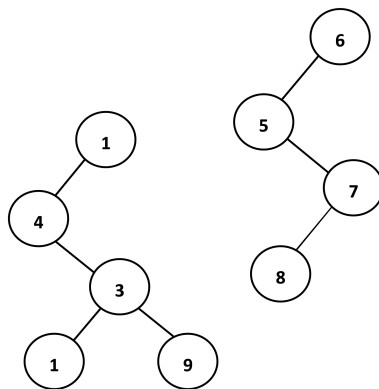


Figure 8.21a Bi-connected component

Hence vertex 2 is an articulation point. A graph is, thus, bi-connected if and only if it contains no articulation points. The sub-graphs obtained by removing the articulation points are called bi-connected components. In the case of a communication network, represented as a graph, it is not desirable to have any articulation points since any damage to the computer systems at the articulation points removes communication connection between several systems. Let  $G = (V, E)$ . We define a maximal bi-connected sub-graph of  $G$  as  $G' = (V', E')$  if and only if there is no sub-graph  $G'' = (V'', E'')$ , which is bi-connected such that  $V' \subset V''$  and  $E' \subset E''$ . A maximal bi-connected sub-graph is a bi-connected component.

Two bi-connected components can have at most one vertex in common and that point is an articulation point. Hence no edge can be in two bi-connected components. The bi-connected components of the previous graph are shown in Figure 8.21b.

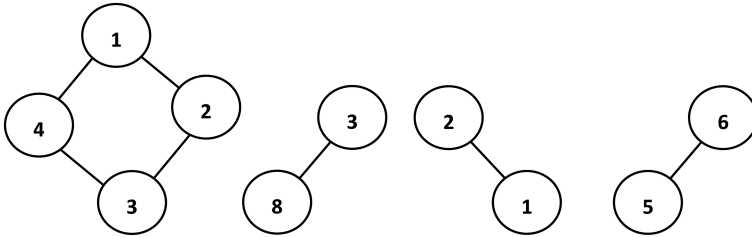


Figure 8.21b Bi-connected component

In order to construct a bi-connected graph, we have to make sure the articulation points are no longer articulation points by adding new edges.

### Algorithm 2

```

1. Algorithm bi-connected(G)
2. for each articulation point a do
3. {
4.     let  $B_1, B_2, \dots, B_k$  be the bi-connected components
       containing the vertex a.
5. let  $v_i \neq a$  be a vertex in  $B_i$ ,  $1 \leq i \leq k$ 
6. add to G the edges  $\langle v_i, v_{i+1} \rangle$ 
7. }
```

This construction converts a graph  $G$  into a bi-connected graph  $G'$ . Now the problem that remains is to identify articulation points. Let us first identify the depth-first spanning tree of  $G$ , as shown in Figure 8.22.

The depth-first algorithm creates a tree with forward edges (edges included in the tree) and back edges (edges not included in the tree). Thus, a vertex  $v$  in this tree is an articulation point, if there is at least one sub-tree, whose vertices are not connected with any of  $v$ 's predecessors by a back edge because none of the predecessor vertices can be reached from the forward vertices. It is only through the back edge (which is not in the tree but is in the graph) that we can reach a predecessor. In Figure 8.22, the spanning tree is shown in double lines. The number outside the nodes, called the depth-first number (dfn), shows the order in which the nodes are visited. The edges shown in dotted lines are known as back edges, which means that they are in the original graph but not in the depth-first spanning tree.

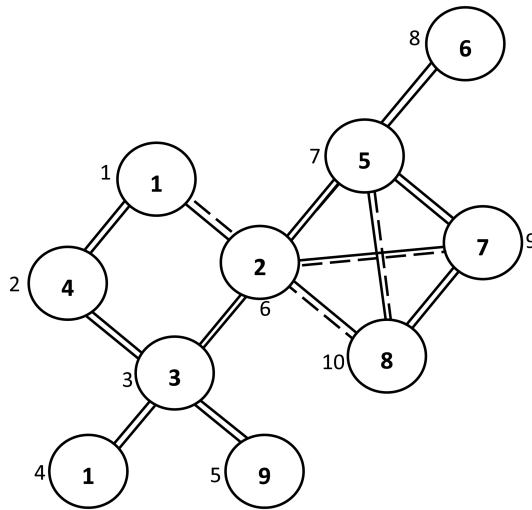


Figure 8.22 Bi-connected component

- The root node of the depth-first spanning tree is an articulation point if and only if it has two children. If we remove the root, the nodes in the left sub-tree cannot reach the nodes in the right sub-tree.
- The leaf node in the depth-first spanning tree is never an articulation point.
- Non-leaf, non-root node  $u$  is an articulation point if and only if no non-tree edge goes above  $u$  from a sub-tree below some child of  $u$ . This is because, if there is a non-tree edge connecting the children to the ancestors, the children remain connected to their ancestors, even when the parent is removed.

The presence of a back edge means the presence of an alternate path to the ancestors.

Let us define  $L(u)$  as the lowest depth-first number that can be reached from  $u$  using a path of its descendants and a back edge. Applying the definition we have  $L[1:10] = [1, 1, 1, 1, 6, 8, 6, 6, 5, 4]$ .

Let us calculate some of these numbers.

$L[1] = 1$ , since it can reach itself through  $1 \rightarrow 4 \rightarrow 3 \rightarrow 2$  and the back edge  $2 \rightarrow 1$ .

$L[2] = 1$ , since we can reach 1 from 2 through the back edge  $2 \rightarrow 1$ .

Similarly,  $L[3] = 1$  and  $L[4] = 1$ .

$L[5] = 6$ , since 5 can reach 2 through its descendants 7 and 8 and the back edge  $8 \rightarrow 2$  and  $\text{dfn}(2) = 6$ .

$L[6] = \text{dfn}(6) = 8$ , since it has no descendants.

$L[7] = 6$ , since we can reach node 2 whose  $\text{dfn}$  is 6 through  $7 \rightarrow 8$  and back edge  $8 \rightarrow 2$ . And similarly,  $L[8] = 6$ .

$L[9] = 5$  and  $L[10] = 4$ , their own depth-first numbers, since they have no descendants and no back edges.

If  $u$  is not a root, then  $u$  is an articulation point if and only if  $u$  has a child  $w$  such that  $L(w) \geq \text{dfn}(u)$ . Vertex 2 is an articulation point since its child 5 satisfies  $L(5) \geq \text{dfn}(2)$ , since  $L(5) = 6$  and  $\text{dfn}(2) = 6$ .

Similarly, nodes 3 and 5 are articulation points.

## 8.6 SHORTEST-PATH PROBLEM

Let  $G = (V, E)$  be a directed graph. The shortest path between two vertices in the graph is the path with shortest length (least number of edges) called the link distance. A breadth-first search discussed earlier is the algorithm for finding the shortest link distance from a single source vertex to all other vertices in the graph. A breadth-first search processes all vertices in the increasing order of distance from the source.

Now, let  $G = (V, E)$  be a weighted graph. Let us use a weight function that maps the edges to real numbers:

$$w: E \rightarrow \mathbb{R}$$

if  $e$  is an edge,  $w(e)$  is called the weight associated with the edge  $e$ . The length of the path from a vertex  $u$  to a vertex  $v$  is the sum of the weights associated with the edges that constitute the path between  $u$  and  $v$ . Let us denote the minimum distance between  $u$  and  $v$  as  $d(u, v)$ , if one exists.

### 8.6.1 Single-source shortest-path problem

Given a di-graph  $G = (V, E)$  with non-negative edge-costs (weights) and a special vertex  $s$ , called the source vertex, determine the shortest path and distance between the source vertex and all other vertices.

#### Dijkstra's algorithm

1. Let  $d(v)$  be the length of the shortest path from ' $s$ ' to each vertex  $v$ .
2. Initialize  $d(v)$  to  $\infty$  for all vertices of the graph.
3. Let  $d(s) = 0$ .
4. Process vertices to find new paths and update  $d(v)$ .

The processing is done as follows:

- There is a shortest path from ' $s$ ' to ' $u$ ' with length  $d(u)$ .
- There is a path from ' $s$ ' to ' $v$ ' with length  $d(v)$ .
- Another path to ' $v$ ' through ' $u$ ', if it exists, has length  $d(u) + w(u, v)$ .
- If  $d(u) + w(u, v) < d(v)$ , then the old path is replaced by the new path and  $d(v)$  is replaced by  $d(u) + w(u, v)$ .

The last statement can be made more precise by the following statement:

```

If( $d(u) + w(u,v) < d(v)$ )
{
     $d(v) = d(u) + w(u,v)$ ;
     $pred(v) = u$ ;
};

```

where  $pred(v)$  means the predecessor of  $v$ .

Let us apply Dijkstra's algorithm to the di-graph in Figure 8.23.

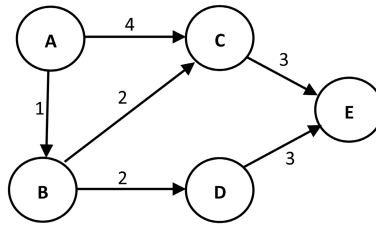


Figure 8.23 Dijkstra's algorithm for the graph

**Step 1:** Let A be the source vertex.

```

 $d(A) = 0$ ;
 $d(B) = d(C) = d(D) = d(E) = \infty$ 

```

**Step 2:** Find adjacent vertices of A.

```

Adj(A) = {B, C};
 $d(B) = 1$  ;  $d(C) = 4$  since they are less than  $\infty$ .
Pred(B) = A and pred(C) = A;

```

**Step 3:** Choose the vertex with a smaller distance value, which is C.

```

Adj(C) = {B, D}.
Update  $d(B)$  and  $d(D)$ 
 $d(B) > d(C) + w(C,B)$  //  $w(C,B)$  is the weight on the edge C->B
Therefore,  $d(B) = d(C) + w(C,B) = 4 + 2 = 6$ ;
Pred(B) = C;
 $d(D) = d(C) + w(C,D) = 4 + 2 = 6$ ;
pred(D) = C;

```

**Step 4**

```

Adj(B) = {E};
 $d(E) = d(B) + w(B,E) = 6 + 3 = 9$ ;
pred(E) = B;

```

### Step 5

$$\begin{aligned}\text{Adj}(D) &= E; \\ d(E) &= d(D) + w(D, E) = 3 + 3 = 6\end{aligned}$$

which is the same value computed earlier and we keep the earlier value and earlier predecessor. We show the shortest paths in the following diagram.

### Step 6

$$\text{Adj}(E) = \varnothing.$$

This means there are no outgoing edges from E. And there are no more vertices. The algorithm terminates.

Using the information about the predecessors of each vertex, we show the shortest paths to each vertex from A (Figure 8.24).

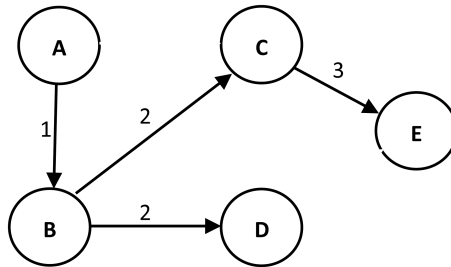


Figure 8.24 Shortest path after Dijkstra's algorithm

## 8.7 TOPOLOGICAL SORTING

**Definition:** A directed graph  $G$  in which vertices represent tasks or activities and edges represent precedence relations between tasks, is called an activity on vertex network (AOV). Topological sort is applied to an activity network to determine the order in which tasks are to be executed.

### Algorithm 3

1. Algorithm topological sort.
2. Input the AOV network and let  $n$  be the number of vertices.
3. for  $i = 1$  to  $n$  do
4. {
- 4.1. If every vertex has a predecessor, then the network has a cycle and is infeasible. Stop
5. Pick a vertex  $v$  that has no predecessors.
6. Output  $v$ .
7. Delete  $v$  and all edges leading out of  $v$ .
8. }



**Example:** Consider the activity network in Figure 8.25a.

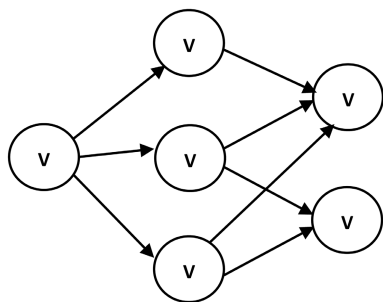


Figure 8.25a Topological sorting for the graph

Since  $v_1$  has no predecessors, remove  $v_1$  and all edges leading out of it (Figure 8.25b).

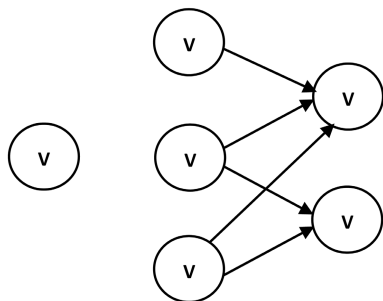


Figure 8.25b Topological sorting after removing edges from  $v_1$

Now neither  $v_2$ , nor  $v_3$ , nor  $v_4$  have any predecessors. Choose any one of them, say,  $v_2$  and remove  $v_2$  and all edges leading out of it (Figure 8.26).

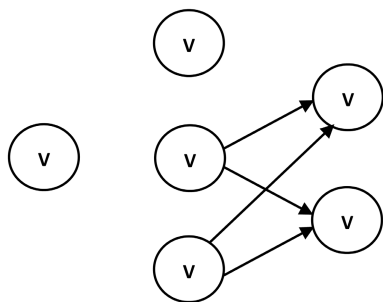


Figure 8.26 Topological sorting after removing edges from  $v_2$

Now remove  $v_3$  and all edges leading out of it (Figure 8.27).

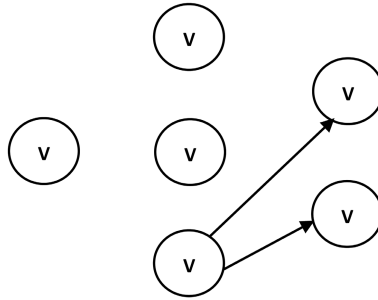


Figure 8.27 Topological sorting after removing edges from  $v_3$

We get  $v_1 v_2 v_3 v_4 v_5 v_6$  as the order in which the activities are to be executed.

### True–false questions

1. A graph is a tree.
  - a. True
  - b. False
2. An edge is a connection between two vertices.
  - a. True
  - b. False
3. A bi-connected graph is not essentially a connected graph.
  - a. True
  - b. False
4. The shortest path between two vertices in the graph is the least number of edges.
  - a. True
  - b. False
5. A queue is used for depth-first search.
  - a. True
  - b. False

### Fill-in-the-blank questions

1. A graph can be defined (and represented) as a set of \_\_\_\_\_ and \_\_\_\_\_.
2. Vertices in \_\_\_\_\_ are stored in an arbitrary order.
3. The graph in which all the nodes have the same degree is called a \_\_\_\_\_.
4. When labels are associated with the edges, a graph with numbers indicating the importance of the edge is called a \_\_\_\_\_.
5. A \_\_\_\_\_ in a graph is a directed path from any vertex to itself.

**Multiple-choice questions**

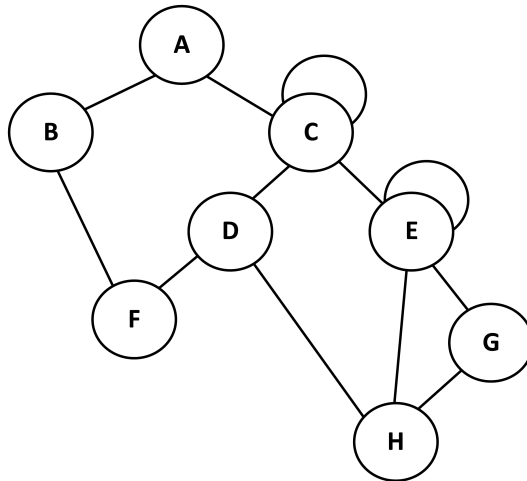
1. An AND/OR graph is a
  - a. Multiway tree
  - b. Binary tree
  - c. Not necessarily a tree
  - d. Not necessarily a binary tree
  
2. The total number of edges associated with a node is called
  - a. In-degree
  - b. Out-degree
  - c. Degree
  - d. None of the above
  
3. Which of the following statements is/are true for undirected graphs?  
P: number of odd degree vertices is even.  
Q: sum of all vertices is even.
  - a. P only
  - b. Q only
  - c. Both P and Q
  - d. None of the above
  
4. A tree with  $n$  vertices has
  - a.  $n$  edges
  - b.  $N-1$  edges
  - c.  $n+1$  edges
  - d. A cycle in the graph
  
5. A graph with one vertex and no edges is a/an
  - a. Isolated graph
  - b. Multigraph
  - c. Trivial graph
  - d. Di-graph
  
6. A graph in which all nodes are of equal degree is known as a
  - a. Di-graph
  - b. Complete graph
  - c. Multigraph
  - d. Regular graph
  
7. A spanning tree of a graph is one that includes
  - a. All edges of the graph
  - b. All vertices of the graph
  - c. Only vertices with even degrees
  - d. None of the above
  
8. The degree of any vertex of a graph is
  - a. The number of edges incident with the vertex
  - b. Number of vertices in the graph
  - c. Number of vertices adjacent to that vertex
  - d. Number of edges in the graph

9. A simple graph that contains exactly one edge between each pair of distinct vertices is a
  - a. Tree
  - b. Degree
  - c. Path
  - d. Complete graph
10. When information about the vertices is more desirable than information about the edges, we use
  - a. Adjacency matrix
  - b. Adjacency list
  - c. Both a and b
  - d. None of the above
11. In an adjacency matrix, if any diagonal element is 1, it indicates
  - a. There is a self-loop.
  - b. There is a path.
  - c. There is a cycle.
  - d. None of the above.
12. What is a sequence of edges that begins at a vertex of a graph and travels along edges of the graph, always connecting pairs of adjacent vertices?
  - a. Leaf nodes
  - b. Internal nodes
  - c. Path
  - d. Cycle
13. What is the removal of a vertex known as when it produces a subgraph with more connected components than in the original graph?
  - a. Connected graph
  - b. Articulation point
  - c. Complete Graph
  - d. Even degree
14. What is the maximum number of edges in a acyclic undirected connected graph with  $n$  vertices?
  - a.  $n/2$
  - b.  $n-1$
  - c.  $n$
  - d.  $n+1$
15. What is the maximum degree of any node in a simple graph with  $n$  vertices?
  - a.  $n/2$
  - b.  $n-1$
  - c.  $n$
  - d.  $n+1$
16. The number of circuits in a tree is
  - a. 0
  - b. 1
  - c. 2
  - d. 3

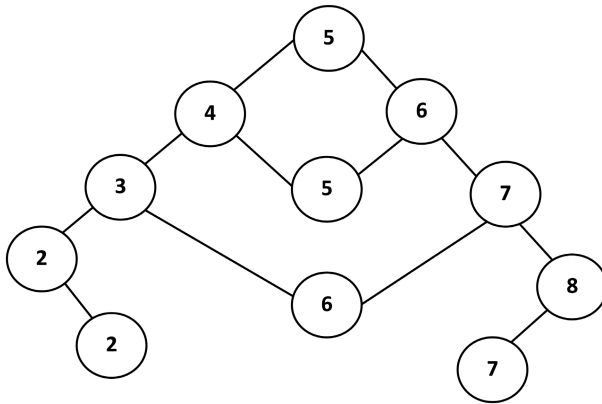
17. The degree of each vertex in  $K_n$  is
- a.  $2n-1$
  - b.  $n$
  - c.  $n-1$
  - d.  $N-2$
18. What is the number of distinct simple graphs with up to three nodes?
- a. 7
  - b. 9
  - c. 10
  - d. 15
19. What is the number of vertices in an undirected connected graph with 27 edges, 6 vertices of degree 2, 3 vertices of degree 4 and the remaining of degree 3?
- a. 10
  - b. 11
  - c. 18
  - d. 19
20. In any undirected graph, the sum of degrees of all the nodes
- a. Must be even
  - b. Twice the number of edges
  - c. Both a and b
  - d. None of the above

### Descriptive questions

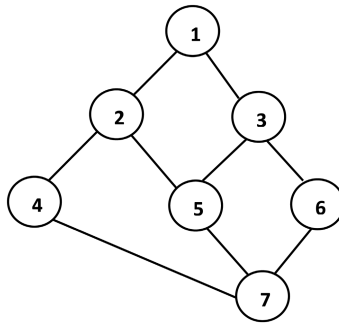
1. Find the degree of each node in the graph given below.



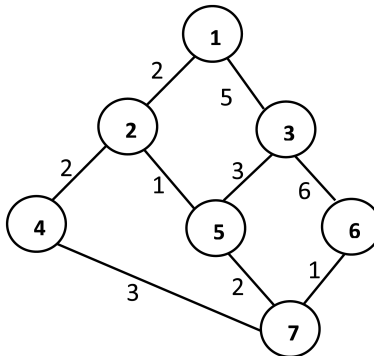
2. Find the bi-connected components of the graph given below.



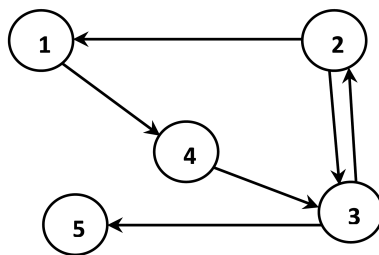
3. Find the order of nodes that is printed in DFS for the following graph.



4. Find the adjacency matrix and adjacency lists for the following graph.



- 5. What is an AND/OR graph and how it is useful in problem-solving?
- 6. Show that a circuitless connected graph with  $n$  vertices has  $n-1$  edges.
- 7. Use the topological sort for the following graph.



- 8. Explain different types of graph traversal techniques with examples.
- 9. What is a directed acyclic graph and what is the usefulness of it?
- 10. Explain the following with examples.
  - i. Weighted graph
  - ii. Edge-labeled graph
  - iii. Cyclic graph

**Answers to true–false questions**

- 1. False
- 2. True
- 3. False
- 4. True
- 5. False

**Answers to fill-in-the-blank questions**

- 1. vertices, edges
- 2. adjacency lists
- 3. complete graph
- 4. weighted graph
- 5. cycle

**Answers to multiple-choice questions**

|     |   |     |   |     |   |     |   |     |   |
|-----|---|-----|---|-----|---|-----|---|-----|---|
| 1.  | c | 2.  | c | 3.  | c | 4.  | b | 5.  | c |
| 6.  | d | 7.  | b | 8.  | a | 9.  | d | 10. | c |
| 11. | a | 12. | c | 13. | b | 14. | b | 15. | b |
| 16. | a | 17. | c | 18. | a | 19. | b | 20. | c |

# Sorting and searching

---

### LEARNING OBJECTIVES

After studying this chapter, the readers will be able to:

- Understand different sorting algorithms
- Implement different sorting algorithms
- Understand different searching algorithms
- Implement different searching algorithms

Sorting is the process of placing an existing group of components in a specific order. A group of integers can be arranged in either ascending (growing) or descending (decreasing) order. If the collection consists of strings, we can arrange them alphabetically (from a to z or z to a) or by string length. For instance, a dictionary will list its words in alphabetical order.

Finding a specific element within a group of components is the definition of searching. Whether an element is present in the collection or not is determined by the search result. If it is, we may also determine where that element is in the specified collection. A key method in computer science is searching. Programmers must be aware of the many methods for searching and retrieving data in order to create algorithms.

In this chapter, we will learn about a few sorting and searching methods and implement them using Python.

### 9.1 INTRODUCTION TO SORTING

Sorting is a fundamental computer science function that entails putting a group of objects or data pieces in a particular order. It is essential to many applications and algorithms because it makes data retrieval, organizing, and analysis more effective. Sorting algorithms are crucial tools in computer programming, whether they are used to arrange a list of names in alphabetical order or to sort a large data set for analysis.



The primary objective of sorting is to rearrange the items according to a predetermined comparison criterion, usually in ascending or decreasing order. This order may be determined by a number of factors, including timestamps, numerical values, alphabetical order or user-defined criteria.

There are numerous types of sorting algorithms, each with distinct advantages and disadvantages. While some algorithms promote effectiveness and ideal performance, others place a higher priority on simplicity and ease of implementation. The amount of the data set, the preferred order and the available processing resources are only a few examples of the variables that influence the choice of sorting algorithm.

Commonly used sorting algorithms include:

1. Bubble sort: A straightforward method that checks nearby elements frequently and swaps them if the order is incorrect.
2. Insertion sort: Through iteratively inserting components into the appropriate locations, this technique creates a sorted subsequence.
3. Selection sort: An algorithm that finds the smallest (or largest) element in an unsorted list and places it in the correct position.
4. Merge sort: This algorithm employs the divide-and-conquer strategy, separating the list recursively into smaller sublists, sorting them and then merging them back together.
5. Quick sort: Choosing a pivot element, dividing the list around it and then recursively sorting the resulting sublists are all aspects of another divide-and-conquer method.
6. Heap sort: This algorithm repeatedly extracts the maximum (or minimum) element from a binary heap data structure to sort the elements.

In this chapter, we will discuss the first three sorting algorithms.

A key component of organizing and arranging data in a certain order is sorting, a fundamental idea in computer science. In numerous applications and algorithms, it is a crucial process that enables effective searching, data analysis and problem-solving. Sorting includes placing a group of items in a particular order, usually according to their values or other predetermined criteria.

Sorting is important when dealing with large amounts of data that need to be sorted, such as a list of names, a database of client data or a collection of numbers. Sorting the data allows us to expedite processes, enhance search results and gain valuable insights from the ordered data.

There are numerous sorting algorithms, and each has advantages and disadvantages in terms of productivity, stability and application for particular data sets. Some of the most well-known sorting algorithms include bubble sort, selection sort, insertion sort, merge sort, quick sort and heap sort. Each algorithm follows a specific set of steps to rearrange the pieces and generate the necessary sorted output.

The size of the data collection, how the data are distributed, memory requirements and desired performance characteristics all play a role in selecting the best sorting method. To choose an algorithm that satisfies the particular needs of the current problem, it is critical to take these elements into account.

### 9.1.1 Bubble sort

The first sorting method we'll learn about is the bubble sort. By continually comparing neighboring elements and swapping them if they are unordered, it sorts a given list of elements. When two items are swapped, their relative positions are also changed. *Pass* is the term used in algorithms to describe each iteration of a list's elements. The bubble sort sorts a list with  $n$  elements over a total of  $n - 1$  passes. The necessary adjacent pairs of list entries will be compared in each pass. The largest element is determined after each pass and positioned at the appropriate location in the list in order to arrange the components in ascending order. This can be considered as the largest element being "bubbled up." Hence the name bubble sort. This sorted element is not considered in the remaining passes and thus the list of elements gets reduced in successive passes.

#### Algorithm 1

```

1. BUBBLESORT( numList, n)
2. SET i = 0
3. WHILE i < n REPEAT STEPS 3 to 8
4.   SET j = 0
5.   WHILE j < n-i-1, REPEAT STEPS 5 to 7
6.     IF numList[j] > numList[j+1] THEN
7.       swap(numList[j], numList[j+1])
8.     SET j=j+1
9. SET i=i+1

```

#### Example

Sort the elements in Figure 9.1a using the bubble sort algorithm.

|       |    |    |    |   |    |   |
|-------|----|----|----|---|----|---|
| List  | 13 | 12 | 18 | 6 | -4 | 9 |
| Index | 0  | 1  | 2  | 3 | 4  | 5 |

Figure 9.1a Bubble sort

Comparison in Pass 1 (Figure 9.1b).

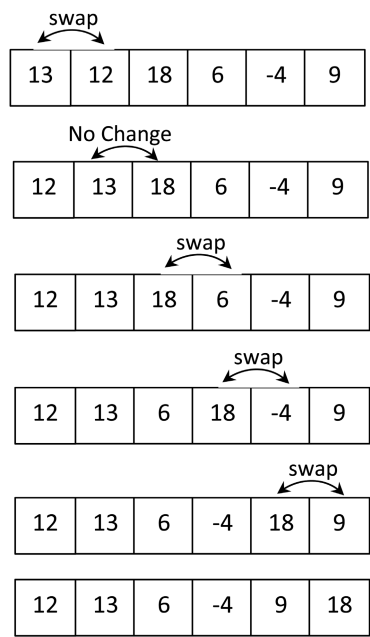


Figure 9.1b Bubble sort – Comparison in Pass 1

Comparison in Pass 2 (Figure 9.1c).

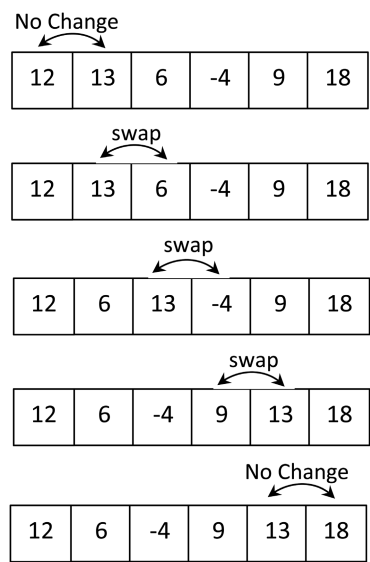


Figure 9.1c Bubble sort – Comparison in Pass 2

Comparison in Pass 3 (Figure 9.1d).

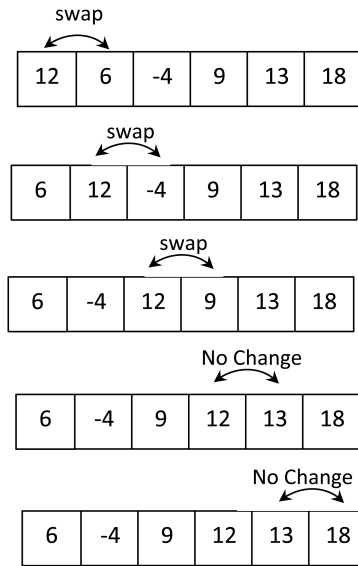


Figure 9.1d Bubble sort – Comparison in Pass 3

Comparison in Pass 4 (Figure 9.1e).

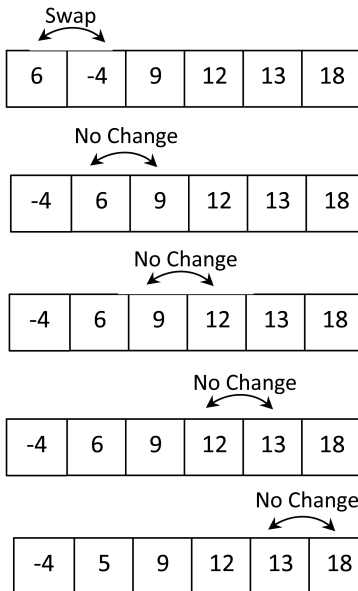


Figure 9.1e Bubble sort – Comparison in Pass 4

Comparison in Pass 5 (Figure 9.1f).

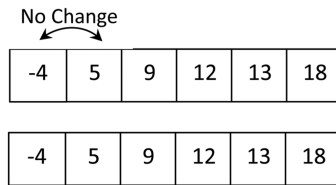


Figure 9.1f Bubble sort – Comparison in Pass 5

### Python program 1

```
def bubblesort(elements):
    swapped = False
    # Looping from size of array from last index[-1] to index [0]
    for n in range(len(elements)-1, 0, -1):
        for i in range(n):
            if elements[i] > elements[i + 1]:
                swapped = True
                # swapping data if the element is less than next
                # element in the array
                elements[i], elements[i + 1] = elements[i + 1],
                elements[i]
        if not swapped:
            # exiting the function if we didn't make a single swap
            # meaning that the array is already sorted.
            return

elements = [13, 12, 18, 6, -4, 9]

print("Unsorted list is,")
print(elements)
bubblesort(elements)
print("Sorted Array is, ")
print(elements)
```

### Output

```
Unsorted list is,
[13, 12, 18, 6, -4, 9]
Sorted Array is,
[-4, 6, 9, 12, 13, 18]
```

### 9.1.2 Insertion sort

Another sorting technique that can place items in a given list in either ascending or descending order is insertion sort. Similar to selection sort,

insertion sort divides the list into two sections: one with sorted entries and the other with unsorted elements. One by one, each item in the unsorted list is taken into account and added to the sorted list in the proper location. The sorted list is walked through once for each pass to locate the potential spot to insert an unsorted element. Thus, the sorting technique is known as insertion sort.

In pass 2, starting with element *e* of the unsorted list in the backward direction, each element of the sorted list will be compared with element *e* of the unsorted list until the proper position for insertion is identified. The components of the sorted list will be moved to the right, creating room for element *e* to be entered.

This keeps happening until every element from unsorted lists has been added to the sorted list at the proper places.

As a consequence, a sorted list is produced, with the components organized in ascending order.

### Algorithm 2

```

INSERTIONSORT( numList, n)
1. i=1
2. WHILE i< n REPEAT STEPS 3 to 9
3. temp = numList[i]
4. j = i-1
5. WHILE j>= 0 and numList[j]>temp, REPEAT STEPS 6 to 7
    numList[j+1] = numList[j]
    j=j-1
6. numList[j+1] = temp #insert temp at position j
7. i=i+1

```

### Example

Sort the data in Figure 9.2a using insertion sort.

|       |    |    |    |   |    |   |
|-------|----|----|----|---|----|---|
| List  | 13 | 12 | 18 | 6 | -4 | 9 |
| Index | 0  | 1  | 2  | 3 | 4  | 5 |

Figure 9.2a Insertion sort

Comparison in Pass 1: At the beginning, 13 was in the list; next 12 is inserted into it (Figure 9.2b).

Comparison in Pass 2: The next data 18 is inserted into the list (Figure 9.2c).

Comparison in Pass 3: The next data 6 is inserted into the list (Figure 9.2d).

Comparison in Pass 4: The next data -4 is inserted into the list (Figure 9.2e).

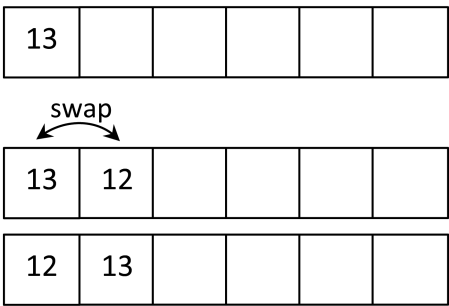


Figure 9.2b Insertion sort – Comparison in Pass 1

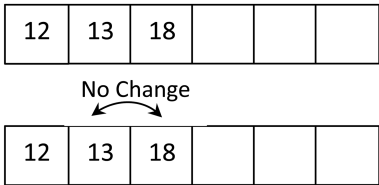


Figure 9.2c Insertion sort – Comparison in Pass 2

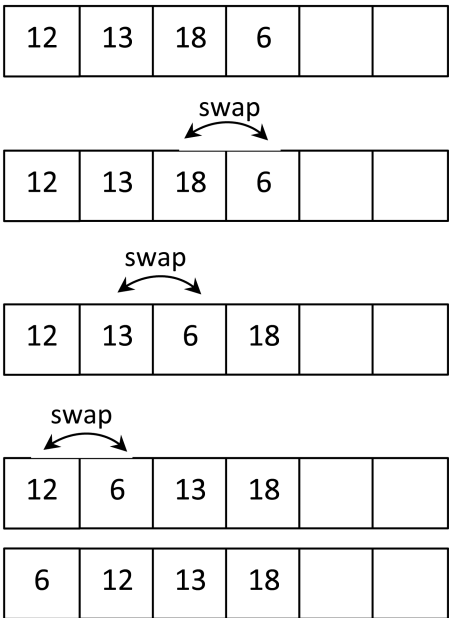


Figure 9.2d Insertion sort – Comparison in Pass 3

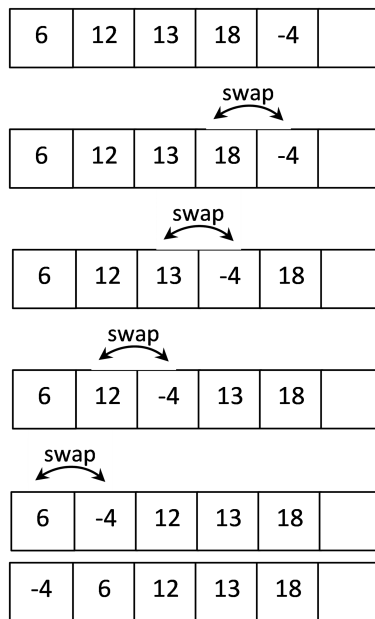


Figure 9.2e Insertion sort – Comparison in Pass 4

Comparison in Pass 5: The next data 9 is inserted into the list (Figure 9.2f).

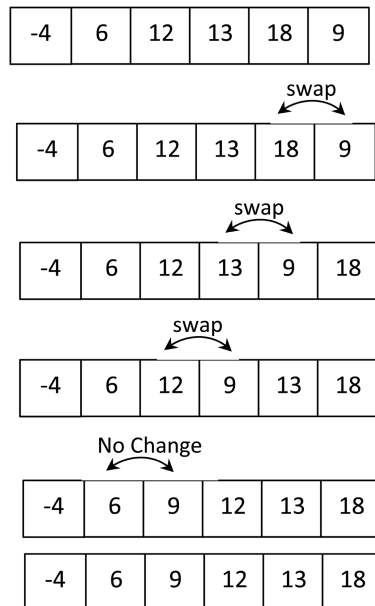


Figure 9.2f Insertion sort – Comparison in Pass 5



## Python program 2

```
# Function to do insertion sort
def insertionSort(arr):

    # Traverse through 1 to len(arr)
    for i in range(1, len(arr)):

        key = arr[i]

        # Move elements of arr[0..i-1], that are
        # greater than key, to one position ahead
        # of their current position
        j = i-1
        while j >=0 and key < arr[j] :
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key

# Driver code to test above
arr = [13, 12, 18, 6, -4, 9]
print("Unsorted list is,")
print (arr)
insertionSort(arr)
print ("Sorted array is:")
print (arr)
```

## Output

```
Unsorted list is,
[13, 12, 18, 6, -4, 9]
Sorted array is:
[-4, 6, 9, 12, 13, 18]
```

### 9.1.3 Selection sort

Another sorting method is selection sort. The selection sort makes  $(n - 1)$  number of trips through the list in order to sort a list with  $n$  elements. The list is thought to be split into two lists, with the right list containing the unsorted elements and the left list carrying the sorted elements. The right list initially has all the elements, but the left list is initially empty.

In the first pass, all of the elements in the unsorted list are scanned to identify the smallest element in order to arrange the elements in ascending order. The leftmost element of the unsorted list is then switched for the smallest element. This element is at the top spot in the sorted list and is skipped over in subsequent passes. The leftmost member of the unsorted list is swapped with the next-smallest element in the second pass, which is chosen from the remaining items in the unsorted list.

This element is the second item in the sorted list, and during the third run, the unsorted list is reduced by one element. This procedure is repeated until the  $n-1$  smallest elements are located and placed in the appropriate locations. The final and previously present ingredient is the  $n$ th one.

**Algorithm 3**

```
SELECTIONSORT( numList, n)
1. i=0
2. WHILE i< n REPEAT STEPS 3 to 11
3.   min = i, flag = 0
4.   j= i+1
5.   WHILE j< num REPEAT STEPS 6 to 10
6.     IF numList[j] < numList[min] THEN
7.       min = j
8.       flag = 1
9.   IF flag = 1 THEN
10.    swap(numList[i],numList[min])
11. i=i+1
```

**Example**

Sort the data in Figure 9.3a using insertion sort.

|       |    |    |    |   |    |   |
|-------|----|----|----|---|----|---|
| List  | 13 | 12 | 18 | 6 | -4 | 9 |
| Index | 0  | 1  | 2  | 3 | 4  | 5 |

Figure 9.3a Selection sort

Comparison in Pass 1 (Figure 9.3b).

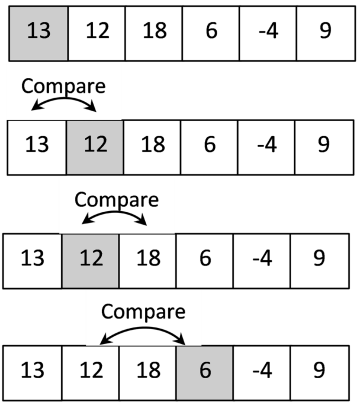


Figure 9.3b Selection sort – Comparison in Pass 1

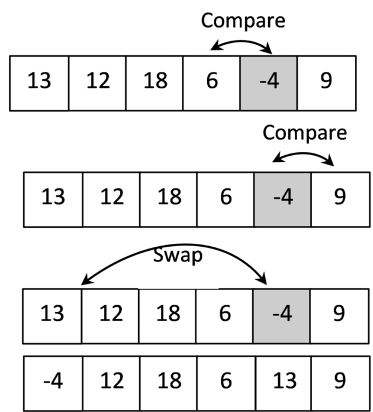


Figure 9.3b Continued

Comparison in Pass 2 (Figure 9.3c).

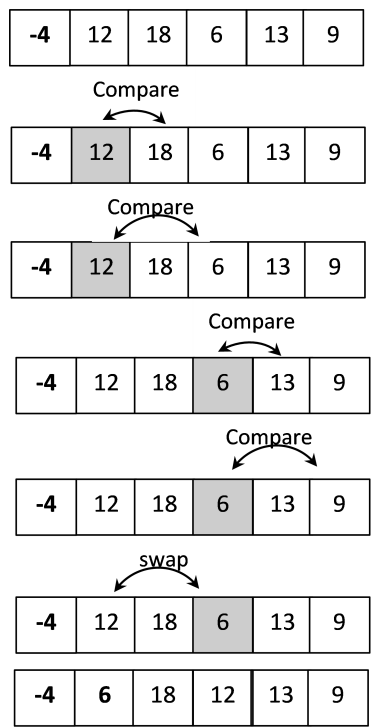


Figure 9.3c Selection sort – Comparison in Pass 2

Comparison in Pass 3 (Figure 9.3d).

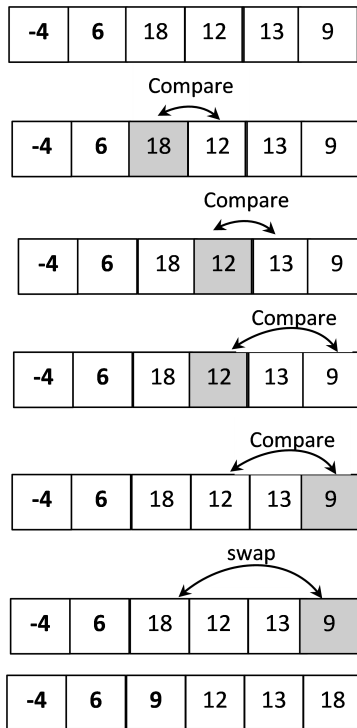


Figure 9.3d Selection sort – Comparison in Pass 3

Comparison in Pass 4 (Figure 9.3e).

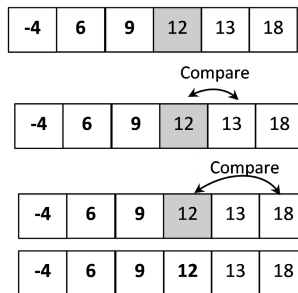


Figure 9.3e Selection sort – Comparison in Pass 4

Comparison in Pass 5 (Figure 9.3f).

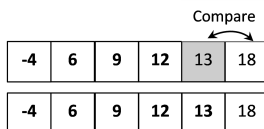


Figure 9.3f Selection sort – Comparison in Pass 5

### Python program 3

```
A = [13, 12, 18, 6, -4, 9]
print ("UnSorted array")
print(A)
# Traverse through all array elements
for i in range(len(A)):

    # Find the minimum element in remaining
    # unsorted array
    min_idx = i
    for j in range(i+1, len(A)):
        if A[min_idx] > A[j]:
            min_idx = j

    # Swap the found minimum element with
    # the first element
    A[i], A[min_idx] = A[min_idx], A[i]

# Driver code to test above

print ("Sorted array")
print(A)
```

### Output

```
UnSorted array
[13, 12, 18, 6, -4, 9]
Sorted array
[-4, 6, 9, 12, 13, 18]
```

## 9.2 INTRODUCTION TO SEARCHING

The practice of choosing specific information based on predetermined criteria from a set of data is called searching. This idea is known to you from your experiences searching the web for sites that include specific words or phrases or from looking up phone numbers in phone books. We limit the

definition of searching in this article to the act of locating a particular item within a collection of data items.

There are several data structures on which the search function can be used. The main topic of this chapter is sequence search, which is locating a specific item inside a sequence by employing a search key. The data items in a collection are identified by a key, which is a distinct value. The values in a collection of basic types, such as integers or reals, are the keys.

It is necessary to determine which particular data component is the key for collections of complicated kinds. Compound keys, as they are sometimes called, are made up of several different parts.

### 9.2.1 Linear search

Among search techniques, linear search is the most basic and straightforward. It is a thorough search method in which each entry in a list is compared to the object you're looking for, also known as the "key." As a result, the key is compared one by one with each member in the list. This procedure is repeated until an element that matches the key is located, at which point we proclaim the search to be successful. We deem the search failed if, after searching the whole list, no element matches the key, indicating that the key is not present in the list. The order in which the items are contained in the list is followed when comparing each item individually, starting with the first and working all the way down to the last. As a result, it is also known as a serial or sequential search. When gathering little, unordered things, this strategy might be helpful.

#### Algorithm 4

```
LinearSearch(numList, key, n)
  1. index = 0
  2. WHILE index < n, REPEAT Step 3
  3.   IF numlist[index] = key THEN
  4.     PRINT "Element found at position", index+1 STOP
  5.   ELSE index = index+1
  6. PRINT "Search unsuccessful"
```

#### Example

Search for key = 6 in the given list.

|       |    |    |    |   |    |   |
|-------|----|----|----|---|----|---|
| List  | 13 | 12 | 18 | 6 | -4 | 9 |
| Index | 0  | 1  | 2  | 3 | 4  | 5 |

Here  $n = 6$  (number of data available) (Table 9.1).

Table 9.1 Linear search

| <i>Index</i> | <i>Index &lt; n</i> | <i>numList[index] = key</i> | <i>Index = index + 1</i> |
|--------------|---------------------|-----------------------------|--------------------------|
| 0            | 0 < 6 ? Yes         | 13 = 6 ? No                 | 1                        |
| 1            | 1 < 6 ? yes         | 12 = 6 ? No                 | 2                        |
| 2            | 2 < 6 ? Yes         | 18 = 6 ? No                 | 3                        |
| 3            | 3 < 6 ? Yes         | 6 = 6 ? yes                 |                          |

Observe that after four comparisons, the algorithm found the key 17 and will display “Element found at position 4”.

Let us search for key = 17 (Table 9.2).

Table 9.2 Linear search

| <i>Index</i> | <i>Index &lt; n</i> | <i>numList[index] = key</i> | <i>Index = index + 1</i> |
|--------------|---------------------|-----------------------------|--------------------------|
| 0            | 0 < 6 ? Yes         | 13 = 17 ? No                | 1                        |
| 1            | 1 < 6 ? Yes         | 12 = 17 ? No                | 2                        |
| 2            | 2 < 6 ? Yes         | 18 = 17 ? No                | 3                        |
| 3            | 3 < 6 ? Yes         | 6 = 17 ? No                 | 4                        |
| 4            | 4 < 6 ? Yes         | -4 = 17 ? No                | 5                        |
| 5            | 5 < 6 ? Yes         | 9 = 17 ? No                 | 6                        |
| 6            | 6 < 6 ? No          |                             |                          |

Observe that after six comparisons, the algorithm does not find the key 17 and will display “Search unsuccessful”.

Python program 4

```
def linear_Search(list1, n, key):  
    # Searching list1 sequentially  
    for i in range(0, n):  
        if (list1[i] == key):  
            return i  
    return -1  
  
list1 = [13, 12, 18, 6, -4, 9]  
key = 6  
  
n = len(list1)  
res = linear_Search(list1, n, key)  
if(res == -1):  
    print("Search unsuccessful")  
else:  
    print("Element",key,"found at index: ", res)
```

Output

Element 6 found at index: 3

### 9.2.2 Binary search

The binary search is a method of rapidly finding a key by using the ordering of the list's members. The list's members can be ordered according to their key values in either ascending or descending order for numeric values. Textual data can either be sorted alphabetically from z to a or from a to z.

In a binary search, the element in the center of a sorted list is compared with the key to be searched. Any one of the following three outcomes might arise from this:

- i) The element in the middle place matches the key.
- ii) The element is larger than the key.
- iii) The element is smaller than the key.

The search is deemed successful and comes to an end if the element in the middle position matches the key.

If the middle element exceeds the key, the key must unquestionably be in the first half of the list, if it is present at all. Therefore, we may skip over the second half of the list right away and limit our search to the first half.

If the middle element is smaller than the key, the key must be in the second half of the list, if it is there at all. Thus, we may skip over the first half of the list right away and limit our search to the second half. Until the key was located or the remaining list had just one item, this splitting and list-size reduction process is carried out. The search fails if that item is not the key since the key is not in the list.

Therefore, it is clear that a binary search can be more effective than a linear search of individual components if the list we need to search is organized in a certain way. In other words, the list must be sorted. The floor division ( $//$ ) operator is used to determine the mid value if the list to be searched has an even number of entries. The midpoint (mid) of the list, if there are 10 entries, is equal to  $10//2$ , or 5. Since the first element in the list has an index value of 0, the sixth element in the list is thus regarded as the middle element. If required, the list is further divided into two parts where the first half contains five elements and the second half contains four elements.

Interestingly, the intermediate comparisons that do not locate the key nonetheless provide information about the section of the list that could contain the key! They indicate whether the key is located before or after the list's current middle position, and we make use of this information to limit the scope of our search. The term *binary search* comes from the fact that every failed comparison cuts the amount of components that need to be searched by half. Now let's talk about the binary search algorithm.



Algorithm 5

```
BinarySearch(numList, key)
1. first = 0, last = n-1
2. mid = (first+last)//2
3. WHILE first <= last REPEAT Step 4
4. IF numList[mid] = key THEN
    PRINT "Element found at position", " mid+1
    STOP
    ELSE IF numList[mid] > key, THEN
        last = mid-1 ELSE first = mid + 1
5. PRINT "Search unsuccessful"
```

Example

Consider a sorted list consisting of 15 elements: numList = [2, 3, 5, 7, 10, 11, 12]. The key = 10, has to be found in numList. The index values of the first, middle and final elements are found in the numList.

Search for key = 10 in the given list. (Also see Tables 9.3 and 9.4.)

|       |   |   |   |   |    |    |    |
|-------|---|---|---|---|----|----|----|
| List  | 2 | 3 | 5 | 7 | 10 | 11 | 12 |
| Index | 0 | 1 | 2 | 3 | 4  | 5  | 6  |

Table 9.3 Binary search – data

|           | First |   |   | Mid |    |    | Last |
|-----------|-------|---|---|-----|----|----|------|
| Index     | 0     | 1 | 2 | 3   | 4  | 5  | 6    |
| numList[] | 1     | 2 | 5 | 7   | 10 | 11 | 12   |

Table 9.4 Binary search – search key =10

|             | First | Last | Mid          | numList<br>[mid]==key ? | Key <<br>numList[mid] ? | First <= last |
|-------------|-------|------|--------------|-------------------------|-------------------------|---------------|
| Iteration 0 | 0     | 6    | (0+6)//2=3   | 7 == 10 No              | 10 < 7 No               | 0 <= 6 Yes    |
| Iteration 1 | 4     | 6    | (4+6)//2=5   | 11 == 10 No             | 10 < 11 Yes             | 4 <= 6 Yes    |
| Iteration 2 | 4     | 5    | (4+5)//2 = 4 | 10 == 10 Yes            |                         |               |

Python program 5

```
def binary_search(arr, low, high, x):
    # Check base case
    if high >= low:
        mid = (high + low) // 2
        # If element is present at the middle itself
        if arr[mid] == x:
            return mid
```

---

```

    # If element is smaller than mid, then it can only
    # be present in left subarray
    elif arr[mid] > x:
        return binary_search(arr, low, mid - 1, x)

    # Else the element can only be present in right subarray
    else:
        return binary_search(arr, mid + 1, high, x)

else:
    # Element is not present in the array
    return -1

# Test array
arr = [2, 3, 5, 7, 10, 11, 12]
x = 10

# Function call
result = binary_search(arr, 0, len(arr)-1, x)

if result != -1:
    print("Element",x,"is present at index", str(result))
else:
    print("Element is not present in array")

```

## Output

Element 10 is present at index 4

## True-false questions

- The `sort()` method is valuable because it keeps elements of parallel arrays in the correct order.
  - True
  - False
- The given array is `arr = {1,2,4,3}`. Bubble sort is used to sort the array elements. Two iterations are required to sort the array with an improvised version.
  - True
  - False
- The advantage of selection sort over other sorting techniques is that it does not require additional storage space.
  - True
  - False
- When the list has only a few elements and when performing a single search in an unordered list, then a linear search is used.
  - True
  - False
- Given `{5,6,7,8,9}` and `key = 8`, three iterations are done until the element is found using binary search.
  - True
  - False

**Fill-in-the-blank questions**

1. For the given array {45,77,89,90,94,99,100} with searching key = 100, \_\_\_\_\_ and \_\_\_\_\_ are the mid values generated in the first and second iterations.
2. The number of iterations in bubble sort and selection sort, respectively, for the array {3,4,5,2,1} are \_\_\_\_\_ and \_\_\_\_\_.
3. For the array {1, 2, 4, 3}, to sort the items using bubble sort \_\_\_\_\_ iterations are required.
4. The linear search (recursive) algorithm is used \_\_\_\_\_.
5. For the array {1,2,3,6,8,10} with search key 17, \_\_\_\_\_ calls are required to confirm an unsuccessful search.

**Multiple-choice questions**

1. For the array {25, 45, 87, 21, 18, 49, 13, 115, 83, 65}, how many searches are required to find 83?
  - a. 8
  - b. 9
  - c. 10
  - d. 11
2. What benefit does a recursive approach have over an iterative one?
  - a. Requires more memory
  - b. Uses less memory
  - c. Requires more code to be written
  - d. Requires less code and is simple to implement
3. Out of the following, which one does not involve the use of binary search?
  - a. Union of intervals
  - b. To look through an unsorted list
  - c. Debugging
  - d. Determining the top and lower bounds of an ordered sequence
4. Linear search is also called
  - a. Perfect search
  - b. Random search
  - c. Sequential search
  - d. None of the above
5. The necessary condition for using binary search in an array is
  - a. The array should of more size
  - b. The array should not be too long
  - c. The array should be sorted
  - d. None of the above

- ## Descriptive questions

1. Consider a list of ten elements: numList = [7, 11, 3, 10, 17, 23, 1, 4, 21, 5]. Display the partially sorted list after three complete passes of bubble sort.
2. Determine the number of swaps needed to sort the following list using bubble and selection sort, and determine which sorting method is more effective in terms of comparisons made (2, 5, 34, 8, 90, -33, 43, -22, 32).

3. Students' names are entered into a course in increasing order at the time of acceptance. The sorting process is completed concurrently with adding entries to a list. Determine the kind of sorting method being employed and create a program with a user-defined function that saves names in the list in ascending order each time a name is entered.
4. Using a linear search, determine the position of 8, 1, 99 and 44 in the list: [1, -2, 32, 8, 17, 19, 42, 13, 0, 44]. Draw a detailed table showing the values of the variables and the decisions taken in each pass of the linear search.
5. Write a program that takes as input a list having a mix of ten negative and positive numbers and a key value. Apply a linear search to find whether the key is present in the list. If the key is present, it should display the position of the key in the list; otherwise it should print an appropriate message. Run the program for at least three different keys and note the result.

#### Answers to true-false questions

1. False
2. True
3. True
4. True
5. False

#### Answers to fill-in-the-blank questions

1. 90, 99
2. 5, 4
3. four
4. when the size of the dataset is low
5. seven

#### Answers to multiple-choice questions

- |    |   |    |   |    |   |    |   |     |   |
|----|---|----|---|----|---|----|---|-----|---|
| 1. | b | 2. | d | 3. | b | 4. | c | 5.  | c |
| 6. | a | 7. | b | 8. | a | 9. | d | 10. | b |

---

# Index

---

- @classmethod, 47, 48
- @staticmethod, 53
- \_\_del\_\_(), 44
- \_\_init\_\_(), 41
- \_\_new\_\_(), 42
- \_\_init\_\_(), 40
- \_\_next\_\_(), 29
- Abstract data type, 72
- Accessing attributes of inner class, 55
- Accessing variables, 48
- addAtIndex(), 100
- Advantages of doubly linked lists, 160
- Algorithms, 73
- Ancestor, 280
- AND/OR graphs, 359
- anonymous function, 36
- Applications of binary search trees, 314
- Applications of Stack, 209
- Argument, 4, 33, 34
- Arithmetic Operators, 14
- Array, 87
- Array as an abstract data type, 88
- Assigning values to variables, 2
- Assignment Operator, 15
- Attribute, 39
- AttributeError, 44, 51
- AVL trees (height-balanced trees), 315
- Big oh notation, 80
- Bi-connected components, 361
- Binary search, 391
- Binary search trees, 295
- Binary trees, 280
- Binary tree traversals, 288
- Bitwise operators, 18
- Boolean data type, 6
- Breadth-first search, 356
- Break, 28
- Bubble sort, 377
- built-in methods, 10
- built-in data structures, 88
- Calling a function, 30
- Child, 279
- Circular linked lists, 160
- Circular queue, 251
- Class, 5, 39
- Class inside a class, 54
- Classification of data structures, 64
- Class method, 52
- Class variables, 46, 47
- class\_suite, 39
- Comment Statement, 5
- Comparison (relational) operators, 16
- Complexity analysis, 79
- Concatenation, 7, 9, 68
- Conditional statements, 20
- Continue, 27
- count, 78
- Defining a function, 30
- Degree, 279
- Delimiter matching, 209
- Depth-first search, 348
- Deque, 230
- derived data type, 72
- Descendent, 280
- Dictionary, 12
- Dictionary methods, 14
- Dictionary methods to access key, 13
- double underscore (\_\_), 50
- Double-ended queue, 257
- Doubly linked lists, 147, 160

- elif statement, 21
- encapsulation, 39
- Enqueue, 228
- Escape characters, 2
- eval(), 3
  
- file, 71
- Float, 66
- Flow control statements, 19
- Forest, 280
- For loop, 25
- Formal and actual arguments, 33
- Functions, 30
- Function decorators, 36
- Function generators, 37
  
- grandparent, 328
- Graph representation, 348
  
- Identifiers, 1
- Identity operator, 18
- if-else clause, 20
- Implementation, 187
- Implementation
  - of array
    - using list, 89
    - using dictionary, 92
    - using import array, 96
    - using NumPy, 98
    - using class, 99
  - of priority queue
    - by importing PriorityQueue
      - package of queue module, 268
    - using class, 263
    - using heapq module, 271
  - of queue
    - using a dictionary, 240
    - using a class, 243
    - using queue module, 246
  - of stack
    - using a dictionary, 196
    - using a class, 199
    - using queue module, 206
  - using a list, 232
- Import, 39, 96
- Importing a module, 38, 89
- Increment Operator, 26
- Indentation, 5, 20
- Internal node, 280
- Infix-postfix conversion, 212
- Initialization, 41, 43
- Initializer, 46, 243
- Inner Class, 54
- input(), 2
- insert(), 10, 130, 259
- Insertion sort, 380
- Instance, 40, 46, 51
- Instance method, 51
- Instantiation, 40, 43, 219
- Integer, 2, 6, 14, 66
- Intersection, 12, 73
- invocation, 219
- exception, 29
- isEmpty(), 187, 188, 199
- isFull(S), 186
- isalpha(), 8
- isdisjoint(), 12
- issubset(), 12
- isupper(), 8
- items(), 13
- iter(), 29
- Iterator, 29
  
- key, 12
- keys(), 13
- Keywords, 2, 49, 67
- keyword argument, 4
  
- lambda, 36
- Leaf node, 280
- len(), 10
- Little omega, 82
- Linear search, 389
- List, 8
- List methods, 10
- Logical operators, 17
- Looping statement, 23
- lower(), 8
  
- map(), 36
- max(), 8, 75
- Membership operators, 18
- Methods, 8, 10, 13, 14, 40, 51
- min(), 8, 10
- Module, 38
- Multidimensional arrays, 104
  
- name(), 40, 53
- Namespace, 39
- Nested loops, 28
- Newline, 4
- None, 13, 14, 65, 67
- Non-linear data structures, 70
- Non-primitive data structures, 68
- Numeric data type, 6

- objects, 39
- Object variables, 46
- Object oriented, 51
- Omega notation, 81
- Operating System, 177
- Operations
  - in a binary tree, 287
  - in a circular linked list, 161
  - in a doubly linked list, 148
  - in a singly linked list, 133
- Operators, 14, 213
- Outer Class, 54
- Parameter, 30, 33
- Parent, 279
- pass, 27
- Pass by reference, 32
- Pass by value, 32
- peek(), 186
- Polynomial manipulation, 105
- pop(), 185
- Positional arguments, 33
- postfix expression, 216
- Precedence of operators, 213
- Priority queue, 262
- PriorityQueue(), 270
- Primitive data structures, 66
- Pseudo-code, 74
- Public, private and protected
  - variables, 49
- push(), 184
- Python operators precedence, 19
- Queue, 69, 227
- Queue ADT operations, 232
- Queue operations, 228
- range() function, 26
- readArray(), 100, 102
- Recursion, 75, 218
- Recursive functions, 35
- reduce(), 36
- Relational Operators, 16, 19, 213
- Relationship, 51, 64, 70
- Repetition, 7
- representing a binary tree, 285
- Retrieval, 63, 375
- Return, 2, 30
- Returning results from a function, 31
- Returning multiple values from a
  - function, 31
- reverse(), 10, 110
- Root, 279
- Searching, 388
- Selection sort, 384
- Self, 40, 41
- Sep, 3, 4
- Sequence, 29, 389
- serial, 389
- Set, 11
- Set operations, 12
- Shortest-path problem, 365
- Siblings, 279
- single leaf, 389
- Single left, 329
- Singly linked lists, 131
- Small oh notation, 82
- sorting, 375
- Space complexity, 77
- Sparse matrices, 111
- Sparse matrix addition, 116
- Splay trees, 328
- Stacks, 183
- Stack operations, 184
- Stack (ADT), 186
- Standard data types, 5
- Static method, 53
- Static variables, 46
- str(), 3
- String data type, 7
- String methods, 8
- Strings, 3, 7
- strong private, 50
- super(), 42
- Symmetric Difference, 12
- Ternary operator, 23
- Theta notation, 81
- Time complexity, 78
- Topological sorting, 367
- Transpose, 114
- Tree, 70, 278
- Tuple, 9
- Type casting, 3
- Types of binary trees, 281
- upper(), 8
- Variables, 1, 2, 45
- Variables and methods, 45
- While loop, 23
- write(), 77, 327
- yield, 2, 37, 38
- zip(), 13